Xiaochun Wang
Xiali Wang
Don Mitchell Wilkes

Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment

西安交通大学出版社
XI'AN JIAOTONG UNIVERSITY PRESS

Springer

Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment

Xiaochun Wang · Xiali Wang ·
Don Mitchell Wilkes

Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment

西安交通大学出版社
XI'AN JIAOTONG UNIVERSITY PRESS

Springer

Xiaochun Wang
School of Software Engineering
Xi'an Jiaotong University
Xi'an, Shaanxi, China

Xiali Wang
School of Information Engineering
Chang'an University
Xi'an, Shaanxi, China

Don Mitchell Wilkes
Department of Electrical Engineering
and Computer Science
Vanderbilt University
Nashville, TN, USA

ISBN 978-981-13-9216-0 ISBN 978-981-13-9217-7 (eBook)
https://doi.org/10.1007/978-981-13-9217-7

Jointly published with Xi'an Jiaotong University Press
The print edition is not for sale in China. Customers from China please order the print book from: Xi'an Jiaotong University Press.

© Xi'an Jiaotong University Press 2020
This work is subject to copyright. All rights are reserved by the Publishers, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publishers, the authors, and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publishers nor the authors or the editors give a warranty, express or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publishers remain neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Singapore Pte Ltd.
The registered company address is: 152 Beach Road, #21-01/04 Gateway East, Singapore 189721, Singapore

Foreword

Learning is an active research topic in robotics. Evidence from cognitive sciences shows that the working memory in primate brains plays a crucial role in the learning process, in part by fixing attention on the most relevant data. Autonomous mobile robot navigation sets out enormous theoretical and applied challenges to advanced robotic systems using machine learning techniques. This monograph addresses mobile robot localization in an unknown environment by machine learning based natural scene recognition. Unlike human brain, artificial information systems often follow human instructions with regard to how they should work about all possible situations and, however, cannot do anything in unexpected situations. The algorithm of choice for mobile robot localization is a generic-based machine learning method that combines reinforcement learning and unsupervised and supervised learning algorithms in a cognitive-plausible fashion and bridges the gap between feature representations and decision-making.

Having been exploited in computer vision for years to take computer vision out of the lab and into the real environments, cameras can be used to reduce the overall cost and maintain high degree of intelligence, flexibility. and robustness. In this book, an object identification model is presented which uses patch-based color histograms for image segmentation and relies on unsupervised learning algorithms (i.e., clustering algorithms) for perceptual grouping in order to detect objects existing in an unknown environment. By this model to represent elemental objects in a 2D scene, not only discriminative features (i.e., the patch-based color histograms) but also their spatial and semantic relationships are used to categorize a scene.

Image understanding is a research area involving both feature extraction and object identification within images from a scene, and a posterior treatment of this information in order to establish relationships between these objects with a specific goal. To infer the semantic content of images and videos for mobile robotic localization tasks, this research exploits machine learning and pattern recognition techniques and presents two different strategies for online learning, namely, a supervised learning strategy where the parsed images are produced based on training data obtained beforehand by the unsupervised learning, and a

semi-supervised learning strategy in which the robot generates training data via exploration while wandering in the environment.

It is realized that humans acquire various behaviors based not only on generic information but also on post-natal learning. Learning motor skills is an active research topic in robotics. However, most solutions are optimized for industrial applications, and thus, few are plausible explanations for learning in the human brain. Increase in computational power of modern computers fosters new approaches to advanced signal processing and there is a trend to shift functional behavior of industrial automation systems from hardware to software to increase flexibility. While it is beneficial to use vision sensors in many vision applications such as object and scene recognition, building bridges between natural and artificial computation is one of the main motivations for this book. Adding human-like flexibility by learning can equip the robot with the ability of generalization, guessing a correct behavior for an unexperienced situation based on learned relationships between behavior and situation. From a biological viewpoint, it has been claimed that the hippocampus region is especially engaged in spatial learning and takes part in forming maps of the external world. In this strategy, the hippocampus, as a network of cells and pathways that receive information from all of the sensory systems, learns about the spatial configural representations of the sensory world and then acts on the motor systems to produce appropriate spatial behavior.

The chapters cover such topics as image segmentation based visual perceptual grouping for the efficient identification of objects composing an unknown environment, classification-based fast object recognition for the semantic analysis of natural scenes in the unknown environment, the present understanding of the Hippocampal working memory mechanism and its biological processes for human-like localization, and the application of this present understanding towards mobile robot localization improvement. The book also features a perspective on bridging the gap between feature representations and decision-making using reinforcement learning, laying the groundwork for future advances in mobile robot navigation research. I hope natural scene recognition based localization for mobile robots will serve as an invaluable reference for mobile robotic researchers for years to come.

Xi'an, P.R. China
May 2019

Xubang Shen
Chinese National Academician

Preface

This book is the final result of two research projects that occurred to us, "ITR: A Biologically Inspired Adaptive Working Memory System for Efficient Robot Control and Learning" (USNSF Grant EIA-0325641, 2003–2008) which happened at Center for Intelligent System of School of Engineering, Vanderbilt University and "A Study on Natural Scene Recognition-Based Localization for Mobile Robots in An Outdoor Unknown Environment" (CNSF Grant 61473220, 2015–2018) which happened at School of Software Engineering of Xi'an Jiaotong University. Autonomous robots are intelligent machines capable of performing tasks in the real world without explicit human control for extended periods of time. A high degree of autonomy is particularly desirable in fields where robots can replace human workers, such as state-of-the-practice video surveillance system and space exploration. However, not having human's sophisticated sensing and control system, two broad open problems in autonomous robot systems are the perceptual discrepancy problem, that is, there is no guarantee that the robot sensing system can recognize or detect objects defined by a human designer, and the autonomous control problem, that is, how the robots can operate in unstructured environments without continuous human guidance. As a result, autonomous robot systems should have their own ways to acquire percepts and control by learning.

In the first project, a computer vision system was used for visual percept acquisition and a working memory toolkit was used for robot autonomous control. Natural images contain statistical regularities which can set objects apart from each other and from random noise. For an object to be recognized in a given image, it is often necessary to segment the image into non-overlapping but meaningful regions whose union is the entire image. Therefore, a biologically based percept acquisition system was developed to build an efficient low-level abstraction of real-world data into percepts. Perception in animals is strongly related to the type of behavior they perform. Learning plays a major part in this process. To solve how the robots can learn autonomously to control their behavior based on percepts they have acquired, the computer vision system was integrated with a software package called the Working Memory Toolkit (WMtk) for decision-making and learning. The WMtk was developed by Joshua L. Phillips and David C. Noelle based on a neuron

computational model of primate working memory system. The success of the whole system was demonstrated by its application to a navigation task in an indoor environment.

This research went on to outdoor environments after the National Natural Science Foundation of China funded research project was awarded. Navigation is an important ability of mobile robots. Localization in an environment is the very first step to achieve it. In this project, based on the extensive research already conducted for known indoor environments, a natural landmark-based localization strategy was designed for a mobile robot working in an unknown outdoor environment. Particularly, a real-time scene recognition scheme was developed so as to use objects segmented in it as the natural landmarks and to explore the suitability of configural representation for automatic scene recognition in robot navigation by conducting experiments designed to infer semantic prediction of a scene from different configurations of its stimuli using a machine learning paradigm named reinforcement learning. Unlike traditional localization approaches, the proposed machine learning based approaches do not require the use of the coordinate information.

The purpose of this book is to provide easy access to our contributions to localization theory. To describe the original contribution, explain and interpret it, the book is organized in four parts. In the introductory part, an overview of some of the many facets of localization problems is first provided. Then a perceptual learning model is described for a vision-based autonomous mobile robot system including visual feature extraction, perceptual binding, object representation and recognition. In the second part, the chapters feature the latest developments in unsupervised learning techniques in general and in image segmentation-based perceptual grouping in specific. The third part devotes to the efficient identification of objects composing an unknown environment using supervised learning. Finally, in the fourth part, the chapters discuss a visual percept-based autonomous behavior learning system with a loose neuro-biological bases for cognitive learning paradigm. First, an entire chapter is devoted to reinforcement learning methodologies in general and an implementation for TD-learning, that is, the Working Memory Toolkit (WMtk), in specific for robot decision-making and learning. The last chapters delve back into the machine learning based solutions to mobile robot localization problems.

It is our hope that graduate students, young and senior researchers, and professionals from both academia and industry will find the book useful for understanding and reviewing current approaches in mobile robot localization research.

Xi'an, China — Xiaochun Wang
Xi'an, China — Xiali Wang
Nashville, USA — Don Mitchell Wilkes

Acknowledgements

First and foremost, the authors would like to thank National Natural Science Foundation of China for its valuable support of this work under award 61473220. Without the support, this work would not have been possible.

The authors gratefully acknowledge the contribution of many people. First of all, they would like to take this opportunity to acknowledge the work of the graduate students of School of Software Engineering at Xi'an Jiaotong University, Yuchao Ma, Chenyu Chang, Yiqin Chen and Jing Wang for their diligence and quality work through this project. More specifically, Y. Ma conducted experiments to demonstrate the advantages of the fast approximate quantization search tree. C. Chang developed a BIRCH tree based fast incremental spectral clustering algorithm. Y. Chen proposed a k-nearest neighbors centroid based density based clustering algorithm. J. Wang accomplished all the localization experiments for an outdoor environment. The authors are also greatly indebted to Dr. Kazuhiko Kawamura at the Electrical Engineering and Computer Science Department of Vanderbilt University and Dr. David C. Noelle at the Department of Cognitive Science of University of California, Merced for their kind suggestions and efforts about the proposed experiments. The authors want to convey special thanks to Dr. Daniel Fleetwood, Dr. James A. Cadzow, Dr. Francis Wells, Dr. Douglas Hardin, Dr. Nilanjan Sarkar, Dr. Bharati Mehrotra for their personal support and help, and Jonathan Hunter, Mert Tucgu, Soradech Krootjohn, Faisal Al-hammadi for their friendships and technical supports. The authors would like to thank Yuan Bao of Xi'an Jiaotong University for her kind cooperation in connection with writing this book.

Finally, the authors wish to express their deep gratitude to their families for their assistance in many ways for the successful completion of the book.

Contents

Part I Introduction

1 Overview and Contributions 3
1.1 Introduction 3
1.2 Research Issues on Learning in Mobile Robotic Localization 4
1.3 Overview of the Book 5
1.4 Contributions 7
1.5 Conclusions 10

2 Developments in Mobile Robot Localization Research 11
2.1 Localization Problems: Its Problem Statement and Its Components 11
2.2 A Short History of the Early Developments in Mobile Robotics 12
2.3 Some Standard Localization Approaches 13
2.3.1 Dead-Reckoning Localization Approach 14
2.3.2 Triangulation Based Absolute Localization Using Landmarks 16
2.3.3 Simultaneous Localization and Mapping (SLAM) 20
2.4 Conclusions 30
References 30

3 A Computer Vision System for Visual Perception in Unknown Environments 35
3.1 Introduction 35
3.2 Information Acquisition 37
3.2.1 Vision Sensors 39
3.2.2 Color Space 40

3.3 Representation . . . 43
3.3.1 Feature Extraction . . . 44
3.3.2 Place Recognition . . . 48
3.3.3 Image Segmentation . . . 49
3.4 Reasoning . . . 51
3.5 A Vision Based Machine Perceptual Learning System . . . 52
3.5.1 Unsupervised Learning for Percept Acquisition . . . 53
3.5.2 Supervised Learning for Object Recognition . . . 55
3.5.3 Reinforcement Learning for Autonomous Control . . . 56
3.6 Conclusions . . . 58
References . . . 59

Part II Unsupervised Learning

4 Unsupervised Learning for Data Clustering Based Image Segmentation . . . 63
4.1 Introduction . . . 63
4.1.1 Factor Analysis . . . 64
4.1.2 Clustering . . . 65
4.2 Partitioning-Based Clustering Algorithms . . . 66
4.2.1 *K*-Means Algorithm . . . 67
4.2.2 *K*-Medoids Algorithm . . . 68
4.3 Hierarchical Clustering Algorithms . . . 69
4.3.1 Agglomerative Algorithms . . . 69
4.3.2 Divisive Algorithms . . . 70
4.4 Density-Based Clustering . . . 71
4.4.1 Density-Based Clustering with DBSCAN . . . 71
4.5 Graph-Based Algorithms . . . 73
4.5.1 Minimum Spanning Tree Based Clustering . . . 74
4.5.2 Spectral Clustering . . . 75
4.6 Distance and Similarity Measures . . . 75
4.7 Clustering Performance Evaluation . . . 76
4.7.1 Internal Validation Criteria . . . 77
4.7.2 External Validation Criteria . . . 79
4.7.3 Cluster Tendency . . . 81
4.8 Summary . . . 82
References . . . 82

5 An Efficient *K*-Medoids Clustering Algorithm for Large Scale Data . . . 85
5.1 Introduction . . . 85
5.2 Existing Work on *K*-Medoids Clustering . . . 86
5.2.1 FastK Algorithm . . . 88
5.2.2 INCK Algorithm . . . 89

5.3 The Proposed Efficient *K*-Medoids Approach . . . 90
5.3.1 A Simple Idea . . . 90
5.3.2 Central Limit Theorem . . . 91
5.3.3 An Improved K-Medoids Algorithm . . . 92
5.3.4 Time Complexity Analysis . . . 93
5.3.5 Pseudocode for the Proposed K-medoids Algorithm . . . 94
5.4 A Performance Study . . . 94
5.4.1 Performance on Large Datasets . . . 95
5.4.2 Performance on Image Data . . . 101
5.4.3 Running Time Performance . . . 105
5.5 Concluding Remarks . . . 107
References . . . 107

6 Enhancing Hierarchical Linkage Clustering via Boundary Point Detection . . . 109
6.1 Introduction . . . 109
6.2 Related Work . . . 110
6.2.1 Linkage Clustering Algorithms . . . 111
6.2.2 Boundary Point Detection . . . 113
6.3 The Proposed Efficient Linkage Clustering Algorithms . . . 115
6.3.1 A Simple Idea . . . 115
6.3.2 Improved Linkage Algorithms . . . 116
6.3.3 Time Complexity Analysis . . . 117
6.4 A Performance Study . . . 117
6.4.1 Performance on Small Real Data . . . 119
6.4.2 Performance on Image Data . . . 120
6.4.3 Performance Comparison . . . 125
6.5 Conclusions . . . 125
References . . . 126

7 A New Fast *K*-Nearest Neighbors-Based Clustering Algorithm . . . 129
7.1 Introduction . . . 129
7.2 Existing Literature on Density-Based Clustering . . . 131
7.3 Basic Notion of the Proposed Algorithm . . . 132
7.3.1 Formal Definition of KNN-Based Centroid . . . 132
7.3.2 A Local-Centroid-Based Notion of Clusters . . . 133
7.4 A *k*NN Centroid-Based Clustering Algorithm . . . 137
7.4.1 The Algorithm . . . 137
7.5 Performance Evaluation . . . 140
7.5.1 Performance on Small Low-Dimensional Data . . . 140
7.5.2 Performance on Large Image Datasets . . . 146
7.6 Conclusions . . . 149
References . . . 150

8 An Efficient EMST Algorithm for Clustering Very High-Dimensional Sparse Feature Vectors . . . 153
8.1 Introduction . . . 153
8.2 Related Work . . . 155
8.2.1 Minimum Spanning Tree Algorithms . . . 155
8.2.2 MST-Based Clustering Algorithms . . . 156
8.2.3 Fast *k*-Nearest Neighbor Query Techniques . . . 157
8.3 The Proposed EMST-Based Clustering Algorithms . . . 159
8.3.1 A Simple Idea . . . 160
8.3.2 Construction of a Simplest iDistance . . . 162
8.3.3 Radius Initialization . . . 163
8.3.4 Our *k*NN Search Structure . . . 163
8.3.5 Proposed EMST-Inspired Clustering Algorithm . . . 165
8.3.6 Time Complexity Analysis . . . 166
8.4 Experiments and Results . . . 166
8.4.1 Experiment I—Perceptual Grouping in an Indoor Environment . . . 167
8.4.2 Experiment II—Perceptual Grouping in an Outdoor Environment . . . 170
8.4.3 Experiment III—Running Time Issue . . . 173
8.5 Conclusions . . . 173
References . . . 174

Part III Supervised Learning and Semi-supervised Learning

9 Supervised Learning for Data Classification Based Object Recognition . . . 179
9.1 Introduction . . . 179
9.2 Decision Trees . . . 181
9.3 Perceptron . . . 182
9.4 Support Vector Machine . . . 185
9.5 Instance-Based Learning . . . 188
9.5.1 *k*-Nearest-Neighbour Classifier (kNNC) . . . 188
9.6 Semi-supervised Learning . . . 189
9.6.1 Learning Methods . . . 190
9.7 Classifier Performance Evaluation . . . 191
9.7.1 Quantification Issues . . . 192
9.8 Summary . . . 193
References . . . 194

10 A Fast Image Retrieval Method Based on A Quantization Tree . . . 195
10.1 Introduction . . . 195
10.2 Related Work . . . 198
10.3 Scalable Recognition with a Quantization Tree . . . 199

10.3.1 Visual Vocabulary Generation . . . 199
10.3.2 Quantization Tree . . . 200
10.4 Experiments and Results . . . 201
10.4.1 Experiment I: Visual Vocabulary Generation in an Indoor Environment . . . 202
10.4.2 Experiment II: Visual Vocabulary Generation in an Outdoor Environment . . . 204
10.5 Conclusions . . . 212
References . . . 213

11 An Efficient Image Segmentation Algorithm for Object Recognition Using Spectral Clustering . . . 215
11.1 Introduction . . . 215
11.2 Background . . . 217
11.2.1 Spectral Clustering Algorithms . . . 217
11.2.2 BIRCH Tree . . . 219
11.3 A BIRCH Clustering Feature Tree Based Spectral Clustering Algorithm . . . 220
11.3.1 General Idea . . . 221
11.3.2 Cluster Merging Based on BIRCH Tree . . . 221
11.3.3 Threshold Selection . . . 222
11.3.4 Tree Insertion . . . 223
11.3.5 Algorithm Description . . . 224
11.3.6 Time Complexity Analysis . . . 224
11.4 Experiments and Results . . . 224
11.4.1 Image Segmentation . . . 225
11.4.2 Running Time Performance . . . 229
11.5 Conclusions . . . 232
References . . . 233

12 An Incremental EM Algorithm Based Visual Perceptual Grouping . . . 235
12.1 Introduction . . . 235
12.2 Backgrounds . . . 237
12.2.1 Expectation and Maximization Clustering Algorithm . . . 237
12.2.2 Instance-Based Learning and a Novelty Detection Method . . . 239
12.3 The Proposed Approach . . . 240
12.3.1 Image Segmentation Model for Object Recognition . . . 240
12.3.2 EM Clustering with the Aid of Silhouette Coefficient . . . 241
12.3.3 Cluster Merging Based on BIRCH Tree . . . 241
12.3.4 Time Complexity Analysis . . . 243

12.4 A Performance Study . . . 244
12.4.1 Performance in an Outdoor Environment . . . 244
12.4.2 Performance in Comparison with a Previous Threshold Model . . . 246
12.4.3 Comparison with a Locality Sensitive-Hashing . . . 246
12.4.4 Other Threshold Values . . . 248
12.4.5 Running Time Issue . . . 249
12.4.6 Discussion . . . 249
12.5 Conclusions . . . 250
References . . . 250

Part IV Reinforcement Learning

13 Reinforcement Learning for Mobile Robot Perceptual Learning . . . 253
13.1 Introduction . . . 253
13.2 Reinforcement Learning . . . 255
13.3 Temporal Difference Learning . . . 258
13.4 Performance Measures . . . 260
13.4.1 Success Rate . . . 260
13.4.2 Speed to Success . . . 260
13.4.3 Optimality of Solution . . . 260
13.4.4 Robustness of the Learning Process . . . 260
13.5 Working Memory . . . 261
13.5.1 Psychological Model . . . 261
13.5.2 Biological Background . . . 261
13.5.3 Episodic Memory . . . 264
13.6 Working Memory Toolkit (WMtk) . . . 265
13.6.1 WMtk Interface . . . 265
13.6.2 Chunk . . . 266
13.6.3 FeatureVector . . . 267
13.6.4 WorkingMemory . . . 267
13.6.5 Processing . . . 270
13.6.6 Conjunctive Coding . . . 271
13.7 Summary . . . 272
References . . . 272

14 A Developmental Robotic Paradigm for Mobile Robot Navigation in an Indoor Environment . . . 275
14.1 Introduction . . . 275
14.2 Related Work . . . 278
14.3 A Vision-Based Autonomous Mobile Robot System . . . 279
14.4 Experiments and Results . . . 282
14.4.1 Experiment I: Learning Open Space . . . 284

14.4.2 Experiment II: Learning Landmarks . . . 285
14.4.3 Experiment III: Learning the Navigation Task . . . 286
14.5 Conclusions . . . 291
References . . . 292

15 An Automatic Scene Recognition Using TD-Learning for Mobile Robot Localization in an Outdoor Environment . . . 293
15.1 Introduction . . . 293
15.1.1 Scene Representation . . . 294
15.1.2 Scene Recognition . . . 295
15.2 Related Work . . . 297
15.3 Mobile Robot Localization with Configural Representation . . . 299
15.4 Experiments and Results . . . 301
15.4.1 Experiment I—Single Percept . . . 301
15.4.2 Experiment II—Multiple Percepts . . . 305
15.4.3 Experiment III—Competing Methods . . . 307
15.5 Conclusions . . . 308
References . . . 309

16 An Autonomous Vision System Based Sensor-Motor Coordination for Mobile Robot Navigation in an Outdoor Environment . . . 311
16.1 Introduction . . . 311
16.2 Related Work . . . 312
16.3 Vision-Based Perceptual Learning . . . 314
16.3.1 A Behaviour Learning Model . . . 314
16.3.2 An Incremental Percept Learning System for Mobile Robot Navigation . . . 315
16.4 Experiments and Results . . . 317
16.4.1 Experiment I: Learning the Road . . . 318
16.4.2 Experiment II: Learning the Subgoal Target Location . . . 322
16.4.3 Experiment III: Learning the Navigation Task . . . 323
16.5 Conclusions . . . 326
References . . . 326

About the Authors

Xiaochun Wang received her BS degree from Beijing University and her Ph.D. degree from the Department of Electrical Engineering and Computer Science, Vanderbilt University. She is currently an associate professor of School of Software Engineering at Xi'an Jiaotong University. Her research interests include machine learning, computer vision, signal processing, and pattern recognition.

Xiali Wang received his Ph.D. degree from the Department of Computer Science, Northwest University, China, in 2005. He is a faculty member in the School of Information Engineering, Changan University, China. His research interests are in computer vision, signal processing, intelligent traffic system, and pattern recognition.

Don Mitchell Wilkes received his BSEE degree from Florida Atlantic, and his MSEE and Ph.D. degrees from Georgia Institute of Technology. His research interests include digital signal processing, image processing and computer vision, structurally adaptive systems, sonar, as well as signal modeling. He is a member of the IEEE and a faculty member at the Department of Electrical Engineering and Computer Science, Vanderbilt University.

Acronyms

AMD	Autonomous mental development
AMR	Autonomous mobile robot
APF	Artificial potential field
BBD	Brain-based device
CCD	Charge coupled device
CDBSCAN	Centroid based DBSCAN
CFAR	Constant false alarm outlier detection
CFSFDP	Clustering by fast search and find of density peaks
CLARA	Clustering large application
CLT	Central limit theorem
CML	Concurrent mapping and localization
CMOS	Complementary metal oxide on silicon
CMU	Carnegie Mellon University
COF	Connectivity-based outlier factor
DB	Distance based outlier
DBSCAN	Density based clustering algorithm
DeLi-Clu	Density-Link-Clustering
DPK	Density peak optimized K-medoids
DP-NMK	Density peak optimized K-medoids with new measure
EKF	Extended Kalman filter
EMST	Euclidean minimum spanning tree
FAST	Features from accelerated segment test
GPP	Global path planning
GPS	Global Positioning System
HAC	Hierarchical agglomerative clustering
HSV	Hue Saturation Value
IHDR	Incremental hierarchical discriminant regression
INCK	Incremental K-medoids
INFLO	INFLuenced Outlierness
KDEOS	Kernel density estimate based outlier score

Kd-tree	K-dimensional tree
kNN	k-nearest neighbors
kNNC	k-nearest-neighbour classifier
KQTree	K-way quantization tree
LDBSCAN	LOF based DBSCAN
LDOF	Local distance-based outlier factor
LNG	Local neighborhood graph
LOCI	Local outlier integral
LOF	Local outlier factor
LPP	Local path planning
LTM	Long-term memory
MDP	Markov decision process
MER	Mars Exploration Rover
MST	Minimum spanning tree
NNC	Nearest-neighbor classifier
NR	Normalized residual
PCA	Principal component analysis
POMDP	Partially-observable Markov decision process
PRT	Positive reward threshold
RBDA	Rank based outlier detection
RDOS	Relative density-based outlier score
RERT	Rapidly exploring random trees
RGB	Red, Green, Blue
RNN	Reverse nearest neighbors
R-tree	Rectangle-tree
SIFT	Scale invariant feature transform
SLAM	Simultaneous localization and mapping
SLOM	Spatial local outlier measure
SMC	Sequential Monte Carlo method
SNN	Shared nearest neighbors
SOFM	Self-organizing feature map
SRI	Stanford Research Institute
SR-tree	Sphere/Rectangle-tree
SRVT	Scalable recognition with a vocabulary tree
SVM	Support vector machine
SVT	Scalable vocabulary tree
TDL	Temporal difference learning
TF-IDF	Term frequency inverse document frequency
UCI	University of California Irvine
VA-file	Vector approximation file
VT	Vocabulary tree
WMtk	NSF ITR Robot-PFC Working Memory Toolkit

Part I
Introduction

Chapter 1
Overview and Contributions



Abstract While it is beneficial to use vision sensors in many vision applications such as object detection, scene recognition, human pose estimation, and activity recognition, machine learning plays an important role in bridging the gap between feature representations and decision-making. In this chapter, an overview of this book is presented. We begin with an introduction to the research issues on learning in mobile robot localization. The content for each chapter is next described. Finally, the contributions are summarized.

Keywords Research issues on learning in mobile robot localization · Content for each chapter · Contributions

1.1 Introduction

Mobile robots play a significant role in a variety of environments, and mobile robotics has grown rapidly within the past decade, producing tools to replace human workers. In general, mobile robots often operate with incomplete information about the environment. Considerable research has been done on learning the environment as the robot moves by means of various types of sensors mounted on the robot. Computer vision addresses any autonomous behavior of a technical system supported by visual sensory information. To provide some methods of analyzing digital image sequences for mobile robotic applications, such as localization and navigation tasks, the understanding of visual information, especially for scenes with no accompanying structural, administrative, or descriptive text information, requires a combination of high-level concept creation as well as the processing and interpretation of inherent visual features. The vision has evolved from the applications of classical pattern recognition and image processing techniques to advanced applications of image understanding, model-based vision, knowledge based vision, and systems that exhibit learning capability, and to approaches that involve analysis and learning of the type of information being sought, the domain in which it will be used, and systematic testing to identify optimal methods.

In recent years, theoretical and practical advances are being made in the field of mobile robot localization by new techniques and process of learning, representation,

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_1

and adaptation. The ability to reason and the ability to learn are the two major capabilities. It is probably fair to claim that learning represents the next challenging frontier for mobile robotic navigation research.

In the following, Sect. 1.2 gives an introduction to the research issues on learning in mobile robotic localization. The content for each chapter is next described in Sect. 1.3. The contributions are summarized in Sect. 1.4. Finally, conclusions are made in Sect. 1.5.

1.2 Research Issues on Learning in Mobile Robotic Localization

The goal of improving the performance of robots by incorporating machine learning techniques has brought new challenges to the field of machine learning. Computer vision is the science and technology of making machines that see. It is concerned with the theory, design, and implementation of algorithms that can automatically process visual data to recognize objects, track, and recover their shape and spatial layout. In recent years, there has been a surge of interest and efforts in developing machine learning techniques for computer vision-based robotic applications.

From the standpoint of vision-based robotic applications, machine learning is one of the current frontiers for computer vision research and can offer effective methods for automating the acquisition of visual models, adapting task parameters, transforming image signals to symbolic representations, focusing attention on target object, and performing perceptual learning associated with a vision system. From the standpoint of machine learning systems, vision-based mobile robot navigation can provide interesting and challenging problems. Solving the problems in visual domains will result in the development of new and more robust machine learning algorithms that will be able to work in more realistic settings. It is in this sense that machine learning technology has strong potential to contribute to the development of flexible and robust vision algorithms that will improve the performance of practical mobile robot systems with a higher level of competence and greater generality, and the development of human-like learning architectures that will provide better performance.

Autonomous mobile robot navigation sets out enormous theoretical and applied challenges to advanced robotic systems using machine learning techniques. One challenging question would be **how to learn models** based on the human visual system rather than handcrafting them. Unlike human brain, artificial information systems often assume that a careful trainer provides internal representations of the observed environment. As a result, little attention is paid to the problem of perception and the handcrafted models limit the use of the system to a specific class of images, which is subject to change in a relatively short time and the system cannot do anything in unexpected situations.

Another challenging question would be **how to apply machine learning algorithms** to vision-based mobile robot localization research. The algorithm of choice is a machine learning method that employs unsupervised and supervised learning algorithms to improve the perception of the surrounding environment (i.e., to improve the transformation of sensed signals into internal representations), and reinforcement learning to bridge the gap between the internal representations of the environment and the representation of the knowledge needed by the system to perform its tasks for decision-making.

A third challenging question would be **how to represent the visual information**. To take computer vision out of the lab and into real environments, cameras can be used to reduce the overall cost, maintaining high degree of intelligence, flexibility, and robustness. In this book, an object identification model is presented, which uses patch-based color histogram for image segmentation and relies on unsupervised learning algorithms (i.e., clustering algorithms) for perceptual grouping in order to detect objects existing in an unknown environments. By this model to represent elemental objects in a 2D scene, not only discriminative features (i.e., the patch-based color histogram) but also their spatial and semantic relationships are used to categorize a scene.

A fourth challenging question would be **what machine learning paradigms and strategies are appropriate to the vision-based localization problem**. There are several important paradigms that are being used. Both supervised and unsupervised learning emerge as the most important learning strategies. In many computer vision applications, feature vectors are used to represent the perceived environment. However, relational descriptions are deemed to be of crucial importance in high-level vision. To capture the structure of both objects and scenes, another emerging paradigm is the use of reinforcement learning models in general and TD-learning models in particular in this book.

Finally, **the design of the criteria for evaluating the quality of the learning processes in vision-based systems** is challenging. Although estimates of the predictive accuracy are considered the main parameters to evaluate the success of a learning algorithm, the comprehensibility of learned models is also deemed an important criterion, especially when domain experts have strong expectations on the properties of visual models. The results of unsupervised learning and supervised learning should be symbolic descriptions of given entities, semantically and structurally similar to those a human expert might produce observing the same entities. The results of reinforcement learning should be descriptions directly interpretable in natural language and should relate quantitative and qualitative concepts in an integrated fashion.

1.3 Overview of the Book

The proposed manuscript is devoted to the theory and development of machine learning based autonomous localization mechanism for mobile robots and covers various aspects which are essential to the goal of bringing learning into mobile robot

localization research, including vision-based sensing, perception, and reasoning. Individual chapters present the theoretical analysis of specific technical problems, often supplemented by numerical analysis, simulation and real experiments on prototypes. This research involves algorithms and architectures for real-time applications in the area of computer vision, image processing, and object-vision recognition.

To bring together high-quality and recent research advances on machine learning based scene recognition for mobile robot localization, we divide the chapters into four parts. In the first part, we also include two survey chapters which overview the prior arts in this field. In general, the study of machine learning can be divided into three broad categories, unsupervised learning, supervised learning, and reinforcement learning, each built upon top of the previous one in the current research. In the proposed framework, the applications should form the basis of the theoretical research leading to interesting algorithms. As a consequence, the rest of the book is divided into three parts. In the second part, five chapters dedicated to the research of patch based 2D-image segmentation for object identification by unsupervised learning. In the third part, there are four chapters describing novel supervised and semi-supervised learning algorithms for object recognition and tracking. The last part consists of four chapters, demonstrating accurate mobile robot localization with the aid of a type of reinforcement learning method, that is, TD-Learning, so as to give a mobile robot the ability to navigate autonomously in an unstructured unknown environment. The theoretical results in this book originate from different practical problems encountered when using machine learning in mobile robot localization problems. The goal of the book is to address some of the challenging questions posed so far. We believe that a detailed analysis of the way by which machine learning theory can be applied through algorithms to real-world applications is very important and extremely relevant to the scientific community. In the following, we summarize all the chapters.

In this chapter, an overview of the book chapters and a summary of contribution are presented. First, the research issues on learning for mobile robot localization are explained. The overview of the book is then followed. Finally, contributions are highlighted. In Chap. 2, recent developments in mobile robot localization research are reviewed. This chapter begins with the statement and components of localization problems. Then, it investigates some standard localization approaches. In Chap. 3, a vision-based machine perception model from the perspective of a system design domain is presented and strategies for extracting information acquired from vision sensors for mobile robot localization tasks are discussed. Finally, the proposed method is briefly introduced.

In Chap. 4, the key ideas underlying the field of unsupervised learning from the perspective of clustering is introduced in a fairly concise manner for image segmentation tasks. A short introduction to distance measures and a review on performance evaluation metrics for clustering are also covered. In Chap. 5, a new efficient K-medoids clustering algorithm is proposed for perceptual grouping-based object identification tasks, which preserves the clustering performance by following the notion of a simple and fast K-medoids algorithm while improving the computational efficiency. In Chap. 6, a new approach for hierarchical linkage hierarchical clustering

is described as a solution for an efficient as well as reliable data clustering problem, which applies the traditional linkage algorithms to cluster a size-reduced version of the original dataset which is obtained via boundary point detection. In Chap. 7, a new density-based clustering algorithm is proposed in which the selection of appropriate parameters is less difficult but more meaningful. In Chap. 8, we present a new fast minimum spanning tree-based clustering algorithm for image segmentation and object detection tasks. The performance is evaluated on an indoor environment and an outdoor environment.

In Chap. 9, we give a short introduction to the fields of supervised learning and semi-supervised learning from the perspective of a tree-based approximate nearest neighbor based classifier for object categorization tasks. At the end, the performance evaluation methods are reviewed. In Chap. 10, a fast approximate nearest neighbor search tree-based classifier is presented together with a performance evaluation for object categorization tasks in an indoor environment as well as in an outdoor environment. In Chap. 11, an efficient incremental spectral clustering with the aid of BIRCH tree is presented for an object recognition based environmental modeling task in an outdoor environment. In Chap. 12, to further manifest the effectiveness of BIRCH tree, another efficient semi-supervised learning algorithm based on incremental EM clustering algorithm is presented for object recognition based environmental modeling tasks in an outdoor environment.

In Chap. 13, to bridge the gap between feature representations and decision-making, robots need to infer semantic prediction of a scene and to acquire control through a learning paradigm inspired by a biological model of human-like localization via a Hippocampal memory mechanism. In Chap. 14, a developmental robotic paradigm using working memory learning mechanism is proposed for mobile robotic navigation tasks in an indoor environment. In Chap. 15, a natural landmark-based localization strategy for a mobile robot working in an unknown outdoor environment is designed based on a configural representation of stimuli detected in the environment with the aid of TD-learning. In Chap. 16, based on the fact that perception in animals is strongly related to the type of behavior they perform, a robot control learning system is applied successfully to a simple sensor-motor coordinated road detection task in an unknown outdoor environment.

1.4 Contributions

This book advances the mobile robot localization research in an unknown environment by machine learning based natural scene recognition methodology. The chapters feature the latest developments in vision-based machine perception and machine learning research for localization applications and cover such topics as image segmentation based visual perceptual grouping for the efficient identification of objects composing an unknown environment, classification-based fast object recognition for the semantic analysis of natural scenes in the environment, the present understanding of the Hippocampal working memory mechanism and its biological processes for

human-like localization, and the application of this present understanding towards mobile robot localization improvement. The volume also features a perspective on bridging the gap between feature representations and decision-making using reinforcement learning, laying the groundwork for future advances in mobile robot navigation research.

Having been exploited in computer vision for years to take it out of the lab and into real environments, cameras can be used to reduce the overall cost, while maintaining high degree of intelligence, flexibility, and robustness. Image understanding is a research area involving both feature extraction and object identification within images from a scene, and a posterior treatment of this information in order to establish relationships between these objects with a specific goal. In this book, an object identification model is presented, which uses patch-based color histogram for image segmentation and relies on unsupervised learning algorithms (i.e., clustering algorithms) for perceptual grouping in order to detect objects existing in an unknown environments. To infer the semantic content of images and videos for mobile robotic localization tasks, this research exploits machine learning and pattern recognition techniques and presents two different strategies for online learning, namely, a supervised learning strategy where the parsed images are produced based on training data obtained beforehand by the unsupervised learning, and a semi-supervised learning strategy in which the robot generates training data via exploration while wandering in the environment.

It is realized that humans acquire various behaviors based not only on genetic information but also on post-natal learning. Localization learning is an active research topic in robotics. However, most solutions are optimized for industrial applications and, thus, few are plausible explanations for learning in the human brain. Increase in computational power of modern computers fosters new approaches to advanced signal processing and there is a trend to shift functional behavior of industrial automation systems from hardware to software to increase flexibility. While it is beneficial to use vision sensors in many vision applications such as object/scene recognition, building bridges between natural and artificial computation is one of the main motivations for this book. Adding human-like flexibility by learning can equip the robot with the ability of generalization, guessing a correct behavior for an unexperienced situation based on learned relationships between behavior and situation. One contribution is an analysis of algorithms that are suitable for localization learning and plausible from a biological viewpoint. It has been claimed that the hippocampus region is especially engaged in spatial learning and takes part in forming maps of the external world. In this strategy, the hippocampus, as a network of cells and pathways that receive information from all of the sensory systems, learns about the spatial configural representations of the sensory world and then acts on the motor systems to produce appropriate spatial behavior.

Therefore, original contributions presented in this book focus on developing algorithms for unsupervised learning, supervised learning, and reinforcement learning in the area of machine learning paradigms. In particular, some key issues addressed in this book are given as follows:

Theory

- A new scene representation model is proposed which combines image segmentation and feature binding using unsupervised and supervised learning.
- A scene recognition model based on transverse patterning is proposed for mobile robot localization which combines symbolic representation and conjunctive coding using reinforcement learning.

Algorithms

- Four new unsupervised learning algorithms are proposed and used for perceptual grouping.
- Two novel supervised learning algorithms are introduced for classification.

Applications

- A novel architecture for mobile robot navigation task is proposed. This architecture allows one to model an unknown environment by combining the three learning methodologies at different phases of perceptual learning pipeline. Empirically, this architecture is observed to be robust to environmental noise and provides good generalization capabilities in different settings.
- A new method for learning the meaning of percepts with regard to action is proposed for detecting open space in videos which is characterized by the road and captured by the weight distribution in neural network after training. More specifically, a new method for recognizing open space or the road path from video sequences using labeled and unlabeled data is introduced.
- A new algorithm for percept-based localization learning using reinforcement learning under various scenarios is presented. We show that configural representation learning for detecting target location using our proposed learning architecture not only yields improved classification results but also opens a door for semantic content interpretation of a scene.

This book concentrates on the application domains of machine learning to vision-based mobile robot localization research. However, the results and algorithms presented in the book are general and equally applicable to other areas including data mining, pattern recognition, and computer vision. Finally, the chapters in this book are mostly self-contained and meant to ease the reading of each chapter in isolation.

1.5 Conclusions

In this book, both an in-depth overview of challenging areas and novel advanced algorithms are provided which exploit machine learning and pattern recognition techniques to infer the semantic content of image sequences and videos. The topics covered by the chapters include visual feature extraction, image segmentation based object detection and recognition, and configural represented based location learning. Each chapter contains key references to the existing literature to provide both an objective overview and an in-depth analysis of the corresponding state-of-the-art research, and covers both theoretical and practical aspects of a real-world localization problem for autonomous mobile robots. We hope that the presented contributions of this work in theoretical development and progress in practical solutions will be useful to those interested in the area.

Chapter 2
Developments in Mobile Robot Localization Research



Abstract In this chapter, recent developments in mobile robot localization research are reviewed. We begin with the statement and components of localization problems. This is followed by a short history of the early developments in mobile robotics. Next, some standard localization approaches are investigated. First, we survey the literature on mobile robot localization approaches in an environment without constructing a map. Then we present the details of a mobile robot localization approach based on construction of a map, the so-called simultaneous localization and mapping. Finally, the conclusions are given.

Keywords Mobile robot localization · Relative localization · Absolute localization · Dead-reckoning approach · Triangulation-based approach · Global positioning system · Simultaneous localization and mapping

2.1 Localization Problems: Its Problem Statement and Its Components

Robots are intelligent machines capable of performing tasks in the real world without explicit human control for extended periods of time (Nattharith 2010). Mobile robots are programmable, self-controlled machines capable of accomplishing tasks while navigating in an environment with or without obstacles based on the sensed information of its own internal state and the external environment using on-board sensors. In mobile robotics, a navigation problem would probably be defined as solving the following question: given some metric space (e.g., some region in an environment) and a set of fixed points (e.g., the locations of some landmarks, known or unknown beforehand), determine a suitable path from a starting point to a goal point using a given representation of the environment. To find an optimal or an approximate optimal collision-free path and to realize a safety movement, there are various scenarios that the robot can follow to reach its goal location. The complete problem can be tackled by achieving a set of subgoals. To this end, Leonard and Durrant-Whyte divide the mobile robot navigation problem into three subproblems, Where am I? (i.e., the localization problem), Where am I going? (i.e., the object recognition problem), and How do I get there? (i.e., the path planning problem) (Leonard and Durrant-Whyte

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_2

1992). Navigation is an important ability of mobile robots. Localization in an environment is the very first step to achieve it.

The ability to navigate is an important embodiment of the intelligent level of a mobile robot. A high degree of autonomy is particularly desirable in fields where robots can replace human workers, such as state-of-the-practice video surveillance systems and space exploration. The combination of navigation and autonomy is the core and key point of intelligent mobile robot research. Before 1980s, mobile robot navigation was mainly based on the simple obstacle avoidance strategies and localization relied mainly on internal sensors. After 1980s, map-based navigation approaches appeared. Particularly, proposed by Smith and Cheesman in 1987 (Smith et al. 1990), the Simultaneous Localization and Mapping (SLAM) has attracted the attention of a large number of researchers, and achieved a lot of practical results (Thrun et al. 1998, 2006; Durrant-Whyte and Bailey 2006; Bailey and Durrant-Whyte 2006). Although SLAM has gradually become an active topic in robot navigation research in the past 20 years, most of the results have focused on the navigation control in a known and deterministic environment. When the space exploration robot of the United States, Sojourner, landed on Mars for the first time to carry out scientific research activities in 1997, the use of mobile robot technologies for space exploration and development has become the world's main competition among countries in the twenty-first century. When mobile robots perform tasks on the surface of Mars or the moon, it will navigate in an unknown environment. Therefore, navigation in an unknown environment has become a key technology in this field for Mars and lunar explorations using mobile robots. In an unknown environment, the creation of a precise map depends on a reliable localization of the mobile robot. Due to the lack of a priori knowledge of the environment and the distribution of obstacles, and particularly when there are no preset landmarks and other absolute references, the mobile robot must rely on the information provided by its on-board sensors to realize self-localization. Vision-based localization is still a difficult problem in the field.

In this book, we deal with localization problems in an unknown environment. In order to realize an accurate localization of a mobile robot in its working environment, many difficulties exist such as the interference from the sensor noise, the impact of the environmental, and so on. Therefore, the research on autonomous localization technology of mobile robotics in an unknown environment is still in its developing stage, and many theories need to be further improved. In the following, some standard localization approaches established in the past are introduced.

2.2 A Short History of the Early Developments in Mobile Robotics

It is generally believed that autonomous mobile robot system is highly self-planning, self-organizing and adaptive, and suitable for working in complex environment. Mobile robots have a long history and mobile robotic research began with Nilssen and

Rosen et al. at SRI's Artificial Intelligence Centre (Stanford Research Institute) who developed Shakey, the world's first mobile robot, in the late 1960s (Nilsson 1984). Shakey was equipped with various sensors and driven by a problem-solving program called "STRIPS," and used algorithms for perception, world modeling, and actuation. Not surprisingly, it has had a substantial influence on present-day robotics. Their purpose was to study the application of artificial intelligence technology to autonomous reasoning, system planning and control in complex environments. Another example of an early robot, CART, was developed at Stanford University in 1977 by Hans Moravec as part of his doctoral thesis (Moravec 1990). However, CART was very slow, not because it was slow-moving by design, but because it was "slow-thinking." Then in the early 1980s, another well-known example of mobile robot, Rover, was developed at Carnegie Mellon University (CMU). Rover was equipped with a camera and was able to perform better than CART. Nevertheless, its thinking and acting were still very slow (Moravec 1990). Equipped with stereo cameras and landed on Mars in 1997, NASA's twin Mars Exploration Rovers (MERs), Spirit and Opportunity, used a local path planner (GESTALT) to guide the rovers around narrow and isolated hazards. To improve, a new technology was proposed at the Jet Propulsion Laboratory where Field D* global path planner had been integrated into rover's software to allow simultaneous local and global planning during navigation (Carsten et al. 2009a, b). A comprehensive review paper about major milestones in the development of computer vision for autonomous vehicles over the last decades can be found in (Matthias et al. 2007). The paper also discussed the design and performance of three computer vision algorithms used on Mars in the NASA/JPL Mars Exploration Rover (MER) mission, namely: stereo vision, visual odometry for the rover, and feature tracking for the lander. Despite the limitations of processors deployed, they performed consistently and made significant contributions to the project. Two important survey papers that have been published review various aspects of the progress made so far in vision for mobile robot navigation (DeSouza and Kak 2002; Bonin-Font et al. 2008).

This book proposes a vision-based method using mapless strategy for vision-based mobile robot localization. Principally, the strategy involves recognizing objects found in an environment and tracking those objects using visual clues or observations.

2.3 Some Standard Localization Approaches

For autonomous navigation, a mobile robot needs to consider its position and orientation within a certain working coordinate system. Localization is the most basic problem for mobile robot to realize autonomous ability. The so-called localization problem is the estimation of the position and orientation of the autonomous mobile robot in the operating environment. It is usually solved by a mixture of different principles contributing to the state variables of position and orientation, called "pose". The robot's position depends on the sensor used. The sensors commonly used by mobile robots include odometry, cameras, laser radar, ultrasonic infrared, microwave

radar, gyroscope, compass, gyroscope, speed, and accelerometers, etc. In corresponding to these sensors, generally two different approaches are being considered: relative localization (relative pose determination) and absolute localization (absolute pose determination). By relative localization, a mobile robot's current position is determined by measuring its current distance and orientation relative to its initial position. Relative pose determination is also called the dead reckoning approach. Absolute localization is based on navigation beacons/markers, active or passive landmarks, map matching, and satellite-based technologies (e.g., GPS), and can achieve high localization precision. However, it seems most logical to present the measurement principles in an order so that the approaches can be put together as a whole localization system, supplementing each other. The two-categorized methods are introduced in the following.

2.3.1 Dead-Reckoning Localization Approach

For the localization problem to make sense, the locations (points) must be related by distance. Again, the mobile robots are very small as compared to the space they are located in. To determine the current position which is within a number of points along the path between two existing points, the robot could memorize the number of steps walked between them and could determine its current position by counting up the number of steps on its way. This is the method referred to as dead-reckoning and can be realized by deriving a robot's trajectory from the summation of wheel velocities which is called odometry. In order to exemplify the techniques involved, the calculations for a differential drive are presented in the following.

To mathematically indicate the coordinates of the locations that are to be determined, the geographical region commonly corresponds to a metric space. An obvious scenario for representing such a region is the two-dimensional or three-dimensional space. Correspondingly, the position of the mobile robot can be described by three-dimensional vector, (x_r, y_r, θ), and six-dimensional vector, $(x_r, y_r, z_r, \alpha, \beta, \gamma)$, respectively. Without loss of generality, here, most prominently, the two-dimensional plane is used. Therefore, in localization problem, the coordinates of a mobile robot in a two-dimensional working environment with respect to global coordinates need to be determined.

As illustrated in Fig. 2.1, the X-axis of the robot's coordinates is the direction towards which it is moving, and the Y-axis is on its left. At the beginning, the robot coordinates coincide with global coordinates. After time t, suppose the robot moves from the origin to the point P_0 then to the point P_1. That is, the position of the robot in the global coordinate system is P_0 (X_t, Y_t). The angle between the forward direction of the robot and the global coordinate X axis is $\theta_0 = \theta_t$. Suppose two wheels roll on the ground with distance W between them and Δt is the fixed time interval of pulse generated by photoelectric encoder. In the same sampling time window Δt, the angle the robot turns is,

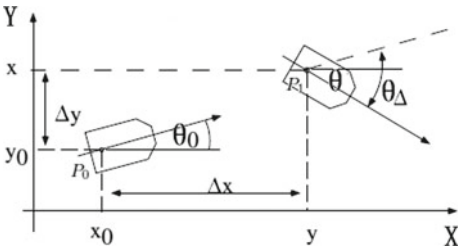


Fig. 2.1 An illustration of the dead reckoning method

$$\Delta\theta = \frac{(V_{\mathrm{L}} \cdot \Delta t - V_{\mathrm{R}} \cdot \Delta t)}{W} \tag{2.1}$$

where V_{L}, V_{R} are the robot speeds for the left and right wheels, respectively. Let their average be,

$$\overline{V} = \frac{V_{\mathrm{L}} + V_{\mathrm{R}}}{2} \tag{2.2}$$

The position change of the robot within Δt is ΔS,

$$\begin{cases} \Delta X = \overline{V}\cos(\theta_t + \Delta\theta) \cdot \Delta t \\ \Delta Y = \overline{V}\sin(\theta_t + \Delta\theta) \cdot \Delta t \end{cases} \tag{2.3}$$

Then at moment $t + \Delta t$, the location information, or the pose, for the robot is,

$$\begin{cases} X_{t+\Delta t} = X_t + \Delta X \\ Y_{t+\Delta t} = Y_t + \Delta Y \\ \theta_{t+\Delta t} = \theta_t + \Delta\theta \end{cases} \tag{2.4}$$

If the distance the wheel moves in the ith sampling period is ΔS_i, the total distance traveled by the wheel during the n sampling periods S, is,

$$S = \sum_{i=1}^{n} \Delta S_i \tag{2.5}$$

The dead reckoning method is a widely used method for mobile robot localization. The advantage of the method is that the robot's posture is self–calculated. It does neither rely on external references nor need to perceive the external environment. A rotating photoelectric encoder is usually installed on the drive wheel to measure the rotation angle. The distance between the robot and the starting point is calculated by using the mechanical characteristics of the robot. The pose information of the robot is thus obtained. In this process, however, error accumulation is inevitable. Its disadvantage is that the drift error will accumulate with time, which is not suitable for accurate localization. In other words, the dead reckoning method is a process of

integrating the movement with smaller errors. As the movement continues, the error will become greater and greater. Therefore, the traditional dead reckoning method is only accurate within a limited time and distance. In practical applications, the method is often not acceptable after the robot moves 10 m.

2.3.2 *Triangulation Based Absolute Localization Using Landmarks*

To indicate the coordinates of the mobile robot that is to be located, Global Positioning System (GPS), the second generation satellite system of the United States, can be utilized. GPS is a high precision navigation and positioning system based on space satellites. The system consists of 24 satellites in 6 orbit levels at a height of 20.051 km. Each satellite covers 1/6 of the earth surface and they are arranged at such orbits that the 24 satellites cover each point on earth fourfold, i.e. at least four satellites can be seen at an arbitrary moment from every point on earth. The satellites are equipped with an atomic clock and synchronized with each other. They are sending out signals containing the number of the satellite, its position at the moment of the timing signal, and a timing signal itself. The signals are picked up by earthbound receivers. Using the signals from four satellites, a receiver can find out its position on earth by measuring the time differences between sender signals, given their positions known at the moment of a timing signal. Evaluating the time differences between the position signals from four satellites, three sources of position information may be gathered, that is, the longitude, the latitude and the height above ground. So at any time at least four satellites should be visible. It operates worldwide with an accuracy of approximately 10 m and is used in navigation systems in cars, ships, and on board of agricultural machines.

To find out position and orientation in places where GPS cannot be used, landmarks are needed. Landmarks are easily detectable features in a scene which make it possible to find the position and orientation of a mobile robot. Natural landmarks could be anything from edges of stationary objects to markers found in the environment or readily recognizable objects. The environment might also be equipped with artificial landmarks, such as lighthouses on a coast or easily recognizable markers brought into the environment to ease finding positions, like barcode strips. Sensors on board the mobile robot must be able to detect these landmarks. The sensor can provide the angle between a symmetry axis of the mobile robot and the source of light as well as the number of the landmarks as output signals. From three angles measured, the position and orientation of a mobile robot with respect to the room can be deduced. Let P_1, P_2 and P_3 be three landmarks with coordinates (x_1, y_1), (x_2, y_2) and (x_3, y_3), respectively, not arranged in a line. The measured angles are α, β and γ. Let P be the position of the mobile robot itself with coordinates (x, y). The difference angles $\varphi_{12} = \beta - \alpha$ and $\varphi_{23} = \gamma - \beta$ are invariants under the rotation of the vehicle. Figure 2.2 depicts the situation described.

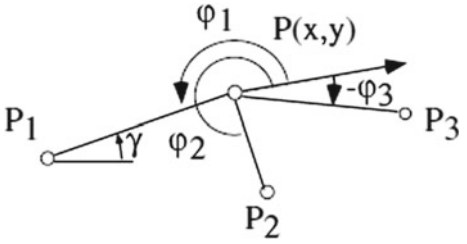


Fig. 2.2 Position from angles to three landmarks

$P(x, y)$ is located on a circle through P_1 and P_2. The radius of the circle is R_1. Let a$*$ be the distance between P_1 and P_2, then

$$R_1 = \frac{a^*}{2\sin\phi_{12}} = \frac{\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}}{2\sin\phi_{12}} \tag{2.6}$$

$$a^* = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \tag{2.7}$$

As illustrated in Fig. 2.3, the midpoint of P_1 and P_2 is P_m with its coordinates,

$$x_\mathrm{m} = \frac{x_2 - x_1}{2} \tag{2.8}$$

$$y_\mathrm{m} = \frac{y_2 - y_1}{2} \tag{2.9}$$

with

$$d = R_1 \cos\phi_{12} \tag{2.10}$$

Let

$$\tan\alpha_1 = \frac{y_2 - y_1}{x_2 - x_1} \tag{2.11}$$

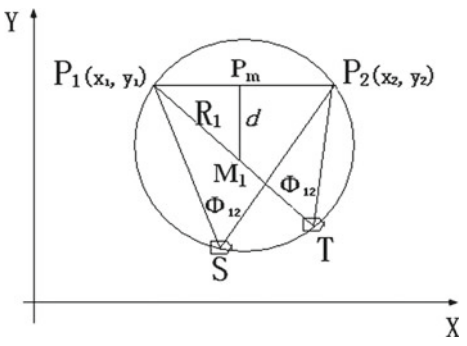


Fig. 2.3 An illustration of the calculation of circle center and circle radius

the coordinates of the midpoint M_1 of the circle are,

$$x_{M_1} = x_{\mathrm{m}} + d\cos\alpha_1 \tag{2.12}$$

$$y_{M_1} = y_{\mathrm{m}} + d\cos\alpha_1 \tag{2.13}$$

The position of P looked for then has coordinates implicitly given by the circle equation,

$$R_1^2 = \left(x - x_{M_1}\right)^2 + \left(y - y_{M_1}\right)^2 \tag{2.14}$$

Accordingly, for the circle through P, P_2, and P_3 including an angle φ_{13} as shown in Fig. 2.4, there is,

$$b^* = \frac{x_3 - x_2}{\sin\alpha_2} \tag{2.15}$$

$$R_2 = \frac{b^*}{\sin\phi_{23}} \tag{2.16}$$

$$\tan\alpha_2 = \frac{y_3 - y_2}{x_3 - x_2} \tag{2.17}$$

The coordinates of the midpoint of the circle through P, P_2, and P_3 are,

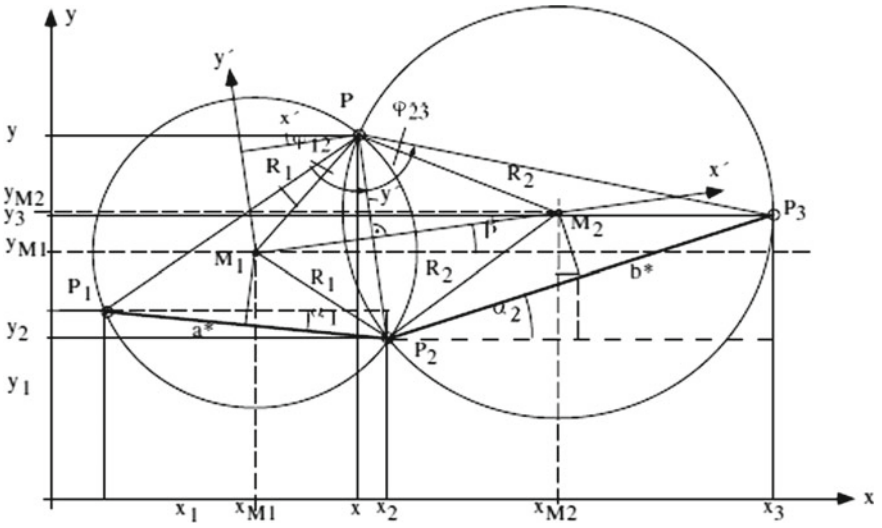


Fig. 2.4 An illustration of triangulation using landmarks

$$x_{M_2} = \frac{x_3 - x_2}{2} - \frac{b^*}{2\cot\phi_{23}\sin\alpha_2} \tag{2.18}$$

$$y_{M_2} = \frac{y_3 - y_2}{2} - \frac{b^*}{2\cot\phi_{23}\cos\alpha_2} \tag{2.19}$$

The calculation of the coordinates of the point P is first done in a coordinate system (x, y) running through the points M_1 and M_2 and with M_1 as center point as shown in Fig. 2.5. Let L be the distance between M_1 and M_2 then,

$$L = \sqrt{\left(x_{M_2} - x_{M_1}\right)^2 + \left(y_{M_2} - y_{M_1}\right)^2} \tag{2.20}$$

The point P lies on a circle with radius R_1 around M_1,

$$\left(x'\right)^2 + \left(y'\right)^2 = R_1^2 \tag{2.21}$$

and also on a circle with radius R_2 around M_2,

$$\left(L - x'\right)^2 + \left(y'\right)^2 = R_2^2 \tag{2.22}$$

The coordinates of point P are thus,

$$x' = \frac{R_1^2 - R_2^2 + L^2}{2L} \tag{2.23}$$

$$y' = \sqrt{R_1^2 - (x')^2} \tag{2.24}$$

The last step in this calculation is the coordinate transformation into the world coordinate system (x, y) according to Fig. 2.5.

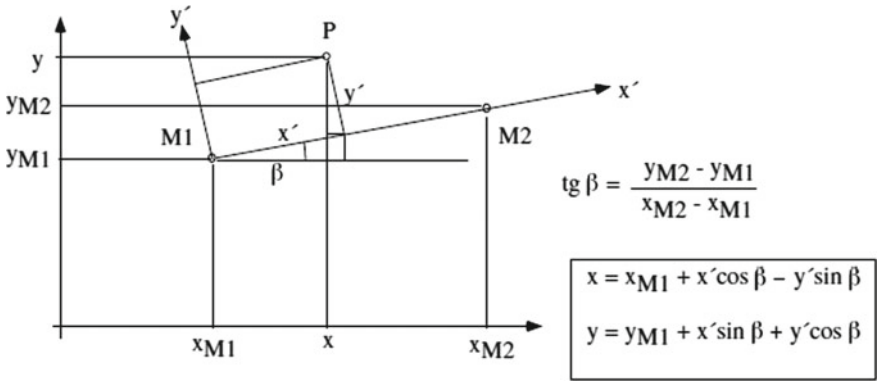


Fig. 2.5 Coordinate transformation

In addition to three angles, three distances measured from point P to three fixed points P_1, P_2, P_3 can also be used to calculate P's coordinates. The global positioning system (GPS) is an example of triangulation based on three distances. To summarize, a landmark has a known fixed position. There are usually three or more landmarks in the operating environment. In other words, there are three or more known fixed locations. Using geometric trigonometry described in the above, the coordinates of a moving mobile robot can be determined in a certain working coordinate system.

2.3.3 Simultaneous Localization and Mapping (SLAM)

When a mobile robot is required to navigate beyond its sensory horizon, it can also employ metric or topological maps to realize localization. Maps may contain certain knowledge of the environment in different degrees of detail, varying from a complete CAD model of the environment to a simple graph of interconnections between the elements in the environment. The main idea behind map-based navigation is essential to provide the robot with a sequence of landmarks expected to be found during navigation. The task of the robot sensor system is then to search for and recognize the landmarks observed in the acquiring sensory information. When the landmarks are recognized, the robot can employ the map to estimate its own position (self-localization) by matching the observation (sensory information) against the expectation (landmark) description in the database. The start position of the robot is unknown under absolute localization methods. Accordingly, the system must provide exact matching between the current and expected data, derived purely from the entire database. This self-localization problem has been solved either using deterministic triangulation or Monte Carlo-type localization (DeSouza and Kak 2002; Bonin-Font et al. 2008). However, this solution entails that a map has previously been drawn.

For an autonomous system operating in an unknown environment, the autonomous mobile robot is activated without precise information about its pose and no map of the environment exists. In this case, the mobile robot has to explore the surroundings, create a map, and track the pose. This somehow paradox situation is called the SLAM (Simultaneous Localization And Mapping) problem (Smith et al. 1990; Leonard and Durrant-Whyte 1991). Simultaneous localization and mapping (SLAM) is also known as concurrent mapping and localization (CML), where a mobile robot can build a map of an environment and at the same time use this map to deduce its location. The SLAM problem has attracted significant attention from the research communities of the autonomous vehicles and mobile robots in the past two decades.

The SLAM problem, essentially, consists of estimating the unknown motion of a moving platform iteratively in an unknown environment and, hence, determining the map of the environment consisting of features (also known as landmarks) and the absolute location of the moving platform on the basis of each other's information (Dissanayake et al. 2001). This is known as a very complex problem as there is always the possibility that both the vehicle's pose estimate and its associated map estimates become increasingly inaccurate in absence of any global position

information (Montemerlo et al. 2003). This situation arises when a vehicle does not have access to a global positioning system (GPS). Hence the complexity of the SLAM problem is manifold and requires a solution in a high dimensional space due to the mutual dependence of the robot pose and the map estimates.

In SLAM problem, initially, both the map and the robot position are not known. The mobile robot has a known kinematic model and it is moving through the unknown environment, which is populated with artificial or natural landmarks. A simultaneous estimation of both robot and landmark locations is carried out based on the observation of the landmarks. The SLAM problem involves finding appropriate representation for both the observation and the motion models (Durrant-White and Bailey 2006).

In this case, features are extracted from its local environment that can be used as identifiers for landmarks. A precondition for the use of the landmark is the knowledge about its pose. The local configuration of these features (e.g. angle and distance due to the robot's coordinates system) yields a local feature map. For small movements it is easy to use these features to track the changing pose of the robot. More specifically, the robot measures the distances to the landmarks using sensors (e.g. a laser scanner). While moving, it can use its odometry to estimate the new pose. This odometry-based robot pose could be used to estimate the new distances to the features extracted from the environment. Comparing the distances between robot and features with the measured ones, the autonomous mobile robot (AMR) is able to correct its estimated pose. So far this is a localization with landmarks within the initial local feature map. By adding new features to this map, it can be incrementally extended until it covers the whole working space of the AMR. If one supposes that the distance measurement is absolutely precise and that enough of the old known features are observed in every extension step, the SLAM problem is solved. In reality, we must assume that even after localization minimal errors in position and orientation persist due to the sensor systems used. These would not propagate through a pure relocalization process. If based on relocalization new features are added to the map however, the corresponding error results in wrong positions. Such errors accumulate during the whole map building process, resulting in unusable maps.

One of the oldest and popular approaches to solve the SLAM problem employs Kalman filter-based techniques. More often than not a system is given with observable outputs $z(t_k)$ at time step t_k but an unobservable internal state $\boldsymbol{x}(t_k)$. The idea of a Kalman filter is to build a model of this system in which the internal state is observable and to correct this state comparing the output of the real system and the output of the model. The system to be modeled is described as follows. With an initial state $x(t_0)$ and matrices $A(t_k)$, $B(t_k)$ and $H(t_k)$, given an input vector $\boldsymbol{u}(t_k)$, the next internal state is $\boldsymbol{x}(t_{k+1})$.

$$x(t_{k+1}) = A(t_k)x(t_k) + B(t_k)u(t_k) + q(t_k) \tag{2.25}$$

$$z(t_{k+1}) = H(t_k)x(t_k) + r(t_k) \tag{2.26}$$

There is inevitable noise in the system, that is, $\boldsymbol{q}(\mathrm{t}_k)$ is the system noise and $\boldsymbol{r}(\mathrm{t}_k)$ the measurement noise. To get the still unknown internal state $\boldsymbol{x}(\mathrm{t}_k)$, a noise-free model is run in parallel to the real system together with matrices A, B, and H and an initial value $\boldsymbol{x}^*(t_0) = 0$. The output of the model is

$$z^*(t_{k+1}) = H(t_k)x^*(t_k) \tag{2.27}$$

Let A, B and H be treated as time invariant. The model internal state $\boldsymbol{x}^*(t_k)$ is enhanced using the measured output $\boldsymbol{z}(t_k)$ and a Kalman amplification $K(t_k)$,

$$x^{**}(t_k) = x^*(t_k) + K(t_k)\big(z(t_k) - z^*(t_k)\big) \tag{2.28}$$

Then the next internal value of the model is calculated as,

$$x^*(t_{k+1}) = A(t_k)x^{**}(t_k) + B(t_k)u(t_k) \tag{2.29}$$

The Kalman amplification is calculated so that the sum of the variances of the error gets minimal. The theory of a Kalman filter shows how to calculate the Kalman amplification from the given variances of the measured output values. This gives a rather good enhancement of the calculation of the vehicle pose and shrinks the remaining error considerably.

An EKF is employed for state estimation in those situations where the process is governed by nonlinear dynamics and/or involves nonlinear measurement relationships. The method employs linearization about the filter's estimated trajectory, which is continuously updated in accordance with the state estimates obtained from the measurements (Brown and Hwang 1997). The state transition can be modeled by a nonlinear function $\boldsymbol{f}(\cdot)$ and the observation or measurement of the state can be modeled by a nonlinear function $\boldsymbol{h}(\cdot)$, given as,

$$\boldsymbol{x}_{k+1} = \boldsymbol{f}(\boldsymbol{x}_k, \boldsymbol{u}_k) + \boldsymbol{q}_k \tag{2.30}$$

and

$$\boldsymbol{z}_{k+1} = \boldsymbol{h}(\boldsymbol{x}_{k+1}) + \boldsymbol{r}_{k+1} \tag{2.31}$$

where $\boldsymbol{x}_k$ is the $(n \times 1)$ process state vector at sampling instant k, $\boldsymbol{z}_k$ is the $(m \times 1)$ measurement vector at sampling instant k and $\boldsymbol{u}_k$ is the control input. The random variables $\boldsymbol{q}_k$ and $\boldsymbol{r}_k$ represent Gaussian white process noise and measurement noise respectively, and $\boldsymbol{P}_k$, $\boldsymbol{Q}_k$ and $\boldsymbol{R}_k$ represent the covariance matrices for $\boldsymbol{x}_k$, $\boldsymbol{q}_k$, and $\boldsymbol{r}_k$, respectively. Until now extensive research works have been reported employing EKF to address several aspects of the SLAM problem (Dissanayake et al. 2001; Smith and Cheeseman et al. 1986; Moutarlier and Chatila 1989; Davison 1998; Bailey 2002; Davison and Murray 2002; Guivant and Nebot 2001, 2003; Williams et al. 2000; Chong and Kleeman 1999). In case of the SLAM problem, the state vector $\boldsymbol{x}$ is composed of the mobile robot states $\boldsymbol{x}_\mathrm{r}$, and the landmarks' states $\boldsymbol{x}_\mathrm{m}$. Hence the

estimates of the total state vector $\boldsymbol{x}$, maintained in the form of its mean vector $\hat{x}$ and the corresponding total error covariance matrix $\boldsymbol{P}$ is given as,

$$\hat{x} = \left[\, \hat{x}_{\mathrm{r}}^{\mathrm{T}} \;\; \hat{x}_{\mathrm{m}}^{\mathrm{T}} \,\right]^{\mathrm{T}} \tag{2.32}$$

$$P = \begin{bmatrix} P_{\mathrm{r}} & P_{\mathrm{rm}} \\ P_{\mathrm{rm}}^{\mathrm{T}} & P_{\mathrm{m}} \end{bmatrix} \tag{2.33}$$

where $\hat{x}_{\mathrm{r}}$ = the mean estimate of the robot states (represented by its pose), $\boldsymbol{P}_{\mathrm{r}}$ = error covariance matrix associated with $\hat{x}_{\mathrm{r}}$, $\hat{x}_{\mathrm{m}}$ = mean estimate of the feature/landmark positions and $\boldsymbol{P}_{\mathrm{m}}$ = error covariance matrix associated with $\hat{x}_{\mathrm{m}}$. The robot pose is defined with respect to an arbitrary base Cartesian coordinate frame. The features or landmarks are considered to be 2D point features. It is assumed that there are n such static point features/landmarks observed in the map. Then,

$$\hat{x}_{\mathrm{r}} = \left[\, \hat{x}_{\mathrm{r}} \;\; \hat{y}_{\mathrm{r}} \;\; \hat{\varphi}_{\mathrm{r}} \,\right]^{\mathrm{T}} \tag{2.34}$$

$$P_{\mathrm{r}} = \begin{bmatrix} \sigma^2_{x_{\mathrm{r}}x_{\mathrm{r}}} & \sigma^2_{x_{\mathrm{r}}y_{\mathrm{r}}} & \sigma^2_{x_{\mathrm{r}}\varphi_{\mathrm{r}}} \\ \sigma^2_{x_{\mathrm{r}}y_{\mathrm{r}}} & \sigma^2_{y_{\mathrm{r}}y_{\mathrm{r}}} & \sigma^2_{y_{\mathrm{r}}\varphi_{\mathrm{r}}} \\ \sigma^2_{x_{\mathrm{r}}\varphi_{\mathrm{r}}} & \sigma^2_{y_{\mathrm{r}}\varphi_{\mathrm{r}}} & \sigma^2_{\varphi_{\mathrm{r}}\varphi_{\mathrm{r}}} \end{bmatrix} \tag{2.35}$$

and

$$\hat{x}_{\mathrm{m}} = \left[\hat{x}_1 \hat{y}_1 \;\; \ldots \;\; \hat{y}_n \hat{y}_n\right]^{\mathrm{T}} \tag{2.36}$$

$$P_{\mathrm{m}} = \begin{bmatrix} \sigma^2_{x_1x_1} & \sigma^2_{x_1y_1} & \cdots & \sigma^2_{x_1x_n} & \sigma^2_{x_1y_n} \\ \sigma^2_{x_1y_1} & \sigma^2_{y_1y_1} & & \sigma^2_{y_1x_n} & \sigma^2_{y_1y_n} \\ \vdots & & \ddots & \vdots & \\ \sigma^2_{x_1x_n} & \sigma^2_{y_1x_n} & \cdots & \sigma^2_{x_nx_n} & \sigma^2_{x_ny_n} \\ \sigma^2_{x_1y_n} & \sigma^2_{y_1y_n} & & \sigma^2_{x_ny_n} & \sigma^2_{y_ny_n} \end{bmatrix} \tag{2.37}$$

The map is defined in terms of the position estimates of these static features and $\boldsymbol{P}_{\mathrm{rm}}$ in (2.33) maintains the robot-map correlation. The off-diagonal elements of $\boldsymbol{P}_{\mathrm{m}}$ signify the cross-correlation and hence interdependence of information among the features themselves. The system is initialized assuming that there is no observed feature as yet, the base Cartesian coordinate frame is aligned with the robot's starting pose and there is no uncertainty in the starting pose of the robot. Mathematically speaking, $\hat{x}_{\mathrm{m}} = \hat{x}_{\mathrm{m}}$ $r = 0$ and $\boldsymbol{P} = \boldsymbol{P}_{\mathrm{r}} = \boldsymbol{0}$.

As the robot starts moving, $\hat{x}_{\mathrm{r}}$ and $\boldsymbol{P}_{\mathrm{r}}$ become non-zero values. In subsequent iterations, when the first observation is carried out, new features are expected to be initialized and $\hat{x}_{\mathrm{m}}$ and $\boldsymbol{P}_{\mathrm{m}}$ appear for the first time. This increases the size of $\hat{x}$ and $\boldsymbol{P}$, and the entries of $\hat{x}$ vector and $\boldsymbol{P}$ matrix are re-calculated. This process is continued

iteratively in three steps, that is, the time update ("predict") step, the measurement update ("correct") step, and the step of initialization of a new feature and deletion of an old feature.

The prediction step calculates the projections of the state estimates and the error covariance estimates from sampling instant k to $(k + 1)$. The robot pose is estimated on the basis of the motion model and the control inputs. Here, it is assumed that the control input vector $\boldsymbol{u}$, under the influence of which the robot moves, is constituted of two control inputs, the steering angle command (s) and the velocity at which the rear wheel is driven (w). Hence, $\boldsymbol{u} = [w\ s]^{\mathrm{T}}$. So the state estimates can be obtained by employing wheel encoder odometry and the robot kinematic model. The control inputs w and s must be considered with their uncertainties involved (e.g. uncertainties due to wheel slippage, incorrect calibration of vehicle controller) and these are modeled as Gaussian variations in w and s from their nominal values. Based on the odometric equation of the mobile robot under consideration here, which assumes that the incremental movement of the robot is linear in nature, $\boldsymbol{f}_{\mathrm{r}}$ can be represented as,

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \varphi_{k+1} \end{bmatrix} = \begin{bmatrix} x_k + \bar{w}\Delta t\cos(\varphi_k + \bar{s}) \\ y_k + \bar{w}\Delta t\sin(\varphi_k + \bar{s}) \\ \varphi_k + \frac{\bar{w}\Delta t\sin(\bar{s})}{W_{\mathrm{B}}} \end{bmatrix} \tag{2.38}$$

$$\begin{aligned} \bar{w} &= w + Q_1 \\ \bar{s} &= s + Q_2 \end{aligned} \tag{2.39}$$

where W_{B} represents the wheel base of the robot and Δt is the sampling time, Q_1 and Q_2 represent measurement noise, respectively. To estimate the error covariance, the Jacobians and the covariance matrix of $\boldsymbol{u}$ are given as

$$A = \begin{bmatrix} \frac{\partial f_{\mathrm{r}}}{\partial x} \\ \frac{\partial f_{\mathrm{r}}}{\partial y} \\ \frac{\partial f_{\mathrm{r}}}{\partial \varphi} \end{bmatrix} = \begin{bmatrix} 1 & 0 & -w\Delta t\sin(\varphi + s) \\ 0 & 1 & w\Delta t\cos(\varphi + s) \\ 0 & 0 & 1 \end{bmatrix} \tag{2.40}$$

$$W = \begin{bmatrix} \frac{\partial f_{\mathrm{r}}}{\partial Q_1} & \frac{\partial f_{\mathrm{r}}}{\partial Q_2} \end{bmatrix} = \begin{bmatrix} \Delta t\cos(\varphi + s) & -w\Delta t\sin(\varphi + s) \\ \Delta t\sin(\varphi + s) & w\Delta t\cos(\varphi + s) \\ \frac{\Delta t\sin(\varphi + Q_2)}{W_{\mathrm{B}}} & \frac{w\Delta t\cos(\varphi)}{W_{\mathrm{B}}} \end{bmatrix} \tag{2.41}$$

$$U = \begin{bmatrix} \sigma_{\mathrm{w}}^2 & 0 \\ 0 & \sigma_{\mathrm{s}}^2 \end{bmatrix} \tag{2.42}$$

Here, $\hat{x}_{\mathrm{m}}$ and $\boldsymbol{P}_{\mathbf{m}}$ in (2.32) and (2.33) remain constant with time, as the features are assumed to remain stationary with time.

In the measurement update step, it is assumed that we observe a feature, which already exists in the feature map, whose position is denoted by that of the ith feature i.e. $\left(\hat{x}_i,\ \hat{y}_i\right)$ and that the mobile robot is also assumed to be equipped with wheel and

steering encoders. The distance measured, in polar form, gives the relative distance between each feature and the scanner (and hence the vehicle). Let this feature be measured in terms of its range (r) and bearing (θ) relative to the observer, given as

$$z = \begin{bmatrix} r & \theta \end{bmatrix}^{\mathrm{T}} \tag{2.43}$$

The uncertainties in these observations are again modeled by Gaussian variations and let $\boldsymbol{R}$ be the corresponding observation/measurement noise covariance matrix, given as

$$R = \begin{bmatrix} \sigma_{\mathrm{r}}^2 & 0 \\ 0 & \sigma_{\theta}^2 \end{bmatrix} \tag{2.44}$$

where we assume that there is no cross-correlation between the range and bearing measurements. In the context of the map, the measurements can be given as

$$z_{i_k} = h_i\left(\hat{x}_k\right) = \begin{bmatrix} \text{range} \\ \text{bearing} \end{bmatrix}_i = \begin{bmatrix} \sqrt{\left(x_i - x_{r_k}\right)^2 + \left(y_i - y_{r_k}\right)^2} \\ \tan^{-1}\left(\frac{y_i - y_{r_k}}{x_i - x_{r_k}}\right) - \varphi_{r_k} \end{bmatrix} \tag{2.45}$$

The corresponding Jacobians is given as

$$S = \begin{bmatrix} \frac{\partial \text{range}}{\partial x_{\mathrm{r}}} & \frac{\partial \text{range}}{\partial y_{\mathrm{r}}} & \frac{\partial \text{range}}{\partial \varphi_{\mathrm{r}}} \\ \frac{\partial \text{bearing}}{\partial x_{\mathrm{r}}} & \frac{\partial \text{bearing}}{\partial y_{\mathrm{r}}} & \frac{\partial \text{bearing}}{\partial \varphi_{\mathrm{r}}} \end{bmatrix} \tag{2.46}$$

$$S = \begin{bmatrix} \frac{x_{\mathrm{r}} - x_i}{r} & \frac{y_{\mathrm{r}} - y_i}{r} & 0 \\ \frac{y_i - y_{\mathrm{r}}}{r^2} & \frac{x_i - x_{\mathrm{r}}}{r^2} & -1 \end{bmatrix} \tag{2.47}$$

Now the Kalman gain, $\boldsymbol{W}_i$, can be calculated assuming that there is correct landmark association between $\boldsymbol{z}$ and $\left(\hat{x}_i,\ \hat{y}_i\right)$. Hence, the *a posterior* augmented state estimate and the corresponding covariance matrix are updated as

$$v_{i_{k+1}} = z_{k+1} - h_i\left(x_{k+1}^{-}\right) \tag{2.48}$$

$$\hat{x}_{k+1}^{+} = \hat{x}_{k+1}^{-} + W_{i_{k+1}}\left(z_{k+1} - h_i\left(x_{k+1}^{-}\right)\right) = \hat{x}_{k+1}^{-} + W_{i_{k+1}} v_{i_{k+1}} \tag{2.49}$$

$$P_{k+1}^{+} = P_{k+1}^{-} - W_{i_{k+1}} S_{i_{k+1}} W_{i_{k+1}}^{\mathrm{T}} \tag{2.50}$$

In this work, we have considered that the features are point-like features, each representing a unique distinct point in the two-dimensional map of the environment. During this iterative procedure of performing prediction and update steps recursively, it is very likely that observations of new features are made time to time. Then these new features should be initialized into the system by incorporating their 2D position coordinates in the augmented state vector and accordingly modifying the covariance

matrix. These features may correspond to points, lines, corners, edges, etc. The deletion of unreliable features is a relatively simple matter. It is only necessary to delete the relevant row entries from the state vector and the relevant row and column entries from the covariance matrix. Since there will be multiple number of landmarks visible at the same time when an observation step is carried out, several independent observations will be carried out in batches. The corresponding SLAM algorithm is based on composite ν, $\boldsymbol{S}$ and $\boldsymbol{W}$ vectors/matrices and the sizes of these vectors/matrices keep changing with time because, at any instant of observation, the total number of visible landmarks keep changing.

To summarize, it can be seen that the "time update" and "measurement update" equations in the formulation of an EKF are obtained by employing linearization of nonlinear functions $\boldsymbol{f}(\bullet)$ and $\boldsymbol{h}(\bullet)$ about the point of the state mean. This linearization is obtained by employing a Taylor series like expansion and neglecting all terms which are of higher order than the first order term in the series. In addition to the process and measurement uncertainties, this manner of approximating a nonlinear system by a first order derivative introduces this additional source of uncertainty in EKF algorithm. In addition to the process and measurement uncertainties, there is an additional uncertainty due to linearization involved in EKF algorithm. In fact, for highly nonlinear functions, these linearized transformations cannot sufficiently and accurately approximate correct covariance transformations and may lead to highly inconsistent uncertainty estimate.

From the description of EKF-based SLAM, however, it can be seen that another problem with the classical full EKF-based SLAM approach is that the computational burden becomes significantly high in the presence of a large number of features, because both the total state vector and the total covariance matrix become large in size. This is because, during the process of navigation, new landmarks are initialized in the state vector at different time instants and, under some specific conditions, some existing landmarks may even be deleted and hence these vector and matrix sizes will keep changing. Hence, the sizes of the state vector and the covariance matrix are time varying in nature. The sizes of these matrices usually grow.

Some alternative approaches to solve SLAM problems have also been proposed which intend to implement some numerical algorithms, rather than employing the rigorous statistical methods as in EKF. Some of these schemes are based on the Bayesian approaches which can dispense with the important assumption in EKF. Several such algorithms have been developed employing Sequential Monte Carlo (SMC) methods that employ the essence of particle filtering (Montemerlo et al. 2003; Grisetti et al. 2005; Montemerlo and Thrun 2003; Hu et al. 2004). In particle filtering based methods, it is expected that a large number of particles are employed so that it can contain a particle that can very closely resemble the true pose of the robot at each sampling time instant (Frese et al. 2005). How to develop an efficient SLAM algorithm, employing particle filtering with small enough number of particles, constitutes an important area of modern-day research. A significant leap in this direction is taken by the FastSLAM1.0 and FASTSLAM2.0 algorithms, which have successfully solved the issue of dimensionality for particle filter based SLAM problems (Montemerlo et al. 2002).

As shown in Fig. 2.6, FastSLAM is a probabilistic approach that describes a solution for the SLAM problem from a Bayesian point of view (Montemerlo et al. 2002). FastSLAM factors the problem into the localization (i.e., the knowledge about the robot's path $s_1, s_2, \ldots, s_t$) and a collection of single landmark estimations θ_k that depend on the robot's estimated pose.

In terms of the probabilistic approach, the robot's poses evolve according to the motion model,

$$p(s_t \mid u_t,\ s_{t-1}) \tag{2.51}$$

where s_t is a probabilistic function of a control u_t and the previous pose s_{t-1}. The landmarks in the environment of the robot are characterized by their location, denoted as θ_k. By serializing the observation of multiple landmarks at the same time, sensor measurements for these landmarks are underlying the measurement model,

$$p(z_t \mid s_t,\ \theta,\ n_t) \tag{2.52}$$

where θ is the set of all landmarks and n_t is the index of the landmark observed as z_t at the time t. To simplify the following description, the correspondence (value of n_t) is assumed to be known.

Given these two models, SLAM can be solved by determining the location of all landmarks and the robot poses based on measurements z_t and control inputs u_t,

$$p\left(s^t,\ \theta \mid z^t,\ u^t,\ n^t\right) \tag{2.53}$$

The superscript t describes a set of variables from time 1 to time t.

All individual landmark estimation problems are independent if the robot's path s_t and the correspondence n_t are known. So the rather difficult solution for (2.53) can be found by solving $k + 1$ more simple problems,

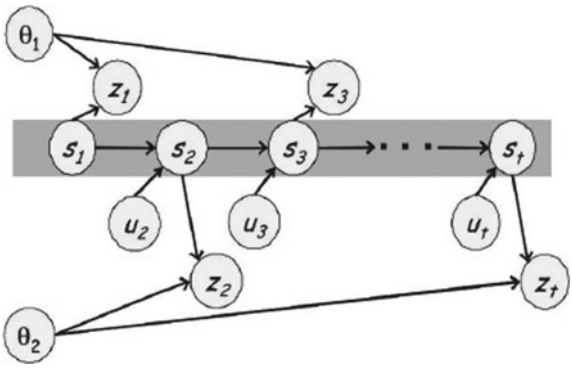


Fig. 2.6 The SLAM problem as Bayesian network

$$p\left(s^t,\ \theta \mid z^t,\ u^t,\ n^t\right) = p\left(s^t \mid z^t,\ u^t,\ n^t\right) \prod_k p\left(\theta_k \mid s^t,\ z^t,\ u^t,\ n^t\right) \tag{2.54}$$

To solve these problems, FastSLAM implements a path estimator,

$$p\left(s^t \mid z^t,\ u^t,\ n^t\right) \tag{2.55}$$

using a particle filter that is similar to Monte Carlo Localization (Thrun et al. 2001). At each point in time, the algorithm maintains a set S_t of particles representing the posterior distribution $p\left(s^t \mid z^t, u^t, n^t\right)$. Each particle $s^{t,[m]}$ is considered as a guess of the robot's path, using the superscript notation to refer to the mth particle in the set. Each particle set S_t is calculated incrementally from the set S_{t-1}, a control u_t and a measurement z_t. This is done by generating a temporary guess $s_t^{[m]}$ using the prior distribution $p\left(s_t \mid u_t, s_{t-1}^{[m]}\right)$. This basically means that the last guess together with the last control command are used to deduce the new guess (a similar process to the dead reckoning case). Assuming that S_{t-1} is distributed according to $p\left(s^{t-1} \mid z^{t-1}, u^{t-1}, n^{t-1}\right)$, the new set S_t is distributed according to $p\left(s^t \mid z^{t-1}, u^t, n^{t-1}\right)$ as a proposal distribution. The new set S_t is then obtained by sampling from the temporary guesses with a probability that is proportional to an importance factor $w_t^{[m]}$. This results in a new distribution:

$$p\left(s^{t,[m]} \mid z^t, u^t, n^t\right) = w_t^{[m]} p\left(s^{t,[m]} \mid z^{t-1}, u^{t-1}, n^{t-1}\right) \tag{2.56}$$

In the following derivation, the conditional Bayes' theorem with the two events x, y and additional information e is applied,

$$p(x \mid y,\ e) = \frac{p(y \mid x,\ e)p(x \mid e)}{p(y \mid e)} \tag{2.57}$$

In addition, $z^t = z^{t-1} \cup z_t$ and $n^t = n^{t-1} \cup n_t$ is used to compute the weights $w_t^{[m]}$,

$$\begin{aligned}
w_t^{[m]} &= \frac{p\left(s^{t,[m]} \mid z^t,\ u^t,\ n^t\right)}{p\left(s^{t,[m]} \mid z^{t-1},\ u^t,\ n^{t-1}\right)} = \frac{p\left(s^{t,[m]} \mid z_t,\ n_t,\ z^{t-1},\ u^t,\ n^{t-1}\right)}{p\left(s^{t,[m]} \mid z^{t-1},\ u^t,\ n^{t-1}\right)} \\
&\overset{\text{Bayes}}{=} \frac{\frac{p\left(z_t,\ n_t \mid s^{t,[m]},\ z^{t-1},\ u^t,\ n^{t-1}\right)}{p\left(z_t,\ n_t \mid z^{t-1},\ u^t,\ n^{t-1}\right)} p\left(s^{t,[m]} \mid z^{t-1},\ u^t,\ n^{t-1}\right)}{p\left(s^{t,[m]} \mid z^{t-1},\ u^t,\ n^{t-1}\right)} \\
&= \frac{p\left(z_t,\ n_t \mid s^{t,[m]},\ z^{t-1},\ u^t,\ n^{t-1}\right)}{p\left(z_t,\ n_t \mid z^{t-1},\ u^t,\ n^{t-1}\right)} \\
&\propto p\left(z_t,\ n_t \mid s^{t,[m]},\ z^{t-1},\ u^t,\ n^{t-1}\right) \\
&\overset{\text{Total_prob}}{=} \int p\left(z_t,\ n_t \mid \theta,\ s^{t,[m]},\ z^{t-1},\ u^t,\ n^{t-1}\right) p\left(\theta \mid s^{t,[m]},\ z^{t-1},\ u^t,\ n^{t-1}\right) \mathrm{d}\theta
\end{aligned}$$

$$
\begin{aligned}
&\overset{\text{Markov}}{=} \int p\left(z_t,\ n_t \mid \theta,\ s^{t,[m]}\right) p\left(\theta \mid s^{t-1,[m]},\ z^{t-1},\ u^{t-1},\ n^{t-1}\right) \mathrm{d}\theta \\
&= \int p\left(z_t \mid \theta,\ s^{t[m]},\ n_t\right) p\left(n_t \mid \theta,\ s^{t[m]}\right) p\left(\theta \mid s^{t-1,[m]},\ z^{t-1},\ u^{t-1},\ n^{t-1}\right) \mathrm{d}\theta \\
&\propto \int p\left(z_t | \theta, s^{t[m]}, n_t\right) p\left(\theta \mid s^{t-1,[m]},\ z^{t-1},\ u^{t-1},\ n^{t-1}\right) \mathrm{d}\theta \\
&= \int p\left(z_t \mid \theta_{n_t}^{[m]},\ s^{t[m]},\ n_t\right) p\left(\theta_{n_t}^{[m]}\right) \mathrm{d}\theta_{n_t}^{[m]}
\end{aligned} \tag{2.58}
$$

The last step assumes $p\left(n_t \,\middle|\, \theta, s_t^{[m]}\right)$ being uniform and the landmark estimation relying on a Gaussian posterior $p\left(\theta_{n_t}^{[m]}\right)$, specified by the mean $u_{n_t}^{[m]}$ and covariance $\sum_{n_t}^{[m]}$ of the estimated posterior of θ_{n_t}. Now the above equation can be solved in closed form. The landmark estimators

$$
p(\theta_k \mid s_t,\ z_t,\ u_t,\ n_t) \tag{2.59}
$$

as the remaining part of Eq. (2.54) are implemented via Kalman filters. These estimators are conditioned on the robot pose, so each particle in S_t is extended by its own set of Kalman filters for the landmark estimators.

	Path	θ_1	θ_2	θ_k
1st Particle	s^t	$\mu_1,\ \Sigma_1$	$\mu_2,\ \Sigma_2$	$\cdots\ \mu_k,\ \Sigma_k$
2nd Particle	s^t	$\mu_1,\ \Sigma_1$	$\mu_2,\ \Sigma_2$	$\cdots\ \mu_k,\ \Sigma_k$
$\vdots$				
mth Particle	s^t	$\mu_1,\ \Sigma_1$	$\mu_2,\ \Sigma_2$	$\cdots\ \mu_k,\ \Sigma_k$

Assume $n_t = k$. This means that the landmark θ_k is visible at time t and the estimation of $\theta_k^{[m]}$ can easily be obtained:

$$
\begin{aligned}
&p(\theta_k \mid s_t,\ z_t,\ u_t,\ n_t) = p\left(\theta_k \mid z_t,\ s^t,\ z^{t-1},\ u^t,\ n^t\right) \\
&\overset{\text{Bayes}}{=} \frac{p\left(z_t \mid \theta_k,\ s^t,\ z^{t-1},\ u^t,\ n^t\right) p\left(\theta_k \mid s^t,\ z^{t-1},\ u^t,\ n^t\right)}{p\left(z_t \mid s^t,\ z^{t-1},\ u^t,\ n^t\right)} \\
&\propto p\left(z_t \mid \theta_k,\ s^t,\ z^{t-1},\ u^t,\ n^t\right) p\left(\theta_k \mid s^t,\ z^{t-1},\ u^t,\ n^t\right) \\
&\overset{\text{Markov}}{=} p(z_t \mid \theta_k,\ s_t,\ u_t,\ n_t) p\left(\theta_k \mid s^{t-1},\ z^{t-1},\ u^{t-1},\ n^{t-1}\right)
\end{aligned} \tag{2.60}
$$

For $n_t \neq k$, meaning landmark θ_k is not visible at time t, the distribution is not changed.

$$
p\left(\theta_k \mid s^t,\ z^t,\ u^t,\ n^t\right) = p\left(\theta_k \mid s^{t-1},\ z^{t-1},\ u^{t-1},\ n^{t-1}\right) \tag{2.61}
$$

The updated Eq. (2.61) can be implemented using an extended Kalman filter, resulting in $O(MK)$ computations for M particles and K landmarks per step t.

Most of the SLAM approaches are oriented towards indoor, well structured and static environment (Zunino and Christensen 2001; Bosse et al. 2004; Dissanayake et al. 2001; Estrada et al. 2005; Guivant and Nebot 2001; Bosse et al. 2002) and give metric information regarding the position of the mobile robot and of the landmarks. A few works have also been attempted for dynamic scenarios and for outdoor environments (Andrade-Cetto and Sanfeliu 2002; Liu and Thrun 2003). Several successful applications of SLAM algorithms have also been developed for underwater applications (Williams et al. 2001; Beall et al. 2010; Kim and Eustice 2013), and underground applications (Thrun et al. 2003), etc.

2.4 Conclusions

To summarize, mapless strategies require an explicit representation of the working environment where navigation is to occur. Map-based techniques basically employ geometric models or topological maps of the environment. Despite providing fast, robust, and consistent solutions to many problems, they are highly dependent on static maps of working environments which limits the operational capability of the algorithms in this group. In map-building-based strategies, sensors are used to construct geometric models or topological models of the environment, which are then used to navigate the robot through corresponding environments. Although they can allow autonomous mobile robots to navigate through dynamic environments (Szendy et al. 2016; Lategahn et al. 2011; Li et al. 2014; Marzat et al. 2017; Latif et al. 2013; Sünderhauf and Protzel 2012; Valiente et al. 2014; Huang et al. 2017; Mur-Artal et al. 2015), algorithms used in this category cost a lot of time and effort to obtain a robust model of the environment.

References

Andrade-Cetto, J., & Sanfeliu, A. (2002). Concurrent map building and localization on indoor dynamic environment. *International Journal of Pattern Recognition and Artificial Intelligence, 16*(3), 361–374.

Bailey, T. (2002). *Mobile robot localization and mapping in extensive outdoor environments* (Ph.D. Thesis), University of Sydney.

Bailey, T., & Durrant-Whyte, H. (2006). Simultaneous localisation and mapping (SLAM): Part II state of the art. *IEEE Robotics and Automation Magazine, 13*(3), 108–117.

Beall, C., Lawrence, B. J., Ila, V. et al. (2010). 3D reconstruction of underwater structures. In *Proceedings of the 2010 IEEE/RSJ International Conference on Intelligent Robots and Systems* (*IROS*) (pp. 4418–4423), Taipei, China.

Bonin-Font, F., Ortiz, A., & Oliver, G. (2008). Visual navigation for mobile robots: A survey. *Journal of Intelligent and Robotic Systems, 53*(3), 263–296.

Bosse, M., Leonard, J., & Teller, S. (2002). Large-scale CML using a network of multiple local maps. In J. Leonard, J. D. Tardós, S. Thrun, & H. Choset (Eds.), *Workshop Notes of the ICRA Workshop on Concurrent Mapping and Localization for Autonomous Mobile Robots (W4)*, Washington, DC, USA.

Bosse, M., Newman, P., Leonard, J., & Teller, S. (2004). Slam in large-scale cyclic environments using the atlas framework. *International Journal of Robotics Research, 23*(12), 1113–1139.

Brown, R. G., & Hwang, P. Y. C. (1997). *Introduction to random signals and applied Kalman filtering* (3rd ed.). USA: Wiley.

Carsten, J., Rankin, A., Ferguson, D., & Stentz, A. (2009a). Global path planning on board the mars exploration rovers. In *Proceedings of the 2007 IEEE Aerospace Conference* (pp. 1–11), Big Sky, MT, USA.

Carsten, J., Rankin, A., Ferguson, D., & Stentz, A. (2009b). Global planning on the mars exploration rovers: Software integration and surface testing. *Journal of Field Robotics, 26*(4), 337–357.

Chong, K. S., & Kleeman, L. (1999). Feature-based mapping in real, large scale environments using an ultrasonic array. *International Journal of Robotic Research, 18*(2), 3–19.

Davison, A. J. (1998). *Mobile robot navigation using active vision* (Ph.D. Thesis), University of Oxford.

Davison, A. J., & Murray, D. W. (2002). Simultaneous localization and map-building using active vision. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 24*(7), 865–880.

DeSouza, G. N., & Kak, A. C. (2002). Vision for mobile robot navigation: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 24*(2), 237–267.

Dissanayake, M. W. M. G., Newman, P., Clark, S., Durrant-Whyte, H. F., & Csorba, M. (2001). A solution to the simultaneous localization and map building (SLAM) problem. *IEEE Transactions on Robotics and Automation, 17*(3), 229–241.

Durrant-White, H., & Bailey, T. (2006). Simultaneous localization and mapping. *IEEE Robotics and Automation Magazine, 13*(2), 99–108.

Estrada, C., Neira, J., & Tardos, J. D. (2005). Hierarchical SLAM: Real-time accurate mapping of large environments. *IEEE Transactions on Robotics, 21*(4), 588–596.

Frese, U., Larsson, P., & Duckett, T. (2005). A multilevel relaxation algorithm for simultaneous localization and mapping. *IEEE Transactions on Robotics, 21*(2), 196–207.

Grisetti, G., Stachniss, C., & Burgard, W. (2005). Improving grid-based SLAM with Rao-Blackwellized particle filters by adaptive proposals and selective resampling. In *Proceedings of the IEEE International Conference on Robotics and Automation* (*ICRA*'05) (pp. 2443–2448), Barcelona, Spain.

Guivant, J. E., & Nebot, E. M. (2001). Optimization of the simultaneous localization and map-building algorithm for real-time implementation. *IEEE Transactions on Robotics and Automation, 17*(3), 242–257.

Guivant, J., & Nebot, E. (2003). Solving computational and memory requirements of feature based simultaneous localization and mapping algorithms. *IEEE Transactions on Robotics and Automation, 19*(4), 749–755.

Hu, W., Downs, T., Wyeth, G., Milford, M., & Prasser, D. (2004). A modified particle filter for simultaneous robot localization and Landmark tracking in an indoor environment. In *Proceedings of Australian Conference on Robotics and Automation* (*ACRA*'04), Canberra, Australia.

Huang, A. S., Bachrach, A., Henry, P., et al. (2017). Visual odometry and mapping for autonomous flight using an RGB-D camera. In: H. I. Christensen, O. Khatib (Eds.), *Robotics research. Springer Tracts in Advanced Robotics* (Vol. 100, pp. 235–252). Cham: Springer.

Kim, A., & Eustice, R. M. (2013). Real-time visual SLAM for autonomous underwater hull inspection using visual saliency. *IEEE Transactions on Robotics, 29*(3), 719–733.

Lategahn, H., Geiger, A., & Kitt, B. (2011). Visual SLAM for autonomous ground vehicles. In *Proceedings of the 2011 IEEE International Conference on Robotics and Automation* (ICRA) (pp. 1732–1737) Shanghai, China.

Latif, Y., Cadena, C., & Neira, J. (2013). Robust loop closing over time for pose graph SLAM. *The International Journal of Robotics Research, 32*(14), 1611–1626.

Leonard, J. J., & Durrant-Whyte, H. F. (1991). Simultaneous map building and localization for an autonomous mobile robot. In *Proceedings of IEEE/RSJ International Workshop on Intelligent Robots and Systems* (*IROS*'91), Osaka, Japan.

Leonard, J., & Durrant-Whyte, H. (1992). Dynamic map building for an autonomous mobile robot. *The International Journal of Robotics Research, 11*(4), 286–298.

Li, R. B., Liu, J. Y., Zhang, L., et al. (2014). LIDAR/MEMS IMU integrated navigation (SLAM) method for a small UAV in indoor environments. In *Proceedings of the 2014 DGON Inertial Sensors and Systems Symposium* (*ISS*'14) (pp. 1–15). Germany: Karlsruhe.

Liu, Y., & Thrun, S. (2003). Results for outdoor-SLAM using sparse extended information filters. In *Proceedings of IEEE Conference on Robotics and Automation* (*ICRA*'03) (pp. 1227–1233), Taipei, Taiwan.

Marzat, J., Bertrand, S., Eudes, A., et al. (2017). Reactive MPC for autonomous MAV navigation in indoor cluttered environments: Flight experiments. *IFAC-Papers On Line, 50*(1), 15996–16002.

Matthias, L., Maimone, M., & Johnson, A. (2007). Computer vision on Mars. *International Journal of Computer Vision, 75*(1), 67–92.

Montemerlo, M., & Thrun, S. (2003). Simultaneous localization and mapping with unknown data association using Fast SLAM. In *Proceedings of IEEE International Conference on Robotics and Automation* (*ICRA*'03), Taipei, Taiwan.

Montemerlo, M., Thrun, S., Koller, D., & Wegbreit, B. (2002). FastSLAM: A factored solution to the simultaneous localization and mapping problem. In *Proceedings of the 18th AAAI National Conference on Artificial Intelligence and 14th Conference on Innovative Applications of Artificial Intelligence* (pp. 593–598), Edmonton, Canada.

Montemerlo, M., Thrun, S., Koller, D., & Wegbreit, B. (2003). FastSLAM 2.0: An improved particle filtering algorithm for simultaneous localization and mapping that provably converges. In *Proceedings of the 18th International Joint Conference on Artificial Intelligence* (*IJCAI*'03) (pp. 1151–1156), Acapulco, Mexico.

Moravec, H. P. (1990). The Stanford cart and the CMU rover. In I. J. Cox & G. T. Wilfong (Eds.), *Autonomous robot vehicles* (pp. 407–419). New York, NY: Springer.

Moutarlier, P., & Chatila, R. (1989). Stochastic multisensory data fusion for mobile robot location and environment modeling. In *Proceedings of the 5th International Symposium on Robotics Research*, Tokyo.

Mur-Artal, R., Montiel, J. M. M., & Tardos, J. D. (2015). ORB-SLAM: A versatile and accurate monocular SLAM system. *IEEE Transactions on Robotics, 31*(5), 1147–1163.

Nattharith, P. (2010). *Mobile robot navigation using a behavioural control strategy* (Ph.D. thesis), Newcastle University, Newcastle, UK.

Nilsson, N. J. (1984). Shakey the robot. In *Artificial Intelligence Center*, *Computer Science and Technology Division*. Menlo Park, California, USA: Standford Research Institute.

Smith, R., & Cheeseman, P. (1986). On the representation and estimation of spatial uncertainty. *International Journal of Robotics Research, 5*(4), 56–68.

Smith, R., Self, M., & Cheesman, P. (1990). Estimating uncertain spatial relationships in robotics. In *Autonomous Robot Vehicles* (pp. 167–193). New York, USA: Springer.

Sünderhauf, N., and Protzel, P. (2012). Towards a robust back-end for pose graph SLAM. In *Proceedings of the 2012 IEEE International Conference on Robotics and Automation* (*ICRA*'12) (pp. 1254–1261). MN, USA: Saint Paul.

Szendy, B., Balázs, E., Szabó-Resch, M.Z., et al. (2016). Simultaneous localization and mapping with TurtleBotII. In *Proceedings of the 16th IEEE International Symposium on Computational Intelligence and Informatics* (CINTI'15) (pp. 233–237), Budapest, Hungary.

Thrun, S., Burgard, W., & Fox, D. (1998). A probabilistic approach to concurrent mapping and localization for mobile robots. *Machine Learning, 31*(1), 29–53.

Thrun, S., Burgard, W., & Fox, D. (2006). *Probabilistic robotics*. New York, Cambridge: The MIT Press.

Thrun, S., Fox, D., Burgard, W., & Dellaert, F. (2001). Robust Monte Carlo localization for mobile robots. *Artificial Intelligence, 128*(1–2), 99–141.

Thrun, S., Hähnel, D., Ferguson, D., Montemerlo, M., Triebel, R., Burgard, W., et al. (2003). A system for volumetric robotic mapping of abandoned mines. In *Proceedings of the IEEE International Conference on Robotics and Automation* (*ICRA*'03) (Vol. 3, pp. 4270–4275).

Valiente, D., Gil, A., Fernández, L., et al. (2014). A comparison of EKF and SGD applied to a view-based SLAM approach with omnidirectional images. *Robotics and Autonomous Systems, 62*(2), 108–119.

Williams, S., Dissanayake, G., & Durrant-Whyte, H. F. (2001). Towards terrain-aided navigation for underwater robotics. *Advanced Robotics, 15*(5), 533–549.

Williams, S. B., Newman, P., Dissanayake, G., & Durrant-Whyte, H. (2000). Autonomous underwater simultaneous localization and map building. In *Proceedings of the IEEE International Conference on Robotics and Automation* (*ICRA*'00) (Vol. 2, pp. 1792–1798), San Francisco, CA.

Zunino, G., & Christensen, H. I. (2001). Simultaneous localization and mapping in domestic environments. In *Multisensor Fusion and Integration for Intelligent Systems* (pp. 67–72).

Chapter 3
A Computer Vision System for Visual Perception in Unknown Environments



Abstract The goal of machine learning research is to equip robots with human-like perception capabilities so that they can sense its working environment, understand the collected data, take appropriate actions, and learn from its experience so as to enhance future performance. As a result, the acquisition of knowledge about its environment is one of the most important tasks of an autonomous system of any kind. This is done by taking measurements using various sensors and then extracting meaningful information from those measurements. In this chapter, we will present a vision based machine perception model from the perspective of a system design domain and discuss strategies for extracting information acquired from vision sensors for mobile robot localization tasks. More specifically, we aim to apply several important machine learning techniques to vision-based mobile robot navigation applications by discussing three issues, namely, information acquisition, environmental representation, and reasoning, leading to a general high-level model of the problem. The model is intended to be generic enough to allow a wide variety of tasks to be performed using a single set of sensory data. It is argued that the model has a direct correspondence with some recent biological evidences and can be applied to solving real-world problems specifically for an autonomous system operating in outdoor unknown environments.

Keywords Computer vision · Machine perception · Mobile robot localization · Information acquisition · Environmental representation · Reasoning · Unsupervised learning · Supervised learning · Reinforcement learning

3.1 Introduction

In robotic navigation task, a mobile robot in general situations has to determine a path to reach a designated goal location, that may in general be arbitrarily remote and out of sight, in an efficient manner with respect to distance, time, energy consumption or other specific criteria related to the context. The basic problem of reaching a goal location requires knowledge about the environment layout, which is generally not available to the robot a priori and must therefore be acquired through perception. Therefore, environment representation appears as one key issue of the navigation

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_3

problem and a prerequisite for motion planning and execution. For how to represent the environment, there has been a great deal of discussions and controversies on the issues in the past twenty years.

Even though several successful approaches have been recently presented, approaches using purely metric maps, where robot position is defined by position and orientation $[x\ y\ \theta]^{\mathrm{T}}$, are vulnerable to inaccuracies in both map-making and odometry abilities of the robot since sensors are indeed always imperfect and their data are incomplete, noisy and inaccurate. Solutions for consistent mapping allowing precise and robust localization in unmodified, dynamic real-world environments are very rare. The problem is highly complex due to the fact that it requires the robot to remain localized with respect to the portion of the environment which has already been explored in order to build a coherent model. Environment representations should therefore explicitly take into account uncertainties to solve the data association problem, i.e., the ability to recognize the same feature from different perceptions.

To correctly relate the newly perceived areas with the already known ones, one means of localization is to recognize known features in the environment and use them as landmarks. Landmark-based approaches, which rely on the topology of the environment and where the robot position is defined by states or places, can better handle this problem, because they only have to maintain topological global consistency, not metric. The robot discovers its environment gradually while moving in it. Objects and regions perceived from a given location must therefore be related to previously perceived ones and integrated with them in a consistent manner. The advantage relies on the fact that topological relationships do not suffer from incremental drift as the metric ones do since they are qualitative, not quantitative.

However, historically, autonomous systems have relied on tightly constraining the complexity and structure of the operating environment through the use of artificial lighting, targets and structured environments. Modeling an unstructured and unknown environment is an extremely important competency that any autonomous system must possess in order to be considered truly useful. The term modeling in this context will be considered to mean the process whereby complex information is gathered, subsequently abstracted and concisely represented. Additionally, the representation should readily support reasoning upon the concisely represented information so that appropriate actions may be taken. Therefore, when designing a perception system for a mobile robot, the following issues must be considered,

- Information acquisition—Natural outdoor environments are characterized as being structurally complex and require significant subtlety of interpretation. The goal is for the information sources (usually sensors) to be capable of measuring the quantities of interest and for a sensor system to reduce the effects of ambiguity through the appropriate selection of sensors and task-specific processing. Without appropriate sensing and interpretative capabilities, objects remain indistinguishable. A lack of precision in interpretation leads inexorably to fragility and increased likelihood of failure.

- Representation—Data from neuroscience indicate that there are neuronal organizations and interactions that amount to representations of the environment's layout. As a result, based on a good understanding of the different kinds of representations in natural systems, it is desired that a compact representation model should be determined with regard to issues such as what knowledge it should convey to the robot, whether it models the underlying uncertainties present in the system and it is computationally tractable for autonomous robots when high resolution is used, and so on.
- Reasoning—The process of reasoning analyses the semantic or meaningful behavior of the low level knowledge and their association, such as, how the representation facilitates reasoning, what mechanisms allow decisions to be made, and whether the designer can be confident that correct decisions are made and many tasks can be carried out using the same data sources. Generally this issue constructs high-level knowledge from acquired information of relatively lower level and organizes it into a structural form for efficient access.

This chapter will examine each of these issues in turn. The issues will be explored in the context of an autonomous mobile robot for which a video camera is used as the vision sensor. The robot must be capable of constructing the model of its environment by extracting relevant information such as the identification of objects for reasoning. This chapter is organized as follows. Information acquisition, representation, and reasoning are introduced in Sects. 3.2, 3.3 and 3.4, respectively. The proposed vision based machine perception system is introduced in Sect. 3.5. The chapter summary is presented in Sect. 3.6.

3.2 Information Acquisition

To reduce the effects of ambiguity, the selection of appropriate sensors and task-specific processing defines a basic problem in environment modeling. Eighty percent of our perceived information about the external world reaches us by way of the eyes. As a result, vision is the primary sensory modality. The human visual system starts from the eye. Its basic structure is shown in Fig. 3.1. The basic function of the eye is to catch and focus light from the target stimuli onto the retina at the back of the eye. Once the image has been focused on the retina, this pattern of light will be transformed into a pattern of neural activity that can accurately represent the image. The transformation or transduction of the light energy into a neural signal is carried out by the light-sensitive receptor cells (photoreceptors) in the retina. There are two types of photoreceptors, the cones, and the rods. The cones are concentrated in a small area of the retina called the fovea. They mediate diurnal visual function and provide high-acuity color vision. In contrast, the rods mediate nocturnal vision and provide only low-acuity monochrome vision. Therefore, the range of possible light intensities is divided between these two photoreceptors, with the cones responding to high intensities and the rods to low intensities, respectively.

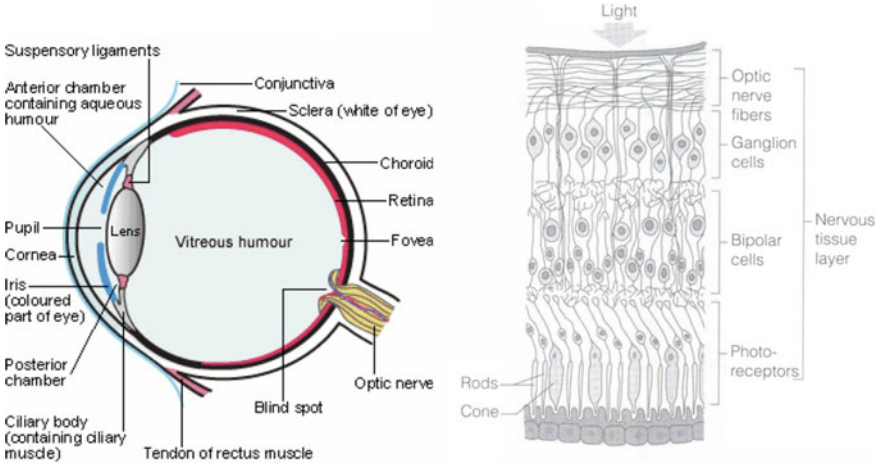


Fig. 3.1 (Left) The structure of the eye; (right) The neural structure of retina

For retina color vision to be possible in humans, there are three types of cones: the blue, red and green cones. Blue cones are between 5 and 10% of the total cone population and form a ring or annulus around the edge of the fovea. The rest are red and green cones which are randomly mixed together in small patches and clusters (Mollon and Bowmaker 1992). The blue or short-wavelength pigment absorbs maximally at 445 nm, the green or middle-wavelength pigment absorbs maximally at 535 nm, and the red or long-wavelength pigment absorbs at 575 nm (Dartnall et al. 1983). The sensitivities of these three cones to light wavelengths are shown in Fig. 3.2. In 1802, Thomas Young correctly proposed the trichromatic (three color) theory, which suggested that, for a human observer, any color could be reproduced

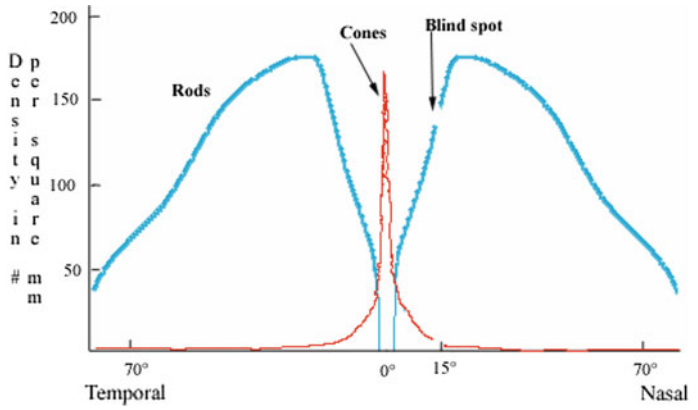


Fig. 3.2 The distribution of the rod and cone cells around fovea. Horizontal axis is the angular separation from the fovea

by various quantities of three colors selected from various points in the spectrum. The responses from the three different cone classes are compared to allow color discrimination in an opponent manner.

3.2.1 Vision Sensors

Providing us with an enormous amount of information about the environment and enabling rich, intelligent interaction in dynamic environments, vision is our most powerful sense. As a result, a great deal of effort has been devoted to providing machines with sensors that mimic the capabilities of the human vision system.

For image sensing devices to capture the light and convert it into a digital image, followed by the processing of the digital image in order to get salient information like depth computation, motion detection, color tracking, feature detection, scene recognition, and so on, vision sensors have become very popular in robotics. As shown in Fig. 3.3, the two main kinds of sensors used in digital still and video cameras today are CCD (charge coupled device) and CMOS (complementary metal

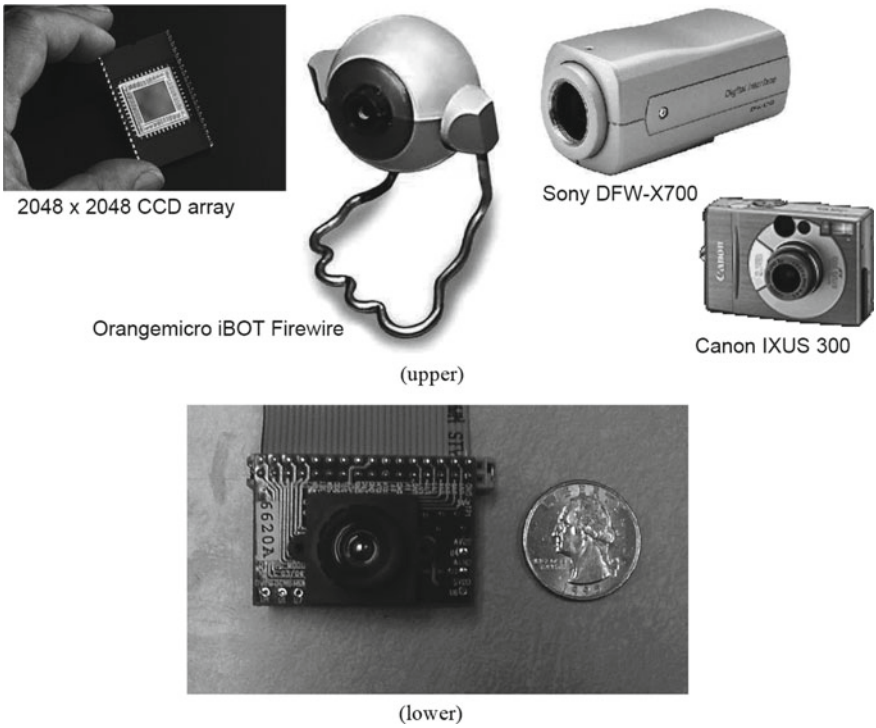


Fig. 3.3 (Upper) Commercially available CCD chips and CCD cameras, (lower) a commercially available, low-cost CMOS camera with lens attached

oxide on silicon). At the highest level, a roboticist may choose instead to utilize a higher-level digital transport protocol to communicate with an imager. Most common are the IEEE 1394 (Firewire) standard and the USB (and USB 2.0) standards. More recently, both CCD and CMOS technology vision systems provide digital signals that can be directly utilized by the roboticist.

To produce a particular image, the light, starting from one or more light sources in a scene and reflecting off one or more surfaces in the world, reaches the camera, passes through the camera's optics (lenses), and finally reaches the imaging sensor where it is usually picked up by an active sensing area, integrated for the duration of the exposure and then passed to a set of sense amplifiers. A digital image I is modeled from a mathematical point of view as a function $I(x, y)$ which maps the locations (x, y) in space to the pixel value,

$$I(x, y) = v \tag{3.1}$$

3.2.2 Color Space

An important aspect of vision sensing is that the vision chip can provide sensing modalities and cues that no other robot sensors can provide. One such novel sensing modality is detecting and tracking color in the environment. Color is an environmental characteristic and represents both a natural cue and an artificial cue that can provide new information to a mobile robot. To make extensive use of color both for environmental marking and for robot localization, color sensing has two important advantages. First, the detection of color is a straightforward function of a single image and, therefore, no correspondence problem needs to be solved in such algorithms. Second, color sensing provides a new and independent environmental cue.

The analysis of images and their processing are two major fields that are known as image processing (Gonzalez and Woods 2008) and computer vision (Szeliski 2010). Image processing is a form of signal processing where the input signal is an image (such as a photo or a video) and the output is either an image or a set of parameters associated with the image. Most image-processing techniques treat the image as a two-dimensional signal $I(x, y)$ where x and y are the spatial image coordinates and the amplitude of I at any pair of coordinates (x, y) is called intensity or gray level of the image at that point. As a scientific discipline, computer vision is concerned with the theory behind artificial systems that extract information from images in order to produce numerical or symbolic information, e.g., in the forms of decisions, to gain high-level understanding from digital images or videos. During the past forty years, we have seen significant advances and new theoretical findings in these fields (Faugeras 1993; Hartley and Zisserman 2004; Ma et al. 2003; Trucco and Verri 1998).

3.2.2.1 RGB Color Space

There are two common approaches for creating color images, using a single chip or three separate chips, respectively. The single chip technology uses the so-called Bayer filter. The pixels on the chip are grouped into 2×2 sets of four, then red, green, and blue color filters are applied so that each individual pixel receives only light of one color. Normally, two pixels of each 2×2 block measure green intensity while the remaining two pixels measure red and blue light intensities. As a result, this one-chip technology has a geometric resolution disadvantage. The number of pixels in the system has been effectively cut by a factor of four, and therefore the image resolution output by the camera will be sacrificed. The three-chip color camera avoids these problems by splitting the incoming light into three complete (lower intensity) copies. Three separate chips receive the light, with one red, green, or blue filter over each entire chip, respectively. Thus, in parallel, each chip measures light intensity for one color, and the camera must combine the chips' outputs to create a joint color image. Resolution is preserved in this solution, although the three-chip color cameras are, as one would expect, significantly more expensive and therefore more rarely used in mobile robotics.

Therefore, images and videos taken from a camcorder are stored for each frame as a RGB (Red, Green, Blue) color image. The RGB color space is a three-dimensional Euclidean space. For RGB color images, each color channel can be represented by an integer in the range [0, …, 255] or by a floating point number. In both cases, v in Eq. (3.1) will be a vector,

$$v = (R, G, B) \tag{3.2}$$

3.2.2.2 HSV Color Space

Being an alternative to the RGB color space, the HSV (Hue Saturation Value) color space can be used. While R, G, and B values encode the intensity of each color, HSV separates the color (or chrominance) measure from the brightness (or luminosity) measure. Thus, a bounding box expressed in HSV space can achieve greater stability with respect to changes in illumination than is possible in RGB space. The Hue describes each color by a normalized number in the range from 0 to 1 starting at red and cycling through yellow, green, cyan, blue, magenta, and back to red. The Saturation describes the vibrancy of the color and represents the purity of a color such as the "redness" of red. The less saturation in a color, the more pale it looks (washed out). The Value describes the brightness of the color (Fig. 3.4).

For normalized RGB values in the ranges from 0 to 1, the conversion to HSV is done in the following manner,

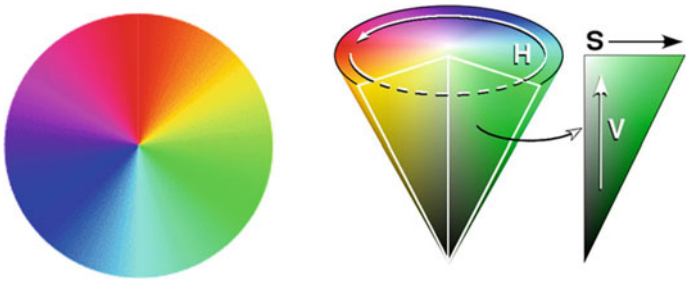


Fig. 3.4 (Left) RGB color space; (right) HSV color space

$$H = \begin{cases} \left(0 + \frac{G-B}{\text{MAX}-\text{MIN}}\right) \times 60 \text{ if } R = \text{MAX}, \\ \left(2 + \frac{B-R}{\text{MAX}-\text{MIN}}\right) \times 60 \text{ if } G = \text{MAX}, \\ \left(4 + \frac{R-G}{\text{MAX}-\text{MIN}}\right) \times 60 \text{ if } B = \text{MAX}, \end{cases} \tag{3.3}$$

$$S = \frac{\text{MAX} - \text{MIN}}{\text{MAX}} \tag{3.4}$$

$$V = \text{MAX} \tag{3.5}$$

where MAX is the maximum value of (R, G, B), and MIN is the minimum. From the above formulas, it can be seen that, if MAX = MIN, H is undefined and $S = 0$, there is no hue and the color lies along the central line of grays, and that, if MAX = 0, $V = 0$ and S is undefined, the color is pure black and there is no hue, saturation and value. As the outputs of the above formulas, the Hue values range from 0 to 360, and the Saturation and Value values range from 0 to 1. The Hue values are next normalized to be in the range [0.00, 1.00]. Orange with a bit of red lies in the range [0.00, 0.05], yellow lies in the range [0.05, 0.14], yellow–green lies in the range [0.14, 0.22], green lies in the range [0.22, 0.28], blue–green lies in the range [0.28, 0.45], blue lies in the range [0.45, 0.54], blue–violet lies in the range [0.54, 0.75], purple lies in the range [0.75, 0.81], red–violet lies in the range [0.81, 0.92], and red lies in the range [0.92, 1.00].

3.2.2.3 Opponent Color Space

The channels of the opponent color space are,

$$\begin{pmatrix} O_1 \\ O_2 \\ O_3 \end{pmatrix} = \begin{pmatrix} \frac{R-G}{\sqrt{2}} \\ \frac{R+G-2B}{\sqrt{6}} \\ \frac{R+G+B}{\sqrt{3}} \end{pmatrix} \tag{3.6}$$

In the opponent color space, the intensity information is represented by channel O_3 and the color information by O_1 and O_2. Due to the subtraction in O_1 and O_2, the offsets will cancel out if they are equal for all channels (e.g., a white light source). This is verified by substituting the unknown illuminant with offset O_1,

$$\begin{pmatrix} O_1 \\ O_2 \end{pmatrix} = \begin{pmatrix} \frac{R^{\mathrm{c}}-G^{\mathrm{c}}}{\sqrt{2}} \\ \frac{R^{\mathrm{c}}+G^{\mathrm{c}}-2B^{\mathrm{c}}}{\sqrt{6}} \end{pmatrix} = \begin{pmatrix} \frac{(R^{\mathrm{u}}+o_1)-(G^{\mathrm{u}}+o_1)}{\sqrt{2}} \\ \frac{(R^{\mathrm{u}}+o_1)+(G^{\mathrm{u}}+o_1)-2(B^{\mathrm{u}}+o_1)}{\sqrt{6}} \end{pmatrix} = \begin{pmatrix} \frac{R^{\mathrm{u}}-G^{\mathrm{u}}}{\sqrt{2}} \\ \frac{R^{\mathrm{u}}+G^{\mathrm{u}}-2B^{\mathrm{u}}}{\sqrt{6}} \end{pmatrix} \tag{3.7}$$

Therefore, these O_1 and O_2 are shift-invariant with respect to light intensity. The intensity channel O_3 has no invariance properties.

3.2.2.4 Transformed Color Space

The RGB color channels are not invariant to changes in lighting conditions. However, by normalizing the pixel value distributions, scale-invariance and shift-invariance can be achieved with respect to light intensity. Because each channel is normalized independently, the color channels are also normalized against changes in light color and arbitrary offsets,

$$\begin{pmatrix} R' \\ G' \\ B' \end{pmatrix} = \begin{pmatrix} \frac{R-\mu_R}{\sigma_R} \\ \frac{G-\mu_G}{\sigma_G} \\ \frac{B-\mu_B}{\sigma_B} \end{pmatrix} \tag{3.8}$$

with μ_{C} (i.e., the mean) and σ_{C} (i.e., the standard deviation) of the distribution in channel C being computed over the area under consideration (e.g., a patch or image). This yields, for every channel, a distribution where $\mu = 0$ and $\sigma = 1$.

3.3 Representation

Recent studies indicate that, in some biological systems, up to 80% of the brain's visually related activity is devoted to maintaining its learned representation of the world (Fiser et al. 2004). This biological analogue strongly suggests that maintaining a world representation is perhaps the most taxing part of the navigation process of robotic systems. To guide the robot's behavior using uncertain sensor inputs, there are three main different widely used representations, namely, appearance, grids, and features (Jefferies and Yeap 2008).

Appearance designates the practically raw data provided by sensors as sets of points with statistical properties, which require minimal processing but are almost impossible to use individually. Grid representations capture the presence or probability of presence of objects in space areas organized as preset grid cells. Features are structures of the environment that require some processing to build such as

pixels with invariance properties. Appearance representations are noisy and difficult to be recognized from one perception to the next, and therefore are rather often used globally. Grid representations are easy to construct but require that grid cells be recognized as such for updating. Feature representations have the advantage that they have some stability that makes them easier to recognize from a perception to another.

An autonomous mobile robot must be able to determine its relationship to the environment by making measurements with its sensors and then using those measured signals. But every sensor is imperfect since measurements always have error and, therefore, uncertainty is associated with them. Therefore, sensor inputs must be used in a way that enables the robot to interact with its environment successfully in spite of measurement uncertainty. Perception allows making an internal representation (model) of the environment, which is then used for moving, avoiding collision, and finding its position and its way to the target. Without a sufficient environment perception, the robot simply cannot make any secure movement. The more unstructured an environment is, the more dependent the robot is on its sensorial system. The representation problem for autonomous mobile robots has been addressed at length in relatively structured internal, and some external, environments.

As the environment structure degree decreases and the environment becomes more complex, however, robot capacity gets limited and fails to conform to such simple parametric models, and these modeling methods often break-down in a fragile manner. This occurs in more dynamic indoor environments and, most importantly, in outdoor environments. In such cases, generalized non-parametric models become essential. Thus, a practical representation should display two simultaneous characteristics, that is, it must provide a reasonable level of compactness and it must also support efficient manipulation.

For map-based navigation, the robot needs to know a geometrical model (i.e., the position and metrics of physical objects) to plan its motions in a given space region and the relationships between regions of space to decide for the general roadmap it will follow. Hence both a geometrical and a topological model are useful and complementary. Although recent work on mapping has rather mostly focused on trying to capture geometry and topology, being an alternate, semantics, which defines the nature of objects or of space regions to manipulate on, would be an important knowledge representation. In the following, we will present an overview of feature extraction based environment representations and place recognition from digital images.

3.3.1 Feature Extraction

A robot, like a human or an animal, does not need to know its precise position with respect to the environment when traveling. Given this, instead of using raw sensor input, it is comparatively easier to extract information from one or more sensor readings first, generating a higher-level percept that can then be used to inform the robot's model and perhaps the robot's actions directly. This process is called feature



Fig. 3.5 The perceptual pipeline from sensor readings to knowledge models

extraction, and it is a step in the perceptual interpretation pipeline as illustrated in Fig. 3.5. In practical terms, mobile robots use feature extraction and scene interpretation to maintain topological global consistence instead of metric for its navigation activity.

For our navigation task, the vision-sensor measurements may pass through the complete perceptual pipeline, being subjected to feature extraction followed by scene interpretation, to minimize the impact of individual sensor uncertainty on the robustness of the robot's navigation skills. The pattern that thus emerges is that, as one moves on into more sophisticated long-term perceptual tasks, the feature-extraction and scene-interpretation aspects of the perceptual pipeline become essential.

3.3.1.1 Feature Definition

Images contain much more than implicit depth information and color blobs. Visual interpretation is an extremely challenging problem to fully solve. Significant research effort has been dedicated over the past several decades to inventing algorithms for understanding a scene based on 2D images, and the research efforts have slowly produced fruitful results.

Features are recognizable structures of elements in an environment. They can usually be extracted from measurements and mathematically described. Good features are always perceivable and easily detectable from the environment. There are two kinds of features, low-level features such as intensity, color, texture, shape, etc., and high-level features (objects) such as doors, tables, or trash cans, and so on. At one extreme, raw sensor data provide a large volume of data and a high conservation of information, but with low distinctiveness of each individual quantum of data. Vision interpretation is primarily about the challenge of reducing information. A CCD camera can output 240 million bits per second, producing too much information, and this over-abundance of information mixes together relevant and irrelevant information haphazardly. At the other extreme, high-level features provide maximum abstraction from the raw data by filtering out poor or useless data, thereby reducing the volume of data as much as possible while providing highly distinctive resulting features, although the abstraction process has the risk of filtering away important information, potentially lowering data utilization. Having some spatial locality, the geometric extent of features can range widely.

Therefore, enabling more compact and robust descriptions of the environment so as to help a mobile robot during localization, features play an especially important role in the creation of environmental models, and a critical decision revolves around choosing the appropriate features for the robot to use. Since the problem of visual feature extraction is largely one of removing the majority of irrelevant information in an image so that the remaining information unambiguously describes specific features in the environment, a number of factors are essential to this decision, such as what features to select for a target environment, whether visual feature extraction can affect a significant computational cost, and whether the features extracted provide information that is consonant with the representation used for the environmental model since feature extraction is an important step toward scene interpretation, among which two key requirements must be met for a visual feature extraction technique to have mobile robotic relevance. First, the method must operate in real time. Mobile robots move through their environment, and so the processing simply cannot be an offline operation. Second, the method must be robust to the real-world conditions outside of a laboratory. This means that carefully controlled illumination assumptions and carefully painted objects are unacceptable requirements.

3.3.1.2 Image Feature Extraction

A local feature is an image pattern that differs from its immediate neighborhood in terms of intensity, color, and texture. Local features can be small image patches (such as regions of uniform color), edges, or points (such as corners originated from line intersections). In the modern terminology, local features are also called interest points, interest regions, or keypoints.

Depending on their semantic content, local features can be divided into three different categories. In the first category, local features are those that have a semantic interpretation such as, for instance, edges corresponding to lanes of the road. In the second category, local features are those that do not have a semantic interpretation. Here, what the features actually represent is not relevant. What matters is that their location can be determined accurately and robustly over time. Finally, in the third category, local features are those that still do not have a semantic interpretation if taken individually, but that can be used to recognize a scene or an object if taken all together. For instance, a scene could be recognized counting the number of feature matches between the observed scene and the query image. This principle is the basis of the visual-word-based place recognition described in Sect. 3.3.2.

For the purpose to generate a model of an environment, more general solution to the problem would be of extracting a large number of feature types from images. Suppose several shots of the scenes in an environment are simply taken with little overlap between adjacent pictures. One way to solve this problem is to extract feature points from adjacent pictures, and some machine learning techniques are used to automatically fuse them all together into an environment representation. To find corresponding pairs according to some similarity measure, and compute the transformation to align them, the key challenge is to identify corresponding regions

between overlapping images by good feature detectors. Important properties of an ideal good feature detector include repeatability, distinctiveness, localization accuracy, quantity of features, invariance, computational efficiency and robustness, which are summarized in the following.

- To detect the same points independently in two images, "repeatability" is probably the most important property of a good feature detector.
- Given two images of the same scene taken under different viewing and illumination conditions, for each point in the first image, to correctly recognize the corresponding one in the second image, the detected features should be of high distinctiveness.
- The detected features should be accurately localized, both in image position and scale, that is, should be of high localization accuracy.
- Quantity of features depends on the application. For object recognition, a sufficiently large number of low-level features is preferred to increase the recognition rate. For semantic interpretation, a small number of high-level features would be enough to recognize a scene.
- Good features should be invariant to changes of camera viewpoint, environment illumination, and scale (like zoom or camera translation).
- It is desirable that features can be detected and matched with computational efficiency. This is important in robotics, where most of the applications need to work in real-time.
- The detected features should be robust to image noise, discretization effects, compression artifacts, blur, deviations from the mathematical model used to obtain invariance, and so on.

3.3.1.3 SIFT Features

The computer vision literature in this field of the local feature detectors is very large, and a comprehensive survey on local feature detectors was provided by Tuytelaars and Mikolajczyk (2007). In the following, only the one most popular feature detector, namely SIFT, is briefly described. Being the simplest structures with invariance properties, SIFT stands for Scale Invariant Feature Transform and is a method to detect and match robust keypoints, which was invented in 1999 by Lowe (1999, 2004). The uniqueness of SIFT is that these features are extremely distinctive and can be successfully matched between images with very different illumination, rotation, viewpoint, and scale changes. Its high repeatability and high matching rate in very challenging conditions have made SIFT the best feature detector so far. It has found many applications in object recognition, robotic mapping and navigation, video tracking, and other related areas. The main advantage of the SIFT features in comparison to all previously explained methods is that a "descriptor" is computed from the region around the interest point, which distinctively describes the information carried by the feature. This descriptor is a vector that represents the local distribution of the image gradients around the interest point. As proven by its inventor, it is actually this descriptor that makes SIFT robust to rotation and small changes of illumination, scale, and viewpoint.

To recognize any object in an image, keypoints on the object can be extracted from a training image to provide a "feature description" of the object, which can then be used to identify the object when attempting to locate it in a test image containing that object. Therefore, Lowe's method for image feature generation transforms an image into a large collection of feature vectors, each of which is invariant to image translation, scaling, and rotation, partially invariant to illumination changes, and robust to local geometric distortion.

For content-based image retrieval, keypoints between the input image and the images in the databases are matched by identifying their nearest neighbors. However, for modern large databases, these methods do not scale well with the size of the image databases, and cannot select a small number of matching images out of the databases in acceptable time. To speed up, Lowe used a modification of the *K-d* tree algorithm called the Best-bin-first (BBF) search method (Lazebnik et al. 2004) that can identify the nearest neighbors with high probability using only a limited amount of computation. However, due to the "curse of dimensionality" problem, its search efficiency could degenerate to that of a sequential search in high-dimensional space.

3.3.2 Place Recognition

Location recognition (or place recognition) describes the capability of naming discrete places in the world. A requirement is that it is possible to obtain a discrete partitioning of the environment into places and a representation of the place, and that the places with the corresponding representations are stored in a database. The location recognition process then works by computing a representation from the current sensor measurements of the robot and searching the database for the most similar representation stored. The retrieved representation can then determine the location of the robot. Location recognition is the natural form of robot localization in a topological environment map as described by many authors (Cummins and Newman 2008; Fraundorfer et al. 2007; Goedeme et al. 2004; Matsumoto et al. 1996; Meng and Kak 1993; Ulrich and Nourbakhsh 2000).

Visual sensors (i.e., cameras) are perfectly suited to create a rich representation that is both descriptive and discriminative. Most visual representations proposed so far can be divided into global representations and local representations. Global representations use the whole camera image as a representation of the place. However, whole-image features are not designed to identify specific spatial structures such as obstacles or the position of specific landmarks. Rather, they serve as compact representations of the entire local region. Local representations instead identify salient regions of the image first and create the representation out of this only. This approach largely depends on the detection of salient regions using interest point or interest region detectors. With the development of many effective interest point detectors, local methods have proven to be practical, are nowadays applied in many systems and, therefore, are the preferred way to location recognition.

3.3.2.1 From Bag of Features to Visual Words

A representation of an image by a set of interest points only is usually called a bag of features. For each interest point, a descriptor that is invariant to rotation, scale, intensity, and viewpoint change, such as SIFT, is usually computed. This set of descriptors is the new representation of the image. It is called a bag of features because the original spatial relation between the interest points is removed and only the descriptors are remembered. The similarity between two sets of descriptors can be computed by counting the number of common feature descriptors. For this, a matching function needs to be defined, which allows us to determine whether two feature descriptors are the same. This matching function usually depends on the type of feature descriptor. But in general a feature descriptor is a high-dimensional vector, and matching features can be found by computing the distance using the L_2 norm.

Visual words are a 1-dimensional representation of the high-dimensional feature descriptor. This means that the visual word for a 128-dimensional SIFT descriptor is just a single integer number. The conversion to visual words creates a bag of visual words instead of a bag of features. For this conversion, the high-dimensional descriptor space is divided into nonoverlapping cells. This division is computed by K-means clustering (Duda et al. 2001). For the clustering, a large number of feature descriptors are necessary. The computed cluster borders form the cell divisions of the feature space. Each of the cells is now assigned a number that will be assigned to any feature descriptor within the cell. This number is referred to as a visual word. Similar feature descriptors will then be sorted into the same cell and therefore get the same visual word assigned. This is a very efficient method of finding matching-feature descriptors. The visual words created by the partitioning are called visual vocabulary. For quantization, a prototype vector for each cell is stored, which is the mean descriptor vector of all training descriptors from the cell. To assign a feature descriptor to its cell, it needs to be compared to all prototype vectors. For a large number of cells, this can be a very expensive operation. It can speed up by creating a hierarchical splitting of the feature space called vocabulary tree (Nistér and Stewénius 2006).

Feature quantization into visual words is one key ingredient for efficient location recognition. However, the set of visual words does not contain the spatial relations anymore, and thus an image that has the same visual words but in a different spatial arrangement would also have high similarity. The spatial relations can be enforced again by a final geometric verification, which can return the desired location in place recognition.

3.3.3 Image Segmentation

One of the early and most successful approaches to place recognition before the advent of the visual words based methods described above is the use of image histograms. A single visual image provides so much information regarding a robot's

immediate surroundings that an alternative to searching the image for spatially localized features is to make use of the information captured by the entire image (i.e., all the image pixels) to extract a whole-image feature or global image feature. Mobile robots have used whole-image histogram features to identify their position in real time against a database of previously recorded images of locations in their environment. Using this whole-image extraction approach, a robot can readily recover the particular place or the particular object in which it is located (Ulrich and Nourbakhsh 2000).

However, from the perspective of robot localization, the goal is to extract one or more features from the images that are correlated well with the robot's position. The identification of specific spatial structures such as obstacles or specific landmarks can be realized by image segmentation techniques that divide an image into its regions or objects by assigning a label to every pixel in an image such that pixels with the same label share certain characteristics in common. In computer vision, image segmentation is the process of partitioning an input image into homogeneous and connected characteristic regions that are more meaningful and easier to analyze according to color, edges, or some other criteria and preparing the content of an image for subsequent "higher-level" specialized operations, such as object detection or recognition. The result of image segmentation is a set of segments that collectively cover the entire image.

From a mathematical point of view, for an image I, the segmentation operation formalism states that the image is decomposed into a number N_R of regions R_i, with $i = 1 \ldots N_R$, which are disjoint nonempty sections of I. Regions are connected sets of pixel locations that exhibit some similarity in the pixel values which can be defined in various ways. The segmentation of an image I into regions is called complete, if the regions exhibit the properties listed in the following,

- The union of all regions should give the entire image, or in other words, all the pixels should belong to a region at the end of segmentation.
- The regions should not overlap.
- Each segment R_i is a connected component or compact, i.e., the pixel locations in a region R_i are connected.
- A certain criterion of uniformity is satisfied for each segment, i.e., pixels belonging to the same region have similar properties.
- The uniformity criterion for any two segments is not satisfied, i.e., pixels belonging to different regions should exhibit different properties.

The result of segmentation is a set of regions $\{R_i\}$, $i \in \{1, \ldots, N_R\}$ which can be represented in several ways. The simple solution used frequently is to create a so-called region label image (I_R) which is a feature image where each location contains the index of the region that this location is assigned to, i.e., $I_R \rightarrow \{1, \ldots, N_R\}$.

Haralick and Shapiro state listed some of the guidelines for achieving a good segmentation: (1) regions of an image segmentation should be uniform and homogeneous with respect to some characteristic such as gray tone or texture; (2) region interiors should be simple and without many small holes; (3) adjacent regions of

a segmentation should have significantly different values with respect to the characteristic on which they are uniform; and (4) boundaries of each segment should be simple, not ragged, and must be spatially accurate (Haralick and Shapiro 1991). Partially, these guidelines are met by the formal properties mentioned above. There are usually two distinguished cases, oversegmentation and undersegmentation. The oversegmentation means that the number of regions is larger than the number of objects in the images, or it is simply larger than desired. This case is usually preferred because it can be fixed by a post-processing stage called region merging. The undersegmentation is the opposite case and usually less satisfying.

Among many existing methods of color image segmentation, four main categories can be distinguished, that is, pixel-based techniques, region-based techniques, contour-based techniques, and hybrid techniques. In pixel-based techniques, clustering in three-dimensional color space on the basis of color similarity is one of the popular approaches in the field of color image segmentation. Being the process of partitioning a set of objects (pattern vectors) into subsets of similar objects called clusters, clustering is often seen as an unsupervised classification of pixels. Generally, a priori knowledge of the image is not used during the clustering process. Many different clustering techniques, proposed in the pattern recognition literature, can be applied to color image segmentation (Singh and Singh 2010; Ladicky et al. 2009). In region-based techniques, pixels are grouped into homogeneous regions. This family of techniques includes region growing, region splitting, region merging, and thresholding. Region growing techniques start with a pixel and go on by adding the pixels based on similarity to the region, until all pixels belong to some region (Ning et al. 2010). In region splitting techniques, initially the whole image is taken as a single region which is repeatedly split until no more splits are possible, and then two regions are merged if they are adjacent and similar until no more merging is possible (Sharma et al. 2012; Dass and Priyanka 2012). Thresholding is a method used to separate the foreground or object from the background and into nonoverlapping sets (Gonzalez and Woods 1992). Gray–level images are converted to binary images by selecting a single threshold value to classify the pixels so that the binary image should contain information about the position and shape of the foreground objects. Being the sign of lack of continuity and ending, edges are local changes in the image intensity and occur on the boundary between two regions. Based on the idea that objects are separated by edges, edge-based or contour-based techniques transform images into edge images for image segmentation using the changes of grey tones in the images (Senthilkumaran and Rajesh 2009).

3.4 Reasoning

Reasoning, like information acquisition and environmental representation, may be thought of as being application specific and is intimately linked to the choice of sensors and representations. Being considered in toto with information acquisition and environmental representation to solve real-world problems such as robot navigation

tasks, in the current discussion, reasoning is defined as the transformation of the sensory stimuli to another more abstract representation to aid the decision making process. Referred to as "Data Condensation", this "transformation" should have the following properties:

- It must be able to incorporate prior knowledge, should it exist.
- The transformation should be considered *optimal* if it retains only the information needed for a particular task, and no other.

These conditions refer to the ability of a reasoning algorithm or process to highlight the aspects of the data that are important to a particular task. The criteria of optimality can only be measured with respect to the task being performed in the sense that no task-specific information is lost during the transformation process. The "prior" knowledge needed to complete a particular task could therefore, in principle, be learnt by posing it as an optimization problem which maximizes task performance.

Navigation is an important ability of mobile robots. Localization in an environment is the very first step to achieve it. In this book, based on the extensive research already conducted for mobile robot localization problem in an indoor known environment, a natural landmark-based localization strategy is designed for a mobile robot working in an outdoor unknown environment. Particularly, the goal is to design a real-time scene recognition scheme so as to use objects segmented in it as the natural landmarks and to explore the suitability of configural representation for automatic scene recognition in robot localization by conducting experiments designed to infer semantic prediction of a scene from different configurations of its stimuli using a machine learning paradigm named reinforcement learning. In this work, the goal is to bring innovative machine learning based solutions to challenging mobile robotic localization and navigation problems by approaching the problems from computational viewpoints.

3.5 A Vision Based Machine Perceptual Learning System

Much work has and continues to be done toward making robots to think, to behave, and to act like human beings. The current research focuses on theoretical development of machine learning algorithms for image, videos and multimodal data to build recognition systems for mobile robot navigation. Autonomous robots are intelligent machines capable of performing tasks in the real world without explicit human control for extended periods of time. A high degree of autonomy is particularly desirable in fields where robots can replace human workers, such as state-of-the-practice video surveillance system and space exploration. However, not having human's sophisticated sensing and control system, two broad open problems in autonomous robot systems are the perceptual discrepancy problem, that is, there is no guarantee that the robot sensing system can recognize or detect objects defined by a human designer, and the autonomous control problem, that is, how the robots can operate in unstructured

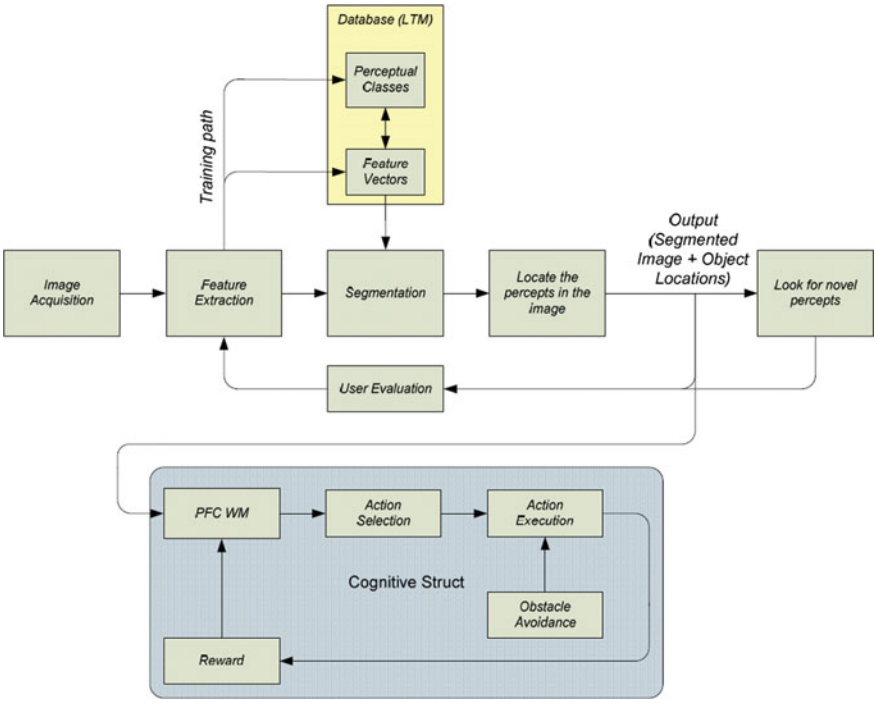


Fig. 3.6 The proposed vision-based machine perceptual learning system

environments without continuous human guidance. As a result, autonomous robot systems should have their own ways to acquire percepts and control by learning.

The proposed model is shown in Fig. 3.6. The model has three significant characteristics based on three most extensively used types of machine learning paradigms, namely, unsupervised learning, supervised learning and reinforcement learning.

3.5.1 Unsupervised Learning for Percept Acquisition

To enable unambiguous reconstruction of an environment model from image data, image segmentation evolved in the last two decades from the initial exploratory approaches mostly in the pixel value space to feature space-based techniques where a computation of a local feature descriptor is conducted and can enforce the spatial relations to return the desired location in place recognition.

To greatly facilitate the image content analysis and interpretation, it is desired that each resulting region or segment represents an object in the original image. Different feature types can provide quantitatively different information for mobile robot localization tasks. Colors, dominated in the image, create dense clusters in the color space in a natural way. For each segment to be semantically meaningful, the

widely used unsupervised learning (also called clustering) techniques are nowadays performed on the feature descriptors extracted from images in order to detect objects in the color space.

In order to associate the segments to terms describing the content of the image, like annotations, or, in other words to map the pixelic content to the semantic image description, currently, the most competitive approaches for image segmentation are formulated as clustering models for probabilistic grouping of distributional feature vectors. In the following, an image segmentation based environmental modeling is presented for the current implementation.

For image segmentation and object identification, instead of using the color information of a single pixel, the color pixels in a small local region of an image are considered. As illustrated in Fig. 3.7, to obtain feature vectors, for a given image, a moving window of size $N \times N$ hops by M pixels in the row and column directions but not to exceed the border of the image. The moving windows are overlapping to allow a certain amount of fuzziness to be incorporated so as to obtain a better segmentation performance. The window size controls the spatial locality of the result and the window hopping step controls the resolution of the result. A decrease in the step gives rise to an increased resolution but an increased processing time.

To extract color features, a histogram of color measurements in the color space chosen is computed as follows. The color channels are broken into different number of bins of equal width, say u, v, w, respectively. Each color can be represented by combining the three bins, one from the u bins, one from the v bins and one from the

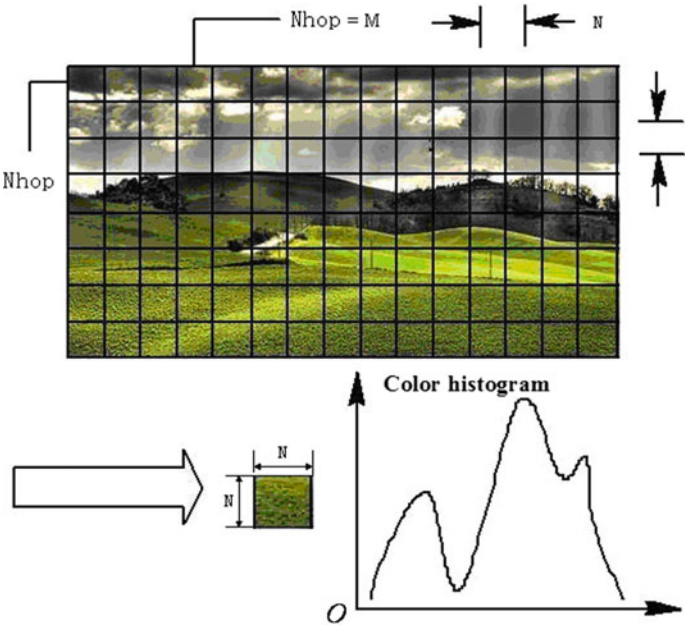


Fig. 3.7 Image segmentation model for environment representation

w bins. All possibilities of the combinations equal $u \cdot v \cdot w$ different feature color bins for the histogram. The histogram can then be constructed for an image patch by looking at each color feature and finding the number of pixels in the patch that corresponds to that feature. After doing this for all the color features in the object, there are $u \cdot v \cdot w$ numbers, each representing the number of pixels of a certain color in the selection patch. The total number of pixels in the selection patch divides these $u \cdot v \cdot w$ numbers, resulting in a feature vector of a dimension as high as $u \cdot v \cdot w$. By this way, the spatial content organization of image objects has been maintained.

Based on the feature vectors extracted from image patches, image segmentation can be realized by partitioning the set of image patches into a number of disjoint clusters or segments to form perceptual groups, and the object identification task reduces to partitioning the feature space. Each feature appears as a point in the feature space and patterns pertaining to different classes will fall into different regions in the feature space. Unsupervised learning is this process of classifying a pattern into the right category. In unsupervised learning, there exists no a priori knowledge of categories into which the patterns are to be classified, nor do we know how many classes there are within the input patterns. The input patterns group themselves by natural association based on some properties in common. It is expected that the degree of natural association is high among members belonging to the same category and low among members belonging to different categories according to some similarity measures. As a result, patterns belonging to the same cluster should be very close together in the pattern space, while patterns in different clusters should be further apart from one another. Then the segmented images consist of object labels, which results in a much size-reduced learned version of the original ones that can be stored separately and used as indexing files.

To build a perceptual model for an environment, usually, the pattern recognition system will first select a small number of training images from an environment and partition them into small overlapping image patches, on which feature vectors in the form of color histograms are exacted and subsequently clustered to obtain a knowledge base consisting of visual vocabulary words.

3.5.2 Supervised Learning for Object Recognition

If each input pattern is given along with the category to which this particular pattern belongs, the learning process to classify new patterns that have not been seen before but belong to the same population of existing labeled patterns is called supervised learning. Therefore, with a set of labeled feature vectors, the next step is to classify new observations using supervised learning. Real-time processing of the information coming from video demands for efficient search structures and algorithms. For modern large databases, traditional nearest neighbor based classifiers are of low efficiency and, therefore, unable to meet the requirements of efficient search. Design of searching methods that scale well with the size of the database and the dimensionality of the data is a challenging task. Trees present an efficient way to index

local image regions, making the nearest neighbor search more efficient by pruning. Randomized *Kd*-trees have been used with significant success in object recognition. However, for the unbalanced nature of our high-dimensional highly sparse data, the forced creation of empty or nearly empty leaves and nodes reduces its performance significantly. Randomized decision trees can offer logarithmic-time coding, however, they use one or a few attributes in the computation of splitting criterion and each path through the tree typically accesses only a few of the feature dimensions. Unlike decision trees, the Vocabulary tree uses all the attributes but is slow for very large databases. For fast image retrieval, the Scalable Vocabulary Tree (SVT) constructed by the repeated hierarchical *K*-means clustering is a way of both improved retrieval accuracy and better retrieval efficiency but remains hard to accelerate in high dimensional descriptor space due to the "curse of dimensionality" problem of high-dimensional space.

To partially circumvent this problem, through studying the vocabulary tree, we propose a *k*-way random quantization tree (QT) method for fast approximate nearest neighbor search which follows the notion of Vocabulary tree but with important variations. Given the obtained database, at the first level, there is only one node, the tree root. At the second level, a set of k representative patterns is randomly selected from the whole database as the cluster centers. Then the whole database is clustered into k subsets by assigning each feature vector to its closest center according to the chosen similarity measure. At the third level, for each of the k clusters obtained at the second level, k feature vectors are selected randomly from its pool of feature vectors as its next level cluster centers, resulting in k^2 cluster tree nodes at this level. This procedure continues until either all the feature vectors in a leaf node (i.e., the tree node that has no child nodes) belong to the same object class (a pure node) or the number of the feature vectors in a leaf node is below some limit, e.g., 50. Every feature vector has a class label associated with it. Given a new feature vector, to search through the tree, its distances to k cluster centers at each level along a certain branch are calculated and the winner is the center among the k's that the new feature vector is nearest to. When a leaf node is reached, if it is pure, assign the associated label and then stop. Otherwise, do a nearest neighbor search over the vectors in the associated cluster, and the winner is the feature vector which gives minimum distance according to the chosen metric.

3.5.3 Reinforcement Learning for Autonomous Control

With the development of computer vision, robots can detect target objects from image sequences for autonomous navigation. To identify targets, the perceptual system of autonomous robots first needs to segment the images into nonoverlapping but meaningful regions based on low-level features such as color, texture measures, and shapes, etc. However, not having a human's sophisticated control system, the autonomous robot systems should have their own ways to acquire control by some kind of learning.

Perception in animals is strongly related to the type of behavior they perform. Learning plays a major part in this process. For higher animals such as humans, evidence suggests that the occurrence of a response depends on its predicted outcome and behaviors are planned on the basis of future possibilities rather than present contingencies alone. If the predicted outcome is a reward, the intended response in the form of a behavior is facilitated. However, if an inverse outcome is predicted, the response is strongly inhibited. Based on the same principle, the robot learning problem addresses the question of making a robot perform certain tasks in the world successfully. If a task can be defined in term of a set of such reward receiving goals, a qualitative measure of robot performance can be the sum of the rewards it receives over time and the robot learning problem is then to improve robot performance through experience. This kind of learning is called reinforcement learning (RL) in machine learning community. Specifically, the learning algorithm we focus on is the temporal difference learning algorithm TD(λ), and the form of knowledge representation is a neural network. There is biological evidence that the adaptive working memory (WM) structures, existing in primate brains, are important to the learning and performing of tasks by providing the embodiment necessary for accumulating rewards for those features most relevant to the current task. Working memory system is a psychological model of the human short-term memory that accounts for a limited capacity system for temporarily storing and manipulating information used to control an ongoing behavior. To solve how the robots can learn to autonomously control their behavior based on percepts they've acquired, the computer vision system is integrated with a learning paradigm, WMtk, for decision making (Fig. 3.8).

The mechanisms by which humans use visually-acquired landmarks to find their way around have proved fascinating. Considerable evidence suggests that animals navigate not only on the basis of the overall geometry of the space but also on the basis of a configural representation of the cues. In contrast to earlier linear models of elemental feature representation, configural representation requires individual stimulus be represented in the context of other stimuli and is typified by nonlinear learning tasks such as the transverse patterning problem. For automatic scene recognition in robot navigation, a landmark learning methodology is developed to infer semantic prediction of a scene from different configurations of its stimuli with the aid of a reinforcement learning which allows reward associations between a target location and the conjunctive representations of its stimuli. Although the reinforcement learning has been applied to real-world problems such as robot control (Smart and Kaelbling 2002), the use of the temporal difference learning for studying mobile robot location learning is a relatively novel approach the success of which has been demonstrated (Wang et al. 2009).

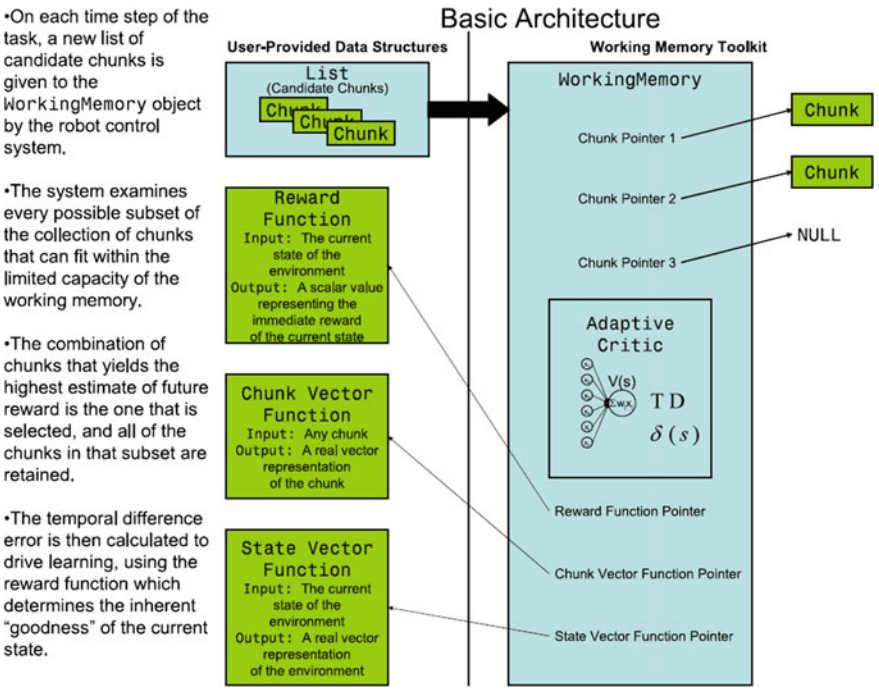


Fig. 3.8 NSF ITR Robot-PFC Working Memory Toolkit (WMtk)

3.6 Conclusions

This chapter has presented a discussion of the various aspects of "a perceptual learning system" from a practical system designers' viewpoint. The task of perception is broken into three main but interrelated parts, those of information gathering, representation and reasoning. It is argued that reasoning is the process of taking abstract sensory data and transforming it (in combination with any prior information) into a more abstract representation that contains only the information relevant to a particular task. Further, it is argued that, in order for this to work effectively, percepts relevant to the task must be sensed and then stored in a representation which may be manipulated efficiently and is compact. Several representations are examined for use in this task. An optimal implementation would be one in which the sensors used are uniquely suited to the task being performed, the representation is computationally efficient and compact, and the reasoning process discards only the information not relevant to the current task. Finally, an architecture and an implementation are presented to illustrate the concepts as applied to an outdoor mobile robotic navigation system.

References

Cummins, M., & Newman, P. (2008). FAB-MAP: Probabilistic localization and mapping in the space of appearance. *The International Journal of Robotics Research, 27*(6), 647–665.

Dartnall, H. J. A., Bowmaker, J. K., & Mollon, J. D. (1983). Human visual pigments: Microspectrophotometric results from the eyes of seven persons. *Proceedings of the Royal Society of London. Series B, 220,* 115–130.

Dass, R., & Priyanka, S. D. (2012). Image segmentation techniques. *The International Journal of Electronics and Communication Technology, 3*(1).

Duda, R. O., Hart, P. E., & Stork, D. G. (2001). *Pattern classification*. New York: Wiley.

Faugeras, O. (1993). *Three-dimensional computer vision: A geometric viewpoint*. Cambridge: The MIT Press.

Fiser, J., Chiu, C., & Weliky, M. (2004). Small modulation of ongoing cortical dynamics by sensory input during natural vision. *Nature, 431,* 573–578.

Fraundorfer, F., Engels, C., & Nister, D. (2007). Topological mapping, localization and navigation using image collections. In *Proceedings of 2007 IEEE/RSJ Conference on Intelligent Robots and Systems* (pp. 3872–3877), San Diego, CA.

Goedeme, T., Nuttin, M., Tuytelaars, T., & Van Gool, L. (2004). Markerless computer vision based localization using automatically generated topological maps. In *Proceedings of the European Navigation Conference GNSS*, *Rotterdam*.

Gonzalez, R. C., & Woods, R. E. (1992). *Digital image processing*. Reading, Mass, USA: Addison Wesley.

Gonzalez, R. C., & Woods, R. E. (2008). *Digital image processing* (3rd ed.). New York: Pearson Prentice Hall.

Haralick, R. M., & Shapiro, L. G. (1991). *Computer and Robot Vision 1*. Reading: Addison-Wesley.

Hartley, R. I., & Zisserman, A. (2004). *Multiple view geometry*. Cambridge, UK: Cambridge University Press.

Jefferies, M. E., & Yeap, W.-K. (Eds.). (2008). Robotics and cognitive approaches to spatial mapping in series. In *Springer tracts in advanced robotics*. Heidelberg: Springer.

Ladicky, L., Russell, C., Philip H. S., & Kohli, P. (2009). Associative hierarchical CRFs for object class image segmentation. In *Proceedings of the 2009 IEEE 12th International Conference on Computer Vision* (*ICCV*'09) (pp. 739–746), Kyoto, Japan.

Lazebnik, S., Schmid, C., & Ponce, J. (2004). Semi-local affine parts for object recognition. In *Proceedings of the British Machine Vision Conference* (Vol. 2, pp. 779–788).

Lowe, D. G. (1999). Object recognition from local scale-invariant features. In *Proceedings of the 7th IEEE International Conference on Computer Vision* (ICCV'99) (Vol. 2, pp. 1150–1157), Kerkyra, Greece.

Lowe, D. G. (2004). Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision, 60*(2), 91–110.

Ma, Y., Soatto, S., Kosecka, J., & Sastry, S. (2003). *An invitation to 3-D vision: From images to geometric models*. New York: Springer.

Matsumoto, Y., Inaba, M., & Inoue, H. (1996). Visual navigation using view sequenced route representation. In *Proceedings of the IEEE International Conference on Robotics and Automation* (*ICRA*'96) (Vol. 1, pp. 83–88), Minneapolis, MN, USA.

Meng, M., & Kak, A. C. (1993). Mobile robot navigation using neural networks and non-metrical environmental models. *IEEE Control Systems Magazine, 13*(5), 30–39.

Mollon, J. D., & Bowmaker, J. K. (1992). The spatial arrangement of cones in the primate fovea. *Nature, 360,* 677–679.

Ning, J., Zhang, L., Zhang, D., & AndWu, C. (2010). Interactive image segmentation by maximal similarity based region merging. *Pattern Recognition, 43,* 445–456.

Nistér, D., & Stewénius, H. (2006). Scalable recognition with a vocabulary tree. In *Proceedings of the IEEE International Conference on Computer Vision and Pattern Recognition* (*CVPR*'06) (Vol. 2, pp. 2161–2168).

Senthilkumaran, N., & Rajesh, R. (2009). Edge detection techniques for image segmentation—A survey of soft computing approaches. *International Journal of Recent Trends in Engineering, 1*(2), 250–254.

Sharma, N., Mishra, M., & Shrivastava, M. (2012). Color image segmentation techniques and issues: an approach. *International Journal of Science and Technology Research, 1*(41), 9–12.

Singh, K. K., & Singh, A. (2010). A study of image segmentation algorithms for different types of images. *International Journal of Computer Science Issues, 7*(5).

Smart, W. D., & Kaelbling, L. P. (2002). Effective reinforcement learning for mobile robots. In *Proceedings of the IEEE International Conference on Robotics and Automation* (*ICRA*'02) (Vol. 4, pp. 3404–3410), May, Washington, D.C.

Szeliski, R. (2010). *Computer vision: Algorithms and applications*. New York: Springer.

Trucco, E., & Verri, A. (1998). *Introductory techniques for 3-D computer vision*. New York: Prentice Hall.

Tuytelaars, T., & Mikolajczyk, K. (2007). Local invariant feature detectors: A survey. *Foundations and Trends in Computer Graphics and Vision, 3*(3), 177–280.

Ulrich, I., & Nourbakhsh, I. (2000). Appearance-based place recognition for topological localization. In *Proceedings of the IEEE International Conference on Robotics and Automation* (*ICRA*'00) (pp. 1023–1029), April, San Francisco.

Wang, X., Tugcu, M., Hunter, J. E., & Wilkes, D. M. (2009). Exploration of configural representation in landmark learning using working memory toolkit. *Patterns Recognition Letters, 30*(1), 66–79.

Part II
Unsupervised Learning

Chapter 4
Unsupervised Learning for Data Clustering Based Image Segmentation



Abstract The purpose of this chapter is to introduce in a fairly concise manner the key ideas underlying the field of unsupervised learning from the perspective of clustering for image segmentation tasks. We begin with a briefly review of fundamental concepts in clustering and a quick tour of its four basic models, namely, partitioning-based, hierarchical, density-based, and graph-based approaches. This is followed by a short introduction to distance measures and a brief review on performance evaluation metrics of clustering algorithms. This introduction is necessarily incomplete given the enormous range of topics under the rubric of clustering. The hope is to provide a tutorial-level view of the field so that many topics covered here can be delved more deeply into and state-of-the-art research will be touched upon in the next four chapters.

Keywords Unsupervised learning · Clustering · Partitioning-based clustering · Hierarchical clustering · Density-based clustering · Graph-based clustering · Distance measures · Internal evaluation index · External evaluation index

4.1 Introduction

Machine learning is the field of research devoted to the formal study of learning systems. The art in machine learning is to develop models with certain desired properties which are appropriate for the data set being analyzed. Unsupervised learning is a sub-field of machine learning. Being a highly interdisciplinary field, it borrows and builds upon ideas from statistics, computer science, engineering, cognitive science, optimization theory and many other disciplines of science and mathematics.

In unsupervised learning, the machine simply receives sensory inputs, but obtains no target outputs from the environment. This input, which is often called the data, could correspond to less obviously sensory data such as an image on the retina or the pixels in a camera. It may seem somewhat mysterious to imagine what the machine could possibly learn given that it does not get any feedback from its environment. However, it is possible to develop of formal framework for unsupervised learning based on the notion that the machine's goal is to build representations of the input that can be used for decision making, predicting future inputs, efficiently communicating

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_4

the inputs to another machine, etc. In a sense, unsupervised learning can be thought of as finding patterns in the data above and beyond what would be considered pure unstructured noise.

Therefore, unsupervised learning can be viewed in terms of learning a probabilistic model of the data. In general, the true distribution of the data from an environment is unknown. However, even when the machine is given no supervision or reward, it may make sense for the machine to estimate a model, $P(x)$, that is learned from data collected from an environment and represents the probability distribution for given sensor reading inputs x. In simpler cases where the order in which the inputs arrive is irrelevant or unknown, the machine can build a model of the data which assumes that the data points are independently and identically drawn from some distribution $P(x)$. As a result, many unsupervised learning algorithms can be seen as finding maximum likelihood or ML parameter estimates. Therefore, the better the model built upon the data, the more effectively the environment can be represented. There is an important link between machine learning and statistics. In the following, probabilistic models that are defined in terms of some latent or hidden variables are briefly reviewed. These models can be used to do dimensionality reduction and clustering, the two cornerstones of unsupervised learning.

4.1.1 Factor Analysis

Let a dataset D consist of N d-dimensional real-valued vectors, $D = \{y_1, \ldots, y_N\}$. In factor analysis, the data is assumed to be generated from the following model,

$$y = \Lambda x + \varepsilon \tag{4.1}$$

where x is a k-dimensional zero-mean unit-variance multivariate Gaussian vector with elements corresponding to hidden (or latent) factors, Λ is a $d \times k$ matrix of parameters, known as the factor loading matrix, and ε is a d-dimensional zero-mean multivariate Gaussian noise vector with diagonal covariance matrix Ψ. Defining the parameters of the model to be $\theta = (\Psi, \Lambda)$, by integrating out the factors, one can readily derive that,

$$p(y|\theta) = \int p(x|\theta)p(y|x,\ \theta)\mathrm{d}x = N\big(0,\ \Lambda\Lambda^{\mathrm{T}} + \Psi\big) \tag{4.2}$$

where $N(\mu, \Sigma)$ refers to a multivariate Gaussian density with mean μ and covariance matrix Σ. For more details, refer to Roweis and Ghahramani (1999).

Factor analysis is an interesting model for several reasons. If the data is very high dimensional (d is large), then even a simple model like the full-covariance multivariate Gaussian will have too many parameters to reliably estimate or infer from the data. By choosing $k < d$, factor analysis makes it possible to model a Gaussian density for high dimensional data without requiring $O(d^2)$ parameters.

Moreover, given a new data point, one can compute the posterior over the hidden factors, $p(x|y, \theta)$. Since x is lower dimensional than y, this provides a low-dimensional representation of the data. Principal components analysis (PCA) is an important limiting case of factor analysis (FA) by assuming that the noise is isotropic (i.e., each element of ε has equal variance) and independent components analysis (ICA) extends factor analysis to the case where the factors are non-Gaussian.

Although the models we have just described are attractive because they are relatively simple to understand and learn, their simplicity is also a limitation since the intricacies of real-world data are unlikely to be well-captured by a simple statistical model. To seek learning in much more flexible models and equip us with many tools, data clustering as a primitive exploration with little or no prior knowledge plays an indispensable role for understanding various phenomena. Cluster analysis consists of research developed across a wide variety of communities.

4.1.2 Clustering

Cluster analysis was originated in anthropology by Driver and Kroeber (1932), introduced to psychology by Zubin (1938) and Tryon in 1939 (Bailey 1994; Tryon 1939), and famously used by Cattell for trait theory classification in personality psychology in 1943 (Cattell 1943). Within the context of machine learning, clusters correspond to hidden patterns in the data, and searching for clusters is typically an unsupervised learning activity. Simple and useful, clustering is an important part of a somewhat wider area of unsupervised learning, where the data to describe is not labeled. In most cases, this is where no prior information is given with regard to what the expected output is. The algorithm only has the data and it should do the best it can. In this case, clustering is performed to separate data into groups (clusters) that contain similar data points, while the dissimilarity between groups is as high as possible.

Cluster analysis can be achieved by various algorithms that differ significantly in their understanding of what constitutes a cluster and how to efficiently find them. The notion of a cluster, as found by different algorithms, varies significantly in its properties. A wide range of models have been developed, each one with its pros and cons regarding what type of data they deal with, time complexity, weaknesses, and so on. No single model is appropriate for all data sets. Understanding these "cluster models" is key to understanding the differences between the various algorithms. Therefore, the notion of a "cluster" cannot be precisely defined, different researchers employ different cluster models, and for each of these cluster models again different algorithms can be given, which is one of the reasons why over 100 clustering algorithms have been published and there is no objectively "correct" clustering algorithm (Estivill-Castro 2002).

Popular notions of clusters include groups with small distances between cluster members, dense areas of the data space, intervals or particular statistical distributions. Clustering can therefore be formulated as a multi-objective optimization problem. The appropriate clustering algorithm and parameter settings (including parameters

such as the distance function to use, a density threshold or the number of expected clusters) depend on the individual data set and intended use of the results.

The most appropriate clustering algorithm for a particular problem often needs to be chosen experimentally, unless there is a mathematical reason to prefer one cluster model over another. It should be noted that an algorithm that is designed for one kind of model will generally fail on a data set that contains a radically different kind of model. Cluster analysis as such is not an automatic task, but an iterative process of knowledge discovery or interactive multi-objective optimization that involves trial and failure. It is often necessary to modify data preprocessing and model parameters until the result achieves the desired properties.

Of course, ultimately, the machine should be able to decide on the appropriate model without any human intervention, but to achieve this in full generality requires significant advances in artificial intelligence. Typical cluster models include partitioning-based models, hierarchical clustering models, density-based models, and graph-based models. Partitioning-based models represent each cluster by a single mean vector. Hierarchical clustering builds models based on distance connectivity. Density models define clusters as connected dense regions in the data space. Graph-based models can be considered as a prototypical form of cluster. In the rest of this chapter, we will survey these clustering algorithms. Several tightly related topics, such as proximity measures and cluster validation indices, are also discussed.

This chapter is organized as follows. Partition based clustering algorithms are introduced in Sect. 4.2. Hierarchical clustering algorithms are introduced in Sect. 4.3. Section 4.4 discusses density-based clustering algorithms. Section 4.5 explains graph-based algorithms. Proximity measures are introduced in Sect. 4.6. Cluster validation indices are discussed in Sect. 4.7. The summary is presented in Sect. 4.8.

4.2 Partitioning-Based Clustering Algorithms

Relying directly on intuitive notions of distance (or similarity) to cluster data points, partitioning based clustering algorithms, also known as representative-based algorithms, are typically done with the use of a set of partitioning representatives. The partitioning representatives may either be created as a function of data points in a cluster (e.g., the mean) or may be selected from existing data points in a cluster. The discovery of high-quality clusters in a dataset is closely related with the discovery of a high-quality set of representatives. Once the representatives have been determined, a distance function can be used to assign the data points to their closest representatives.

Typically, it is assumed that the number of clusters, denoted by k, is specified by the user. Partitioning-based clustering is an optimization problem, that is, finding the k cluster centers and assigning the objects to their nearest cluster center such that the sum of the squared distances from the cluster centers are minimized. Consider a data set D containing N data points denoted by $x_1 \ldots x_N$ in d-dimensional space. The goal is to determine k representatives $o_1 \ldots o_k$ that minimize

Algorithm *GenericRepresentative*
Input (Database: $\mathcal{D}$, Number of Representatives: k)
begin
  Initialize representative set S;
  repeat
    Create clusters $(\mathcal{C}_1 \ldots \mathcal{C}_k)$ by assigning each point in $\mathcal{D}$ to closest representative in S using the distance function $Dist(\cdot, \cdot)$;
    Recreate set S by determining one representative o_j for each $\mathcal{C}_j$ that minimizes $O = \sum_{i=1}^{N} \min_j \left(dist(x_i, o_j)\right)$;
  until convergence;
  return $(\mathcal{C}_1 \ldots \mathcal{C}_k)$;
end

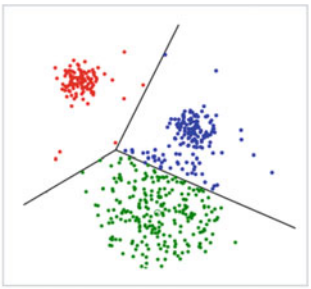


Fig. 4.1 (Left) Generic representation algorithm; (right) *K*-means algorithm separates data into Voronoi-cells, which assumes equal-sized clusters

the following objective function O,

$$O = \sum_{i=1}^{N} \min_j \left(\text{dist}\left(x_i,\ o_j\right)\right) \tag{4.3}$$

where $dist(x, y)$ denotes a distance measure between two data objects or simply $d(x, y)$. In other words, the sum of the distances of different data points to their closest representative needs to be minimized. The representatives and the optimal assignment of data points to representatives are unknown a priori. The generic framework for representative based algorithms with an unspecified distance function is illustrated in the pseudocode of Fig. 4.1.

The optimization problem itself is known to be NP-hard, and thus the common approach is to search only for approximate solutions. A particularly well-known approximate method is Lloyd's algorithm (Lloyd 1982), often referred to as *K*-means algorithm. It does however only find a local optimum, and is commonly run multiple times with different random initializations.

4.2.1 K-*Means Algorithm*

Being the most famous partitioning based clustering algorithm, *K*-means algorithm is an effective, widely used and all-around way to perform clustering. After setting the number of clusters, k, the algorithm begins by selecting k random points as starting centroids ('centers' of clusters). Then, the following two steps are repeated iteratively,

1. Assignment step. For each data point from the dataset, calculate its distances to every centroid, and simply assign it to the cluster with the least distance.
2. Update step. From the previous step, a set of k clusters are obtained. Now, for each cluster, a new centroid is calculated as the mean of all points in the cluster.

After each iteration, the centroids are slowly moving, and the total sum of the distances from each point to its assigned centroid becomes smaller and smaller. The two steps are alternated until convergence, that is, until the same set of points are assigned to each centroid, therefore leading to the same centroids again. *K*-means has a number of interesting theoretical properties. First of all, it is guaranteed to converge to a local optimum, which, however, may not necessarily be the best overall solution (global optimum). The final clustering result can depend on the selection of initial centroids. One simple solution is just to run *K*-means a couple of times with random initial assignments and then to select the best result by taking the one with the minimal sum of distances from each point to its cluster centroid—the error value that we are trying to minimize in the first place. Other approaches to selecting initial points can rely on selecting distant points, that is, points that are further away will have higher probability to be selected as starting centroids (Arthur and Vassilvitskii 2007; Bahmani et al. 2012). Further, most types of *K*-means algorithms require the number of clusters, *k*, to be specified in advance, which is considered to be one of the biggest drawbacks of these algorithms. Last but not least, it partitions the data space into a structure known as a Voronoi diagram. In other words, the algorithms prefer clusters of approximately similar size, as they will always assign an object to the nearest centroid. This often leads to incorrectly cut borders of clusters (which is not surprising since the algorithm optimizes cluster centers, not cluster borders).

Variations of *K*-means often include *K*-medoids clustering algorithms which promote such optimizations as restricting the centroids to be members of the data set, *K*-medians clustering algorithms which choose medians instead of the means, or fuzzy c-means algorithms which allow a fuzzy cluster assignment.

4.2.2 K-*Medoids Algorithm*

Since the representative of a *K*-means cluster may be distorted by outliers in that cluster and it is sometimes difficult to compute the optimal central representatives of a set of data points of a complex data type, *K*-medoids algorithms have been proposed to select the representatives always from the data points in the dataset, *D*, and this difference from *K*-means algorithms necessitates changes to the basic structure of the representative-based algorithms. As long as a representative object is selected from each cluster, the approach will provide reasonably high quality results. A key property of *K*-medoids algorithm is that it can be defined virtually on any data type, as long as an appropriate similarity or distance function can be defined on the data type. Therefore, *K*-medoids methods directly relate the problem of distance function design to clustering. The *K*-medoids approach uses a generic hill-climbing strategy, in which the representative set *S* is initialized to be a set of points from the original dataset *D*. Subsequently, this set *S* is iteratively improved by exchanging a single point from set *S* with a data point selected from the dataset *D*. Each exchange can be viewed as a hill-climbing step. Clearly, in order for the clustering algorithm to be

successful, the hill-climbing approach should at least improve the objective function of the problem to some extent.

4.3 Hierarchical Clustering Algorithms

When clusters are not necessarily of circular (or hyper-spherical) shapes and the number of clusters is not known in advance, hierarchical clustering (or linkage clustering) algorithms come in handy. Hierarchical clustering, also known as connectivity-based clustering, is based on the core idea of objects being more related to nearby objects than to objects farther away and can be classified into agglomerative algorithms which start with single elements and aggregate them into clusters and divisive algorithms which start with the complete dataset and divide it into partitions.

4.3.1 Agglomerative Algorithms

The agglomerative hierarchical clustering algorithms connect "objects" to form "clusters" based on their distance. A cluster can be described largely by the maximum distance needed to connect parts of the cluster. At different distances, different clusters will form, which can be represented using a dendrogram and explains where the common name "hierarchical clustering" comes from. These algorithms do not provide a single partitioning of the data set, but instead provide an extensive hierarchy of clusters that merge with each other at certain distances. In a dendrogram, the y-axis marks the distance at which the clusters merge, while the objects are placed along the x-axis such that the clusters do not mix. With hierarchical agglomerative clustering, we can easily decide the number of clusters afterwards by cutting the dendrogram (tree diagram) horizontally wherever we find suitable. It is also repeatable (i.e., it always gives the same answer for the same dataset), but is also of a higher complexity (quadratic). The generic framework for agglomerative hierarchical algorithms with an unspecified distance function is illustrated in the pseudocode of Fig. 4.2.

Being a whole family of clustering methods, hierarchical agglomerative algorithms start with each point being a separate cluster and work by joining two closest clusters in each step until everything is in one big cluster. Apart from the usual choice of distance functions, the user also needs to decide on the linkage criterion since a cluster consists of multiple objects and there are multiple candidates to compute the distance to use. Popular choices include single-linkage clustering (i.e., the minimum of object distances) (Sibson 1973), complete linkage clustering (i.e., the maximum of object distances) (Defays 1977), and UPGMA or WPGMA (Unweighted or Weighted Pair Group Method with Arithmetic Mean), also known as average linkage clustering (Sokal and Michener 1958; Sneath and Sokal 1973).

These methods will not produce a unique partitioning of the data set, but a hierarchy from which the user can choose appropriate clusters. They are not very robust

Algorithm *AgglomerativeMerge*
Input (Data: $\mathcal{D}$)
begin
  Initialize $n \times n$ distance matrix M using $\mathcal{D}$;
  repeat
    Pick closest pair of clusters i and j using M;
    Merge clusters i and j;
    Delete rows/columns i and j from M and create
      a new row and column for newly merged cluster;
    Update the entries of new row and column of M;
  until termination criterion;
  return current merged cluster set;
end

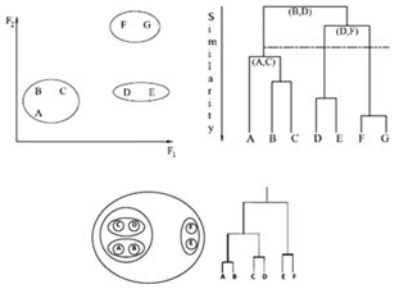


Fig. 4.2 (Left) Generic agglomerative merging algorithm with an unspecified merging criterion; (right) an illustration of a dendrogram (tree diagram) formation

towards outliers, which will either show up as additional clusters or even cause other clusters to merge (known as chaining phenomenon in particular with single-linkage clustering). In the general case, the time complexity is $O(N^3)$ for agglomerative clustering, which makes them too slow for large data sets. For some special cases, optimal efficient methods of complexity $O(N^2)$ have been developed for single-linkage algorithms (Sibson 1973) and complete-linkage algorithms (Defays 1977). In the data mining community, these methods are recognized as a theoretical foundation of cluster analysis and provide inspiration for many later methods such as density based clustering.

4.3.2 Divisive Algorithms

The top-down divisive hierarchical algorithm initializes the tree at the root node containing all the data points. In each iteration, the data set at a particular node of the current tree is split into multiple nodes (clusters). By changing the criterion for node selection, one can create trees balanced by height or trees balanced by the number of clusters. The overall approach for top-down clustering uses a general-purpose flat-clustering algorithm A as a subroutine. If the algorithm A is randomized, such as K-means algorithm (with random seeds), it is possible to use multiple trials of the same algorithm at a particular node and select the best one. The algorithm recursively splits nodes with a top-down approach until either a certain height of the tree is achieved or each node contains fewer than a predefined number of data objects. Unlike bottom-up agglomerative methods which are typically distance-based methods, top-down hierarchical methods can be viewed as general-purpose meta-algorithms that can use almost any clustering algorithm as a subroutine. In the general case, the time complexity of divisive clustering algorithms is $O(2^{N-1})$, which makes them too slow for large data sets (Everitt et al. 2011).

4.4 Density-Based Clustering

Density-based clustering algorithms group points that are closely packed together, expand clusters in any direction where there are nearby points, and thus can deal with different shapes of clusters (Kriegel et al. 2011). In density-based clustering, clusters are defined as areas of higher density than the remainder of the data set. Objects in the sparse areas (that are required to separate clusters) are usually considered to be noise and border points.

The most popular density-based clustering method is DBSCAN (Density-Based Spatial Clustering of Applications with Noise) (Ester et al. 1996). In contrast to many newer methods, it features a well-defined cluster model called density-reachability. Similar to linkage based clustering, it is based on connecting points within certain distance thresholds. However, it only connects points that satisfy a density criterion, which in the original variant is defined as a minimum number of other objects within a radius. A cluster consists of all density-connected objects (which can form a cluster of an arbitrary shape in contrast to many other methods) plus all objects that are within these objects' range.

4.4.1 Density-Based Clustering with DBSCAN

The core idea in density-based algorithms is to first identify fine-grained dense regions in the data. These form the building blocks for constructing the arbitrarily-shaped clusters. In DBSCAN, the density of a data point is defined by the number of points, denoted by τ, that lie within a radius, denoted by *Eps*, of that point (including the point itself). The densities of these spherical regions are used to classify data points into core, border, or noise points. These notions are defined as follows,

- Core point. A data point is defined as a core point, if it contains at least τ data points within a radius *Eps*.
- Border point. A data point is defined as a border point, if it contains less than τ points, but it also contains at least one core point within a radius *Eps*.
- Noise point. A data point that is neither a core point nor a border point is defined as a noise point.

Examples of core points, border points, and noise points are illustrated in right plot of Fig. 4.3 for $\tau = 10$. The data point A is a core point because it contains 10 data points within the illustrated radius *Eps*. On the other hand, data point B is a border point because it contains only 6 points within a radius of *Eps* and contains the core point A. The data point C is a noise point because it contains only 4 points within a radius of *Eps* but does not contain any core point.

After the core, border, and noise points have been determined, the DBSCAN clustering algorithm proceeds as follows. First, a connectivity graph is constructed with respect to the core points, in which each node corresponds to a core point

Algorithm *DBSCAN*
Input (Data: $\mathcal{D}$, Radius: *Eps*, Density: τ)
begin
Determine core, border and noise points of $\mathcal{D}$ at level (Eps, τ);
Create graph in which core points are connected
if they are within *Eps* of one another;
Determine connected components in graph;
Assign each border point to connected component
with which it is best connected;
return points in each connected component as a cluster;
end

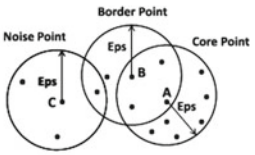


Fig. 4.3 (Left) The basic DBSCAN clustering algorithm; (right) an illustration of core, border and noise data points

and an edge is added between a pair of core points if and only if they are within a distance of *Eps* from one another. Next, all connected components of this graph are identified to be the clusters constructed on the core points. Then the border points are then assigned to the cluster with which they have the highest level of connectivity. Finally, the resulting groups are reported as clusters and noise points are reported as outliers. The basic DBSCAN algorithm is illustrated on the left of Fig. 4.3. It is noteworthy that the DBSCAN algorithm may be viewed as an enhancement of single-linkage agglomerative clustering algorithms when applied only to the core points and by treating marginal (border) and noisy points specially with termination-criterion of *Eps*-distance. This special treatment can reduce the outlier-sensitive chaining characteristics of single-linkage algorithms without losing the ability to create clusters of arbitrary shape. The bridge of noisy data points will not be used in the agglomerative process if *Eps* and τ are selected appropriately. In such cases, DBSCAN will discover the correct clusters in spite of the noise in the data.

The major time complexity of DBSCAN is in finding the neighbors of different data points within a distance of *Eps*. For a database of size N, the time complexity can be $O(N^2)$ in the worst case. Fortunately, the use of a spatial index for finding the nearest neighbors can reduce this complexity to approximately $O(N\log N)$. However, the $O(\log N)$ query performance is realized only for low-dimensional data, in which nearest neighbor indices work well.

The DBSCAN method can discover clusters of arbitrary shape, and it does not require the number of clusters as an input parameter. However, DBSCAN assumes clusters of similar density, and may have problems separating nearby clusters. OPTICS is a DBSCAN variant that can handle different densities much better. Being a generalization of DBSCAN, OPTICS was proposed to remove the need to choose an appropriate value for the range parameter ε and can produce a hierarchical result related to that of linkage clustering (Ankerst et al. 1999). The key drawback of DBSCAN and OPTICS is that they expect some kind of density drop to detect cluster borders. On data sets with, for example, overlapping Gaussian distributions, the cluster borders produced by these algorithms will often look arbitrary, because the cluster density decreases continuously. On a data set consisting of mixtures of

Gaussians, these algorithms are nearly always outperformed by methods such as EM clustering that are able to precisely model this kind of data.

Other more sophisticated density-based methods, such as DENCLUE, use gradient ascent on the kernel-density estimates to create the building blocks (Hinneburg and Keim 1998). Proposed in 2006 to eliminate the ε parameter entirely, DeLi-Clu, Density-Link-Clustering, combines ideas from single-linkage clustering and OPTICS and can offer performance improvements over OPTICS by using an R-tree index (Achtert et al. 2006a, b, c). Proposed in 2007, LDBSCAN combines ideas from local outlier detection and DBSCAN to deal with clusters of different densities (Duan et al. 2007). In comparison to other density-based clustering algorithms, it takes advantage of the k-nearest neighbors based LOF (Local Outlier Factor) to detect the noisy points and outliers and can discover different density clusters existing in different regions of data space. But it needs a user to input four parameters which have a significant influence on the clustering result but are hard to determine.

4.5 Graph-Based Algorithms

In graph-based methods, data of virtually any type can be converted to similarity graphs for analysis. This transformation is the key that allows the implicit clustering of any data type by performing the clustering on the corresponding transformed graph.

The notion of this transformation is defined with the use of a neighborhood graph. Given a set of data objects $D = \{x_1 \ldots x_N\}$, and a distance function defined on these objects, a neighborhood graph can be constructed as,

1. A single node is defined for each object in D to form the node set containing N nodes, where the node i corresponds to the object x_i.
2. An edge exists between x_i and x_j, if the distance $dist(x_i, x_j)$ is less than a particular threshold. A better approach is to compute the k-nearest neighbors of both x_i and x_j, and add an edge when either one is a k-nearest neighbor of the other. The weight w_{ij} of the edge (i, j) is equal to the distance between the objects x_i and x_j, so that larger weights indicate greater dissimilarity.

After the neighborhood graph has been constructed, many clustering algorithms can be used to cluster the nodes in the neighborhood graph. The obtained clusters on the nodes can be used to map back to clusters on the original data objects. Therefore, the graph-based approach should be treated as a generic meta-algorithm that can use any community detection algorithm in the final node clustering step. The overall meta-algorithm for graph-based clustering is provided in Fig. 4.4.

One interesting property of graph-based algorithms is that the approach can discover clusters of arbitrary shape. This is because the neighborhood graph encodes the relevant local distances (or k-nearest neighbors), and therefore the communities in the induced neighborhood graph are implicitly determined by agglomerating locally dense regions. As with density-based clustering, the agglomeration of locally dense

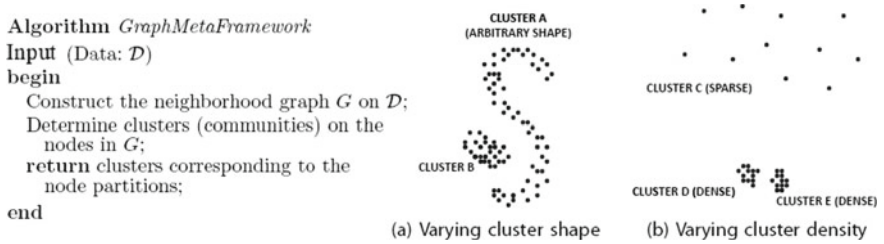


Fig. 4.4 (Left) Generic graph-based meta-algorithm; (right) an illustration of its application areas. Therefore, all three clusters will be found by a community detection algorithm on the k-nearest neighbor graph in spite of their varying density

regions corresponds to arbitrarily shaped clusters. Further, graph-based methods can provide better results than algorithms such as DBSCAN because of their ability to adjust to varying local density in addition to their ability to discover arbitrarily shaped clusters. However, high computational costs are the major drawback of graph-based algorithms. It is often expensive to apply the approach to an $N \times N$ matrix of similarities. Nevertheless, because similarity graphs are sparse, many recent community detection methods can exploit this sparsity to provide more efficient solutions.

4.5.1 Minimum Spanning Tree Based Clustering

Given an undirected and weighted connected graph G with node set V and edge set E, that is, $G = (V, E)$, a minimum spanning tree (MST) is a subgraph that spans over all the vertices but with no cycles, and has the minimal total weight among all such subgraphs. When the weight associated with each edge denotes a distance between two end points, any edge in the minimum spanning tree will be the shortest distance between two subtrees that are connected by that edge. Therefore, removing the longest edge will theoretically result in a two-cluster grouping. Removing the next longest edge will result in a three-cluster grouping, and so on. This corresponds to choosing the breaks where the maximum weights occur in the sorted edges. For cases where clusters are well separated from each other and the number of clusters is significantly smaller than the number of data points, in-cluster edges have weights that are significantly smaller than those edges corresponding to cluster breakers.

Based on this fact, first proposed by Zahn (1971), MST-based clustering begins by constructing an MST over a given weighted graph G with a time complexity of $O(E\log V)$ and then proceeds to remove inconsistent edges (e.g., edges corresponding to maximum weights) to create connected components. One good point of MST-based clustering resides in its taking distances between data points into account when clustering. However, the existence of noise may reduce cluster separations and thus the clustering quality.

Given a set of N data points and a distance measure defined upon them, modern MST-based clustering algorithms usually begin by constructing an MST with a time complexity of $O(N^2)$ (Cormen et al. 2009). This time factor limits the application of MST-based clustering methods to massive data sets.

4.5.2 *Spectral Clustering*

Spectral clustering goes back to Donath and Hoffman (1973), who first suggested to compute graph partitions based on eigenvectors of the adjacency matrix (Donath and Hoffman 1973). Being a competitive clustering method, spectral clustering is based on spectral graph theory. In the machine learning community, spectral clustering became popular by the works of Shi and Malik (2000), Ng et al. (2001), Meila and Shi (2001), and Ding (2007). A huge number of papers have subsequently been published, dealing with various extensions, new applications, and theoretical results on spectral clustering (Jia et al. 2014). Compared with traditional clustering algorithms, spectral clustering algorithms have several advantages. For one example, it is important to note that the spectral clustering algorithm directly works on the Laplacian matrix of feature vectors, and therefore, unlike K-means algorithm, the spherical clusters naturally found by the Euclidean-based K-means in the new embedded space may correspond to arbitrarily shaped clusters in the original space. This behavior is a direct result of the way in which the similarity graph and objective function O are defined and also one of the main advantages of using a transformation to similarity graphs. For another example, the computational complexity of the spectral clustering algorithm depends only on the number of data points but has nothing to do with the dimensionality of the data, and therefore can deal with datasets of high dimensionalities. Finally, spectral clustering is very easy to implement, and can use the standard linear algebra methods for fast solutions.

4.6 Distance and Similarity Measures

The clustering process usually searches to maximize the heterogeneity between regions and the homogeneity inside each region. Before clustering, a distance function between data points should be defined. In this section, we review some of the most popular distance or similarity measures used in clustering.

Being one of the mostly used distances, the Minkowski distance of order p between two data points, $x = (x_1, x_2, \ldots, x_d)$ and $y = (y_1, y_2, \ldots, y_d)$, is defined as,

$$\text{dist}(x,\ y) = \left(\sum_{i=1}^{d} |x_i - y_i|^p \right)^{\frac{1}{p}} \tag{4.4}$$

Minkowski distance is typically used with p being 1 or 2. The latter is the Euclidean distance, while the former is sometimes known as the Manhattan distance or the city-block distance. In the limiting case of p reaching infinity, we obtain the Chebyshev distance,

$$\lim_{p\to\infty}\left(\sum_{i=i}^{d}|x_i - y_i|^p\right)^{\frac{1}{p}} = \max_i|x_i - y_i| \tag{4.5}$$

In information theory, the Hamming distance between two vectors of equal length is the number of positions at which the corresponding symbols are different. In other words, it measures the minimum number of substitutions required to change one vector into the other, or the number of errors that transform one vector into the other.

The Mahalanobis distance is a useful way of determining the similarity of an unknown sample set to a known one. The fundamental difference from the Euclidean distance is that it takes into account the correlation between variables, being also scale-invariant. Formally, the Mahalanobis distance can be defined as a dissimilarity measure between two random vectors X and Y of the same distribution with the covariance matrix Σ,

$$\text{dist}_M(x,\ y) = \sqrt{(x - y)^{\mathrm{T}}\Sigma^{-1}(x - y)} \tag{4.6}$$

If the covariance matrix is the identity matrix, the Mahalanobis distance reduces to the Euclidean distance. If the covariance matrix is diagonal, then the resulting distance measure is called the normalized Euclidean distance,

$$\text{dist}_M(x,\ y) = \sqrt{\sum_{i=1}^{d}\frac{(x_i - y_i)^2}{\sigma_i^2}} \tag{4.7}$$

where σ_i is the standard deviation of the x_i over the sample set.

4.7 Clustering Performance Evaluation

The evaluation of the performance of a clustering algorithm can be conducted based on two situations. In the first case, named internal evaluation, when no cluster labels are provided, a particular evaluation metric should take into account the assumptions such that members belong to the same class are more similar than members of different classes according to some similarity measure. In the second case, named external evaluation, where the ground truth class assignments of the samples of the data used for clustering is known, the evaluation index as that of evaluating a supervised classification algorithm can be employed to count the number of errors.

Since the determination of the number of clusters in a data set is not easy and validation of clustering results is as difficult as the clustering itself, neither of these approaches can therefore ultimately judge the actual quality of a clustering. Being an alternative, "manual" evaluation by a human expert can evaluate the utility of the clustering in its intended application.

4.7.1 Internal Validation Criteria

When a clustering result is evaluated based on the datasets that have had no cluster label associated with each data item, this is called internal evaluation. These methods usually assign the best score to the algorithm that produces clusters with high similarity within a cluster and low similarity between clusters. More than a dozen of internal evaluation measures exist, usually based on the intuition that items in the same cluster should be more similar than items in different clusters. To name a few, the following methods based on internal criterion can be used to assess the quality of clustering algorithms.

4.7.1.1 Davies–Bouldin Index

The Davies–Bouldin index can be calculated by the following formula,

$$\mathrm{DB} = \frac{1}{n}\sum_{i=1}^{n}\max_{j\neq i}\left(\frac{\sigma_i + \sigma_j}{\mathrm{dist}(c_i,\ c_j)}\right) \tag{4.8}$$

where n is the number of clusters, c_{i} is the centroid of cluster i, σ_i is the average distance of all elements in cluster i to centroid c_i, and $dist(c_i, c_j)$ is the distance between centroids c_i and c_j (Davies and Bouldin 1979; Halkidi et al. 2001). Zero is the lowest possible score. Values closer to zero indicate a better partition. Since algorithms that produce clusters with low intra-cluster distances (high intra-cluster similarity) and high inter-cluster distances (low inter-cluster similarity) will have a low Davies–Bouldin index, the clustering algorithm that produces a collection of clusters with the smallest Davies–Bouldin index (i.e., a model with best separation between the clusters) is considered the best algorithm based on this criterion. However, the Davies-Bouldin index is generally higher for convex clusters than other concepts of clusters (such as density based clusters), and the usage of centroid distance limits the distance metric to Euclidean space. In other words, a good value reported by this method does not imply the best information retrieval.

4.7.1.2 Dunn Index

The Dunn index aims to identify dense and well-separated clusters. It is defined as the ratio between the minimal inter-cluster distance to maximal intra-cluster distance. For each cluster partition, the Dunn index can be calculated by the following formula,

$$D = \frac{\min\limits_{1 \le i \le j \le N} \text{dist}(i,\ j)}{\max\limits_{1 \le k \le N} \text{dist}'(k)} \tag{4.9}$$

where $\text{dist}(i, j)$ represents the distance between clusters i and j, and $\text{dist}'(k)$ measures the intra-cluster distance of cluster k (Dunn 1974). The inter-cluster distance $\text{dist}(i, j)$ between two clusters may be any number of distance measures, such as the distance between the centroids of the clusters. Similarly, the intra-cluster distance $\text{dist}'(k)$ may be measured in a variety of ways, such as the maximal distance between any pair of elements in cluster k. Since internal criterion seek clusters with high intra-cluster similarity and low inter-cluster similarity, algorithms that produce clusters with high Dunn index are more desirable.

4.7.1.3 Silhouette Coefficient

The silhouette coefficient contrasts the average distance to elements in the same cluster with the average distance to elements in other clusters (Rousseeuw 1987). The Silhouette Coefficient is defined for each sample to be,

$$s(o) = \frac{b(o) - a(o)}{\max\{a(o),\ b(o)\}} \tag{4.10}$$

where $a(o)$ denotes the mean distance between a sample and all other points in the same class and $b(o)$ denotes the mean distance between a sample and all other points in the next nearest cluster. Objects with a high silhouette value are considered well clustered, while objects with a low value may be outliers. The Silhouette Coefficient for a set of samples is given as the mean of the Silhouette Coefficient for each sample. In normal usage, the Silhouette Coefficient is applied to the results of a cluster analysis. The score is bounded between -1 for incorrect clustering and $+1$ for highly dense clustering. Scores around zero indicate overlapping clusters. The score is higher when clusters are dense and well separated, which relates to a standard concept of a cluster. Unfortunately, the Silhouette Coefficient is generally higher for convex clusters (such as clusters obtained by K-means) than other concepts of clusters (such as clusters obtained through DBSCAN). This index can also be used to determine the optimal number of clusters.

4.7.2 External Validation Criteria

In external evaluation, clustering results are evaluated based on data that are not used for clustering, such as datasets with known class labels and external benchmarks which consist of a set of pre-classified items often created by (expert) humans and can be thought of as a gold standard for evaluation. These types of evaluation methods measure how close the clustering is to the predetermined benchmark classes.

As with internal evaluation, a number of external evaluation measures exist which are adapted from variants used to evaluate classification tasks and are introduced in the following.

4.7.2.1 Rand Measure

The Rand index (RI) computes how similar the clusters (returned by the clustering algorithm) are to the benchmark classifications (Rand 1971). In place of counting the number of times a class is correctly assigned to a single data point (known as true positives), the Rand index can be viewed as pair counting metrics which assess whether each pair of data points that is truly in the same cluster is predicted to be in the same cluster. If C is a ground truth class assignment and K denotes the clustering results, the Rand index can be computed using the following formula,

$$R = \frac{a+b}{a+b+c+d} = \frac{a+b}{\binom{N}{2}} \tag{4.11}$$

where a is the number of pairs of elements that are in the same set in C as well as in the same set in K, b is the number of pairs of elements that are in different sets in C and in different sets in K, $\binom{N}{2}$ is the total number of possible pairs in the dataset (without ordering).

4.7.2.2 Adjusted Rand Index

However, the RI score does not guarantee that random label assignments will get a value close to zero (especially if the number of clusters is in the same order of magnitude as the number of samples). To counter this effect, Hubert and Arabie proposed an adjustment, Adjusted Rand index (Hubert and Arabie 1985), to discount the effect of random labelings by defining the adjusted Rand index as follows,

$$R_{\text{adj}} = \frac{\sum_{i=1}^{k}\sum_{j=1}^{l}\binom{m_{ij}}{2} - t_3}{\frac{1}{2}(t_1+t_2) - t_3} \tag{4.12}$$

where

$$t_1 = \sum_{i=1}^{k} \binom{k}{2}, \; t_2 = \sum_{j=1}^{l} \binom{l}{2}, \; t_3 = \frac{2t_1 t_2}{N(N-1)}$$

Given the knowledge of the ground truth class assignments and the clustering algorithm assignments of the same samples, the adjusted Rand index is a function that measures the similarity of the two assignments, ignoring permutations and with chance normalization. Furthermore, adjusted rand score is symmetric and swapping the argument does not change the score. It can thus be used as a consensus measure.

The scores are in the bounded range [−1, 1]. Negative values are bad (independent labelings), while similar clusterings have a positive ARI. Perfect match will have a score of 1.0. Bad (e.g. independent labelings) have negative or close to 0.0 scores (which is not the case for raw Rand index). Finally, no assumption is made on the cluster structure and it can be used to compare clustering algorithms (such as *K*-means which assumes isotropic blob shapes) with results of spectral clustering algorithms which can find clusters with "folded" shapes.

4.7.2.3 Jaccard Index

The Jaccard index is used to quantify the similarity between two datasets (Jaccard 1901, 1912) and is defined by the following formula,

$$\text{Jaccard} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}} \tag{4.13}$$

where TP is the number of True Positive (i.e. the number of pair of points that belong to the same clusters in both the true labels and the predicted labels), FP is the number of False Positive (i.e. the number of pair of points that belong to the same clusters in the true labels and not in the predicted labels) and FN is the number of False Negative (i.e. the number of pair of points that belongs in the same clusters in the predicted labels and not in the true labels).

This is simply the number of unique elements common to both sets divided by the total number of unique elements in both sets. Also note that TN (i.e., the number of True Negative) is not taken into account and can vary from 0 upward without bound. The Jaccard index takes on a value between 0 and 1. An index of 1 means that the two dataset are identical, and an index of 0 indicates that the datasets have no common elements.

4.7.2.4 Fowlkes–Mallows Index

The Fowlkes–Mallows index computes the similarity between the clusters returned by the clustering algorithm and the benchmark classifications (Fowlkes and Mallows 1983). It can be computed using the following formula,

$$\mathrm{FM} = \sqrt{\frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}} \cdot \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}} \tag{4.14}$$

The mutual information is an information theoretic measure of how much information is shared between a clustering and a ground-truth classification that can detect a non-linear similarity between two clusterings. Normalized mutual information is a family of corrected-for-chance variants of this that has a reduced bias for varying cluster numbers.

The score ranges from 0 to 1. The higher the value of the Fowlkes–Mallows index, the more similar the clusters and the benchmark classifications are. FMI scores close to 0.0 indicate two label assignments that are largely independent, while values close to one indicate significant agreement. Perfect labeling is scored 1.0. As with the case of Jaccard index, no assumption is made on the cluster structure and it can be used to compare clustering algorithms such as *K*-means with results of spectral clustering algorithms.

4.7.3 Cluster Tendency

To measure cluster tendency is to measure to what degree clusters exist in the data to be clustered, and may be performed as an initial test, before attempting clustering. One way to do this is to compare the data against random data. On average, random data should not have clusters.

4.7.3.1 Hopkins Statistic

There are multiple formulations of the Hopkins statistic (Hopkins and Skellam 1954). A typical one is as follows (Banerjee 2004). Let X be the set of N data points in d-dimensional space. Consider a random sample (without replacement) of $m \ll N$ data points with members x_i. Also generate a set Y of m uniformly and randomly distributed data points. Now define two distance measures, u_i to be the distance of $y_i \in Y$ from its nearest neighbor in X and w_i to be the distance of $x_i \in X$ from its nearest neighbor in X. We then define the Hopkins statistic as,

$$H = \frac{\sum_{i=1}^{m} u_i}{\sum_{i=1}^{m} u_i + \sum_{i=1}^{m} w_i} \tag{4.15}$$

With this definition, uniform random data should tend to have values near to 0.5, and clustered data should tend to have values nearer to 1. However, data containing just a single Gaussian will also score close to 1, as this statistic measures deviation from a uniform distribution, not multimodality, making this statistic largely useless in application (as real data are never remotely uniform).

4.8 Summary

In this chapter, we introduce unsupervised learning methods for clustering based image segmentation so as to produce percepts for mobile robotic navigation applications. We begin with a short review on several unsupervised learning methods including *K*-means algorithm, agglomerative hierarchical linkage clustering algorithms, DBSCAN clustering algorithm and MST based clustering algorithm, which is followed by a short introduction to several distance measures. Finally, measures for performance evaluation of clustering are described. Although a broad range of tools is available for us to use, ultimately, the machine should be able to decide on the appropriate model without any human intervention. However, to achieve this in full generality requires significant advances in artificial intelligence. As a result, the most appropriate clustering algorithm for a particular problem often needs to be chosen experimentally. One challenge we have to overcome is the trade-off between the issue of accuracy and the issue of computational cost. Fortunately, there are actually not too many algorithms to explore for our purpose. As is always the case, the challenge is to find it.

References

Achtert, E., Böhm, C., Kriegel, H.-P., Kröger, P., Müller-Gorman, I., & Zimek, A. (2006a). Finding hierarchies of subspace clusters. In *LNCS*: *Knowledge discovery in databases*. *Lecture notes in computer Science* (Vol. 4213, pp. 446–453).

Achtert, E., Böhm, C., & Kröger, P. (2006b). DeLi-Clu: Boosting robustness, completeness, usability, and efficiency of hierarchical clustering by a closest pair ranking. In *LNCS*: *Advances in knowledge discovery and data mining*. *Lecture notes in computer science* (Vol. 3918, pp. 119–128).

Achtert, E., Böhm, C., Kröger, P., & Zimek, A. (2006c). Mining hierarchies of correlation clusters. In *Proceedings of the 18th International Conference on Scientific and Statistical Database Management* (*SSDBM*'06) (Vol. 1, pp. 119–128).

Ankerst, M., Breunig, M. M., Kriegel, H.-P., & Sander, J. (1999). OPTICS: Ordering points to identify the clustering structure. In *Proceedings of the ACM SIGMOD International Conference on Management of Data* (*SIGMOD/PODS*'99) (pp. 49–60), PA,USA.

Arthur, D., & Vassilvitskii, S. (2007). k-means++: The advantages of careful seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*(*SIAM*'07) (pp. 1027–1035), Philadelphia, PA, USA.

Bahmani, B., Moseley, B., Vattani, A., Kumar, R., & Vassilvitskii, S. (2012). Scalable K-means++. In *Proceedings of the VLDB Endowment* (*PVLDB*'12) (Vol. 5, no. 7, pp. 622–633).

Bailey, K. (1994). Numerical taxonomy and cluster analysis. *Typologies and Taxonomies*, 34.

Banerjee, A. (2004). Validating clusters using the Hopkins statistic. *Proceedings of the IEEE International Conference on Fuzzy Systems, 1,* 149–153.

Cattell, R. B. (1943). The description of personality: Basic traits resolved into clusters. *Journal of Abnormal and Social Psychology, 38*(4), 476–506.

Cormen, T. T., Leiserson, C. E., & Rivest, R. L. (2009). Introduction to algorithms. *Resonance, 1*(9), 14–24.

Davies, D. L., & Bouldin, D. W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 1*(2), 224–227.

Defays, D. (1977). An efficient algorithm for a complete-link method. *The Computer Journal, British Computer Society, 20*(4), 364–366.

Ding, C. (2007). A tutorial on spectral clustering. *Journal of Statistics and Computing, 17,* 395–416.

Donath, W. E., & Hoffman, A. J. (1973). Lower bounds for the partitioning of graphs. *IBM Journal of Research and Development, 17,* 420–425.

Driver, H. E., & Kroeber, A. L. (1932). Quantitative expression of cultural relationships. *University of California Publications in American Archaeology and Ethnology, 31,* 211–256.

Duan, L., Xu, L., Guo, F., Lee, J., & Yan, B. (2007). A local-density based spatial clustering algorithm with noise. *Information Systems, 32*, 978–986.

Dunn, J. (1974). Well separated clusters and optimal fuzzy partitions. *Journal of Cybernetics, 4,* 95–104.

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining* (*KDD*'96) (pp. 226–231), Portland, OR, USA: AAAI Press.

Estivill-Castro, V. (2002). Why so many clustering algorithms—A position paper. *ACM SIGKDD Explorations Newsletter, 4*(1), 65–75.

Everitt, B., Landau, S., Leese, M., & Stahl, D. (2011). *Cluster Analysis*. Chichester, West Sussex, U.K.: Wiley Ltd.

Fowkles, E. B., & Mallows, C. L. (1983). A method for comparing two hierarchical clusterings. *Journal of the American Statistical Association, 78*(383), 553–569.

Halkidi, M., Batistakis, Y., & Vazirgiannis, M. (2001). On clustering validation techniques. *Journal of Intelligent Information Systems*, 17(2–3), 107–145.

Hinneburg, A., & Keim, D. A. (1998). An efficient approach to clustering in large multimedia databases with noise. In *Proceedings of 4th International Conference on Knowledge Discovery and Data Mining* (*KDD*'98) (pp. 58–65), New York City, NY, USA.

Hopkins, B., & Skellam, J. G. (1954). A new method for determining the type of distribution of plant individuals. *Annals of Botany, 18*(2), 213–227.

Hubert, L., & Arabie, P. (1985). Comparing partitions. *Journal of Classification, 2*(1), 193–218.

Jaccard, P. (1901). Étude comparative de la distribution florale dans une portion des Alpes et des Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles, 37,* 547–579.

Jaccard, P. (1912). The distribution of the flora in the alpine zone. *New Phytologist, 11*, 37–50.

Jia, H., Ding, S., Xu, X., & Nie, R. (2014). The latest research progress on spectral clustering. *Neural Computing & Applications, 24*, 1477–1486.

Kriegel, H.-P., Kröger, P., Sander, J., & Zimek, A. (2011). Density-based clustering. *Data Mining and Knowledge Discovery, 1*(3), 231–240.

Lloyd, S. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory, 28*(2), 129–137.

Meila, M., & Shi, J. (2001). A random walks view of spectral segmentation. In *Proceedings of the 8th International Workshop on Artificial Intelligence and Statistics* (*AISTATS*'01).

Ng, A., Jordan, M., & Weiss, Y. (2001). On spectral clustering: Analysis and an algorithm. *Advances in Neural Information Processing Systems* (pp. 849–856).

Rand, W. M. (1971). Objective criteria for the evaluation of clustering methods. *Journal of the American Statistical Association, 66*(336), 846–850.

Rousseeuw, Peter J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Computational and Applied Mathematics, 20,* 53–65.

Roweis, S. T., & Ghahramani, Z. (1999). A unifying review of linear Gaussian models. *Neural Computation, 11*(2), 305–345.

Shi, J., & Malik, J. (2000). Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 22*(8), 888–905.

Sibson, R. (1973). SLINK: an optimally efficient algorithm for the single-link cluster method. *The Computer Journal. British Computer Society, 16*(1), 30–34.

Sneath, P. H., & Sokal, R. R. (1973). *Numerical taxonomy*. San Francisco: W. H. Freeman and Company.

Sokal, R., & Michener, C. (1958). A statistical method for evaluating systematic relationships. *University of Kansas Science Bulletin, 38,* 1409–1438.

Tryon, R. C. (1939). *Cluster analysis: Correlation profile and orthometric (factor) analysis for the isolation of unities in mind and personality*. Ann Arbor, MI: Edwards Brothers.

Zahn, C. T. (1971). Graph-theoretical methods for detecting and describing gestalt clusters. *IEEE Transactions on Computers, c-20*, 68–86.

Zubin, J. (1938). A technique for measuring like-mindedness. *The Journal of Abnormal and Social Psychology, 33*(4), 508–516.

Chapter 5
An Efficient *K*-Medoids Clustering Algorithm for Large Scale Data



Abstract *K*-medoids clustering is a popular partition-based clustering technique to identify usual patterns in a data set. Recently, much work has been conducted with the goal of finding fast algorithms for it. In this paper, we propose an efficient *K*-medoids clustering algorithm which preserves the clustering performance by following the notion of a simple and fast *K*-medoids algorithm while improving the computational efficiency. The proposed algorithm does not require pre-calculating the distance matrix and therefore is applicable to large scale datasets. When a simple pruning rule is used, it can give near linear time performance. To this end, the complexity of this proposed algorithm is analyzed and found to be lower than that of the state of the art *K*-medoids algorithms. We test our algorithm on real data sets with millions of examples and experimental results show that the proposed algorithm outperforms state-of-the-art *K*-medoids clustering algorithms.

Keywords Clustering · Patition based clustering · *K*-means algorithm · *K*-medoids algorithm · INCK algorithm

5.1 Introduction

Clustering aims to partition a set of data objects into groups with common properties and is an important data mining technique (Han et al. 2001). Being applied in many important and different fields ranging from statistics, computer science, biology to social sciences or psychology (Malik et al. 2001; Bach and Jordan 2004; Weiss 1999), it has generated enormous interests, and various techniques have been proposed for it in recent years, including partitioning based approaches, connectivity-based approaches (hierarchical clustering), distribution-based approaches, density-based approaches, graph-based approaches and recent developments for high dimensional data. Being a partitioning based approach, *K*-means clustering method (or sometimes called Lloyd-Forgy method) was developed by James MacQueen in 1967 as a simple centroid-based method (MacQueen 1967) and has become one of the most widely used algorithms for clustering (Jain 2008). Given a data set, *K*-means algorithm iteratively finds k centroids and assigns every object to its nearest centroid, until the mean

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_5

of the coordinates of the objects in each cluster has no change. *K*-means algorithm is quite efficient in terms of the computational time for large datasets. However, it is well known that it is very sensitive to outliers. For this reason, *K*-medoids clustering algorithm was proposed where representative objects called medoids are considered instead of centroids (Kaufman and Rousseeuw 1987). Because it is based on the most centrally located object in a cluster, it is less sensitive to outliers in comparison with *K*-means algorithm. Therefore, many algorithms have been developed for *K*-medoids clustering, among which partitioning around medoids (PAM) proposed by Kaufman and Rousseeuw in 1990 is known to be the most powerful (Kaufman and Rousseeuw 1990). However, despite its many practical application domains (Arumugam et al. 2011; Ohnishi et al. 2014; Amorèse et al. 2015; Khatami et al. 2017), PAM works inefficiently for large data sets due to its high time complexity (Han et al. 2001). To improve, other researchers, beginning with the work by Park and Jun (2009), proposed to precompute the distance matrix and keep it in the memory so as to run *K*-medoids clustering algorithms as efficiently as *K*-means algorithms (Yu et al. 2018). Although the precomputation of distance matrix is a simple and effective approach to *K*-medoids clustering, the drawback is the amount of computation resources consumed (i.e., $O(N^2)$ for both time and space complexity). As a result, this major limitation has restricted the ability to apply this type of methods to large scale real-world databases which typically have millions of data objects.

The goal of this research is to develop a *K*-medoids algorithm that scales well to large real data sets not only in time complexity but also in space complexity. More specifically, we show in this paper that one can modify the most recently proposed simple and fast *K*-medoids algorithm (Yu et al. 2018), which would normally have a quadratic scaling behavior based on the procomputed distance matrix, by a simple pruning rule to yield near linear time and space mining on real and large data sets. The result of a complexity analysis suggests that under certain conditions, the time to process a large amount of data points using *K*-medoids algorithm does not depend on the size of the data set.

The remainder of this paper is organized as follows. In Sect. 5.2, we review some related work on *K*-medoids clustering. In Sect. 5.3, we introduce the proposed algorithm and analyze its time and space complexities by explaining that, although our simple algorithm has a poor $O(N^2)$ worst case scaling property, for many large high-dimensional real data sets, the actual performance is extremely good and is close to linear in time and space. In Sect. 5.4, we present experiments to evaluate how the algorithm works on real datasets. Finally in Sect. 5.5, we conclude this paper by discussing limitations and directions for future work.

5.2 Existing Work on *K*-Medoids Clustering

Although Partitioning Around Medoid (PAM) method is known to be the most powerful and popular among many algorithms developed for *K*-medoids clustering, it works inefficiently for large data sets due to its high time complexity (Han et al. 2001). To

address this issue, CLARA was proposed in 1990 also by Kaufman and Rousseeuw which performs PAM on objects sampled from the original whole dataset. However, CLARA may result in degraded clustering performance (Kaufman and Rousseeuw 1990). To improve, Lucasius et al. proposed in 1993 a new approach of *K*-medoid clustering by using a genetic algorithm (Lucasius et al. 1993) while Ng and Han proposed in 1994 an efficient PAM-based algorithm by updating new medoids from some neighboring objects (Ng and Han 1994). Instead of minimizing the sum of distances to the closest medoid used in PAM as an optimization criterion, in 2003, van der Laan et al. tried to maximize the silhouette (Wei et al. 2003; van der Laan et al. 2003). To further reduce the computational time, in 2005, Zhang and Couloigner proposed to utilize the concept of triangular irregular network in the swap step of PAM (Zhang and Couloigner 2005). To provide improved results in both the effectiveness and the efficiency, a simple and fast *K*-medoids algorithm (referred to as the FastK algorithm in the following) was proposed by Park and Jun (2009). In this algorithm, all pairwise distances are first precomputed. Next, the density of each object is calculated and then k data points with the top density values are selected as the initial medoids. However, the initial medoids optimized by this algorithm usually appear in the same cluster, which reduces the final clustering performance. To prevent the algorithm from becoming trapped in a local optimum, in 2013, Zadegan et al. introduced a new function that ranks objects according to their similarities and their hostility values so as to select the new updated medoids, which can aid in finding all the Gaussian-shaped clusters (Zadegan et al. 2013). To currently be the best algorithm for both selecting the medoids and determining the number of clusters simultaneously, a density peaks clustering algorithm was proposed by Rodriguez and Laio (2014). To determine the number of clusters (which is an important question but usually assumed to be given as an input), the variance enhanced *K*-medoids clustering algorithm was proposed in 2011 by Lai and Hu (2011) and the modified silhouette width plot in conjunction with PAM (mPAM) algorithm was proposed by Ayyala and Lin (2015), which iteratively increase the number of clusters until the evaluation index value is lower than a given threshold. To further improve the clustering performance, especially clustering accuracy, in 2015, Broin et al. used the genetic algorithm (GA) to both perform a global search and provide multiple initializations for the *K*-medoids algorithm, reducing the potential impact of poorly chosen starting medoids (Broin et al. 2015). For the optimization of the initial medoids, in 2016, Xie and Qu proposed two more improved *K*-medoids clustering algorithms, the density peak optimized *K*-medoids (DPK) algorithm and the density peak optimized *K*-medoids with new measure (DP-NMK) algorithm. However, both algorithms require a cut-off distance based on density peaks and the number of nearest neighboring objects, which would significantly affect the clustering performance, but for which Xie and Qu did not provide a general solution (Xie and Qu 2016). More recently in 2018, Yu et al. proposed an improved *K*-medoids clustering algorithm (referred to as the INCK algorithm in the following) which preserves the computational efficiency and simplicity of FastK while improving its clustering performance (Yu et al. 2018). The proposed algorithm requires determining the candidate medoids subsets and calculating the distance matrix, next uses them to first select two initial medoids and

then to incrementally increase the number of clusters and new medoids from 2 to k. Experimental results on both real and artificial data sets showed that the proposed algorithm outperforms three state-of-the-art *K*-medoids algorithms.

To summarize, all the previous approaches try to improve the *K*-medoids clustering performance and computational efficiency, whether one is optimizing the initial medoids selection or optimizing the medoids updating. However, most of these algorithms are based on pairwise distance calculations, so the computational burden still remains.

5.2.1 FastK Algorithm

In FastK algorithm, a dataset consisting of N d-dimensional objects, $D = \{x_1, x_2, \ldots, x_N\}$, and the number of clusters, k, are given and the output is a set of clusters, $C = \{C_1, C_2, \ldots, C_k\}$, formed based on the minimization of the total cost, E, as

$$E = \sum_{i=1}^{k} \sum_{x \in C_i} dist(o_k, x)^2 \tag{5.1}$$

where $\{o_1, o_2, \ldots, o_k\}$ denotes the set of k medoids, and the Euclidean distance between object i and object j is used as

$$\mathrm{dist}(x_i, x_j) = \sqrt{\sum_{m=1}^{d} (x_{im} - x_{jm})}, \; i, j = 1, \ldots, N \tag{5.2}$$

To select the initial k medoids, a density for every object is calculated according to,

$$v_j = \sum_{i=1}^{N} \frac{\mathrm{dist}(x_i, x_j)}{\sum_{l=1}^{N} \mathrm{dist}(x_i, x_l)}, \; j = 1, \ldots, N \tag{5.3}$$

Objects with k top density values are selected as the initial medoids and each of the rest objects is assigned to its nearest medoid, resulting in k initial clusters. Then for each cluster, the data object which minimizes the total sum of its distances to all other objects in its cluster is used to be an update for its current medoid. This process continues until the sum obtained by Eq. (5.1) has no changes.

5.2.2 INCK Algorithm

To overcome the problem with FastK, a step increasing and optimizing medoids algorithm, named as INCK, is recently proposed. To exclude some objects that are not suitable to act as medoids, INCK first defines a candidate medoids subset S_m based on the following definitions. Given a data set D, the centroid (i.e., the object mean) and the variance of D are defined to be,

$$\overline{x} = \frac{1}{N}\sum_{i=1}^{N} x_i \tag{5.4}$$

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \mathrm{dist}(x_i, \overline{x})^2} \tag{5.5}$$

The variance for every data object is defined to be,

$$\sigma_i = \sqrt{\frac{1}{N}\sum_{j=1}^{N} \mathrm{dist}\left(x_i, x_j\right)^2}, i = 1, \ldots, N \tag{5.6}$$

Based on Eqs. (5.5) and (5.6), the candidate medoids subset S_m can be defined to be,

$$S_m = \{x_i, |\sigma_i \leq \lambda\sigma, \quad i = 1, \ldots, N\} \tag{5.7}$$

where λ is a stretch factor. The INCK algorithm starts with two initial medoids, then increases the number of medoids in a step-wise fashion, ending at k medoids. The first medoid o_1 is determined according to the following condition,

$$o_1 = \arg\min_{x_i \in S_m}\{\sigma_i | i = 1, \ldots, N\} \tag{5.8}$$

The two initial medoids should be as far away from each other as possible. The second medoid o_2 is determined as

$$o_2 = \arg\max_{x_i \in S_m}\{\mathrm{dist}(x_i, o_1) | i = 1, \ldots, N\} \tag{5.9}$$

To select the rest medoids, a candidate medoid set $O' = \{o'_1, o'_2, \ldots, o'_i\}$ is determined as

$$o_i^{'} = \arg\max_{x_l \in C_i \cap S_m}\{\mathrm{dist}(x_l, o_i) | l = 1, \ldots, N\} \tag{5.10}$$

A new medoid, o_{i+1}, $i + 1 \leq k$, can be selected as

$$o_{i+1} = \underset{o_j \in O'}{\text{argmax}} \left\{ dist\left(o_j, o_j^{'}\right) | j = 1, \ldots, i \right\} \tag{5.11}$$

This process continues until all k initial medoids are selected. The inputs to the INCK algorithm include the dataset D, the desired number of clusters, K, and the stretch factor λ, and the output is a set of clusters, $\{C_1, C_2, \ldots, C_k\}$. The algorithm runs as follows:

1. Calculate the distance between each pair of objects using Eq. (5.2).
2. Calculate the data set D variance σ using Eq. (5.5), and the object variance σ_i using Eq. (5.6), then determine the candidate medoids subset S_m using Eq. (5.7).
3. Select two initial medoids $O = \{o_1, o_2\}$ using Eqs. (5.8) and (5.9).
4. Assign each object to the nearest medoid and calculate the total cluster cost E using Eq. (5.1).
5. For k from 2 to $K - 1$
6. Calculate the new increased medoids o_{k+1} using Eq. (5.11) and generate a new medoids set $O \leftarrow O \cup \{o_{k+1}\}$.
7. Repeat
8. Assign each object to the nearest medoid based on the nearest distance principle.
9. Update the medoids set O similar to FastK.
10. Calculate the total cluster cost E using Eq. (5.1).
11. Until the total cluster cost E no longer changes.
12. End for

5.3 The Proposed Efficient *K*-Medoids Approach

From the previous section, it can be seen that *K*-medoids clustering algorithms have been widely studied. Manually tuning the medoids selection can be tedious. The main challenges of *K*-medoids algorithms for large-scale datasets aim to develop fully automatic algorithm that does not require task-specific tuning to locate medoids fast and accurately. Therefore, the INCK algorithm is simple and elegant. However, it depends on all pairwise distance computations and storage in the main memory. Fortunately, this may not all be necessary for large scale datasets. In this section, we describe a new pruning scheme designed especially to facilitate efficient *K*-medoids clustering for modern large databases.

5.3.1 A Simple Idea

To locate a data point with a potential possibility of being the true cluster medoid, a precise criterion should be obtained from empirically estimated statistical parameters (e.g., mean or centroid, median and midrange), although frequently it is impossible

to anticipate possible distributions for large scale multi-dimensional data. Being the most prominent probability distribution in statistics to describe real-valued random variables that cluster around a single mean value, a normal distribution is often used as a first approximation to complex phenomena.

Basically, the proposed algorithm follows the idea of the "central limit theorem". Before proceeding, we give a formal presentation of the theorem.

5.3.2 Central Limit Theorem

Theorem 1 In most situations, when independent random variables are added, their properly normalized sum tends toward a normal distribution even if the original variables themselves are not normally distributed.

The central limit theorem (CLT) is a key concept in probability theory because it implies that probabilistic and statistical methods that work for normal distributions can be applicable to many problems involving other types of distributions. For example, suppose that a sample is obtained containing a large number of observations with each observation being randomly generated in a way that does not depend on the values of other observations, and that the arithmetic average of the observed values is computed. If this procedure is performed many times, the central limit theorem says that the computed values of the average will be distributed according to a normal distribution. Motivated by this theorem, in this paper, we show that a simple fast *K*-medoids algorithm in conjunction with a pruning rule gives state-of-the-art performance. The proposed algorithm is based on the following three observations.

First, in the initialization stage, the FastK algorithm requires all the pairwise distances be computed to calculate densities for each initial medoid selection while the INCK algorithm eliminates this requirement by using the sum of a data object's distances to all other data points to form an initial medoid subset, which is still quadratic. Therefore, for the medoid selection based on a distance measure with respective to the whole database, the central tendency measures can be a good approximate choice to start the search for the first corresponding medoid. Central tendency measures estimate the location of the middle or center of a data distribution. There are various ways to measure the central tendency of data, including the mean, the median, and the midrange. The most common and effective numeric measure of the "center" of a set of data is the arithmetic mean. Although it is the single most useful quantity for describing a data set, the mean is not always the best way of measuring the center of data. A major problem with the mean is its sensitivity to extreme values (e.g., outlier). Even a small number of extreme values could corrupt the mean. For skewed (asymmetric) data, a better measure of center of the data is the median, which is the middle value in a set of ordered data values. It is the value that separates the higher half of a data set from the lower half. Suppose that a given dataset of N values for an attribute is sorted in increasing order. If N is odd, then the median is the middle value of the ordered set. If N is even, then the median is not unique but by convention is

taken as the average of the two middle-most values. The midrange can also be used to access the center tendency of a numeric data set. It is the average of the largest and smallest values in the set for an attribute. To find a suitable medoid candidate, in our approach, the *d*-dimension mean or centroid, median, and midrange are calculated in a database wide manner. Then data points in a small neighborhood around the three are used to locate the first medoid by calculating their sums of distances to all other data points and selecting the one with the smallest sum.

Second, for the selection of the rest medoids in the initialization stage, to be less sensitive to outliers, we put a constraint on the minimum cluster sizes. The advantage of this constraint is the avoidance of a few largest clusters and an unnecessary large number of small clusters. Given a dataset to cluster, a loose estimate of minimum and maximum numbers of data items in a cluster can usually be available. To this end, we propose that the rest $k - 1$ initial medoids should be determined with an additional condition that they are selected such that the sizes of all formed clusters are larger than a loosely estimated lower-bound number of objects.

Finally, with the set of k medoids being initialized, the next step is to cluster objects. Design of searching methods for medoids that scale well with the size of the database is a challenging task. Central Limit Theory in this sense provides an efficient way to determine a local region, making the medoid search more efficient by pruning. In the update stage, according to central limit theorem, the center of a large cluster could be very close to its cluster mean, median and midrange. Better clustering effectiveness and runtime efficiency can be achieved if a very small neighborhood around the mean or centroid, median and midrange would be sufficient to check to locate the cluster medoid. Real-time clustering process demands for efficient search structures and algorithms. To improve, the proposed *K*-medoids algorithm uses only a small neighborhood near the three center measures in its cluster.

With all these ideas in mind, a new fast *K*-medoids clustering algorithm is developed in the following.

5.3.3 An Improved K-Medoids Algorithm

Suppose that N objects having d variables each should be grouped into k ($k < N$) clusters, where k is assumed to be given. The Euclidean distance is used as a dissimilarity measure in this study although other measures can be adopted. Given a loose estimate of minimum and maximum numbers of data items in a cluster, the proposed algorithm is composed of the following two steps.

Step 1: (Select initial medoids)

1. To find the first medoid candidate, and to scale well with the size of large databases, calculate the mean (i.e., the centroid), median and midrange points of all objects in the whole database. Then for each of the mean, median and midrange points, search for the data object with the smallest value according to Eq. (5.6) within a small neighborhood (i.e., m number of data objects) of these

three center measures. The data object with the smallest value is chosen as the first medoid candidate.
2. Calculate the distances between the first medoid candidate and all other data objects, which is denoted by $\{v_j, j = 1, \ldots, N\}$, based on the chosen dissimilarity measure.
3. Sort v_j's in an ascending order. Next, starting from the last object having the largest v_j value, with these two medoids, do a clustering. If both the resulted clusters have a size larger than the lower bound (i.e., the estimated minimum number of objects), select it as the second initial medoid. This process proceeds until k objects are selected as initial medoids such that all the resulted initial clusters each have a size large than the lower bound.
4. Obtain the initial clustering result by assigning each object to the nearest medoid.
5. Calculate the sum of distances from all objects to their nearest medoids as the initial total cost.

Step 2: (Update medoids)

6. To find a new medoid for each cluster, which is the object minimizing the total sum of distances to other objects in its cluster, calculate the mean (i.e., the centroid), median and midrange points of all objects in a cluster. Then for each of the mean, median and midrange points, search the data point with the smallest value according to Eq. (5.6) within a small neighborhood (i.e., m) of the three estimated local central points. The data object with the smallest value is chosen as the new medoid candidate.
7. Update the current medoid in each cluster by replacing it with the new medoid candidate.
8. Assign each object to the nearest medoid and obtain the clustering result.
9. Calculate the sum of distances from all objects to their medoids. If the sum is equal to the previous one, stop. Otherwise, go back to Step 6.

5.3.4 Time Complexity Analysis

From the description in the previous subsections, it can be seen that our algorithm mainly consists of two phases. In the initialization step of the proposed approach, instead of calculating the distance between every pair of all objects, we perform the distance computation of the data points only in a small local neighborhood (i.e., m) that are the closest to the mean, median and midrange of the whole database. The time complexity for the initialization step includes $3N$ for the mean, median and midrange points' calculations, $3mN$ for the first medoid localization, $(k-1)N$ for the rest $(k-1)$ medoids candidates' localization, and, therefore, is $O(3N + 3mN + (k-1)N)$. In the update step, for each cluster, instead of calculating the distance between every pair of all objects in a cluster, we perform the distance computation only in a small local neighborhood of the data points (i.e., m) that are around the mean, median and

midrange of one of the k clusters. The time complexity is $O(3a + 3am)$, where a is the maximum size of k clusters. Assume that the maximum number of iterations is t, the run-time complexity of the proposed algorithm is $O(3N + 3mN + (k - 1)N) + t((3a + 3am)k + kN))$. If $m \ll N$, $t \ll N$ and $k \ll N$, the time complexity is linear in the number of data points, which is very convenient for modern large databases.

5.3.5 Pseudocode for the Proposed K-medoids Algorithm

The implementation of the proposed approach is through the design of a C++ data structure called Pam. The Pam data structure has several member variables that remember the indices of the data items in the database or in a cluster, the three central tendency measures (the mean, median and midrange points), the indices of its k medoids, the cluster label, the cost for each data item, and the total costs, and member functions that initialize the k medoids and update them to generate k clusters. The outputs of the Pam data structure are k medoids and the corresponding clusters.

The proposed clustering approach starts with creating a Pam instance followed by doing the proposed *K*-medoids clustering. The Pam class is summarized in Table 5.1. The clustering procedures for initialization and updating are given in Tables 5.2 and 5.3, respectively.

5.4 A Performance Study

In this section, we present the results of an experimental study performed to evaluate the proposed fast *K*-medoids clustering algorithm.

First, we check the effectiveness of the proposed algorithm by performing comparison with INCK, CLARA, and *K*-means clustering algorithms on 4 large datasets from UCI machine learning repository (http://archive.ics.uci.edu/ml/datasets.html). For this comparison, we would like to show that the proposed fast *K*-medoids clustering algorithm can outperform the INCK, CLARA and *K*-means algorithms in the classification accuracy. Second, the clustering performance of the proposed method is evaluated on an image dataset for four color spaces because of their visual convenience for the performance evaluation. For this task, the performance is compared with those of CLARA and MST-based clustering algorithms. Finally, we evaluate the performance of our algorithm in the execution time which is also compared with those of the INCK, CLARA, *K*-means, and MST-based clustering algorithms to check the technical soundness of this study.

We implemented all the algorithms in C++. All the experiments were performed on a computer with Intel Core i7 2.3 GHz CPU and 8 GB RAM. The operating system running on this computer is Ubuntu Linux. We use the timer utilities defined in the *C* standard library to report the CPU time. In our evaluation, the total execution time in second accounts for all the phases of the proposed *K*-medoids clustering algorithm,

Table 5.1 The data structure

Name	Explanation
Public data members	
sampleNumbers;	An array holding the indices of all sample in the database/cluster
centroids;	An array holding the coordinate mean for each dimension
median;	An array holding the coordinate median for each dimension
midrange;	An array holding the coordinate midrang for each dimension
medoids;	An array holding the indices of medoids samples
cluster;	An array holding the cluster label for each sample in the dataset
numofsamples;	The number of samples in the database/the current cluster
costs;	An array holding the distance values to the cluster medoids
total_costs;	The sum of values in costs
Public methods	
find_centErs	Calculate and return centroids, medians, and midrange points;
PAM_INIT	Find the predefined number of initial k medoids
Pam_Swap	Update the medoids for each cluster

including those spent on the initialization and the updates. The results show the superiority of the proposed *K*-medoids algorithm over the other algorithms.

5.4.1 Performance on Large Datasets

To cluster a data set using our algorithm, we need to find the medoid for each cluster. Therefore, before delving into this set of experiments, we would first focus our study on the distribution of the first medoid determined by INCK and the relative distributions of the mean, median and midrange in a database wide manner. Particularly, we want to show how distributionally close the first medoid is to the cluster centroid, median and midrange for several small 2-D datasets so as to show the impact of using the cluster centroid, median and midrange to search the medoid on the performance of our algorithm. Presented in Fig. 5.1 are plots of small 2-D datasets and the locations of the centroid, median, midrange and the first medoid of the whole datasets marked with red cross, magenta square, green circle, and red star, respectively. Blue

Table 5.2 The PAM_INIT member function

Function Name: PAM_INIT			
Inputs:			
data	the input data set		
k	the predefined number of clusters		
topM	the size of local neighborhood around *centroids*, *median*, *midrange* points		
minClustersize	the minimum size of clusters		
threshold	the value used to filter		
Output:			
medoids	the initialized set of k medoids		
Begin			
	calculate *centroids*, *median*, *midrange* and their distances to all the data points in *data*;		
	sort the three arrays of distances in a non-decreasing order;		
	for each sample i in the topM local neighborhood around *centroids*, *median*, *midrange* {		
		calculate its distances to all the data points in *data* and the corresponding sum;	
		if (sum < threshold) { threshold = sum; *medoids*[0] = *i*; }	
	calculate the distances, dist, of medoids[0] to all the points in *data* and sort them descendantly;		
	while (*medoids*.size() < *k*)		
		For each sample i in the descendantly sorted order from 1 to n {	
			clustering with initialized medoid and sample i;
			if (the resulted clusters all have number of samples > *minClustersize*) { add i to medoids} }
			clustering with initialized medoid and sample i;
	costs.resize(numofsamples ,1.0e14);cluster.resize(numofsamples ,-1); total_cost s= 0;		
	for (int i=0; i<numofsamples; i++) {		
		for (int j=0; j<k; j++) { calculate the distance between sample i and medoids[j], distance;	

(continued)

Table 5.2 (continued)

Function Name: PAM_INIT				
			if (distance < costs[i]) { costs[i] = distance; cluster[i] = j; } }	
		total_costs = total_costs + costs[i]; }		
	}			
End				

dots denote the data points. From the plot, we can see that all three approximate centers are very close to the first starting medoid even for these small datasets of draftly different distributions. That is, the first starting medoid is in the near local region of the mean, median and midrange estimates from the whole datasets.

With some sensible knowledge of how close the mean or centroid, median and midrange points are to the first medoid chosen by INCK method, to have a more convincing demonstration of how the proposed model would perform in large scale datasets and to justify the proposed *K*-medoids clustering method is realistic and applicable in practice, we calculated four evaluation indices for four algorithms (INCK, *K*-means, CLARA, and the proposed algorithm referred to as CTL in the following) so as to further test the performance of the proposed method on 4 real large datasets from UCI. Its extension to large scale datasets in a more general setting in modern data mining tasks substantially increases the technical soundness of the proposed approach. All the data sets used in this set of experiments are briefly summarized in Table 5.4. For this set of experiments, the local neighborhood around the mean, median and midrange, m, is set to 10 for all the datasets. The minimum numbers of data items in a cluster used in our clustering technique, minPts, are listed in the last column of Table 5.4 (as well as Table 5.10).

The clustering performance on each of 4 UCI datasets by the four algorithms is evaluated using four popular external clustering validity indices (defined to measure the degree of correspondence between a clustering result and the prespecified partitions), Rand, Adjusted-Rand, Jaccard coefficient, and Fowlkes and Mallows (FM). The results obtained for this round of experiments are given in Tables 5.5, 5.6, 5.7 and 5.8 for four indexes.

Table 5.9 gives the number of datasets in which each of the four algorithms showed top clustering performance in four evaluation indices in this round of experiments. In each evaluation index, the proposed *K*-medoids algorithm showed the best clustering performance. This demonstrates that the proposed *K*-medoids algorithm is effective regardless of the evaluation index chosen.

Table 5.3 The PAM_Swap member function

Function Name: PAM_Swap	
Inputs:	
data	the input data set
k	the predefined number of clusters
topM	the size of local neighborhood around *centroids*, *median*, *midrange* points
minClustersize	the minimum size of clusters
threshold	the value used to filter
Output:	
medoids	the final set of k medoids
Begin	
	vector<int> temp_cluster = cluster;
	vector<float> temp_costs = costs;
	temp_total_costs = total_costs;
	total_costs = 0;
	while (1) {
	for (int j=0; j<k; j++) {
	calculate *centroids*, *median*, *midrange* and their distances to all other samples in cluster j;
	sort the three arrays of distances in a non-decreasing order;
	for each sample i in the *topM* local neighborhood around *centroids*, *median*, *midrange*
	calculate its distances to all the data points in the cluster and the sum;
	if (sum < threshold) { threshold = sum; *medoids*[j] = *i*; }
	}
	costs.resize(numofsamples ,1.0e14);
	cluster.resize(numofsamples ,-1);
	total_cost s= 0;
	for (int i=0; i<numofsamples; i++) {
	costs[i] = 1.0e14;
	for (int j=0; j<k; j++) {
	distance = dist(sample i, medoids[j]);
	if (costs[i]>distance) { costs[i] = distance;}
	}
	total_cost s= total_cost s+ costs[i];

(continued)

Table 5.3 (continued)

Function Name: PAM_Swap		
		}
		if (total_cost s== temp_total_costs) { break; }
		temp_cluster = cluster;
		temp_costs = costs;
		temp_total_costs = total_costs;
	}	
End		

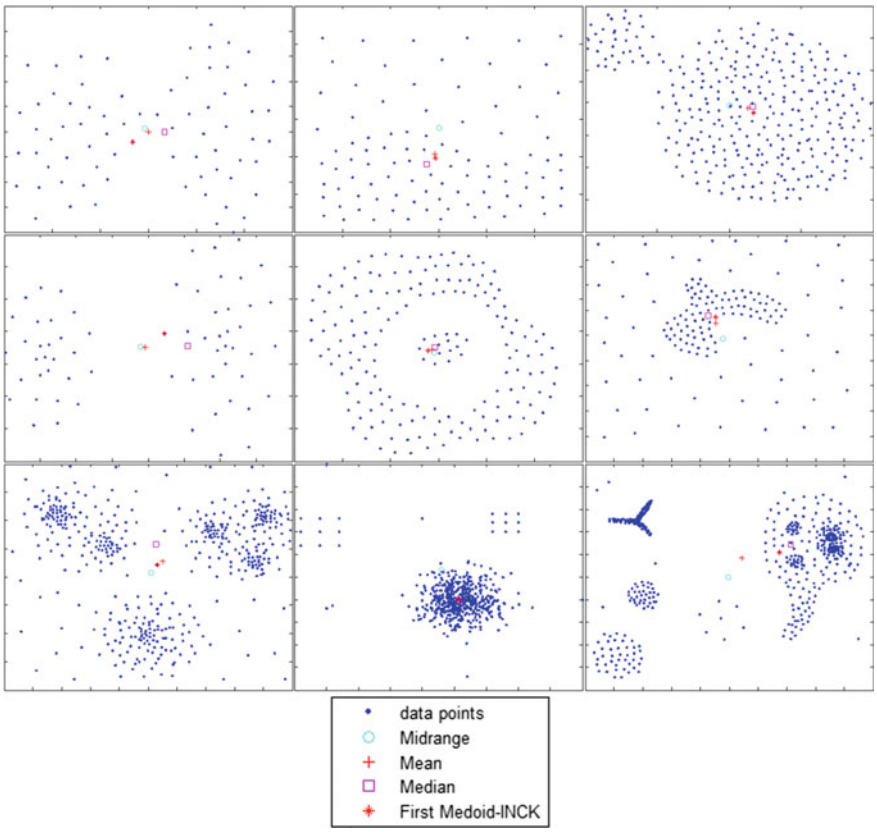


Fig. 5.1 The locations of the centroid, median, midrange, and the first medoid of several small 2-D datasets

Table 5.4 The sets of data

Data name	Size	# of attributes	# of classes	minPts
Localization	164,860	3	11	10,000
MiniBooNE	129,595	50	2	44,700
Skin	245,057	3	2	44,700
Pokerhand	1,000,000	10	10	50

Table 5.5 Performance evaluation in Rand

Data name	INCK	CTL	*K*-means	CLARA
Localization	0.7682	0.7607	0.7666	0.7583
MiniBooNE	0.6994	0.7001	0.6829	0.7574
Skin	0.5059	0.5061	0.5053	0.5190
Pokerhand	0.5548	0.5548	0.5545	0.5500

Table 5.6 Performance evaluation in Jaccard

Data name	INCK	CTL	*K*-means	CLARA
Localization	0.1194	0.1266	0.1180	0.0869
MiniBooNE	0.6242	0.6260	0.6288	0.5201
Skin	0.4332	0.4337	0.4301	0.4056
Pokerhand	0.0928	0.0928	0.0889	0.1108

Table 5.7 Performance evaluation in adjusted Rand

Data name	INCK	CTL	*K*-means	CLARA
Localization	0.09483	0.09568	0.09121	0.0404
MiniBooNE	0.3481	0.3484	0.2851	0.4892
Skin	−0.04397	−0.04422	−0.03961	0.01954
Pokerhand	0.00294	0.00294	0.00110	−0.00004

Table 5.8 Performance evaluation in FM

Data name	INCK	CTL	*K*-means	CLARA
Localization	0.2263	0.2337	0.2236	0.1718
MiniBooNE	0.7712	0.7729	0.7803	0.6863
Skin	0.6062	0.6066	0.6035	0.5771
Pokerhand	0.2149	0.2149	0.2092	0.2367

Table 5.9 Top performance evaluation in four indices

Index name	INCK	CTL	*K*-means	CLARA
Rand	2	1	0	2
Jaccard	0	2	1	1
Adjusted Rand	1	2	0	2
FM	0	2	1	1

5.4.2 Performance on Image Data

Outdoor environments are less structured. In this set of experiments, we exclusively choose images taken from outdoor scenes. Our working dataset, imageData, consists of 35 pictures taken around an outdoor environment, using SONY Digital Handycam DCR VX2000 digital video camera recorder. To accurately express the image content, each of the images are divided into smaller equally spaced patches for which RGB color histograms, HSV color histograms, Opponent color histograms, and Transformed color histograms are extracted. Each color channel is evenly divided into four intervals. All the interval combination can generate 64 different color bins for the creation of color histogram for each image patch. Then, all the pixels in the image patch are assigned to the corresponding possibility, and a feature vector (color histogram) is obtained. To summarize, for the set of images, the image patch is of size 27×27 and shifted by 18, resulting totally 95,550 color histogram-based feature vectors. The dataset used in this experiment is briefly summarized in Table 5.10. For this set of experiments, the local neighborhood, m, is set to 10.

The proposed *K*-medoids algorithm is applied to cluster these RGB, HSV, Opponent, Transformed color histograms. The clustering performances are presented in Fig. 5.2 for five images in order to keep the amount of data presented reasonable, but consistent performance results are obtained for other images. There are five columns in Fig. 5.2. The first row of Fig. 5.2 displays the five testing images used. The second row of Fig. 5.2 displays the clustering results of the proposed method for HSV color histograms. The third row of Fig. 5.2 displays the clustering results of the proposed method for Opponent color histograms. The fourth row of Fig. 5.2 displays the clustering results of the proposed method for RGB color histograms. Finally, the last row of Fig. 5.2 displays the clustering results of the proposed method for Transformed color histograms. To display results, in the segmented images, four close but slightly different sets of colors are used to denote the percepts.

From the original images of imageData, it is easy to identify that the major percepts include dark green trees, tree shadows, wide grey concrete road and manhole covers on the concrete road. There are 5 clusters obtained from this dataset by the proposed

Table 5.10 The set of data

Data name	Size	# of attributes	minPts
ImageData	95,550	64	10,000

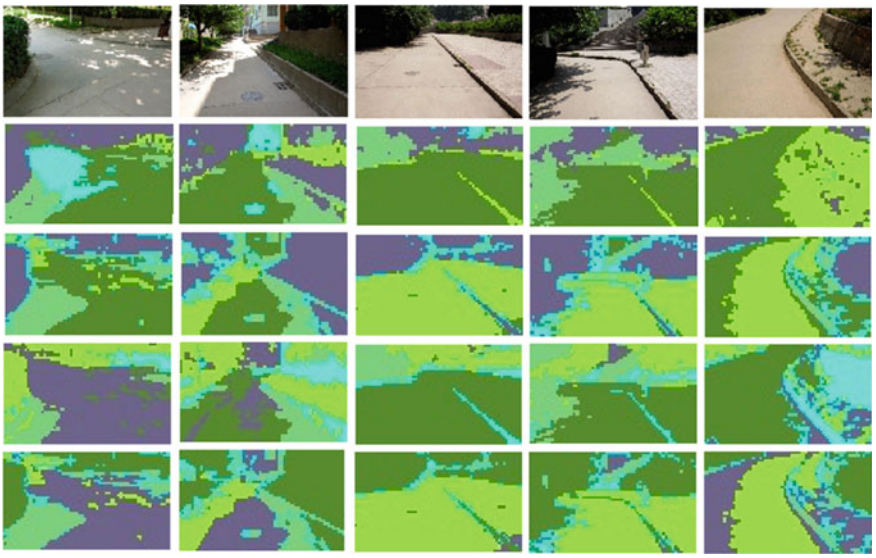


Fig. 5.2 (The first row) Original images, segmentation results for (the second row) HSV color histograms, (the third row) Opponent color histograms, (the fourth row) RGB color histograms, (the last row) Transformed color histograms

method. Typical stable percepts detected (and the corresponding color representing them) include several different more finely segmented concrete road percepts, several different grass and tree percepts, the tree and wall shadows, and the manhole covers.

From the segmented images as shown in the second row of Fig. 5.2, it can be seen that, for this outdoor environment, it is a little harder for the HSV color space to identify the concrete road in the first image (which is oversegmented into two percepts) while segregating the green flower bed in the fifth image from the pavement (which are bundled into one percept). From the segmented images as shown in the third row of Fig. 5.2, the proposed method performs a little better in the Opponent color space (in which situation, the foreground grey concrete road in the first image is not oversegmented and the green flower bed in the fifth image can be clearly segregated from the surrounding pavement) than the HSV color space. From the segmented images as shown in the fourth row of Fig. 5.2, the performance of our proposed method in the RGB color space is similar to that in the Opponent color space except that it cannot identify the manhole cover on the foreground road in the second image. From the last row of Fig. 5.2, it can be seen that the performance in the Transformed color space is similar to that in the Opponent color space. To summarize, overall, our algorithm can discover all the major percepts most of time and discriminate percepts in four color spaces.

Next, three clustering algorithms, the proposed *K*-medoids algorithm, CLARA clustering algorithm, and MST-based clustering algorithm, are applied to the HSV, Opponent, RGB and transformed color histograms for imageData. Since MST-based clustering algorithm is well known to be good at detecting clusters with irregular

boundaries, the performance of the proposed algorithm is also compared with that of its. The results for five images from the dataset are presented in Figs. 5.3, 5.4, 5.5, 5.6 for HSV color space, Opponent color space, RGB color space, and Transformed color space, respectively, in order to keep the amount of data presented reasonable, but consistent performance results are obtained for other images. There are four columns in each figure. The pictures shown in the first columns are the original images. The pictures shown in the second columns are the segmented images using the proposed *K*-medoids method. The pictures shown in the third columns are the segmented images obtained by running the CLARA algorithm on imageData. The pictures shown in the fourth columns are segmented images obtained by running the MST-based clustering algorithm on imageData. The results show that different methods have different optimal segmentation results in different color spaces. The number of clusters is set to 5 for the proposed *K*-medoids method, CLARA, and MST-based clustering algorithm.

The results of image segmentation in HSV color space are shown in Fig. 5.3. From the segmented images in Fig. 5.3, it can be seen that, for the first two scenes, the proposed algorithm performs the best while both CLARA and MST-based method

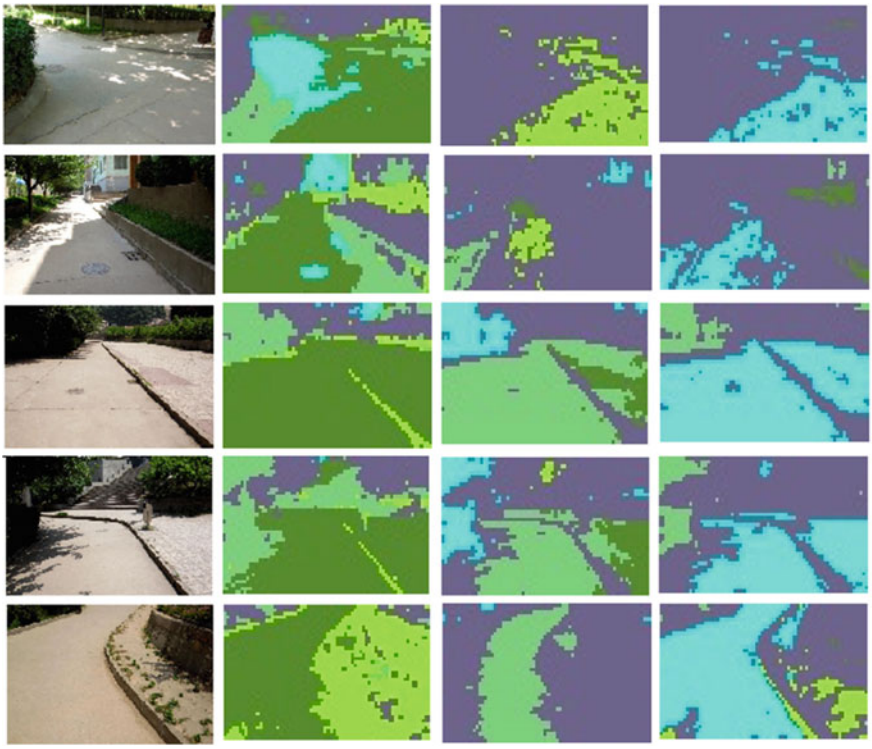


Fig. 5.3 (Left) Original image, results of HSV color histograms for (middle left) our proposed algorithm, (middle right) standard CLARA algorithm, and (right) standard MST algorithm

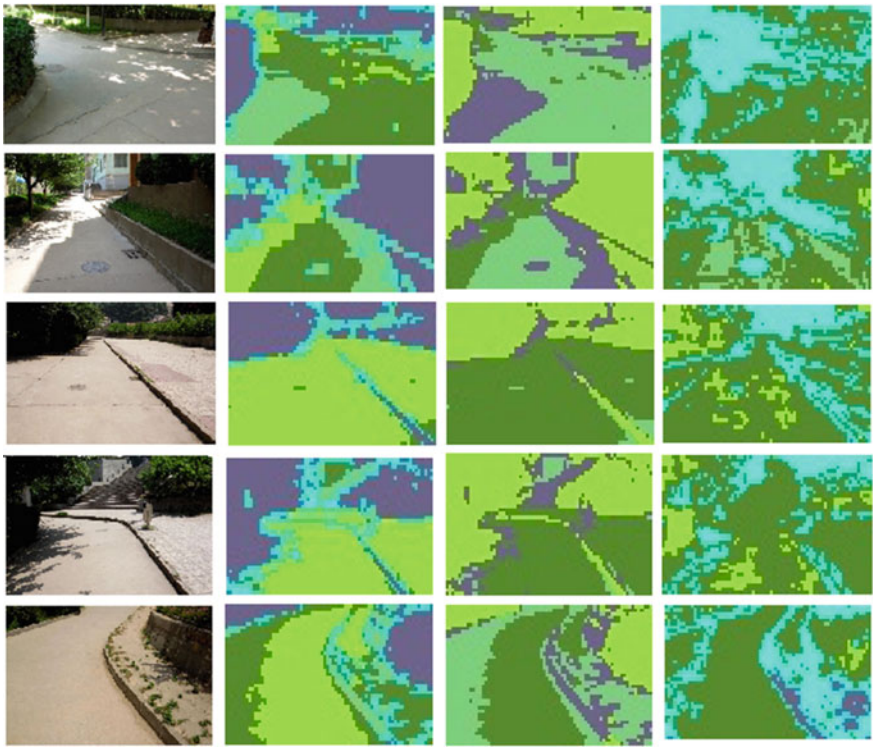


Fig. 5.4 (Left) Original image, results of Opponent color histograms by (middle left) the proposed algorithm, (middle right) CLARA algorithm, and (right) MST-based algorithm

undersegment the major percepts. All three methods perform similarly for the third and fourth scenes. For the last scene, MST-based method performs the best by segregating the green flower bed from the pavement. However, the proposed method cannot separate the two percepts but performs better than CLARA.

The results of image segmentation in Opponent color space are shown in Fig. 5.4. From the segmented images in the figure, it can be seen that, in this color space, CLARA and the proposed algorithm perform similarly for the first four scenes. For the last scene, the proposed algorithm can better segregate the green flower bed from the step pavement while CLARA can better segregate the step pavement from the road than ours. Both methods do a better job than MST based method for all the scenes.

The results of image segmentation in RGB color space are shown in Fig. 5.5. From the segmented images in Fig. 5.5, it can be seen that, in this color space, the proposed algorithm performs the best for the first and the last scenes and the MST based method performs slightly better than ours for the second, the third, and the fourth scenes. CLARA method oversegments the grey concrete road percepts in the

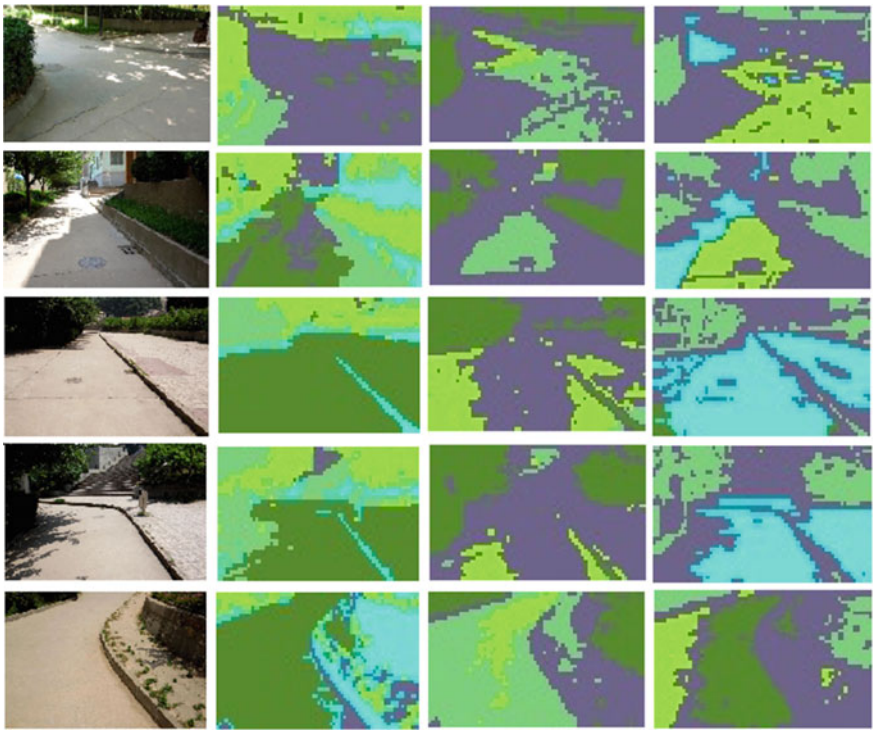


Fig. 5.5 (Left) Original image, results of RGB color histograms for (middle left) our proposed algorithm, (middle right) standard CLARA algorithm, and (right) standard MST algorithm

first, third, fourth, and fifth images and performs slightly better than ours for the second scene.

The results of image segmentation in Transformed color space are shown in Fig. 5.6. From the segmented images in Fig. 5.6, it can be seen that, in this color space, CLARA performs slightly better than ours for the first scene for which the proposed *K*-medoids method oversegments the grey concrete road percepts. For the rest of the scenes, both methods perform similarly and better than MST based method for all the scenes.

Overall, the results presented above offer good evidence that our method performs very well on a variety of images from different outdoor domains.

5.4.3 Running Time Performance

To evaluate the efficiency of the proposed method, more experiments are performed on the same test data sets as in the previous two subsections. The running time performances of our proposed algorithm, INCK, CLARA, *K*-means algorithms for

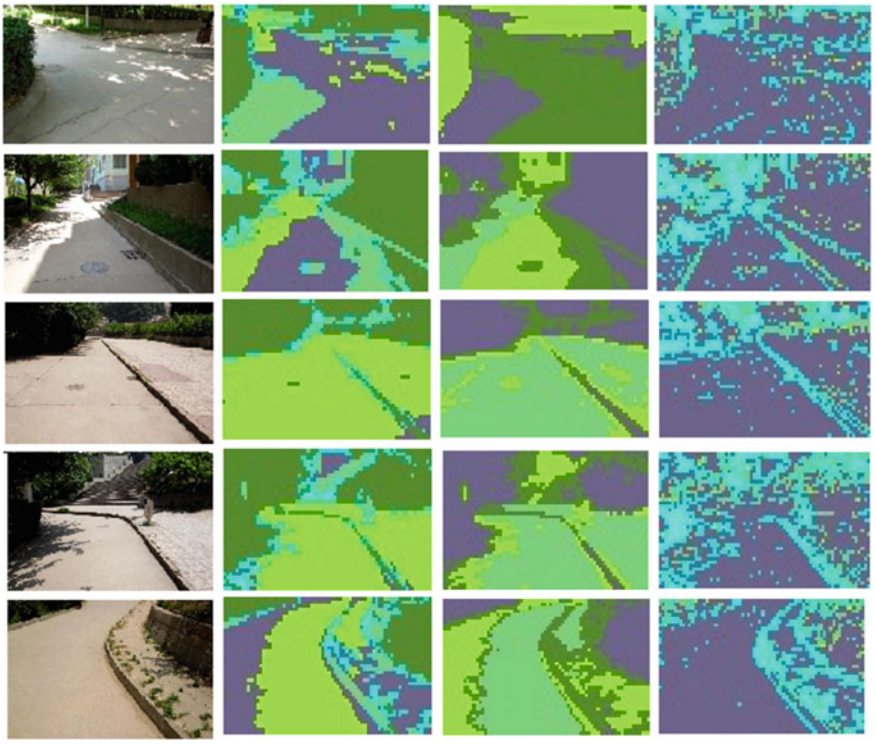


Fig. 5.6 (Left) Original image, results of Transformed color histograms for (middle left) our proposed algorithm, (middle right) standard CLARA algorithm, and (right) standard MST algorithm

4 large datasets from UCI, and those of our proposed algorithm, CLARA and MST based algorithm for the image dataset are summarized in Tables 5.11 and 5.12, respectively. For this set of experiments, the local neighborhood in our algorithm is set to be 10. The running time of the *K*-means algorithm is the average of ten executions.

It can be seen from the tables that our method runs the fastest on all the datasets except on the MiniBooNE dataset for which *K*-means runs faster than our method. The INCK and standard MST-based clustering algorithms require a dissimilarity matrix to be constructed in the first place and, therefore, are time consuming.

Table 5.11 Running time performance for 4 UCI datasets

Data name	INCK	CTL	*K*-means	CLARA
Localization	3619	11	31	1812
MiniBooNE	26,620	43	10	315
Skin	11,192	3	4	765
Pokerhand	281,772	49	475	1951

Table 5.12 Running time performance forimage data

Method name	HSV	Opponent	RGB	Transformed
CTL	27	29	23	28
CLARA	3154	3151	3146	3153
MST	1655	1927	1572	1938

5.5 Concluding Remarks

We have proposed a fast *K*-medoids algorithm and applied it to various clustering tasks. Basically, our algorithm is based on Central Limit Theory and can offer near-linear time performance by accessing only a few data objects in the neighborhood around the cluster mean, median and midrange to reach the true cluster medoids. The algorithm significantly reduces the expense of the computation while retaining good control on the clustering accuracy. Evaluation on various real datasets shows that a significant speedup for *K*-medoids clustering can be achieved by our algorithm in comparison to several state-of-the-art *K*-medoids algorithms. Future work would focus on further improving the clustering accuracy.

References

Amorèse, D., Bossu, R., & Mazet-Roux, G. (2015). Automatic clustering of macroseismic intensity data points from internet questionnaires: Efficiency of the partitioning around medoids (PAM). *Seismological Research Letters, 86,* 1171–1177.

Arumugam, M., Raes, J., & Pelletier, E. (2011). Enterotypes of the human gut microbiome. *Nature, 506,* 174–180.

Ayyala, D., & Lin, S. (2015). GrammR: Graphical representation and modeling of count data with application in metagenomics. *Bioinformatics, 31,* 1648–1654.

Bach, F.R., & Jordan, M.I. (2004, December). Blind one-microphone speech separation: a spectral learning approach. In *Proceedings of the 17th International Conference on Neural Information Processing Systems* (*NIPS'04*), (pp. 65–72). MIT Press.

Broin, P. Ó., Smith, T., & Golden, A. (2015). Alignment-free clustering of transcription factor Binding motifs using agenetic-*k*-medoids approach. *BMC Bioinformatics, 16,* 1–12.

Han, J., Kamber, M., & Tung, A.K.H. (2001). Spatial clustering methods in data mining: aaurvey. In H. J. Miller & J. Han (Eds.), *Geographic data mining and knowledge discovery*. Taylor & Francis.

Jain, A.K. (2008). Data clustering: 50 years beyond Kmeans. In: W. Daelemans, B. Goethals & K. Morik (Eds.). *Machine learning and knowledge discovery in databases*. ECML PKDD 2008. Lecture notes in computer science, Vol. 5211, pp. 3–4, Springer, Berlin, Heidelberg.

Kaufman, L., & Rousseeuw, P.J. (1987). Clustering by means of medoids. In Y. Dodge (Ed.). *Statistical data analysis based on the norm and related methods* (pp. 405–416). North-Holland.

Kaufman, L., & Rousseeuw, P. J. (1990). *Finding groups in data: An introduction to cluster analysis*. New York: Wiley.

Khatami, A., Mirghasemi, S., Khosravi, A., Lim, C. P., & Nahavandi, S. (2017). A new PSO-based approach to fire flame detection using *K*-medoids clustering. *Expert Systems with Applications, 68,* 69–80.

Lai, P.-S., & Hu, H.-C. (2011). Variance enhanced K-medoids clustering. *Expert Systems with Applications, 38,* 764–775.

Lucasius, C.B., Dane, A.D., & Kateman, G. (1993). On K-medoid clustering of large data sets with the aid of agenetic algorithm: background, feasibility and comparison. *Analytica Chimica Acta*, *282*, 647–669.

MacQueen, J.B. (1967). Some methods for classification and analysis of multivariate observations. In *Proceedings of the 5th Berkeley Symposium on Mathematical Statistics and Probability* (pp. 281–297). University of California Press.

Malik, J., Belongie, S., Leung, T., et al. (2001). Contour and texture analysis for image segmentation. *International Journal of Computer Vision, 43,* 7–27.

Ng, R., & Han, J. (1994). Efficient and effective clustering methods for spatial data mining. In *Proceedings of the 20th International Conference On Very Large Databases* (pp. 144–155). Santiago, Chile.

Ohnishi, Y., Huber, W., & Tsumura, A. (2014). Cell-to-cell expression variability followed by signal reinforcement progressively segregates early mouse lineages. *Nature Cell Biology, 16,* 27–37.

Park, H.-S., & Jun, C.-H. (2009). A simple and fast algorithm for K-medoids clustering. *Expert Systems with Applications, 36,* 3336–3341.

Rodriguez, A., & Laio, A. (2014). Clustering by fast search and find of density peaks. *Science, 344,* 1492–1496.

van der Laan, M. J., Pollard, K. S., & Bryan, J. (2003). A new partitioning around medoids algorithm. *Journal of Statistical Computation and Simulation, 73*(8), 575–584.

Wei, C.-P., Lee, Y.-H., & Hsu, C.-M. (2003). Empirical comparison of fast partitioning-based clustering algorithms for large data sets. *Expert Systems with Applications, 24*(4), 351–363.

Weiss, Y. (1999, February). Segmentation using eigenvectors: Aunified view. In *Proceedings of the 7th IEEE International Conference on Computer Vision* (pp. 975–982).

Xie, J., & Qu, Y. (2016). K-medoids clustering algorithms with optimized initial seeds by density peaks. *Journal of Frontiers of Computer Science and Technology, 9,* 230–247.

Yu, D., Liu, G., Guo, M., & Liu, X. (2018). An improved K-medoids algorithm based on step increasing and optimizing medoids. *Expert Systems with Applications, 92,* 464–473.

Zadegan, S. M. R., Mirzaie, M., & Sadoughi, F. (2013). Ranked K-medoids: afast and accurate rand-based partitioning algorithm for clustering large datasets. *Knowledge-Based Systems, 39,* 133–143.

Zhang, Q., & Couloigner, I. (2005). Anew and efficient K-medoid algorithm for spatial clustering. *Lecture Notes in Computer Science, 3482,* 181–189.

Chapter 6
Enhancing Hierarchical Linkage Clustering via Boundary Point Detection



Abstract In various disciplines, hierarchical clustering has been an effective tool for data analysis. However, traditional hierarchical clustering algorithms are not scalable to very large databases because of their high computational cost. To partially circumvent this drawback, in this paper, we propose a new algorithm for hierarchical linkage clustering as a solution for an efficient as well as reliable data clustering problem. Basically, our algorithm consists of two stages. In the first stage, the traditional linkage algorithms are applied to cluster a size-reduced version of the original dataset via boundary point detection. In the second stage, a *k*-nearest neighbors based classifier is employed to assign a cluster label to the remaining data points. Finally, evaluation is performed to show that the proposed algorithms can obtain good results not only in terms of the consumption of reasonable run times but also with better accuracy.

Keywords Agglomerative hierarchical clustering · Distance-based outlier detection · Density-based outlier detection · Boundary point detection · *K*-nearest neighbors search

6.1 Introduction

Being a popular and an important data mining technique, data clustering aims to group data points into meaningful subclasses, known as clusters, such that the intra-differences are minimized and inter-differences of these subclasses are maximized (Flynn et al. 1999). Being one of the most popular data clustering techniques, hierarchical clustering (HC) algorithms produce a set of clusters as well as a tree-like relationship among data points that can be demonstrated visually with a dendogram. There are two main categories of hierarchical clustering algorithms, agglomerative and divisive. Agglomerative algorithms start with clusters each containing only one object, and iteratively choose to merge two clusters to form one larger cluster. The process is repeated until only one large cluster is left, which contains all the objects. Divisive algorithms work in the opposite way. In this chapter, we will focus on the former class of hierarchical clustering algorithms. There are many agglomerative algorithms depending on the criterion used to decide which two clusters to merge,

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_6

with popular ones being single linkage (nearest neighbor), complete linkage (furthest neighbor), unweighted pair-group average (UPGMA), weighted pair-group average (WPGMA), unweighted pair-group centroid (UPGMC), weighted pair-group centroid (median) and Ward's method.

Agglomerative hierarchical clustering methods are popular. However, the weakness of these methods is their low scalability. In some contexts, such as in the case of high-dimensional data, the computation of all object-to-object distances can be a very expensive task and dominates the overall cost of the clustering process, and so any attempt to improve performances should aim at saving a significant portion of distance computations. The development of simple pruning strategies come in handy that improves their performances by reducing the number of object-to-object distances to compute without affecting the results.

Taking this into account, in this chapter, a simple divide and conquer approach is proposed to reduce the time complexity by a two-stage algorithm for the linkage clustering methods. In the first stage, a size-reduced version of the original dataset is provided using k-nearest neighbors and a boundary point detection method. The traditional linkage clustering algorithms are then applied to cluster this size-reduced version of the original datasets. In the second stage, a k-nearest neighbors classifier is employed to assign a label to the remaining data points. Basically, our method starts by finding k-nearest neighbors for each data point. Next a boundary point factor is computed upon a data point's k-nearest neighbors and two thresholds are determined based on the boundary factors. The data points within the two thresholds and some in-cluster data points are retained upon which standard linkage methods are applied. Finally, a k-nearest neighbors classifier is used to assign the rest data points a cluster label. To be as general as possible, the proposed methodology has no specific requirements on the dimensionality of data sets and can be applied to linkage clustering for large high-dimensional data sets. To verify the proposed method, we thoroughly tested it with a number of small and large-scale real data sets. Experimental results demonstrate the robustness and effectiveness of the proposed approach in comparison with standard linkage clustering algorithms.

The rest of this chapter is organized as follows. We provide a short description of the linkage clustering algorithms and some background of boundary point detection methods in Sect. 6.2. In Sect. 6.3, we present the proposed methodology. In Sect. 6.4, an empirical study is conducted to evaluate the performances of the proposed methodology. Finally, conclusions are made in Sect. 6.5.

6.2 Related Work

Work related to the methods presented in this paper falls into two main categories, linkage clustering algorithms and boundary point detection methods.

6.2.1 Linkage Clustering Algorithms

Starting with singleton clusters, agglomerative hierarchical clustering algorithms repeatedly search two closest groups and then merge them until one cluster is left. The general structure of an agglomerative hierarchical clustering algorithm is summarized in Table 6.1.

To complete the clustering of N objects, a linkage method should be defined to calculate the distance between two clusters and $N - 1$ merges should occur. The key operation in the general schema which still needs to be instantiated is the choice of the best couple of clusters to merge. Each different instantiation of these two operations results into a different agglomerative clustering algorithm.

Single linkage method defines the clusters' distance as the distance between the closest pair of objects in the two clusters. In other words, the distance of two clusters is the value of the smallest link from one cluster to another cluster. For two clusters C_1 and C_2, their distance, $f(C_1, C_2)$, is computed as,

$$f(C_1, C_2) = \min_{x \in C_1, y \in C_2} \text{dist}(x, y) \tag{6.1}$$

where dist(x, y) is the Euclidean distance between two data points, x and y.

Complete linkage method defines the clusters' distance as the distance between two most distant objects in the two clusters. In other words, the distance of two clusters is the value of the largest link from one cluster to another cluster. For two clusters C_1 and C_2, their distance, $f(C_1, C_2)$, is computed as,

$$f(C_1, C_2) = \max_{x \in C_1, y \in C_2} dist(x, y) \tag{6.2}$$

where dist(x, y) is the Euclidean distance between two data points, x and y.

Table 6.1 General schema for hierarchical agglomerative clustering algorithms

Algorithm: Hierarchical Agglomerative Clustering			
Input: a dataset D			
Output: a tree structure T			
1.	C := {{o}| o∈D} and T = Ø;		
2.	while |C|>1 do		
3.		Select best couple (a,b) s.t. a,b∈C;	
4.		Create a new cluster c=a∪b, and let a and b be children of c in T;	
5.		C := C∪{c}\{a,b};	
6.		for each x∈C do	
7.			Compute the distance between x and c;
8.	return *T*.		

Centroid linkage score is the distance between the two group centroids (i.e., group averages) as,

$$f(C_1, C_2) = \text{dist}\left(c_{C_1}, c_{C_2}\right) \tag{6.3}$$

where c_{C_1} and c_{C_2} represent the centroids of cluster C_1 and cluster C_2, respectively, and dist(c_{C_1}, c_{C_2}) returns the distance between centroids of the two clusters.

Average linkage measures the average dissimilarity over all pairs of objects in two clusters and merges the two clusters that have the smallest cost to merge as per the equation below,

$$f(C_1, C_2) = \frac{1}{N_{C_1} N_{C_2}} \sum_{i=1}^{N_{C_1}} \sum_{j=1}^{N_{C_2}} \text{dist}\left(x_i, y_j\right) \tag{6.4}$$

where N_{C_1} and N_{C_2} represent the cardinalities of cluster C_1 and cluster C_2, respectively, and dist (x_i, y_j) is the distance between two data points, x_i and y_j, in cluster C_1 and cluster C_2, respectively. Ward linkage method merges the two clusters that have the smallest cost to merge as per the equation below,

$$f(C_1, C_2) = \frac{N_{C_1} N_{C_2}}{N_{C_1} + N_{C_2}} \text{dist}\left(c_{C_1}, c_{C_2}\right) \tag{6-5}$$

where N_{C_1} and c_{C_1} represent the cardinality and centroid of cluster C_1, respectively, while N_{C_2} and c_{C_2} represent the same for cluster C_2, and dist(c_{C_1}, c_{C_2}) is a function returning the distance between the centroids of each of the two clusters.

A run of linkage clustering algorithms requires multiple iterations, each of which needs to compute and update the pairwise distances between all intermediate clusters. The standard algorithm for hierarchical agglomerative clustering (HAC) has a time complexity of $O(N^3)$ and requires $O(N^2)$ memory (with N being the number of objects in the dataset), which makes the exact linkage algorithms inevitably suffer from high time and space complexities and unsuitable and unfeasible for processing datasets containing more than 20,000 objects. However, for some special cases, optimal efficient agglomerative methods (of complexity $O(N^2)$) are known, SLINK for single-linkage (Sibson 1973) and CLINK for complete-linkage clustering (Defays 1977). With a heap, the runtime of the general case can be reduced to $O\ (N^2 \log N)$ at the cost of further increasing the memory requirements. In many programming languages, the memory overheads of this approach are too large to make it practically usable.

Therefore, there exists a significant need to develop a hierarchical clustering algorithm for large datasets. To address large-scale data and to reduce the computational cost of the linkage algorithms, there have been some efforts in developing efficient algorithms for them. However, most of these algorithms are based on the whole dataset, so the computational burden still remains for modern large-scale datasets.

6.2.2 Boundary Point Detection

Boundary points and outliers represent valid, interesting and potentially valuable patterns and their detection is an interesting problem. Boundary points sit on the extremes of a class region, i.e., near free pattern space. The border-edge pattern selection method (BEPS), proposed by Li and Maguire (2011), plays an important role in identifying boundary points. However, there is no universally accepted definition of a boundary point. A general intuition of what constitutes an outlier was given by Hawkins in 1980 as "An outlier is an observation that deviates from other observations so much as to arouse suspicions that it is generated by a different mechanism (Hawkins 1980)". Although outliers and boundary points are different by definition, they are generally located around the margin of densely distributed data such as a cluster. Many approaches have been proposed in the literature to identify outliers (Aggarwal and Yu 2001; Li et al. 2015a, b; Kutsuna and Yamamoto 2014), and some techniques are employed for boundary points (Qiu and Cao 2016; Xia et al. 2006; Li and Maguire 2011; Li 2008, 2011; Ding et al. 2014, 2015).

In the past, outlier detection methods were mainly based on distribution. However, there are more and more data sets with unknown distributions. Unlike distribution-based statistical methods which rely on known distributions, state-of-the-art k-nearest neighbors (kNN) based outlier detection algorithms are simple to implement and can locate the small number of outliers quickly. These methods usually compute a kNN-based outlier score or factor for each data point, rank data points according to their scores, and finally return data points with top n scores as outliers. The general problem of kNN-based outlier detection has been addressed by very diverse approaches that can roughly be classified into distance-based methods and density-based methods.

Originally proposed by Knorr and Ng in 1998, distance-based outlier detection method aims to locate observations that deviate significantly from the rest of the data points by taking the whole dataset into consideration as, given a distance measure defined on a feature space, "an object O in a dataset T is a Distance-Based outlier (i.e., DB(p, D)-outlier) if at least a fraction p of the objects in T lies greater than distance D from O" (Knorr and Ng 1998). Shortly after, two kNN-based variants were developed to suit for different theoretical and practical purposes (Ramaswamy et al. 2000; Angiulli and Pizzuti 2002). Given two integers, n and k, one uses the distance to a data point's k-th nearest neighbor as a measure to rank and returns top n largest ones as outliers (referred to as "DB-Max" method in the following) (Ramaswamy et al. 2000) while the other uses the average distance to their kNN as a measure to rank and returns top n largest ones as outliers (referred to as "DB" method in the following) (Angiulli and Pizzuti 2002). Although distance-based outlier definition is simple and elegant, when it comes to data sets with complex structures and consisting of clusters with different densities as illustrated in Fig. 6.1, it is beyond its ability to effectively mine local outliers.

For an observation in a data set whose density significantly deviates from the local area in which it occurs, in 2000, Breunig et al. proposed a density-based outlier score for each data item, called Local Outlier Factor (LOF), which is a ratio between

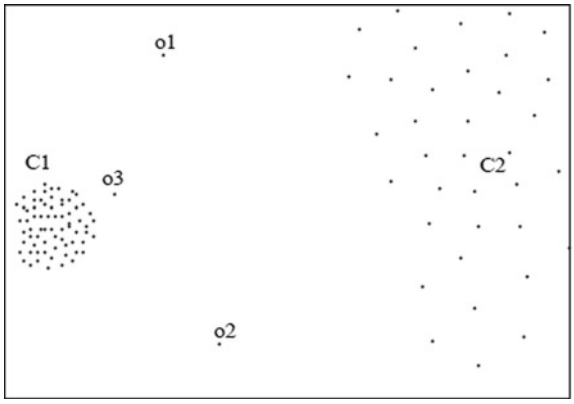


Fig. 6.1 A classic example of a local outlier

the local density of an object and the average of those of its k-nearest neighbors (Breuning et al. 2000). A LOF value of approximately 1 corresponds to objects that are located within a region of homogeneous density. The higher the LOF value of an object o is, the more distinctly is o considered an outlier. The LOF method works by first calculating the LOF value for each object, next ranking data objects by their LOF values, and, finally, returning objects with top n largest LOF values as outliers. Following the notion of local outlier, density-based outlier detection becomes an active research area and many density-based outlier detection methods have been proposed. In 2002, Tang et al. proposed a connectivity-based outlier factor (COF) to deal with "isolativity" of outliers (Tang et al. 2002). In 2003, Papadimitriou et al. proposed a multi-granularity deviation factor (MDEF) based local outlier detection schema called Local Outlier Integral (LOCI) (Gibbons et al. 2003). In 2004, Sun and Chawla proposed a spatial local outlier measure called SLOM (Sun and Chawla 2004). Taking the union of a data point's k-nearest neighbors and its reverse nearest neighbors (RNN) into consideration, Jin et al. proposed an INFLuenced Outlierness (INFLO) method in 2006 (Jin et al. 2006). For detecting outliers in scattered datasets, in 2009, Zhang et al. proposed a Local Distance-Based Outlier Factor (LDOF) (Zhang et al. 2009), which is the ratio of the average of distances from a data point to its k-nearest neighbors over the average of pairwise distances among these $k + 1$ data points. In 2013, Huang et al. proposed a rank-based outlier detection (RBDA) approach, which eliminates the problem of density calculation in the neighborhood of a point but with a high computation cost (Huang et al. 2013). In 2014, Schubert et al. proposed a method called KDEOS, which uses a z-score of local density estimate (Schubert et al. 2014; Terrell and Scott 1992). In 2016, Xiaohu et al. proposed the NR (Normalized Residual-based) outlier detection algorithm based on the concepts of KDE and CFAR (Ru et al. 2016). In 2017, Tang and He proposed a relative density-based outlier detection algorithm (RDOS) (Tang and He 2017). The focus of this algorithm is on the determination of relevant points, including not only kNN but also RNN and SNN (shared nearest neighbors).

6.3 The Proposed Efficient Linkage Clustering Algorithms

6.3.1 A Simple Idea

From the study presented in the previous section, we have obtained the following observations. First of all, to generate the dendogram tree, there is no need to compute all pairwise distances and, sometimes, finding k-nearest neighbors is enough for linkage clustering to merge in-cluster data objects. To measure the distance of two clusters to merge, only the boundary points contribute most to the distance cost function and must be involved, and boundary point detection methods come in handy. Therefore, for inliers, a sampling process is performed based on k-nearest neighbors to reduce the number of data objects which take part in linkage clustering.

Second, without loss of generality, some random noise often exits in the data, which is not good for clustering. In other words, we propose to denoise in some way in the first place before clustering can proceed so that, after denoising, we are left with only the well-separated data points. An illustration of this situation is shown in Fig. 6.2. In the figure, the blue dots represent the noisy outliers, the red colored symbols denote the boundary points, and the green dots denote the core data points or inliers.

Thirdly, both outliers and boundary points locate at the margin of clusters, that is, both outliers and boundary points stay in sparse regions. To prevent the false positives from happening, there must be some ways to differentiate between boundary points and outliers existing in a dataset in the first place. To do so, as a first-degree approximation, the outlier factors of data points within a cluster can be assumed to follow a Normal distribution and the corresponding mean and standard deviation can thus be calculated. The sum of the mean and several standard deviations can be used to differentiate between normal data objects, boundary points, and outliers.

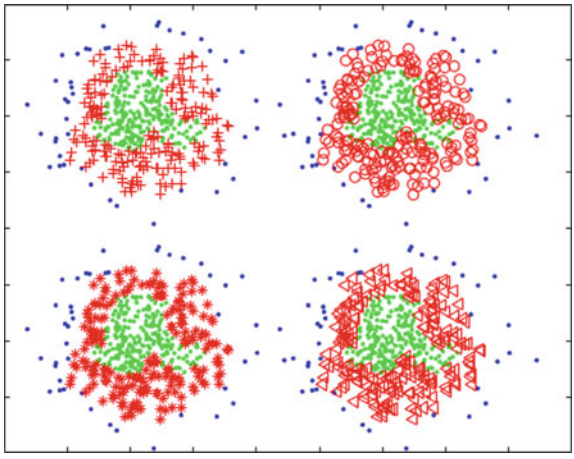


Fig. 6.2 An illustration of noisy outliers, boundary points, and inliers

For the sample dataset shown in Fig. 6.2, the blue dots denote those data points whose outlier factors are large than Threshold2 obtained as the sum of the mean and three standard deviations, and the red ones denote the data points whose outlier factors are larger than Threshold1 obtained as the sum of mean and one standard deviation and smaller than Threshold2.

$$\text{Threshold1} = \text{mean} + \text{std} \tag{6.6}$$

$$\text{Threshold2} = \text{mean} + 3 \times \text{std} \tag{6.7}$$

Based on these observations, the working ideas behind our effective and efficient linkage clustering algorithm are formularized in the following subsections.

6.3.2 Improved Linkage Algorithms

Definition 1 (k-Distance of an object p). For any positive integer k, the k-Distance of object p, denoted as k-Distance(p), is defined as the dist(p, o), or simply, $d(p, o)$, between p and an object $o \in D$ such that:

(1) for at least k objects $o' \in D\backslash\{p\}$, $d(p, o') <= d(p, o)$;
(2) for at most $k - 1$ objects $o' \in D\backslash\{p\}$, $d(p, o') < d(p, o)$.

Definition 2 (k-Nearest Neighbors of an object p). For any positive integer k, given k-Distance(p), k-nearest neighbors of p contain the first k closest objects whose distance from p is not greater than k-Distance(p), denoted as $\text{kNN}_{k\text{-Distance}(p)}(p)$, for which, kNN($p$) is used as shorthand.

Given N objects having d variables each should be grouped into k ($k < N$) clusters, where k is assumed to be given, the proposed algorithm is composed of the following two steps.

Step 1: (Obtain the size reduced dataset and cluster it)

1.1. Calculate k-nearest neighbors for all the objects based on the chosen dissimilarity measure (the Euclidean distance in this case).
1.2. Calculate a density-based outlier factor, v_j, for object j.
1.3. Calculate two thresholds according to Eqs. (6.6) and (6.7).
1.4. For boundary points, sort v_j's in an ascending order and select data objects whose outlier factor is larger than Threshold1 and smaller than Threshold2.
1.5. For inliers, calculate a distance-based outlier factor, dbv_j, for object j. Sort dbv_j's in an nondecreasing order, select in this order the data object (with the smallest value and whose density-based outlier factor is below Threshold1), retain it into a size-reduced subset and remove from the dataset those of its k-nearest neighbors which are neither noisy outliers nor boundary points. Repeat this process for the next such data point in order until none is left.

1.6. Obtain the initial clustering result by applying the linkage clustering algorithms to the size-reduced subset of data objects.

Step 2: (Assign a cluster label to each unclassified object)

2.1. Assign each object a label with the majority of its k nearest neighbors' labels and obtain the clustering result.

To summarize, the numerical parameters the algorithm needs from the user include the data set, the number of nearest neighbors to keep for each data item, while the outputs will be a label array that remembers the cluster label each data item belongs to. Physically, the resource consumed by our algorithm includes the space for holding the whole data set in memory, space to store their k-nearest neighbors, and some space for the outlier scores. To improve the readability, the proposed improved linkage algorithm is presented in a pseudo code format in Table 6.2, and the time complexity analysis is presented in the following.

6.3.3 Time Complexity Analysis

From the description in the previous sections, it can be seen that our algorithm mainly consists of two phases. In the first phase, kNN for each data point is first computed and the corresponding outlier factor is calculated for every data point to identify the relatively small number of boundary points and outliers. Next, a size reduced subset of the data points is located and clustered by standard linkage algorithms. In the second phase, a nearest neighbor classifier is used to assign a cluster label to the rest data points. It can be seen that the quality of the proposed clustering algorithm depends on the efficiency of fast k-nearest neighbors search. With the help of some index structures (Yu et al. 2001; Jagadish et al., 2005; Wang et al. 2013), we expect the first stage of the proposed clustering algorithm to run with an $O(N\log N)$ average case time complexity (although its worst time complexity still could be $O(N^2)$) and $O(2kN)$ memory. We expect the second stage of the proposed approach to run with an $O(M^2\log M)$ time and $O(M^2)$ memory, where M is the number of data points sampled in the first stage. Therefore, the whole time complexity could be $(N\log N + M^2\log M)$ although the worst case time complexity of our proposed linkage clustering algorithms could still be quadratic.

6.4 A Performance Study

In this section, we present the results of experiments conducted to evaluate the proposed methodology. First, we evaluate the performance of the proposed linkage clustering algorithms on several real small low-dimensional UCI data sets, and compare it with that of the standard complete linkage clustering algorithm. Second, we evaluate the proposed linkage clustering algorithms on a real high dimensional large

Table 6.2 The proposed efficient linkage algorithm

Input:	*data,* a set of *N d*-dimension data points, *k,* the number of nearest neighbors
Output:	*C*, a set of *K* clusters
Begin	
1:	for each point p of data
2:	find *k*NN of p; db[p] ← find distance-based outlier factor for p;
3:	end for
4:	for each point p of data
5:	lof[p] ← find local outlier factor for p;
6:	end for
7:	Threshold-1 = mean(lof)+std(lof);
8:	Threshold-2 = mean(lof)+3*std(lof);
9:	dataNew=[]; indexNew=[];
10:	for each point p of data
11:	if (lof[p]>threshold-1) && (lof[p]<threshold-2)
12:	indexNew=[indexNew; p];
13:	end if
14:	end for
15:	[dbv, dbi]=sort(db);//sort db in a nondecreasing order
16:	Done[1:N]=false;
17:	for i=1:N
18:	t=db[i];
19:	if ((lof[t]<= threshold-1) && (Done[t]==0))
20:	for j=1:k
21:	if (lof[t.jNN]) <= threshold-1)
22:	Done[t.jNN]=1;
23:	end if
24:	end for
25:	indx = [indx;t];
26:	end if
27:	end for
28:	indx=sort(indx);
29:	leni=length(indx);
30:	for i=1:leni
31:	dataNew = [dataNew;data[indx[i]]];
32:	end for
33:	T=HC(dataNew);//apply HC to dataNew for label T;
34:	labelFinal←{0};

(continued)

Table 6.2 (continued)

35:	for i=1: indexNew.size()			
36:		labelFinal(indexNew(i)) = T(i);		
37:	end for			
38:	for j=1:k			
39:		for each point p of data		
40:			if (labelFinal[p]>0) continue;	
41:			if (labelFinal[p.jNN]>0)	
42:				labelFinal[p]= labelFinal[p.jNN]
43:			end if	
44:		end for		
45:	end for			
46:	**return** labelFinal;			
End				

q.kNN*is the kth smallest distance of q*
Distance(q, p)calculates the distance between q and p

data sets using LOF-based outlier detection to check the technical robustness of this study. Finally, the clustering results obtained in the second experiment are also compared to those of the standard linkage clustering algorithms. The results show that, overall, the proposed linkage clustering algorithms are feasible for large scale datasets. We implemented all the algorithms in Matlab. All the experiments were performed on a computer with Intel Core 2 Duo Processor E6550 2.33 GHz CPU and 2 GB RAM. The operating system running on this computer is Windows 10.

6.4.1 Performance on Small Real Data

In this subsection, the clustering performance of the proposed approaches is evaluated on eight small real datasets obtained from UCI machine learning repository. All the data sets used are briefly summarized in Table 6.3. The effectiveness of the proposed algorithms employing six different outlier definitions are compared with standard complete linkage clustering algorithm. For this comparison, we would like to show that the proposed clustering methodology is competent with classic linkage clustering algorithms in the classification accuracy.

Being one of the most popular agglomerative clustering methods, complete link clustering is very popular and successful in a wide variety of applications (King 1967; Rieck et al. 2011; Ghaemmaghami et al. 2012; Cole et al. 2013). As this method takes the cluster structure into consideration, it is non-local in behavior and generally obtains compact shaped clusters. Therefore, in this set of experiments, the complete linkage clustering is chosen and its performance using the proposed

Table 6.3 The sets of data

Data name	Size	# of attributes	# of classes
banknote	1372	5	2
haberman	306	3	2
hayes-roth	160	5	3
ionosphere	351	34	2
iris	150	4	3
waveform210	5000	21	3
wdbc	569	30	2
wine	178	13	3

Table 6.4 Performance evaluation by complete linkage in Rand

Data name	Complete	cfar	cof	inflo	ldof	lof	rdos
banknote	0.5426	**0.7315**	**0.5666**	**0.5564**	**0.5661**	**0.7227**	**0.5645**
haberman	0.6126	0.5418	0.5026	0.5498	0.514	0.6095	0.6095
hayes-roth	0.548	0.4929	**0.5545**	0.5453	**0.5508**	0.503	**0.5536**
ionosphere	0.5677	**0.5882**	0.5556	**0.606**	0.5412	0.5428	0.5481
iris	0.8368	**0.8679**	**0.9055**	**0.8923**	**0.8923**	**0.8415**	**0.8415**
segmentation210	0.6782	**0.8**	**0.7596**	**0.8139**	**0.8172**	**0.7801**	**0.783**
wdbc	0.552	**0.7342**	**0.6726**	**0.7671**	**0.7963**	**0.7832**	**0.8103**
wine	0.7145	**0.7312**	0.657	0.687	0.7104	0.6879	0.6799

The bold numbers in the tables indicate that the corresponding index value obtained using an outlier definition is larger than that of the complete linkage clustering results

methodology on each of eight UCI datasets by six outlier definitions is evaluated using four popular external clustering validity indices (defined to measure the degree of correspondence between a clustering result and the prespecified partitions): Rand (Milligan et al. 1983), Adjusted-Rand (Theodorodis and Kouttoumbas 2009), Jaccard coefficient (Warrens 2008), and Fowlkes and Mallows (FM) (Yeung et al. 2001). The local neighborhood, k, is set to 10 for all the datasets in this set of experiments. The results in comparison with the standard complete linkage algorithm are shown in Tables 6.4, 6.5, 6.6 and 6.7 for the four indices, respectively.

6.4.2 Performance on Image Data

In this experiment, our datasets consist of feature vectors extracted from 35 pictures taken around an outdoor environment using SONY digital video camera recorder. To obtain the feature vectors, the images are first converted from RGB color space into HSV color space. Then, for each image, a moving window of size 27×27 is shifted by 18 pixels in the row and column directions of the image to obtain image patch based color histograms. The chromaticity H, the saturation S and the brightness V

Table 6.5 Performance evaluation by complete linkage in Jaccard

Data name	complete	cfar	cof	inflo	ldof	lof	rdos
banknote	0.491	**0.585**	**0.495**	0.4772	0.4898	**0.5722**	0.4824
haberman	0.5853	0.4286	0.3881	0.4529	0.4362	**0.6043**	**0.5992**
hayes-roth	0.2028	**0.2654**	**0.219**	**0.2136**	**0.2044**	**0.2522**	**0.2101**
ionosphere	0.5035	0.435	**0.5386**	0.4702	**0.5378**	**0.5378**	**0.5379**
iris	0.6223	**0.6774**	**0.753**	**0.7248**	**0.7248**	**0.6296**	**0.6296**
segmentation210	0.1993	**0.2769**	**0.2222**	**0.3054**	**0.3024**	**0.2899**	**0.2894**
wdbc	0.5312	**0.5504**	0.4712	**0.605**	**0.6679**	**0.6797**	**0.7032**
wine	0.4179	**0.4454**	0.3592	**0.4791**	0.4011	**0.4833**	**0.4339**

The bold numbers in the tables indicate that the corresponding index value obtained using an outlier definition is larger than that of the complete linkage clustering results

Table 6.6 Performance evaluation by complete linkage in adjusted Rand

Data name	complete	cfar	cof	inflo	ldof	lof	rdos
banknote	0.0781	**0.4628**	**0.1277**	**0.1076**	**0.1269**	**0.4452**	**0.1241**
haberman	0.0745	0.0688	−0.0032	0.0643	−0.0301	0.0143	0.0278
hayes-roth	−0.0057	**0.0033**	**0.0178**	**0.002**	−0.0008	−0.0021	**0.0081**
ionosphere	0.0985	**0.1755**	0.047	**0.2053**	0.0089	0.0135	0.0275
iris	0.6423	**0.7074**	**0.788**	**0.7592**	**0.7592**	**0.6516**	**0.6516**
segmentation210	0.168	**0.319**	**0.228**	**0.3609**	**0.3587**	**0.3278**	**0.3282**
wdbc	0.0521	**0.476**	**0.3551**	**0.5393**	**0.593**	**0.5607**	**0.6179**
wine	0.3708	**0.4099**	0.2612	**0.394**	0.3536	**0.3984**	0.3483

The bold numbers in the tables indicate that the corresponding index value obtained using an outlier definition is larger than that of the complete linkage clustering results

Table 6.7 Performance evaluation by complete linkage in FM

Data name	Complete	cfar	cof	inflo	ldof	lof	rdos
banknote	0.6794	**0.7382**	0.6776	0.6585	0.6713	**0.7279**	0.6628
haberman	0.7503	0.6012	0.561	0.6236	0.6076	**0.7742**	**0.7678**
hayes-roth	0.3372	**0.429**	**0.3593**	**0.3521**	**0.3395**	**0.4087**	**0.3473**
ionosphere	0.6806	0.6065	**0.7281**	0.6397	**0.732**	**0.7314**	**0.7298**
iris	0.7686	**0.8084**	**0.8592**	**0.8407**	**0.8407**	**0.7739**	**0.7739**
segmentation210	0.3671	**0.4441**	**0.3771**	**0.4782**	**0.4727**	**0.472**	**0.4698**
wdbc	0.7214	0.7192	0.6497	**0.7597**	**0.8015**	**0.811**	**0.8259**
wine	0.5897	**0.6167**	0.5299	**0.6672**	0.5725	**0.6723**	**0.6137**

The bold numbers in the tables indicate that the corresponding index value obtained using an outlier definition is larger than that of the complete linkage clustering results

Table 6.8 The set of data

Data name	Size	# of attributes	# of nearest neighbors k
Imagedata1	95,550	50	50
Imagedata2	95,550	100	50
Imagedata3	95,550	250	50

are evenly divided into u, v and w intervals, respectively. All the interval combination can generate $u \times v \times w$ different color bins for the generation of color histogram for each image patch. Then, all the pixels in the moving window are assigned to the corresponding possibility, and a sparse feature vector (color histogram) is obtained. The datasets used in this experiment are briefly summarized in Table 6.8.

The clustering performances are presented in Figs. 6.3, 6.4 and 6.5 for feature vectors of three different dimensions, that is, 50, 100, and 250, respectively. In order to keep the amount of data presented reasonable, five images and their segmentation results are given but consistent performance results are obtained for other images. There are five columns in each figure. The left column displays the testing images

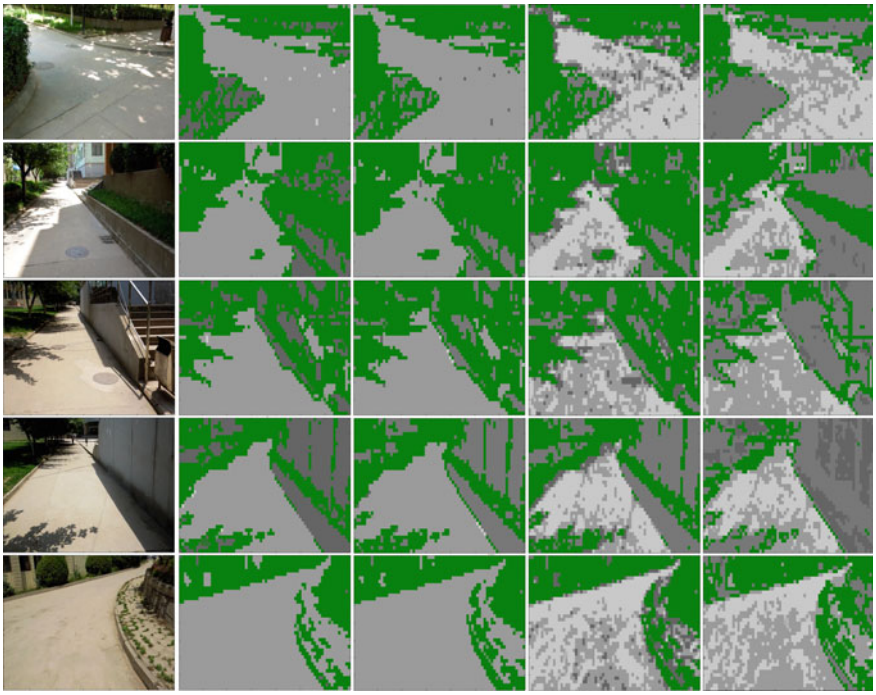


Fig. 6.3 Image data with 50 dimension. (left) original image, results of (middle left) average linkage algorithm and (middle) centroid linkage algorithm, (middle right) complete linkage algorithm (right) ward linkage algorithm

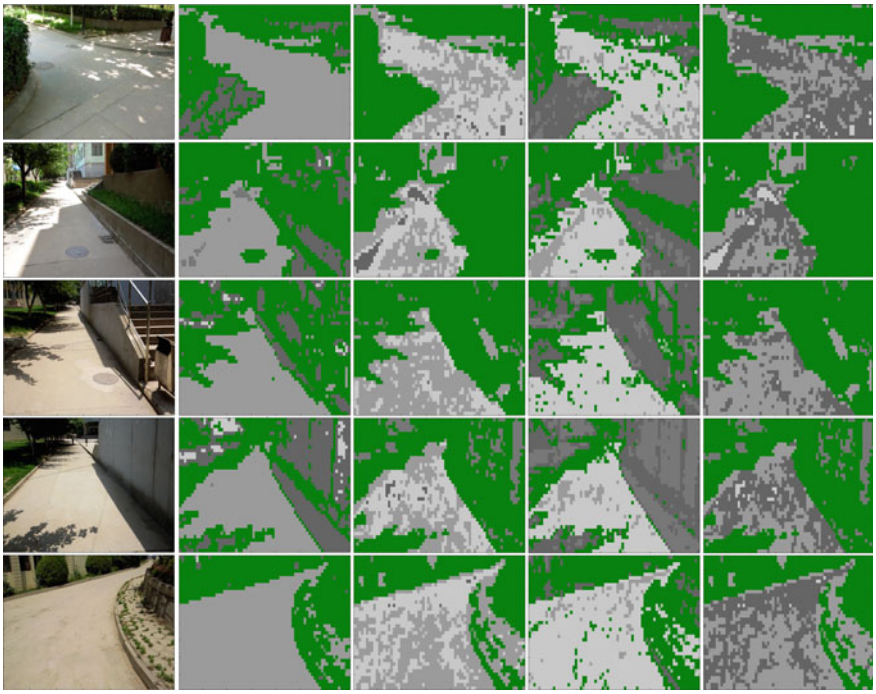


Fig. 6.4 Image data with 100 dimension. (left) original image, results of (middle left) average linkage algorithm and (middle) centroid linkage algorithm, (middle right) complete linkage algorithm (right) ward linkage algorithm

used. The middle left column displays the clustering results of the average linkage clustering using the proposed methodology. The middle column displays the clustering results of the centroid linkage clustering using the proposed methodology. The middle right column displays the clustering results of the complete linkage clustering using the proposed methodology. Finally the last (right) column displays the clustering results of the ward linkage clustering using the proposed methodology. To ease visual examination, in the segmented images, a color is chosen to denote the percepts. To display results from the four different methods, twelve close but slightly different sets of colors are used. The number of clusters is set to 5 for all the methods. The results show that different methods have different optimal segmenting results.

From the original images, we can see that the major percepts include grey concrete road occupying most lower-half part or lower left-half part of images and extending into the far end background, light grayish white concrete buildings and their shadows to the right of road in the second, the third and fourth scenes, concrete building steps on the right part of the third scene, manhole covers on the concrete road in the second and third images, wide brick pavement on the right of the second scene, several dark green trees and light green grass mingled on the left and in the far end of the first scene, several dark green trees (with shadows) on the left of the second, third and

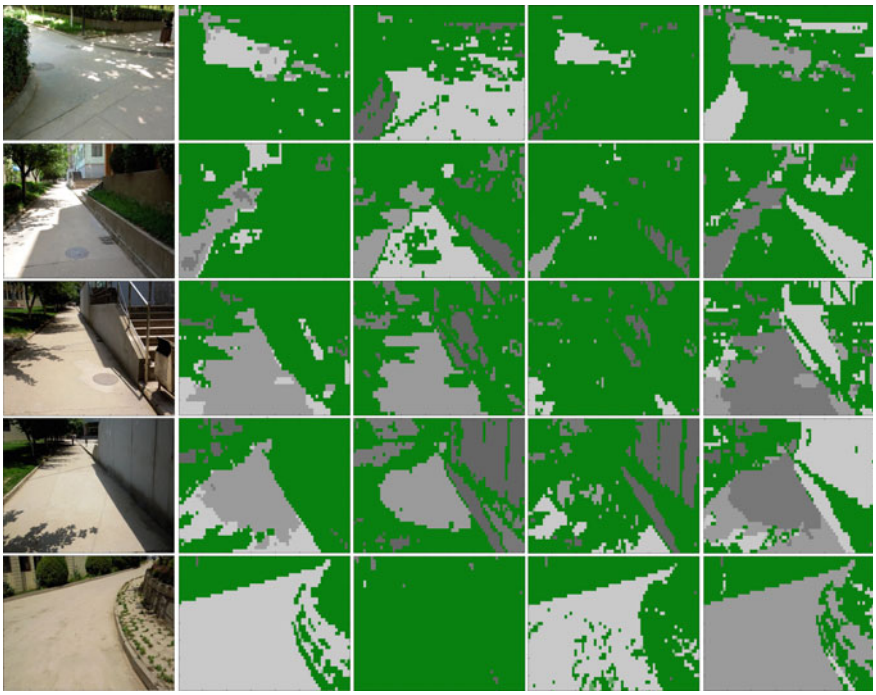


Fig. 6.5 Image data with 250 dimension. (left) original image, results of (middle left) average linkage algorithm and (middle) centroid linkage algorithm, (middle right) complete linkage algorithm (right) ward linkage algorithm

fourth scenes, and several dark green trees in the far end and green grass on the right side of the last scene, a building on the right of the second, third and fourth scenes, and a little garbage can standing at the upper-right and lower-right corner of the first and third scenes. In this segmentation task, the concrete road together with the manhole covers, the green trees and grass and their shadows, and the concrete building are the most discriminative foreground patterns that can be easily captured and separated from each other and from the cluttered background.

From the segmented figures, it can be seen that typical stable percepts detected by our method includes a bundled dark green percept (corresponding to dark green trees and their shadows, green grass and building steps), concrete building walls, concrete road, and manhole covers.

For this outdoor environment, it is a little harder to segregate the more detailed percepts bundled into dark green. From the segmented images as shown in Fig. 6.3, it can be seen that the average and centroid linkage algorithms correctly identifies the foreground road, the building steps, and the background dark green trees, and it is in this sense that these two methods perform better than the other two which can clearly oversegment the concrete road into more percepts.

From the segmented images as shown in Fig. 6.4, it is clearly seen that, for image data of dimension 100, only the average linkage algorithm does a better job (even on the little things such as the manhole covers) than the other three algorithms which oversegment some percepts while undersegment others. From the segmented images as shown in Fig. 6.5, it can be seen that no linkage algorithms work reasonably well on all images for this image data of dimension 250, although the average and ward linkage algorithms both do a good job on the last scene. To summarize, for this image data, the average linkage algorithm works the best. Further, as the dimensions increase, the performance of the average linkage algorithm becomes better and can discover all the major percepts most of time and discriminate percepts that other linkage methods can not. However, when the dimension further increases, percepts are bundled together in the feature space, and the linkage clustering algorithms can not separate them properly and the performance degrades.

6.4.3 Performance Comparison

In this subsection, more experiments are conducted to compare the performance of the proposed methodology for hierarchical linkage algorithms with the standard linkage clustering algorithms. More specifically, for this comparison, standard linkage algorithms are applied to feature vectors of 100 dimensions extracted from each image for single image segmentation. In this set of experiments, the number of clusters as an input to the clustering algorithms vary from 2 to 5. The best results obtained for five test images are presented in Fig. 6.6. From the figure, it can be seen that all four linkage algorithms have similar performance and work reasonably well. Some of the results in this round of experiments are comparable to those of the proposed methodology for large scale datasets consisting of feature vectors extracted from all 35 images.

Overall, the results presented above offer good evidence that the proposed methodology works reasonably well on a variety of images from this outdoor domain.

6.5 Conclusions

We have proposed an efficient hierarchical linkage clustering algorithms and applied it to various clustering tasks. Basically, our algorithm rely on k-nearest neighbors based sampling and can offer comparable clustering performance by accessing only a relatively small number of data objects in the clusters and around the boundaries. Performance evaluation conducted on various real datasets show that the proposed algorithm significantly reduces the time and space expenses of the computation while retaining good control on the clustering accuracy as standard hierarchical linkage clustering can achieve.

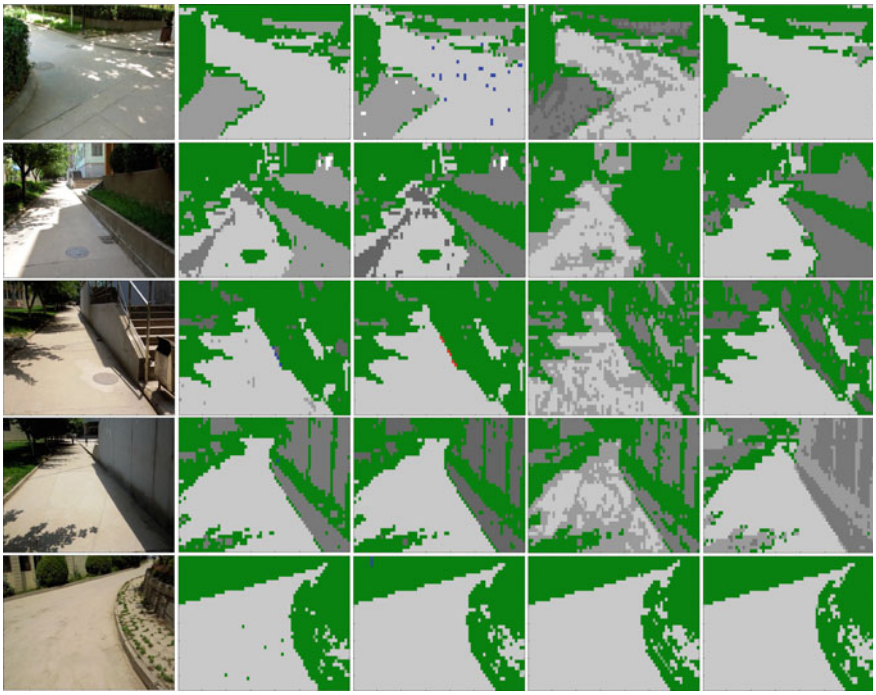


Fig. 6.6 Image data with 100 dimension. (left) original image, results of (middle left) average linkage algorithm and (middle) centroid linkage algorithm, (middle right) complete linkage algorithm (right) ward linkage algorithm

References

Aggarwal, C. C., & Yu, P. S. (2001). Outlier detection for high dimensional data. *ACM Sigmod Record, 30,* 37–46.

Angiulli, F., & Pizzuti, C. (2002). Fast outlier detection in high dimensional spaces. In *Proceedings of the Sixth European Conference on the Principles of Data Mining and Knowledge Discovery* (pp. 15–26).

Breuning, M.M., Kriegel, H.P., Ng, R.T., & Sander, J. (2000). LOF: Identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data* (*SIGMOD*'00) (pp. 93–104).

Cole, J. R., Wang, Q., Fish, J. A., Chai, B., McGarrell, D. M., Sun, Y., et al. (2013). Ribosomal database project: Data and tools for high throughput rRNA analysis. *Nucleic Acids Research, 42(D1),* D 633–D 642.

Defays, D. (1977). An efficient algorithm for a complete link method. *The Computer Journal. British Computer Society, 20*(4), 364–366.

Ding, X., Li, Y., Belatreche, A., & Maguire, L. (2014). A locally adaptive boundary evolution algorithm for novelty detection using level set methods. In *Proceedings of the 2014 IEEE International Joint Conference on Neural Networks* (*IJCNN*'14) (pp. 1870–1876).

Ding, X., Li, Y., Belatreche, A., & Maguire, L. P. (2015). Novelty detection using level set methods. *IEEE Transactions on Neural Networks and Learning Systems, 26*(3), 576–588.

Flynn, P. J., Jain, A. K., & Murty, M. N. (1999). Data clustering: A review. *ACM Computing Surveys, 31*(3), 264–323.

Ghaemmaghami, H., Dean, D., Vogt, R., & Sridharan, S. (2012). Speaker attribution of multiple telephone conversations using a complete-linkage clustering approach. In *Proceedings of the 2012 IEEE International Conference on Acoustics, Speech and Signal Processing* (*ICASSP*'12) (pp. 4185–4188).

Gibbons, P.B., Papadimitriou, S., Kitagawa, H., & Faloutsos, C. (2003). LOCI: Fast outlier detection using the local correlation integral. In *Proceedings of the IEEE 19th International Conference on Data Engineering*, Bangalore, India (pp. 315–328).

Hawkins, D. M. (1980). *Identification of outliers. Monographs on applied probability and statistics*. London: Chapman and Hall.

Huang, H., Mehrotra, K., & Mohan, C. K. (2013). Rank-based outlier detection. *Journal of Statistical Computation and Simulation, 83*(3), 1–14.

Jagadish, H. V., Chin Ooi, B., Tan, K. L., Yu, C., & Zhang, R. (2005). iDistance: An adaptive B + -tree based indexing method for nearest neighbor search. *ACM Transactions on Data Base Systems(ACM TODS), 30*(2), 364–397.

Jin, W., Tung, A.K.H., Han, J., & Wang, W. (2006). Ranking outliers using symmetric neighborhood relationship. In *Proceedings of the 10th Pacific-Asia Conference on Knowledge Discovery and Data Mining* (PAKDD), Singapore (Vol. 3918, pp. 577–59).

King, B. (1967). Step-wise clustering procedures. *Journal of the American Statistical Association, 62*(317), 86–101.

Knorr, E.M., & Ng, R.T. (1998). Algorithms for mining distance-based outliers in large datasets. In *Proceedings of the 24th Very Large Databases Conference* (*VLDB*'98), New York, USA (pp. 392–403).

Kutsuna, T., & Yamamoto, A. (2014). Outlier detection based on leave-one out density using binary decision diagrams. In V. S. Tseng, T. B. Ho, & Z.-H. Zhou (Eds.), *Advances in knowledge discovery and data mining* (pp. 486–497). Berlin: Springer.

Li, L., Lv, J., & Yi, Z. (2015a). A non-negative representation learning algorithm for selecting neighbors. *Machine Learning, 102,* 133–153.

Li, X., Lv, J.C., & Cheng, D. (2015b). Angle-based outlier detection algorithm with more stable relationships. In *Proceedings of the 18th Asia Pacific Symposium on Intelligent and Evolutionary Systems* (Vol. 1). Springer.

Li, Y. (2008). A surface representation approach for novelty detection. In *Proceedings of International conference on information and automation*(ICIA) (pp 1464–1468).

Li, Y. (2011). Selecting training points for one-class support vector machines. *Pattern Recognition Letters, 32*(11), 517–1522.

Li, Y., & Maguire, L. P. (2011). Selecting critical patterns based on local geometrical and statistical information. *IEEE Trans Pattern Analysis and Machine Intelligence, 33*(6), 1189–1201.

Milligan, G. W., Soon, S. C., & Sokol, L. M. (1983). The effect of cluster size, dimensionality, and the number of clusters on recovery of true cluster structure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 5, 40–47.

Qiu, B., & Cao, X. (2016). Clustering boundary detection for high dimensional space based on space inversion and hopkins statistics. *Knowledge Based System, 98(C),* 216–225.

Rieck, K., Trinius, P., Willems, C., & Holz, T. (2011). Automatic analysis of malware behavior using machine learning. *Journal of Computer Security, 19*(4), 639–668.

Ramaswamy, S., Rastogi, R., & Shim, K. (2000). Efficient algorithms for mining outliers from large data sets. In *Proceedings of the 2000 ACM SIGMOD Conference* (pp. 427–438).

Ru, X., Liu, Z., Huang, Z., et al. (2016). Normalized residual-based constant false-alarm rate outlier detection. *Pattern Recognition Letters, 69,* 1–7.

Schubert E, Zimek A, & Kriegel, H.P. (2014). Generalized outlier detection with flexible kernel density estimates. In *Proceedings of the 2014 Siam International Conference on Data Mining*.

Sibson, R. (1973). SLINK: An optimally efficient algorithm for the single-link cluster method. *The Computer Journal. British Computer Society, 16*(1), 30–34.

Sun, P., & Chawla, S. (2004). On local spatial outliers. In *Proceedings of the 4th International Conference on Data Mining* (*ICDM*'04), Brighton, UK (pp. 209–216).

Tang, J., Chen, Z., Fu, A.W.C., & Cheung, D.W. (2002). Enhancing effectiveness of outlier detections for low density patterns. In *Proceedings of the 6th Pacific-Asia Conference on Knowledge Discovery and Data Mining* (*PAKDD*'02), Taipei, Taiwan (Vol. 2336, pp. 535–548).

Tang, B., & He, H. (2017). A local density-based approach for outlier detection. *Neurocomputing, 241,* 171–180.

Terrell, G. R., & Scott, D. W. (1992). Variable Kernel density estimation. *Annals of Statistics, 20*(3), 1236–1265.

Theodorodis, S., & Kouttoumbas, K. (2009). *Pattern Recognition* (4th ed.). Amsterdam: Academic Press.

Wang, X., Wang, X. L., Chen, C., & Wilkes, D. M. (2013). Enhancing minimum spanning tree-based clustering by removing density-based outliers. *Digital Signal Processing, 23*(5), 1523–1538.

Warrens, M. J. (2008). On the equivalence of Cohen's kappa and the Hubert-Arabie adjustedrand index. *Journal of Classification, 25*(2), 177–183.

Xia, C., Hsu, W., Lee, M. L., & Ooi, B. C. (2006). Border: Efficient computation of boundary points. *IEEE Transactions on Knowledge Data Engineering, 18*(3), 289–303.

Yeung, K. Y., Fraley, C., Murua, A., & Raftery, A. E. (2001). Model-based clustering and data transformations for gene expression data. *Bioinformatics, 17*, 977–987.

Yu, C., Chin Ooi, B., Tan, K.L., & Jagadish, H.V. (2001). Indexing the distance: an efficient method to KNN processing. In *Proceedings of the 27th International Conference on Very large Databases* (*VLDB*'01), Roma, Italy (pp. 421–430).

Zhang, K., Hutter, M., & Jin, H. (2009). A new local distance-based outlier detection approach for scattered real-world data. *Advances in Knowledge Discovery and Data Mining, 5476,* 813–822.

Chapter 7
A New Fast *K*-Nearest Neighbors-Based Clustering Algorithm



Abstract Density-based clustering algorithms are well known for identifying clusters of very different local densities and existing in different regions of data space. However, the parameters required by most popular density-based clustering algorithms, such as DBSCAN, are hard to determine while having significant impacts on the clustering results. In this paper, we present a new density-based clustering algorithm in which the selection of appropriate parameters is less difficult but more meaningful. Experimental results show the effectiveness and the efficiency of the proposed approach.

Keywords Density-based clustering · *K*-nearest neighbors · Centroid of *k*-nearest neighbors · Fast *k*-nearest neighbor search · Indexing structures

7.1 Introduction

Being an important branch of data mining techniques, clustering has found numerous applications and many clustering algorithms have been introduced in the past few decades, including partition-based clustering (Bereta and Burczynski 2009; Chiang et al. 2011), density-based clustering (Ester et al. 1996a, b; Hinneburg and Keim 1998a, b), grid-based clustering (Agrawal et al. 1998a, b; Wang et al. 1997), graph-based clustering (Zahn 1971; Lee et al. 2008) and hierarchical clustering (Chaoji et al. 2008, 2011), etc. By taking neighborhood characteristics into account, density-based clustering algorithms count to be the most advanced and robust approach and embody no principle with clearly defined algorithmic properties. To find clusters of different shapes, size, and densities, recent efforts focus on their effectiveness and ability to discover clusters of increasingly complex shapes (Chaoji et al. 2008, 2011), and the scalability to the size of the large datasets to be clustered (Stein and Busch 2005).

Compared with other clustering algorithms, traditional density-based clustering algorithms using only global parameter settings have difficulty revealing the skewed distribution often associated with modern large multidimensional real-world data sets, as illustrated in the left plot of Fig. 7.1. The situation as shown in the middle plot of Fig. 7.1 is referred to in this chapter as density-separated clustering problems.

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_7

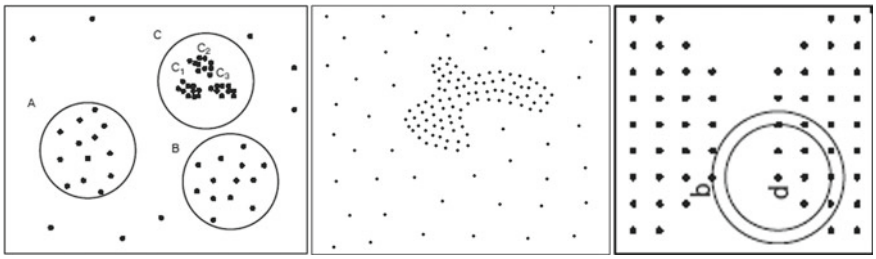


Fig. 7.1 (Left) Clusters with skewed density distributions; (middle) a density separated clustering problem; (Right) a distance separated clustering problem

In this illustration, there are two kinds of points near the boundary, those whose k nearest neighbors are all data points in the same cluster and therefore have similar densities, and those which reside on the touching side of two clusters of different densities and, therefore, whose k nearest neighbors come from different clusters and may have different densities. Further, the values for input parameters required by modern density-based clustering algorithms are usually difficult to determine, and, for slightly different parameter settings, very different partitions of the data set can be produced, as manifested in the right plot of Fig. 7.1, which is referred to in the following as distance separated clustering problems. In this case, two clusters of similar density exist. If, for example, k is selected to be no more than 8, two clusters can be separated easily and formed by a connected component labeling approach upon k nearest neighbors. However, if k is set to be 16, k-nearest neighbors based clustering algorithm cannot set these two clusters apart.

Motivated by these limitations, in this chapter, we propose a new algorithm for local-density-based cluster analysis. In this new algorithm, the concept of k-nearest neighbors based centroid for each object in a data set is introduced whose distance to the data object is used to represents the degree of how well a data point is surrounded by others, and is next used to determine a suitable value for k which is subsequently used to detect clusters in a data set. The next key challenge our work needs to meet is how to define a density measure for data points residing in clusters of different densities. For this matter, we propose the concept of the first neighborhood of a data point in this chapter.

In the following, we first provide a brief discussion of some related work on density-based clustering in Sect. 7.2. We then present the concept of k-nearest neighbors based centroid and the proposed clustering strategy in Sects. 7.3 and 7.4, respectively. In Sect. 7.5, a performance evaluation of the effectiveness of the proposed algorithm is conducted. Finally, conclusions are made in Sect. 7.6.

7.2 Existing Literature on Density-Based Clustering

Being a very popular branch in data clustering, density-based clustering algorithms rely on a local density criterion to form clusters and can detect clusters of different shapes, size, and densities. Early density-based clustering approaches partition the data space into a number of nonoverlapping grid cells and regard cells containing a relatively large number of objects as potential cluster centers surrounded by the low-density boundary cells (Jain and Dubes 1988). The size of the cells must be specified by the user and can have direct impact on the outcome of this method. Cells of small volume can give a very "noisy" estimate of the density, whereas large cells tend to overly smooth the density estimate. Other popular grid-based algorithms include WaveClusters (Sheikholeslami et al. 1988), DENCLUE (Hinneburg and Keim 1998a, b), and CLIQUE (Agrawal et al. 1998a, b), proposed in 1988, 1998, and 1998, respectively. Recently, a new shifting grid clustering algorithm (Ma et al. 2004) was proposed in 2004 which uses the concept of shifting grid. Without requiring users inputting parameters, the algorithm is a nonparametric type, and divides each dimension of the data space into certain intervals to form a grid structure in the data space. Based on the concept of sliding window, it utilizes shifting of the whole grid structure to obtain a more descriptive density profile, and clusters data in terms of cell rather than points.

Another important branch of density-based clustering begins with DBSCAN, which was proposed by Ester et al. (1996a, b) and allows users to determine two parameters, the neighborhood of a given radius (ε) and a minimum number of points (MinPts) in the neighborhood, to complete the clustering procedure. More specifically, the algorithm decides for each point of a cluster that its neighborhood of a given radius (ε) has to contain at least a minimum number of points (MinPts) where ε and MinPts are input parameters. However, modern large real-world data sets often have a much skewed distribution that is difficult to reveal by using only one global parameter setting. For example, in the data set depicted in the left plot of Fig. 7.1, it is impossible to detect clusters A, B, C_1, C_2, and C_3 simultaneously using one global density parameter. To solve this problem, OPTICS was proposed in 1999 (Ankerst et al. 1999). Basically, it creates an augmented ordering of the dataset representing its density-based clustering structure, which contains information equivalent to the density-based clustering corresponding to a broad range of parameter settings. However, it only creates an augmented ordering of the dataset representing its density-based clustering structure instead of producing clusters of a data set explicitly. In addition, it is short at generating the overlapping clusters appropriately. To discover clusters with arbitrary shapes and densities as well as to be insensitive to noise data, a local-density-based spatial clustering algorithm with noise relying on a local-density-based notion of clusters was proposed by Duan et al. (2007). It takes advantage of the LOF (Local Outlier Factor) to detect the noises and outliers in comparison to other density-based clustering algorithms. It can discover clusters of different densities existing in different regions of data space. But it needs a user to input four parameters which have a significant influence on the clustering

result but are hard to determine. Recently, an efficient density-based clustering algorithm was proposed by Alex and Laio (2014). It works by fast search and discovery of density peaks and is referred to as CFSFDP in the following. CFSFDP is based on two assumptions. First, the central point of a cluster has higher density compared to its neighbors. Second, the cluster center is relatively far from other cluster centers compared to its local data points. CFSFDP computes a local density for each data point and a distance relative to the nearest high-density point, and has the ability to create arbitrary shaped clusters by fast search of cluster centers. The effectiveness of CFSFDP depends greatly on the accurate estimation of density and the cutoff distance which is an essential parameter for estimating the accurate density of a data point and identifying border points of a cluster.

7.3 Basic Notion of the Proposed Algorithm

From the study of the related work presented in the previous section, there are two major kinds of points in the standard density-based clustering algorithm DBSCAN, the core points, and the points which are not core points but directly local-density-reachable to the core points. The former are data points deep inside a cluster in the sense that they are well surrounded by other data items. The latter are those close to or on the border of a cluster. One of the key problems with the current density based methods is that they are very sensitive to k, that is, the number of nearest neighbors to use in the algorithms. To partially circumvent this problem, in this chapter, we propose to use the concept of k-nearest neighbors based centroid presented in the following to measure how well a data point is surrounded by others.

7.3.1 Formal Definition of KNN-Based Centroid

To represent the degree of an object being inside a cluster and the local-density of the object in a setting of k nearest neighbors, the formal definition for kNN-based centroid is introduced in the following.

Definition 1 (k-Distance of an object p). For a given data set D and a positive integer k, the k-Distance of an object $p \in D$, denoted as k-distance (p), is defined as the distance (p, o), or simply, $d(p, o)$, between p and an object $o \in D$ such that:

(1) for at least k objects $o' \in D\backslash\{p\}$, it holds that $d(p, o') \leq d(p, o)$
(2) for at most $k - 1$ objects $o' \in D\backslash\{p\}$, it holds that $d(p, o') < d(p, o)$.

Definition 2 (k-Distance neighborhood of an object p). Given the k-Distance of p, the k-Distance neighborhood of p contains every object whose distance to p is not greater than the k-Distance(p), i.e. $N_{k\text{-Distance}(p)}(p) = \{q \in D\backslash\{p\} | d(p, q) \leq k\text{-Distance}(p)\}$. These objects are called the k-Distance neighborhood of object p. For simplicity, $N_k(p)$ is used as a shorthand for $N_{k\text{-Distance}(p)}(p)$.

Definition 3 (Centroid of k-Distance neighborhood of an object p). Given a k-Distance neighborhood of an object p, the Centroid of k-Distance neighborhood of p is defined as,

$$\text{Centroid}_{N_{k-\text{Distance}(p)}}(p) = \frac{\sum_{i=1}^{k} q_i}{k} \tag{7.1}$$

where $q_i \in N_{k\text{-Distance}(p)}(p)$ and $1 \leq i \leq k$.

7.3.2 A Local-Centroid-Based Notion of Clusters

For data points well surrounded by other points, such as data point p which is deeply inside a sample 2-D dataset, shown in Fig. 7.2, we have several observations.

First of all, the distance between data point p and its centroid of 4-Distance neighborhood is zero and, in this sense, it can be said that data point p is well surrounded by its neighboring four points. And 4 is 2 times the dimension of the dataset. To summarize, if k is selected right, the distance between an object p and its Centroid of k-Distance neighborhood can capture a degree how well p is embedded in a cluster and surrounded by others and is therefore defined to be a Local Inlier Factor in the following.

Definition 4 (Local Inlier Factor (LIF) of an object p). The LIF of an object p is defined as,

$$\text{LIF}_k(p) = \text{dist}\big(p, \text{Centroid}_{N_{k-\text{Distance}(p)}}(p)\big) \tag{7.2}$$

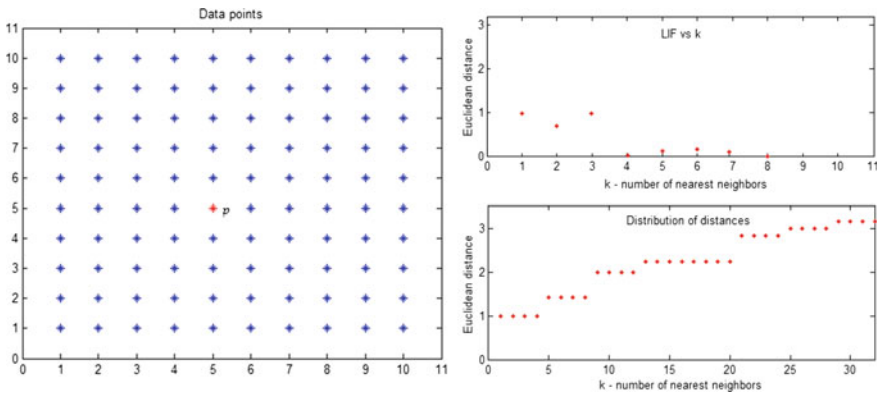


Fig. 7.2 (Left) A sample dataset and (right) the distribution of the distances of several k-Distance neighborhoods of p to p

where dist(x, y) denotes a distance measure between x and y, or simply $d(x, y)$.

Secondly, the local inlier factors of an object p, i.e., $\text{LIF}_k(p)$'s, are calculated for $k = 1, 2, 3, 4, 5, 6, 7, 8, 9$, and 10, and presented in the second plot of Fig. 7.2. It is not difficult to see from the figure that, for this ideal case, $\text{LIF}_k(p)$ can be zero for $k = 4, 8, \ldots$, etc., or $k = 2d, 4d, \ldots$, etc., where d is the dimension of the dataset. It can be imagined that, for more realistic situations, the smaller the LIF of a data object p is for some k, the further the object p is away from the cluster boundaries. Since the LIF represents the degree an object is inside a cluster, we regard object p to belong to a certain cluster if the minimum LIF value with respect to parameter k is among the smallest.

Thirdly; k-Distance neighborhoods of data object p's for $k = 4, 8, \ldots$, etc., not only have zero LIF's, but also are separated into groups of data objects in the form of several layers around data object p separated by gaps as shown in the lower right plot of Fig. 7.2. For example, 8-Distance neighborhood consists of two groups of data objects in the form of two layers, one of which is 4-Distance neighborhood. Larger such neighborhoods exhibit more complex structures. However, the inner most layer, that is, the 4-Distance neighborhood, has included all the necessary data points for all the data points in a cluster to be connected and higher layers have no more direct contributions. Therefore, we call this inner most neighborhood the first neighborhood and the corresponding distance to data point p the reachability distance, which is defined in the following.

Definition 5 (First neighborhood of an object p). Given a dataset of size N, the first neighborhood of an object p is defined to be the k-Distance of an object p where k satisfies the following two conditions:

(1) $\min\{\text{distance}(p, \text{Centrod}_{N_{k-\text{Distance}(p)}(p)})\}, k \in \{1, 2, \ldots, N\}$;
(2) $\Delta_{k+1} >> \Delta_k$, where $\Delta_k = k\text{-distance}(p)\text{-}(k-1)\text{-distance}(p)$.

Condition (1) corresponds to the first two observations in the above, that is, k is selected in such a way that $\text{LIF}_k(p)$ reaches its first minimum and the corresponding k is denoted by MinPts in the rest of this paper. Condition (2) corresponds to the third observation. For the sample dataset in the left part of Fig. 7.2, it can be easily seen that the first neighborhood corresponds to $k = 4$, instead of $k = 2$ or $k = 3$. It is in this sense that the first neighborhood (i.e., 4-nearest neighbors) can be a more accurate local density measure for clustering than k-nearest neighbors (e.g., 2-nearest neighbors or 3-nearest neighbors) based clustering.

Definition 6 (Reachability distance of an object p). Given a dataset of size N, the reachability distance of an object p is defined to be the MinPts-Distance of object p, and is denoted as reach-dist(p).

For distance-separated datasets as the one shown on the right of Fig. 7.1, the concept of first neighborhood is enough for clusters' separation. To illustrate the use of the concept of the first neighborhood in density-separated datasets, five data points, A, B, C, D, and E, are highlighted in red color in Fig. 7.3, and their corresponding first neighborhoods are highlighted in green color. It can be easily recognized that

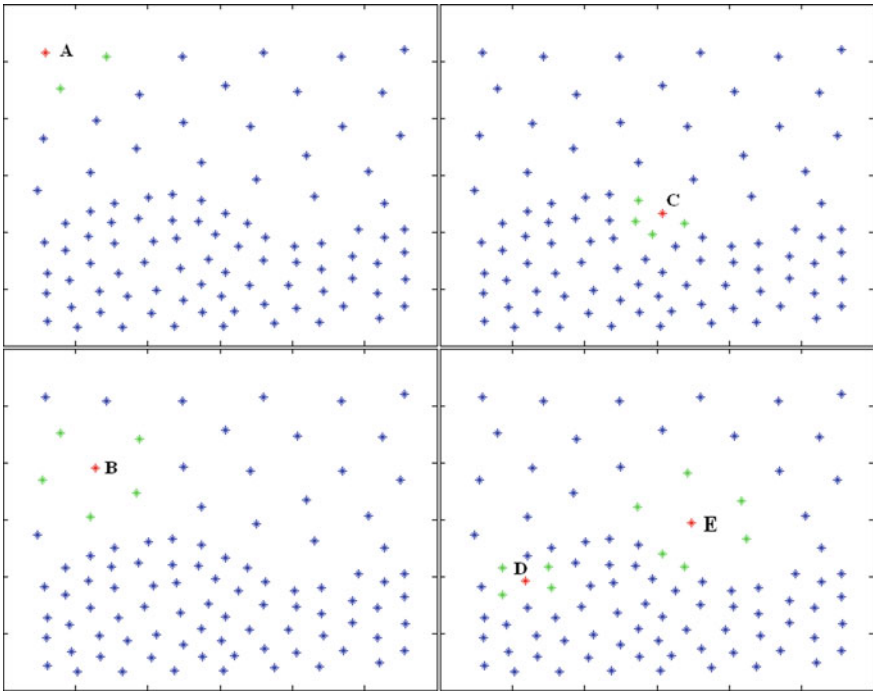


Fig. 7.3 An illustration of five different kinds of data points

A, C, and E are boundary points surrounded by two, four, and five different nearest neighbors (in green color), respectively, and that B and D are within-cluster points which are well surrounded by five and four other data points. These first neighborhoods are chosen since they result in the smallest distances between the data points and their kNN-based centroids. In other words, in the design of our centroid-based clustering algorithm, the main reason why the number of nearest neighbors is not fixed but optimized to minimize the distance between each data point and its kNN-based centroid is that, deep inside each cluster, we have a typical Local Inlier Factor (LIF) for a data point which is significantly smaller than those of the boundary points of a cluster.

In the following, this intuitive notion of "clusters" and "noise" will be formalized. The key idea is that, for any point p, if $\text{LIF}(p) \leq \text{LIFUB}$, the data point p belongs to a certain cluster and can be regarded as a core point in the density-based clustering domain. Further, if data point q is within the reachability distance of p and has a similar density value to that of p, q belongs to the same cluster as that of p.

Definition 7 (Core point). A point p is a core point w.r.t. LIFUB if $\text{LIF}(p) \leq \text{LIFUB}$, where LIFUB is a user-defined parameter.

Definition 8 (Directly local-density-reachable). A data object p is directly local-density-reachable from a data point q w.r.t. MinPts (i.e., the first neighborhood) if data point q is within the first neighborhood of object p.

Definition 9 (Local-density-reachable). A point p is local-density-reachable from the point q w.r.t. MinPts (i.e., the first neighborhood) if there is a chain of points p_1, p_2, …, p_n, $p_1 = q$, $p_n = p$ such that p_{i+1} is directly local density-reachable from p_i.

Local-density-reachability is a canonical extension of direct local-density-reachability. These two relations are transitive, but not symmetric.

Definition 10 (Local-density-connected). A point p is local-density-connected to a point q from o w.r.t. MinPts if there is a point o such that both $p$and q are local-density-reachable from o w.r.t. MinPts.

Compared to local-density-reachability, from the definition, local-density-connectivity is a symmetric relation. Now we are able to define the notion of cluster as a set of local-density-connected points which is maximal w.r.t. local-density-reachability. Noises are defined relatively to be the set of points in the dataset that does not belongs to any of its normal clusters.

Definition 11 (Cluster). Let D be a database of points, and point o be a selected core point with the smallest LIF value of a cluster C, i.e. $o \in C$ and $\mathrm{LIF}(o) = \min(\mathrm{LIF}(C))$, w.r.t. parameter MinPts. A cluster C is a non-empty subset of D satisfying the following conditions:

(1) $\forall\, p$: p is local-density-reachable from o w.r.t. MinPts, then $p \in C$. (Maximality);
2) $\forall\, p, q \in C$: p is local-density-connected to q by o w.r.t. MinPts. (Connectivity).

Definition 12 (Distance between two distance-separated clusters). Let C_1 and C_2 be two distance-separated clusters of a database D. The distance between C_1 and C_2 is defined as,

$$\mathrm{Distance}(C_1, C_2) = \min\{d(p, q) | p \in C_1, q \in C_2\} \tag{7.3}$$

Definition 13 (Distance between two density-separated clusters). Let C_1 and C_2 be two density-separated clusters of a database D and of density den_p and density den_q, respectively. The distance between C_1 and C_2 is defined as,

$$\mathrm{Distance}(C_1, C_2) = \min\left\{\mathrm{den}_p - \mathrm{den}_q | p \in C_1, q \in C_2\right\} \tag{7.4}$$

From the definitions given in the above, it can be seen that, instead of using a fixed value for MinPts, we propose to use a varying value which is chosen according to an optimization to minimize the LIF values for each data point. Using our method,

clusters can be found with a two-step approach. First, data points from the database satisfying the distance separated condition are grouped. Second, data points that are local-density-separated from within the same distance separated groups are further clustered to obtain the final clusters.

7.4 A *k*NN Centroid-Based Clustering Algorithm

In this section, we present the proposed clustering algorithm, CDBSCAN, Centroid-Based DBSCAN.

7.4.1 The Algorithm

Our CDBSCAN clustering algorithm goes by a two-step processing, first to discover distance-separated clusters and then to discover density-separated clusters.

To find distance-separated clusters, unlike LDBSCAN clustering algorithm which starts with an arbitrary point, our CDBSCAN starts with the data point which has the smallest LIF value with respect to its MinPts. Therefore, in the first step, the minimum LIF value and the corresponding MinPts for each data point are calculated and the resulted LIF values are sorted in a nondecreasing order. The MinPts are determined on the fly as a by-product to obtain the smallest distance between a data point and its *k*NN-based centroid and may be different for different data points. The data point with the smallest LIF value is selected as the first core point. Then all the data points that are local-density-reachable from the core point are retrieved. After this cluster is formed. This process is repeated for the next data point with the smallest LIF value in the line that is not clustered yet. This process continues until all the clusters are discovered. Finally, if it is not for outlier detection purpose, all the remaining data points in small clusters are assigned to the nearest cluster of theirs already found. It is easy to see that, in this process, no parameter needs to be specified by user but adaptively chosen with respective to the optimization criterion. A basic version of our CDBSCAN for distance-separated clusters' retrieval is presented in Table 7.1. The function SetOfPoints.get(LIF(i)) returns the i-th element of SetOfPoints in the order according to sorted LIFs. The most important function used by the first step of CDBSCAN is ExpandCluster which is a depth-first-search through those data points. A recursive version of ExpandCluster is presented in the lower part of Table 7.1.

Thereafter, to find density-separated clusters, CDBSCAN calculates the average density of all the reachability distances, the corresponding standard deviation for each cluster obtained in the first step and the ratio of the standard deviation over the average density. If the ratio is larger than a threshold, say 0.5, the CDBSCAN for identifying the density-separated clusters starts with the point p having the smallest reachability distance value among the data points in the distance-separated clusters and retrieves all points local density-reachable from p w.r.t. MinPts. The process

Table 7.1 Our CDBSCAN-Distance separated clustering algorithm

Function Name: CDBSCAN-DiSCA	
Input:	
SetOfPoints	a set of *N* data points
Output:	
LabelOfPoints	an array with a label for each data point
LIF	an array of local inlier factors for each data point
MinPts	an array with number of neighbors for each data point
NumofClusters	the number of clusters resulted
LRD	a label array with Reachability distance label for each data point
Begin	
	InitSet (SetOfPoints); // calculate LIF and MinPts of eachpoint and sort LIF's in non-decreasing order
	ClusterID := 0;
	For (int i=1; i<SetOfPoints.size; i++) {
	Point := SetOfPoints.get(LIF(i));
	If Point = UNCLASSIFIED {
	ClusterID := ClusterID + 1;
	ExpandCluster(SetOfPoints, Point, ClusterID, MinPts)
	}
	}
End	
Function Name: ExpandCluster	
Input:	
SetOfPoints	a set of *N* data points
Point	the current point
ClusterID	its label
MinPts	its number of nearest neighbors
Begin	
	Point.ClusterID = ClusterID;
	For (int i=1; i<Point.MinPts; i++) {
	currentP := Point.Neighbor(i);
	If currentP. ClusterID = UNCLASSIFIED {
	ExpandCluster(SetOfPoints, currentP, ClusterID, MinPts);
	}
	}

(continued)

Table 7.1 (continued)

Function Name: CDBSCAN-DiSCA	
End	
Function Name: CDBSCAN-DiSCA	
Input:	
SetOfPoints	a set of *N* data points
Output:	
LabelOfPoints	an array with a label for each data point
LIF	an array of local inlier factors for each data point
MinPts	an array with number of neighbors for each data point
NumofClusters	the number of clusters resulted
LRD	a label array with Reachability distance label for each data point
Begin	
	InitSet (SetOfPoints); //calculate LIF and MinPts of eachpoint and sort LIF's in non-decreasing order
	ClusterID: = 0;
	For (int i = 1; i < SetOfPoints.size; i ++) {
	Point : = SetOfPoints.get(LIF(i));
	If Point = UNCLASSIFIED {
	ClusterID : = ClusterID + 1;
	ExpandCluster(SetOfPoints, Point, ClusterID, MinPts)
	}
	}
End	
Function Name: ExpandCluster	
Input:	
SetOfPoints	a set of *N* data points
Point	the current point
ClusterID	its label
MinPts	its number of nearest neighbors
Begin	
	Point.ClusterID = ClusterID;
	For (int i = 1; i < Point.MinPts; i ++) {
	currentP : = Point.Neighbor(i);
	If currentP. ClusterID = UNCLASSIFIED {
	ExpandCluster(SetOfPoints, currentP, ClusterID, MinPts);
	}
	}
End	

proceeds continuously by checking the next point in the current distance-separated cluster with the smallest reachability distance value but not clustered yet. A basic version of our CDBSCAN for density-separated clusters' retrieval is presented in Table 7.2.

From the above description, it can be seen that there is no LIFUP required specifically in the algorithm and the MinPts' can be determined according to some optimization criterion. For density-separated datasets as the one shown in the middle of Fig. 7.1, the first neighborhoods of the data points in the denser cluster do not contain any data points in the coarser cluster, thus separating the two clusters automatically.

7.5 Performance Evaluation

In this section, we present the results of two sets of experiments conducted to evaluate our proposed clustering algorithm. In the first set of experiments, we perform our centroid-based CDBSCAN clustering algorithm on four 2-D synthetic datasets and several real small low-dimensional UCI data sets, and compare the performance with LDBSCAN and CFSFDP clustering algorithms to manifest its ability to produce clusters of different densities. In the second set of experiments, we evaluate our CDBSCAN algorithm on two real large data sets without any assumptions on the data distribution. The results are also compared with those of LDBSCAN and CFSFDP to check the technical robustness of this study. All the datasets are briefly summarized in Table 7.3.

We implement all the algorithms in C++. All the experiments were performed on a computer with Intel Core 2 Duo Processor E6550 2.33 GHz CPU and 2 GB RAM. The operating system running on this computer is Windows XP. At this juncture, we would like to point out that the source codes for LDBSCAN and CFSFDP are obtained from the original author.

7.5.1 Performance on Small Low-Dimensional Data

To cluster a data set using density-based clustering algorithms, a set of parameters, such as a suitable value of k for computing the k-nearest neighbors and therefore the suitable reachability distances should be determined to detect clusters. Therefore, we first focus our study on the behavior of our centroid-based clustering algorithm with various datasets and parameters. Particularly, we want to show the impact of the input value k determined using our algorithm on the performance of our algorithm.

Four datasets employed to illustrate distance-separated, density-separated, touching clustering problems, and a composite clustering problem, respectively, are shown in the first column of Fig. 7.4. We use 2-dimensional data sets because of their visual convenience for the performance illustration. From the geometric shapes shown in the first column of Fig. 7.4, it can be seen that the first dataset consists of two

Table 7.2 Our CDBSCAN-density separated clustering algorithm

Function Name: CDBSCAN-DeSCA	
Input:	
SetOfPoints	a set of *N* data points
NumofClusters	the number of clusters resulted
LabelOfPoints	a label array obtained in the first step
LIF	an array of local inlier factors for each data point
MinPts	an array with number of neighbors for each data point
LRD	a label array with Reachability distance for each data point
Output:	
LabelOfPoints	an array of final labels for each data point
Begin	
	ClusterID := *NumofClusters*;
	Ratios(*NumofClusters,* 0);
	For (int i=1; i<*NumofClusters*; i++) {
	Average := 0; Std : =0; Count := 0;
	For (int j=1; j<SetOfPoints.size; j++) {
	If (*LabelOfPoints*[j] = = i) {
	Average := Average + *LRD*[j];
	Std : = Std + *LRD*[j]**LRD*[j];
	Count := Count + 1;
	}
	}
	Average = Average/ Count;
	Std = sqrt(Std / Count- Average*Average);
	Ratios[i] = Std/Average;
	}
	sort *LIF*'s in non-decreasing order
	For (int i=1; i<*NumofClusters*; i++) {
	If Ratios[i] < 0.5 { continue;}
	For (int j=1; j<SetOfPoints.size; j++) {
	Point :=SetOfPoints.get(LIF(j));
	ClusterID := ClusterID + 1;
	If (*LabelOfPoints*[j] = = i) {
	LabelOfPoints[j] = ClusterID;
	}
	ExpandCluster(SetOfPoints, Points, ClusterID, MinPts);

(continued)

Table 7.2 (continued)

Function Name: CDBSCAN-DeSCA		
		}
		update Ratios
	}	
End		

Table 7.3 The set of data

Dataset name	Data size	Dimension	Std/mean
Dataset1	174	2	0.1312
Dataset2	142	2	0.7241
Dataset3	307	2	0.1625
Dataset4	473	2	0.9270
australian	690	17	/
banknote	1372	5	/
bupa	345	7	/
ecoli	336	8	/
haberman	306	3	/
ionosphere	351	34	/
sonar	208	60	/
wdbc	569	32	/
Image Data1	95,550	20	/
Image Data2	95,550	50	/
Image Data3	95,550	100	

clusters of different sizes, shapes, and orientation. And the smaller cluster is inside the space enclosed by the outer cluster. It is evident this data set is an example of well distance-separated clustering problem. If k is chosen correctly, that is, with the largest value being 6 obtained from our experiment, the dataset is naturally set apart into two interconnected groups. The second data set is a typical example of density separated clusters which are very close to each other and have different densities. A particular challenging feature of the third data set is that clusters are very close to each other and have similar densities, the so-called chaining effect. The plots in the second column denote the reachability distances for each of the first three datasets using Definition 6. For the first and the third datasets, the reachability distances have similar values, while for the second dataset, the reachability distances manifest two groups with difference densities and therefore can solve the clustering problem of neighboring clusters of different densities. The plots in the third column highlight by the red color data points that have the k-th distances larger than the average value of these k-th distances. It can be seen that the k-th distances may not be any useful to well distance-separated clusters, however, suitable values for k chosen using our

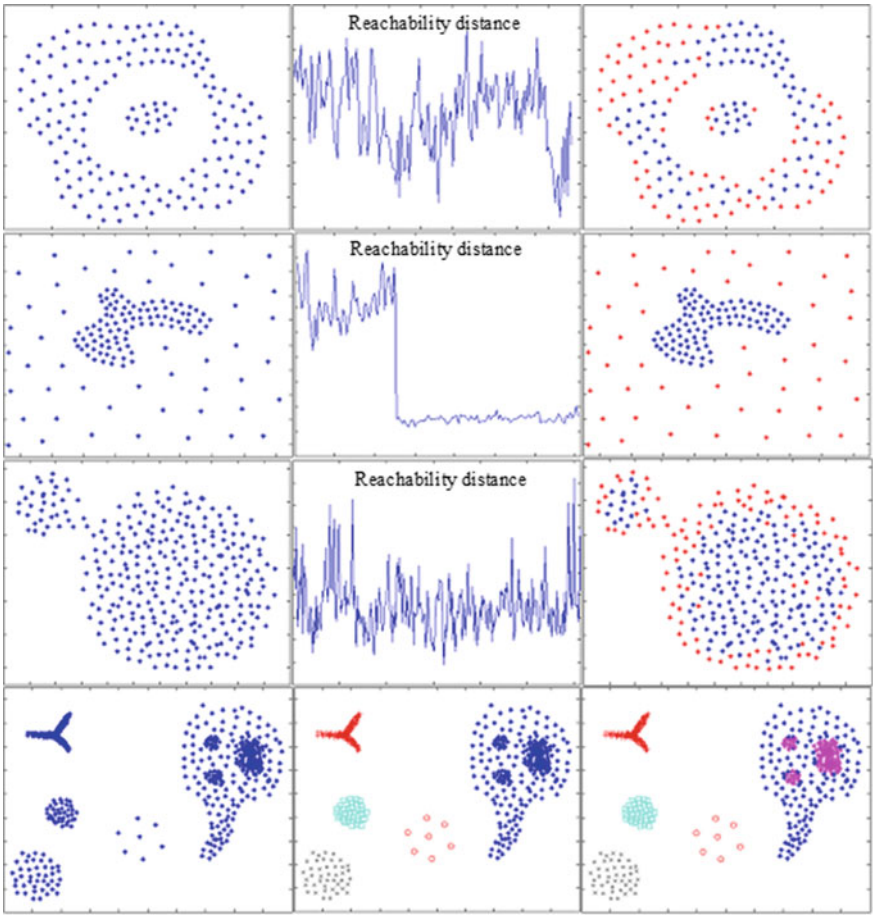


Fig. 7.4 Clustering results for (upper) well distance-separated clusters, (middle upper) well density-separated clusters, (middle lower) touching clusters, (lower) composite clusters

criterion can help to solve the chaining effect in this third dataset by widening the clusters' separation.

Before proceeding, we would like to show quantitatively the ratio of the standard deviation over the mean of the reachability distances for each type of the clustering problems shown in Fig. 7.4 in the last column of Table 7.3. It is clear from the table and the figure that Dataset1 and Dataset3 have relatively uniform density while Dataset2 and Dataset4 have density-separated clusters and therefore have large ratios. The clustering performance of the proposed method is next evaluated on the fourth 2-D synthetic dataset obtained from (Duan et al. 2007). The dataset and the clustering results using our distance-separated and density-separated clustering methods are presented on the lower right corner of Fig. 7.4, respectively. From the left-most figure on the last line, it can be seen that the dataset contains 5 well

distance-separated clusters and 4 touching clusters of different densities. The middle plot shows the resulted clusters obtained using our suitable k's to generate five well distance-separated clusters. The rightmost plot shows the final results where the reachability distances between data points are utilized as the density measure. These results show the competence of our density-based clustering algorithm.

To further test the performance of our proposed method, eight real datasets from UCI machine learning repository are employed, which are well-known benchmarks for machine learning research. The LDBSCAN algorithm, the CFSFDP algorithm and our algorithm are used to cluster these UCI datasets. The clustering performance is evaluated using four clustering validity indices, Rand, Adjusted-Rand, Jaccard coefficient, and Fowlkes and Mallows (FM) (Milligan et al. 1983; Theodorodis and Kouttoumbas 2009; Warrens 2008; Yeung et al. 2001). The evaluation results on these datasets are shown in Tables 7.4 through 7.7, respectively. The bold numbers in the tables indicate that the corresponding index value obtained by using our algorithm is larger than the index values of the other two competing density-based clustering algorithms.

From Tables 7.4, 7.5, 7.6 and 7.7, it is evident that the proposed method has the best performance than the other two methods for all four indices on ecoli, haberman, and wdbc data. For the rest five datasets, all three methods provide better clustering performance on some datasets while lose its performance on others. The results from this set of experiments show that our CDBSCAN clustering algorithm is capable of detecting clusters of arbitrary shape and different densities and on the whole outperforms LDBSCAN and CFSFDP. To further test the capability of the proposed algorithm, we next conduct experiments on real-life datasets to evaluate the performance the proposed clustering method.

Table 7.4 Clustering performance evaluation in Rand

Data name	LDBSCAN	CFSFDP	Ours
australian	0.504	0.511	**0.5111**
banknote	0.527	0.56	0.5116
bupa	0.505	0.518	0.513
ecoli	0.705	0.754	**0.8707**
haberman	0.578	0.509	**0.6221**
ionosphere	0.514	0.55	0.5418
sonar	0.533	0.507	0.498
wdbc	0.763	0.554	**0.7681**

The bold numbers in the tables indicate that the corresponding index value obtained by using our algorithm is larger than the index values of the other two competing density-based clustering algorithms

Table 7.5 Clustering performance evaluation in Jaccard

Data name	LDBSCAN	CFSFDP	Ours
australian	0.062	0.372	**0.499**
banknote	0.065	0.422	**0.4994**
bupa	0.38	0.377	**0.4914**
ecoli	0.398	0.302	**0.6356**
haberman	0.528	0.393	**0.6097**
ionosphere	0.227	0.529	**0.5331**
sonar	0.211	0.361	**0.484**
wdbc	0.589	0.426	**0.6677**

The bold numbers in the tables indicate that the corresponding index value obtained by using our algorithm is larger than the index values of the other two competing density-based clustering algorithms

Table 7.6 Clustering performance evaluation in Adjusted Rand

Data name	LDBSCAN	CFSFDP	Ours
australian	0.017	0.014	0.0125
banknote	0.064	0.116	0.0127
bupa	−0.008	0.02	0.0036
ecoli	0.353	0.306	**0.6867**
haberman	0.021	−0.009	**0.0594**
ionosphere	0.062	0.012	0.0127
sonar	0.057	−0.007	−0.0037
wdbc	0.526	0.092	**0.5284**

The bold numbers in the tables indicate that the corresponding index value obtained by using our algorithm is larger than the index values of the other two competing density-based clustering algorithms

Table 7.7 Clustering performance evaluation in FM

Data name	LDBSCAN	CFSFDP	Our
australian	0.195	0.543	**0.7**
banknote	0.255	0.595	**0.7003**
bupa	0.552	0.548	**0.6878**
ecoli	0.582	0.472	**0.7791**
haberman	0.695	0.566	**0.7762**
ionosphere	0.403	0.717	**0.7254**
sonar	0.38	0.531	**0.6855**
wdbc	0.75	0.598	**0.8037**

The bold numbers in the tables indicate that the corresponding index value obtained by using our algorithm is larger than the index values of the other two competing density-based clustering algorithms

7.5.2 Performance on Large Image Datasets

It can be seen that our CDBSCAN clustering algorithm is a competent density-based clustering method. In this subsection, we provide experiments on real world image data in order to demonstrate that there is some semantic meaning behind the proposed clustering method. We choose to apply it to a color image segmentation task because of their visual convenience for the performance judgment. Being a difficult and challenging problem in computer vision, image segmentation is a learning mechanism used to identify groups of similar image primitives. Here the image primitives can be local features which are mathematically represented by vectors in a feature space. Images taken from a camcorder are stored in RGB color space and then converted to HSV color space. To generate our data sets for identifying objects, we consider color pixels in regularly spaced (with distance of M) image patches of size $N \times N$, for which a color histogram is extracted and used for segmenting images. To obtain the color features, we equally divide the chromaticity H, the saturation S and the brightness V into u, v and w bins, respectively. Each feature vector can be generated by combining these three bins. All possibilities of the combinations equal $u \times v \times w$ different color bins (i.e., the dimension of the feature vectors) for the histogram.

All the data sets used in this set of experiments are briefly summarized on the last three rows of Table 7.3. The image data is from pictures taken around an outdoor working environments, using SONY Digital Handycam DCR VX2000 digital video camera recorder. For the image data, $N = 27$ and $M = 18$. To build our percept databases, the datasets consist of 95,550 unlabeled feature vectors which are extracted from 35 images. Since the number of clusters is not given beforehand, the proposed algorithm stops when the performance levels off, that is, the ratio of standard deviation to the average of the reachability distance values within a cluster produced is smaller than 0.5. The clustering performances are presented in Fig. 7.5 for clustering three image data of three difference dimensions using the proposed algorithm and in Fig. 7.6 for clustering Image Data2 using two competing algorithms, respectively. We only show 7 out of 35 images in order to keep the amount of data presented reasonable, but consistent performance results are obtained for other images.

There are four columns in Fig. 7.5. The first (i.e., left) column displays the original images used. The second (i.e., middle left) column displays the clustering results of the proposed method on Image Data1 of dimension 20. The third (i.e., middle right) column displays the clustering results of the proposed method on Image Data2 of dimension 50. Finally the fourth (i.e., right) column displays the clustering results of the proposed method on Image Data3 of dimension 100. To ease visual examination, in the segmented images, a color close to the real one is chosen to denote the percepts. From the original images, it is easy to identify that the major percepts include dark green trees and their shades on the road under the sun, wide grey concrete road occupying most lower-half part or lower left-half part of images and extending into the far end background and with a couple of trees on the left, light grayish building side wall to the right of road, wide brick pavement on the right, a background consisting



Fig. 7.5 Results for the image data (left) original image; results of (middle left) our algorithm on Image Data1 of dimension 20; (middle right) our algorithm on Image Data2 of dimension 50; (right) our algorithm on Image Data3 of dimension 100

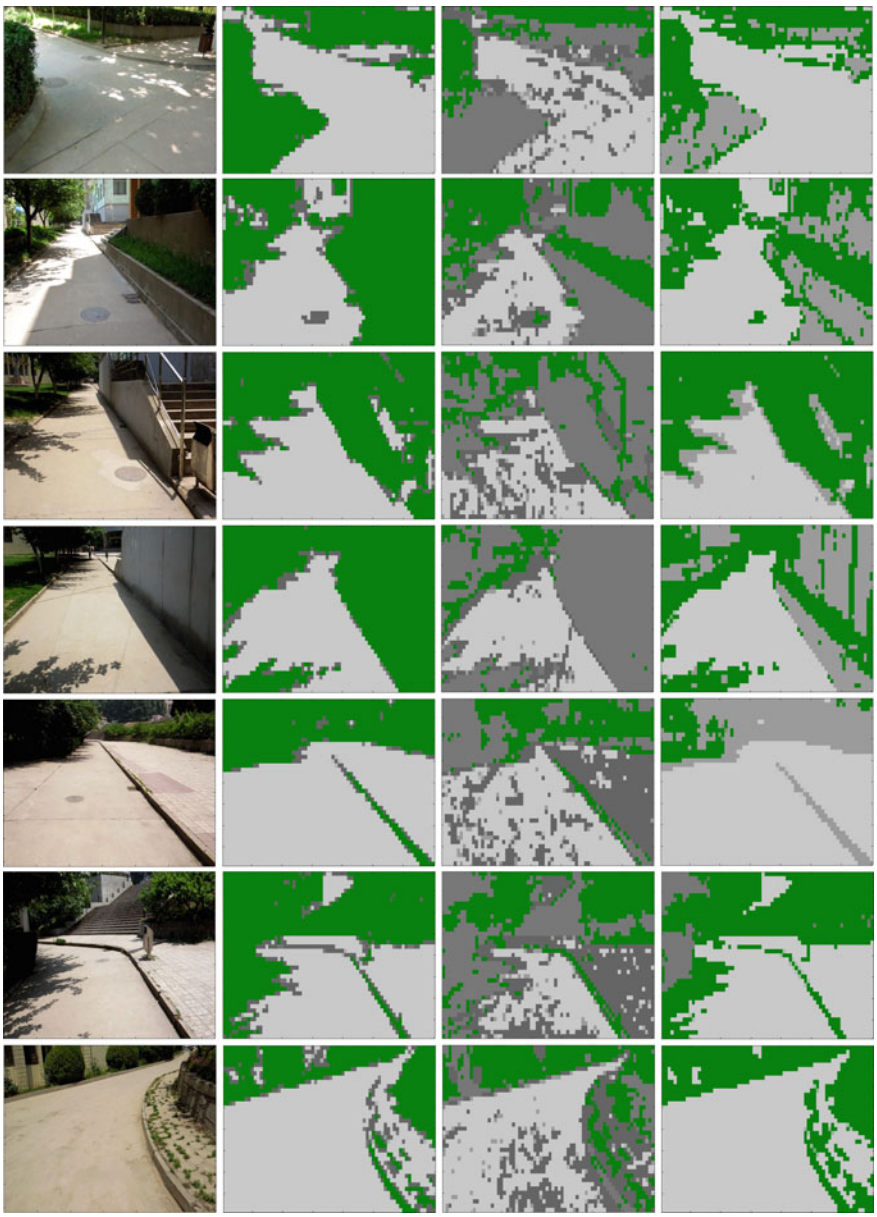


Fig. 7.6 Results for image data (left) original image, clustering results of (middle left) our algorithm on Image Data2 of dimension 50; (middle right) CFSFDP algorithm on Image Data2 (right) CFSFDP algorithm performance on each of 7 single images of Image Data2

of brown bushes, yellow leaves and concrete footsteps mingled in the far end, and a greenish flower bed on the lower-right part. There are 5 clusters obtained for these three datasets by the proposed method, respectively. Typical stable percepts detected (and the corresponding color representing them) include segmented grey road percept (denoted by a color close to white), and grass and tree percepts (denoted by a color close to green). The most interesting comparison here is the segmentation results of the second image as shown in the second row of Fig. 7.5. It is clearly seen that, for Image Data2 of dimension 50, the manhole cover is separated from the road and the tree percepts, while it is mixed with the tree percept in the segmentation result for Image Data1 of dimension 20 and missed in the segmentation result for Image Data3 of dimension 100. All other image segmentation results for three image data obtained by the proposed method are similar.

There are four columns in Fig. 7.6. The first (i.e., left) column displays the original images used. The second (i.e., middle left) column displays the clustering results of the proposed method on Image Data2 of dimension 50. The third (i.e., middle right) column displays the clustering results of the CFSFDP algorithm on Image Data2 of dimension 50. Finally the fourth (i.e., right) column displays the clustering results of the CFSFDP algorithm on each individual image of Image Data2 of dimension 50. To ease visual examination, in the segmented images, a color close to the real one is chosen to denote the percepts. From all the segmented images, it is evident that the proposed method does a better job on segregating the concrete roads than the CFSFDP algorithm while the CFSFDP algorithm does a better job on the second image in Fig. 7.6. The most interesting comparison here is between our method and the CFSFDP algorithm on each individual image of Image Data2 of dimension 50. From the segmentation results as shown in the second and the fourth columns of Fig. 7.6, it can be seen that, in this case, the CFSFDP algorithm performs similarly as the proposed method for the third and the last three images while does a little better job for the first two images and the fourth image by identifying more percepts than our method.

Overall, the results presented above offer good evidence that our method performs very well on this set of images from outdoor domains.

7.6 Conclusions

Density-based clustering mechanism based on kNN was initiated by Ester et al. (1996a, b) and abundant developments in the field have made it a very active branch of modern density-based clustering techniques. However for data sets with clusters of different densities, the determination of its parameters is difficult and may not reveal its instinct nature. In this chapter, we propose a new density-based clustering based on a distance measure between a data point and its centroid, which does not specify the fixed number of nearest neighbors beforehand but let it be variable upon which an optimization is obtained. Experiments conducted to evaluate our algorithm's performance show some promising results.

References

Agrawal, R., Gehrke, J., Gunopulos, D., & Raghavan, P. (1998). Automatic subspace clustering of high dimensional data for data mining applications. In *Proceedings of ACM SIGMOD International Conference on Management of Data*, *Seattle*, *WA*, *ACM*, *New York* (p. 105).

Agrawal, R., Gehrke, J., Gunopulos, D., & Raghavan, P. (1998). Automatic subspace clustering of high dimensional data for datamining applications. In *Proceedings of ACM-SIGMOD Conference on the Management of Data* (pp. 94–105).

Ankerst, M., Breunig, M.M., Kriegel, H.-P., & Sander, J. (1999). OPTICS: Ordering points to identify the clustering structure. In *Proceedings of ACM SIGMOD International Conference on Management of Data*, *Philadelphia*, *PA*, *ACM*, *New York* (pp. 49–60).

Bereta, M., & Burczynski, T. (2009). Immune k-means and negative selection algorithms for data analysis. *Information Sciences*, *179*, 1407–1425.

Chaoji, V., Hasan, M., Salem, S., & Zaki, M.J. (2008). SPARCL: Efficient and effective shape-based clustering. In *Proceedings of the 8th IEEE International Conference on Data Mining* (*ICDM*'08), *Washington*, *DC* (pp. 93–102).

Chaoji, V., Li, G., Yildirim, H., & Zaki, M.J. (2011). ABACUS: Mining arbitrary shaped clusters from large datasets based on backbone identification. In *Proceedings of the 11th SIAM International Conference on Data Mining*(*SDM*'11), *Arizona*, *USA* (pp. 295–306).

Chiang, M. C., Tsai, C. W., & Yang, C. S. (2011). A time-efficient pattern reduction algorithm for k-means clustering. *Information Sciences, 181,* 716–731.

Duan, L., Xu, L., & Guo, F. (2007). A local-density based spatial clustering algorithm with noise. *Information Systems, 32,* 978–986.

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial data sets with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, Portland, OR, USA (pp. 226–231).

Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, Port Land, OR, AAAI, Menlo Park, CA (pp. 226–231).

Hinneburg, A., & Keim, D.A. (1998). An efficient approach to clustering in large multimedia databases with noise. *Knowledge Discovery and Data Mining*, pp. 58–65.

Hinneburg, A., & Keim, D.A. (1998). An efficient approach to clustering in large multimedia databases with noise. In *Proceedings of Fourth International Conference on Knowledge Discovery and Data Mining*, New York, NY, AAAI, Menlo Park, CA (pp. 58–65).

Jain, A. K., & Dubes, R. C. (1988). *Algorithms for clustering data*. Englewood Cliffs, NJ: Prentice-Hall.

Katayama, N., & Satoh, S. (1997). The SR-tree: An index structure for high-dimensional nearest neighbor queries. In *Proceedings of ACM SIGMOD International Conference on Management of Data*, Tucson, Arizona, USA (pp. 369–380).

Lee, C. H., Zaïane, O. R., Park, H.-H., Huang, J., & Greiner, R. (2008). Clustering high dimensional data: A graph-based relaxed optimization approach. *Information Sciences, 178,* 4501–4511.

Ma, W. M., Eden, C., & Tommy, W. S. (2004). A new shifting grid clusting algorithm. *Pattern Recognition, 37*(3), 503–514.

Milligan, G. W., Soon, S. C., & Sokol, L. M. (1983). The effect of cluster size, dimensionality, and the number of clusters on recovery of true cluster structure. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 5,* 40–47.

Rodriguez, A., & Laio, A. (2014). Clustering by fast search and find of density peaks. *Science, 344*(6191), 1492–1496.

Sheikholeslami, G., Chatterjee, S., & Zhang, A. (1988). WaveCluster: a multi-resolution clustering approach for very large spatial databases. In *Proceedings of 24th International Conference on Very Large Data Bases* (*VLDB*'88), New York, NY, Morgan Kaufmann, Los Altos, CA (pp. 428–439).

Stein, B., & Busch, M. (2005). Density-based cluster algorithms in low-dimensional and high-dimensional applications. In *Proceedings of the Second International Workshop on Text-Based Information Retrieval* (TIR'05), University of Koblenz, Germany (pp. 45–56).

Theodorodis, S., & Kouttoumbas, K. (2009). *Pattern Recognition* (4th ed.). Amsterdam: Academic-Press.

Wang, W., Yang, J., & Muntz, M. (1997). STING: A statistical information grid approach to spatial datamining. In *Proceedings of the International Conference on Very Large Data Bases* (pp. 186–195).

Warrens, M. J. (2008). On the equivalence of Cohen's kappa and the Hubert-Arabie adjustedrand index. *Journal of Classification, 25,* 177–183.

Yeung, K. Y., Fraley, C., Murua, A., & Raftery, A. E. (2001). Model-based clustering and data transformations for gene expression data. *Bioinformatics, 17,* 977–987.

Zahn, C. T. (1971). Graph-theoretical methods for detecting and describing gestalt clusters. *IEEE Transactions on Computer, C-20,* 68–86.

Chapter 8
An Efficient EMST Algorithm for Clustering Very High-Dimensional Sparse Feature Vectors



Abstract In this chapter, we present a fast minimum spanning tree-based clustering algorithm for image segmentation and object recognition tasks. We begin with an introduction to the concept of the minimum spanning tree in general and its application to clustering in specific. The proposed clustering method is next described. Finally, the performance evaluation of the proposed method is conducted on an image-patch-based visual percept detection task for an indoor environment and an outdoor environment.

Keywords Clustering · Minimum spanning tree-based clustering · Image segmentation · Object recognition

8.1 Introduction

Organizing image parts into clusters based on psychologically plausible similarity measure, image segmentation generally starts by extracting important visual information from image patches about a scene in the form of feature vectors and subsequently employs clustering algorithms to realize its goals. Clustering is an exploratory data analysis tool that deals with grouping together objects that are similar to each other and has been extensively studied. To find clusters of different shapes, sizes, and densities, many algorithms have been introduced in the literature. Compared to other classic clustering techniques, such as partition-based clustering (Han and Kamber 2001; Ng and Han 1994), density-based clustering (Ester et al. 1996; Ankerst et al. 1999; Hinneburg and Keim 1998), grid-based clustering (Wang et al. 1997; Sheikholeslami et al. 1998), fuzzy clustering (Li et al. 2004) and hierarchical clustering (Zhang et al. 1996; Karypis et al. 1999; Jain et al. 1999; Guha et al. 1998, 1999), minimum spanning tree (MST) based clustering techniques do not assume that data points are grouped around centers or separated by regular geometric curve, but use direct tree edge information (instead of distances to cluster centers) to partition data points into clusters, and are well-known for their ability to detect clusters with irregular boundaries. Recent efforts focus on the scalability and efficiency of these algorithms and their ability to discover clusters with different characteristics.

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_8

The minimum spanning tree method is a graphical analysis of an arbitrary set of data points. In such a graph, two points or vertices can be connected either by a direct edge or by a sequence of edges called a path. The length of a path is the number of edges on it. The degree of the link of a vertex is the number of edges that link to this vertex. A loop in a graph is a closed path. A connected graph has one or more paths between every pair of points. A tree is a connected graph with no closed loops. A spanning tree is a tree that contains every point in a data set. If a value is assigned to each edge in the tree, the tree is called a weighted tree. The weight of a tree is the total sum of the edge weights in the tree. The minimum spanning trees are the spanning trees that have the minimal total weight. Two properties used to identify edges provably in an MST are the cut property and the cycle property. The cut property states that the edge with the smallest weight crossing any two partitions of the vertex set must belong to an MST. The cycle property states that the edge with the largest weight in any cycle in a graph cannot be in an MST. As a result, when the weight associated with each edge denotes a distance between two end points, any edge in a minimum spanning tree will be the shortest distance between two subtrees that are connected by that edge. Therefore, removing the longest edge will theoretically result in a two-cluster grouping. Removing the next longest edge will result in a three-cluster grouping, and so on. This corresponds to choosing breaks where the maximum weights occur in the sorted edges. Based on this finding, MST-based clustering algorithms were initially proposed by Zahn (1971) and have so far been extensively studied.

In traditional MST problems, an undirected, connected and weighted graph with a set of N vertices and a set of E edges is given and the time complexity of standard solutions is $O(E \log N)$. The first minimum spanning tree algorithm was proposed by Borůvka (1926). Since then, optimal exact MST algorithms, fast and approximate MST algorithms, distributed MST algorithms (Khan and Pandurangan 2008), and parallel MST algorithms (Pettie and Ramachandran 2002) have been developed. Minimum spanning tree algorithms have so far been broadly applied in image segmentation (An et al. 2000; Xu and Uberbacher 1997), clustering (Zahn 1971; Xu et al. 2002; Zhong et al. 2010), classification (Juszczak et al. 2009), etc.

In modern MST tasks, usually, a set of N d-dimensional data points is given and the problem is commonly solved in the Euclidean setting, giving rise to the so-called Euclidean minimum spanning tree (EMST) problem. In this case, for a complete graph with $V = N$ vertices and $E = N(N - 1)/2$ edges, standard MST algorithms, such as Kruskal's (1956) and Prim's (1957), have a time complexity roughly equal to $O(dN^2)$ and are practical only for small low dimensional datasets. Thus, the computational cost of building an EMST for a large high-dimensional dataset is often the bottleneck of efficiency for many practical applications.

To find exact EMSTs quickly, especially on a single modern central processing unit, the goal is to lessen the number of distance computations used to construct an EMST as much as possible. Although finding minimum spanning trees in linear computational time is not practical for large high-dimensional datasets, nearest neighbor search between data points in high-dimensional space can be conducted fast in Euclidean space. Therefore, attention has been paid to the design of more efficient

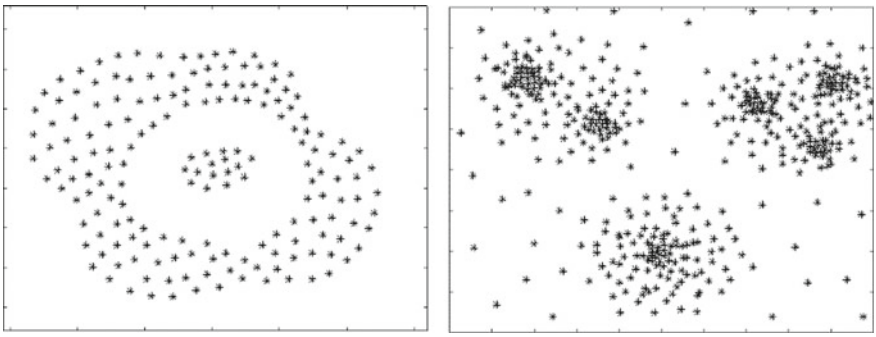


Fig. 8.1 Sample datasets for the proposed EMST-based clustering algorithm (left) well distance-separated clusters, (right) relatively connected clusters

EMST-based clustering algorithms using faster k-nearest neighbors search methods. For well distance-separated clusters, as illustrated in the left plot of Fig. 8.1, since the inter-cluster distances are significantly larger than the intra-cluster distances, a k-nearest neighbors graph is enough to find clusters in terms of connected components. For relatively connected clusters, as illustrated in the right plot of Fig. 8.1, since the inter-cluster distances are comparable to the intra-cluster distances, a k-nearest neighbors graph is enough to construct an EMST.

Based on these observations, in this chapter, we propose a new fast Euclidean minimum spanning tree algorithm for large high-dimensional datasets by determining those neighboring edges in an EMST within much smaller a neighborhood instead of a global neighborhood so as to save the number of distance computations tremendously. Experiments conducted on sample datasets demonstrate the efficiency of the proposed method.

The rest of this chapter is organized as follows. In Sect. 8.2, we review some existing work on MST algorithms, MST-based clustering algorithms and fast k-nearest neighbor search algorithms. We next present our proposed approach in Sect. 8.3. In Sect. 8.4, we verify the efficiency and effectiveness of the proposed methods with experiments. Finally, conclusions are made and future work is discussed in Sect. 8.5.

8.2 Related Work

8.2.1 Minimum Spanning Tree Algorithms

For a given connected and weighted graph $G = (E, V)$, Borůvka's algorithm begins with each vertex of a graph being a tree, and for each consecutive iteration, it selects the shortest edge from a tree to another tree and combines them. This process continues until all the trees are combined into one tree (Borůvka 1926). Proposed independently by Jarník (1930), Prim (1957) and Dijkstra (1959) in 1930, 1957 and 1959,

respectively, the famous Prim's algorithm first arbitrarily selects a vertex as a tree, and then repeatedly adds the shortest edge that connects a new vertex to the tree, until all the vertices are included. Proposed in 1956, Kruskal's algorithm starts with sorting all the edges by their weights in a non-decreasing order, treats each vertex as a tree, and iteratively combines the trees by adding edges in the sorted order excluding those leading to a cycle until all the trees are combined into one tree (Kruskal 1956). The time complexity of these classic MST algorithms is $O(E \log V)$.

To construct an MST in the Euclidean setting, standard Prim's algorithm requires a quadratic running time. To be more efficient, in 1978, Bentley and Friedman proposed to use a *Kd*-tree in Prim's algorithm to enhance the search for the next edge to add to the tree, which can reach an $O(N \log N)$ running time for most data distributions (Bentley and Friedman 1978). In 1985, Preparata and Shamos gave a lower bound of $(N \log N)$ for the EMST problem, which has been the tightest known lower bound (Preparata and Shamos 1985). In 1993, Callahan and Kosaraju proposed Well-Separated Pair Decomposition (WSPD) which forms the basis of most recent EMST algorithms (Callahan and Kosaraju 1993). The WSPD partitions data points into a set of pairs of tree nodes such that the nodes in any pair are farther apart than the diameter of either node. It can be shown that the WSPD has $O(N)$ pairs of nodes, and that an MST is a subset of the edges formed between the closest pair of points in each pair of nodes. In 2000, Narasimhan et al. applied WSPD to compute neighbors of components for Borůvka's MST algorithm (Narasimhan et al. 2000). However, the constant in the $O(N)$ size of the WSPD grows exponentially with the data dimension and is often very large in practice. In 2010, March et al. presented a new dual-tree algorithm (referred to in the following as FEMST algorithm) for efficiently computing an EMST (March et al. 2010), which is superficially similar to the method in (Narasimhan et al. 2000) except that the WSPD is replaced by a new dual-tree data structure. They used adaptive algorithm analysis to prove the tightest (and possibly optimal) runtime bound for the EMST problem to-date. Experiments conducted demonstrated the scalability of their method to astronomical datasets.

8.2.2 MST-Based Clustering Algorithms

MST-based clustering was first proposed by Zahn (1971). The inherent separation was used in the paper to emphasize that the separation solely relies on inter-point distances within a dataset. As a result, it is weak in processing clusters with large density variations. To be less sensitive to outliers, Laszlo and Mukherjee presented an MST-based clustering algorithm (referred to as the LM algorithm in the following) that puts a constraint on the minimum cluster size rather than on the number of clusters (Laszlo and Mukherjee 2005). In their algorithm, when cutting edges in a sorted non-increasing order, only when those edges, removing which can result in two clusters whose sizes are both larger than the minimum cluster size, are reached can the operation proceed.

Given a set of N data points and a distance measure defined upon them, modern MST-based clustering algorithms usually begin by constructing an EMST. The time complexity of popular EMST algorithms, such as the Prim's algorithm, is $O(N^2)$ (Cormen et al. 2009). This time factor limits the application of MST-based clustering methods to modern massive data sets. Several studies have been done to solve these problems. An efficient MST-inspired clustering algorithm was proposed which works by finding a local density factor (LOF) for each data point during an efficient construction of an MST (Wang et al. 2013). By using a centroid based nearest neighbor rule, an MST-based clustering algorithm was introduced which first generates a sparse Local Neighborhood Graph (LNG) and then constructs an approximate MST from LNG (Jothi et al. 2015). Experimental results demonstrated that the computational time was reduced significantly while maintaining the quality of the clustering. Zhong et al. (2015) proposed a two-stage scheme to produce an approximate MST, referred to in the following as the FAEMST algorithm. Experiments conducted demonstrated its efficiency in comparison with other traditional MST algorithms. However, it is not good for complex-shape clusters.

Relying on a compressed quadtree data structure to compute approximate EMSTs, an algorithm was proposed to eliminate the exponential dependencies to reduce the time consumption (Arya and Mount 2016). This algorithm achieves its efficiency by being sloppier in approximating bichromatic closest pairs between well-separated pairs in situations where a significant EMST weight in the vicinity of the well-separated pair could be inferred. Hierarchical and density-based techniques are also employed to reduce the computational complexity in the construction of an MST and to improve the efficiency of clustering data distributed in different shapes and densities (Pappula 2017). Recently, Jothi et al. (2018) proposed two efficient algorithms, namely partition-based near neighbor graph using Bi-means partitioning (BNNG) and partition-based near neighbor graph using K-means partitioning (KNNG), to obtain MST in the context of clustering in a less than quadratic time. A novel centroid based nearest neighbor rule was presented in this paper and the experiment results revealed the good performance of the algorithm.

8.2.3 Fast k*-Nearest Neighbor Query Techniques*

In a multidimensional space, the k-nearest neighbors (kNN) search or query technique is the problem of finding k vectors closest to a given vector (called the query vector) among N data vectors (called the candidate vectors) placed in d-dimensional space based on a distance measure. The query vector can either be one of the candidate vectors or one that has not been seen. In a simple brute force search, the query vector is compared sequentially with all vectors in a database, resulting in a time complexity of $O(dN)$. To support a more efficient kNN search in large databases, multidimensional index structures are developed to reduce the number of candidate vectors encountered in the search process.

Proposed in 1970, the *Kd*-tree divides the search space into disjoint subregions along coordinate planes (Bentley 1970). Being a height-balanced tree, its time complexity is logarithmic to the size of the database. Using bounding rectangles, bounding spheres or both to associate regions with nodes and leaves, R-tree (Rectangle-tree) (Guttman 1984), its most successful variant, R*-tree (Beckmann et al. 1990), SS-tree (White and Jain 1996) and SR-tree (Sphere/Rectangle-tree) (Katayama and Satoh 1997), proposed next one after the other, rely on the amount of overlap among sibling regions, not the tree height, for efficient search.

Working well for low dimensional small-sized data sets, these methods have a dramatically degraded performance for high dimensional larger-sized data sets, due to the so-called curse of dimensionality (Chaoji et al. 2008). Since, in a high-dimensional space, there may not be much difference in the distances from a point to its nearest and farthest points, search areas cannot be restricted and the computational complexity eventually approaches that of a linear search. Based on this intuition, the VA-file (vector approximation file) algorithm (Blott and Weber 1997) was proposed in 1997 as an alternative approach to space partitioning based on multidimensional indexing techniques. Basically, it is a sequential search. To save computations, multidimensional data vectors are first approximated by a low-order bit representation, on which similarity search is first performed. The exact distances are computed only between the query vector and the filtered size-reduced set.

Proposed first in 2001 (Yu et al 2001) and more comprehensively presented in 2005 (Jagadish et al. 2005), iDistance is an efficient B^+-tree-based indexing method, designed to process *k*NN queries more efficiently in high-dimensional spaces. It has been used in many applications and proven to be especially good for skewed data distributions. To build an iDistance, there are two steps: (1) a number of reference points in the data space should be first chosen; (2) the distance between a data point and its closest reference point is then calculated, which plus a scaling value is called the point's iDistance. By this means, points in a multi-dimensional space are mapped to one-dimensional values, and the nearest neighbors can be obtained earlier with a very high probability than a sequential search. The basic search process for a *k*NN of a query point q is summarized in a pseudocode format in Table 8.1.

Table 8.1 Algorithm I: basic kNN search algorithm

Input	p, a data point; data, a set of N data points, and search structure
Output	*k* nearest neighbors of p
1	start with a small search sphere centered at query point
2	search by checking all partitions that intersect the current query space
3	if *k* nearest neighbors are found
4	exit
5	else
6	enlarge search space
7	goto 2

Noticing most unnecessary distance computations can be terminated earlier, in 2008, Ajioka et al. proposed to first compute the distances between the query vector and k data vectors and store them in a priority queue, and then to use only partial sum in the distance computations for decision making for each remaining candidate to save time (Ajioka et al. 2008). To enhance early termination, the authors proposed two strategies, referred to as Fast-DSORT and Fast-PCA, respectively in its paper. In the first one, the dimensions are reordered by their variances in a non-increasing way. In the second one, preliminary transformations of the data are made so that the coordinates are arranged in the order of principal components.

Similar to iDistance but with important variations, Liu et al. proposed a novel polar coordinate-based indexing method, called iPoc, for efficient kNN search in high-dimensional space (Liu et al. 2010). Unlike iDistance, during searching, iPoc can effectively prune query-unrelated data points by estimating the lower and upper bounds not only in radial coordinate but also in angle coordinate. The highest dimension of data they tested was 64. For higher dimensions, their performance is not clear.

8.3 The Proposed EMST-Based Clustering Algorithms

To facilitate MST-based clustering in modern large databases, in this section, we propose a kNN-based EMST-inspired clustering algorithm by eliminating the unnecessary distance computations involved in the construction of an EMST. Basically, the design of this more efficient scheme is motivated by the following two observations.

(1) According to Borůvka's algorithm that at least half of the edges in an EMST correspond to the distance between a data point and its nearest neighbor. In particular, we are interested in EMST construction for cases as illustrated by the right plot of Fig. 8.1 where edges in an EMST can be found quickly by fast k-nearest neighbors search algorithms.

(2) For large-scale high-dimensional datasets, standard EMST algorithms will lose its time performance. To improve the efficiency of solutions for problems such as finding EMST or k nearest neighbors (kNN), an intuitive way is to remove the unnecessary distance computations. For example, to find k nearest neighbors of a point in a data set, it is not necessary to search the whole data set but a small local portion. In this subsection, we, therefore, discuss a fast EMST algorithm which is based on an observation (as illustrated by a 2-dimensional situation in Fig. 8.2) about the maximum number of edges leading into/leaving from a vertex in a minimum spanning tree. From the two-dimensional data in Fig. 8.2, we can see that a data point can have at most 6 nearest neighbors (on the boundary of a unit circle) in an EMST. If a seventh data point, say P, is added outside of the circle, it will be closer to one of O's nearest neighbors, that is Q, instead of O.

Therefore, if there is any efficient index structure available on the dataset to be used in our method, our proposed EMST algorithm can be more CPU efficient with good time complexity of $O(N \log N)$.

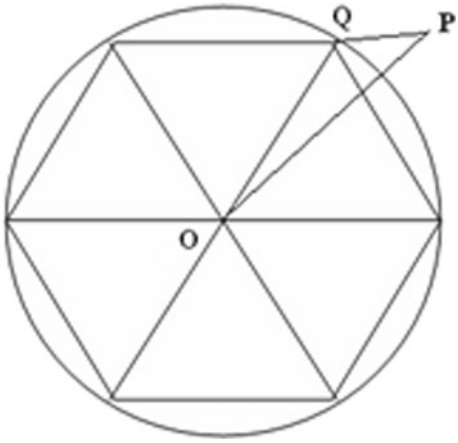


Fig. 8.2 An example of a maximum number of edges leading into/leaving from a vertex in a minimum spanning tree

In d-dimensional space, there exists a distance between each pair of a data set. An exhaustive search for kNN takes $O(dN^2)$. In the following, an efficient method is developed for database based kNN search. It is of the iDistance variety with important distinctions. Thereafter, a new fast MST-inspired clustering algorithm is introduced for large high-dimensional datasets.

8.3.1 A Simple Idea

It is true that, with the help of iDistance index structure, kNN search for each data point can be conducted in any order. However, iDistance is lossy in the sense that multiple data points in a high-dimensional space may be mapped to the same value in the search space. That is, different points within a partition that are equidistant from the reference point have the same transformed value. When most data reside in dense clusters, a large number of false positives may exist during the search, which can make the index structure rather inefficient. Fortunately, every data point in the database has some kind of locality. If a data item has seen all the data points which have it in their kNN, it can retire from the rest search of kNNs for other data points, thus reducing the number of false positives. To modify iDistance for our purpose, we have the following four observations:

(1) If a data point has the largest distance value to its k-th nearest neighbor among all the data points, this data point can be removed from the searching structure from being encountered later on. To prove this, suppose we have a data point p which has the largest k-th nearest distance, and there exists a data point q that is not in p's k-nearest neighborhood but has p in its k-nearest neighborhood. This is impossible because otherwise, q should be the one that has the largest distance to its k-th nearest neighbor since its k-th nearest distance is larger than p's k-th

nearest distance. Therefore, for this effect to be utilized, the kNN search should proceed in some order so that the data involved in the search can be diminishing.

(2) To utilize the above observation, a favorable property of kNN is that the k-th nearest distance to each data point is a monotonic nonincreasing function of the portion of the dataset already explored (Angiulli and Pizzuti 2005; Angiulli et al. 2006; Angiulli and Fassetti 2009).

(3) As shown in Fig. 8.3, when searching for kNN for a data point, say p, it could be more efficient if more than one close data points are searched together, say all the data points no more than r distance away from p, so that the average number of distance computations can be much smaller than that when only one is searched. This is where triangular inequality can be of some help. Suppose that p and q are R distance apart. Data point q could have only one distance computation with p for it to be pruned from its distance computations with all those data points r distance from p, if $R > 3r$. Further, if R is larger than the sum of $3r$ plus the current k-th distance (not necessarily k-th nearest distance) of q, the k neighbors of q could be pruned from search as well.

(4) Although in original iDistance, the reference points are fixed. If a certain number of data points whose k-th nearest distances are among the consecutive top ones are searched and can then retire, data distribution may change and the index structure can be replaced with new reference points and re-constructed so that the search process can be more effective.

Therefore, in our search process, first a search order should be established, the next data points can be processed and some can retire on the flying if satisfying the requirement, and then after a certain number of data points retire, the search structure can be updated with a smaller one. Based on these ideas, the following search structure is designed.

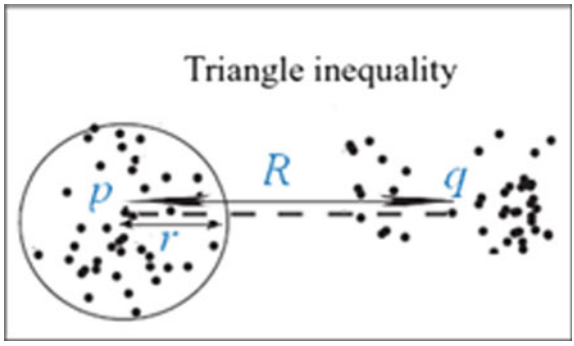


Fig. 8.3 Illustration of triangle inequality

8.3.2 Construction of a Simplest iDistance

With no a priori assumptions on the distribution model underlying the data, for simplicity, we consider only one reference point here, which is not freely chosen. To find it, we first take a random data point and calculate its distance to the rest of the database. The data point with the largest distance is selected as the reference point. By doing so, the largest distance span of the data distribution can be obtained and the worst case when all the data items have equal distances to the reference point will be diminished.

To construct the search facility, the distances between the reference point and the rest of the database are calculated as the index. Next, the data points are sorted according to their distances to the reference point in an increasing order and their original sequential position in the database are remembered. This process is summarized in Algorithm II shown in Table 8.2.

To be used as a search structure, given a data point, the search starts from its position in the sorted distances and proceeds bi-directionally, and only those data points which are within a searching radius are considered. Data points outside of the radius can be excluded in the calculation. If you have a smaller radius during the

Table 8.2 Algorithm II: a simple search structure

Input	data, a set of N data points		
Output	a distance array **Values**, two index arrays **Indexs** and **Edges**		
Let VectDist(data1, data2) return their distance			
Let Sort(V,I) sort N data points by values in V			
Let Values denote N distances			
Let Indexs denote N data indices			
Let Edge denote data point's position in Indexs			
1	start with a random data point p		
2	calculate its distances to all N data points		
3	select the reference point to be farthest to p, say q		
4	calculate the distances of q to all N data points		
		for i=1:N	
			Values[i]=VectDist(data[q],data[i]);
			Indexs[i]=i;
		end	
5	sort N data points by these distances		
6	remember data points' positions in **Indexs**		
		for i=1:N	
			Edges[Indexs [i]]=i
		end	
7	return **Values**, **Indexs** and **Edges**		

searching process, the radius can be updated. Thus, the use of the search structure is expected to reduce the $O(N)$ time full search by a faster $O(n < N)$ time partial search. It can be easily seen the search structure can be constructed by any one-dimensional sorting algorithm. The description of this search algorithm in pseudocode is given in Algorithm III shown in Table 8.3.

8.3.3 Radius Initialization

Before starting the search algorithm, an initial search radius for each data point has to be supplied, and the smaller the better. The distances of each data point to its k immediate predecessors or successors can be calculated when the data points are read in and these distances, whatever they are, provide an upper bound for the distances of each data item to its k nearest neighbors. However, if the data are randomly stored, the sequential initialization (SI) could be far from being optimal. In order to quickly provide the tight distance upper bounds, repeated invocations of a divisive hierarchical clustering algorithm (known as DHCA) have been shown to be effective to suit our purpose (Wang et al. 2009).

Essentially, given a data set, the DHCA starts with h randomly selected centers, and then assigns each point to its closest center, creating h partitions. At each stage in the iteration, for each of these h partitions, DHCA recursively selects h random centers and continues the clustering process, forming at most h^n partitions for the nth stage. In our implementation, the procedure proceeds until the number of elements in a partition is below $h + 2$, at which time, a pure nearest neighbor search is done with all the data items in that partition.

To remember k neighbors, in the implementation, there exist two arrays: an distance array, which is actually an array of priority queues of size $k + 1$ used to record the distances of each data point to some other data points in the sequentially stored data set, and an index array, which is actually an array of integer arrays of size k used to record the indices of k neighboring data items corresponding to the data entries in the distance array.

8.3.4 Our k*NN Search Structure*

Based on the findings presented in the previous subsections, our kNN search strategy can be developed as,

(1) Sequentially read the data set in, initialize the k neighbors of each data item from its immediate predecessors or successors, followed by running DHCA one time, then compute the 1-dimensional array of the k-th distances, the so-called search radius.

Table 8.3 Algorithm III: kNN search algorithm

Input	p, a data point, data, a set of N data points
	kNearN, a data structure holding data points' k neighbors
	Values, Indexs, Edges, the search structure
Output	k nearest neighbors of p and its k neighbors
Let VectDist(data1, data2) return their distance	
Let indexVector denote positions of k neighbors in the dataset	
Let valueVector denote distances of p to its k neighbors	
Let **needChecking** denote data points searched together with p	
1	decide ndata points that are searched with p
	needChecking.push_back(p);
	for i=1:k
	nn_index= kNearN[p].**indexVector**[i];
	if nn_index has not been searched
	needChecking.push_back(nn_inde);
	end
	end
	calculate distance r illustrated in Fig. 8.2;
2	determine p's position in the search structure and search
	m=Edges[p]; m=Edges[p]; up=true; down=true;
	while(up or down) {
	if (down) {
	m=m+1
	calculate distance R between p and Indexs[m]
	R=VectDist(data[p], data[Indexs[m]]
	if m<=N and R<=r
	update members of needChecking to **Indexs**[m]
	else if (m<=N & R>r & |Values[Edges[p]]-Values[m]|<r)
	continue;
	else
	down=false;
	} }
	if (up) {
	n=n-1;
	calculate distance R between p and Indexs[n]
	R=VectDist(data[p], data[Indexs[n]]
	if n>=1 and R<=r
	update members of NeedChecking to Indexs[n]

(continued)

Table 8.3 (continued)

				else if (n>=1 & R>r & |Values[Edges[p]]-Values[n]|<r)	
					continue;
				else	
					up=false;
				}	}
		}			
3	Return needChecking‘sindexVector&valueVector				

(2) To establish the search structure, take a random data point, and calculate its distances to the rest of the database. The data point with the largest distance is selected as the reference point. The distances between the reference point and the rest of the database are calculated as the index. Next, the data points are sorted according to their distances to the reference point in an increasing order and their original sequential index in the database are remembered.
(3) The search radii are sorted in non-increasing order and the search begins with the data item which has the largest distance to its current k-th neighbor and has not been searched yet.
(4) After each search, if the consecutive top ones in the sorted search radii are all searched, they can be eliminated from the search structure, if the number of the eliminated ones are larger than a predefined number, say $N/10$, go to (2); if the number of data points to be searched is larger than 0, go to (3); otherwise, exit.

Physically, the resource consumed by our algorithm includes the space for the whole data set to reside in memory, the space to store the nearest neighbors for each data item, the space for the run of DHCA and the space for the indexing structure.

8.3.5 Proposed EMST-Inspired Clustering Algorithm

Basically, our method works by first finding k nearest neighbors for each data point. Based on k-nearest neighbors distances, a minimum spanning tree is constructed for each connected component of the data. Since MST-based clustering algorithms can be more accurate and efficient if outliers can be correctly identified and quickly removed, next, the edges in the tree are sorted and cut in decreasing order until the number of feature vectors in the largest cluster is below a user-specified threshold determined through error-and-trial. Then any cluster which has more than 100 feature vectors is believed to represent a percept. Finally, a nearest neighbor labeling algorithm is used to assign labels to the remaining points to build up the database.

Suppose we are given a dataset D consisting of N objects with a dimension of d and the Euclidean distance is adopted in this study. The proposed algorithm is composed of the following three steps.

Step 1: (Preprocess the data D by performing k-nearest neighbors search)

1. Construct a simplest iDistance, and conduct k-nearest neighbors search on the given dataset consisting of N objects to produce a k-nearest neighbor graph P.

Step 2: (Construct an EMST)

2. Apply standard Prim's algorithm to construct an EMST on P. The resulted EMST(P) is contained in two arrays, one for the edge weights and the other for the parent index of each node.

Step 3: (Clustering upon the EMST)

3. The edges of the tree are sorted in a non-increasing manner and removed until the desired number of clusters are formed. Finally, a nearest neighbor labeling algorithm is used to assign labels to the remaining points to build up the database.

The above algorithm is a local heuristic that runs fast especially for high dimensional datasets. This method is simple to implement. In Step 1, we employ a kNN search method to produce a k-nearest neighbors graph P for connected component identification. In Step 2, an exact EMST is constructed on P. In Step 3, with the tree nodes' relationship being obtained in Step 2, an EMST-based clustering is performed by deleting tree edges until the desired number of clusters shown up. Finally, the nearest neighbor labeling algorithm is used to assign labels to the remaining points to build up the database. We would like to mention that, for low dimensional datasets, the proposed method may not outperform state-of-the-art EMST algorithms since the saved computations can be marginal.

8.3.6 Time Complexity Analysis

We note that since the Prim's algorithm is used for the construction of an EMST in our algorithm, the running time complexity of the proposed method is proportional to the number of nearest neighbors, that is, $O(kN \log N)$. For this reason, the scalability of the algorithm is proportional to the number of edges in k nearest neighbors graph of the data, instead of the size of the data set (that is, $O(N^2)$). At the same time, since the effect of dimension is essentially only dissimilarity measure dependent, it is possible to apply the proposed method to high-dimensional datasets.

8.4 Experiments and Results

In order to understand and manipulate the world, labeling local descriptors extracted from training images to target categories through machine learning methodologies is often a major task in the building of intelligent systems. In this section, we empirically

investigate the performance of the proposed EMST-inspired clustering algorithm by conducting three sets of experiments. In the first and second sets of experiments, images in an indoor environment and an outdoor environment are collected and segmented using the proposed algorithm to build a visual model of normality for mobile robot applications, respectively. In the third set of experiments, the efficiency of the proposed fast EMST algorithm is evaluated.

We implement all the algorithms in C++ and perform all the experiments on a computer with AMD A6-4400M Processor 2.70 GHz CPU and 4.00G RAM. The operating system running on this computer is Windows 7. In our evaluation of the runtime performances of EMST algorithms on different data sets, the timer utilities defined in the C standard library is used to report the CPU time. The results show that, overall, our proposed algorithm opens a door for a superior performance over state-of-the-art EMST algorithm on high dimensional datasets.

8.4.1 Experiment I—Perceptual Grouping in an Indoor Environment

In this subsection, we apply the proposed algorithm to segment images collected in an indoor environment. As shown in the left column of Fig. 8.4, the first working environment is an indoor hallway. Typical percepts along the hallway include Wood Panel, White Floor, Yellow Floor, Blue Wall, Blue Railing, Black Stripe Tile, Light Window. The images were obtained by a video camera. To generate a model of the environment so that the robot vision system can identify the major percepts existing in this indoor environment, and to build a percept database, 20 pictures of size 720-by-480 along the hallway were taken by the robot, from which 66,740 unlabeled feature vectors were extracted. Each feature vector contained a 10,000-dimensional color histogram and 41 more texture measures and was highly sparse small-valued data (the value in each dimension is in the range 0–1).

To organize patterns into "sensible" clusters, a major issue is to define the "similarity" between two feature vectors. In the multidimensional scaling literature, the city-block distance and the Euclidean distance are believed to represent two types of processing. For unitary or holistic stimuli, such as hue, saturation and brightness of colors, the closest approximation to an invariant relation between data and distances has uniformly been achieved in a space endowed with the familiar Euclidean metric, while for analyzable or separable stimuli, such as size and brightness difference (orientation), the closest approach to invariance has generally been achieved with the city-block metric (Shepard 1964, 1987; Garner 1974). To simulate this rule, the Euclidean distance is used for the HSV color histogram portion of the feature vector and the city-block distance is used for the texture measure portion of the feature vector. That is,

$$d_{ij} = \sqrt{\sum_{k=1}^{10,000} \left(x_{ik} - x_{jk}\right)^2} + \sum_{k=10001}^{10,040} |x_{ik} - x_{jk}| \tag{8.1}$$

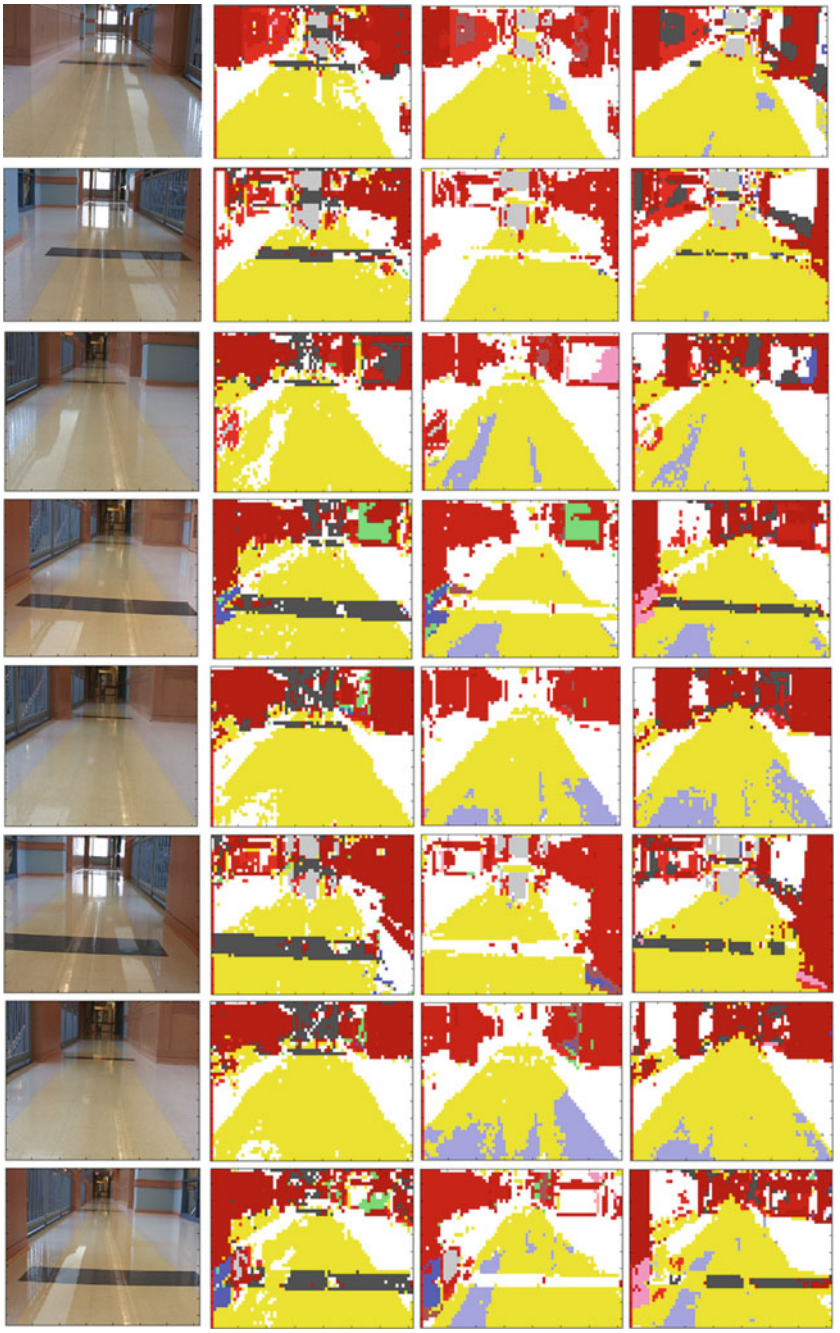


Fig. 8.4 (Left) The original images; (middle left) the results using the proposed metric; (middle right) the results using the Euclidean metric; (right) the results using the city-block metric

Table 8.4 Major percepts and their semantic meanings

Percept name	Percept symbol	Denoting color
Black Stripe	Object1	Black
Wood Panel	Object2	Light red
Light Window	Object3	Light grey
Wood Panel	Object4	Light red
White Floor	Object5	Dark white
Wood Panel	Object6	Dark red
Wood Panel	Object7	Red
Wood Panel	Object8	Light red
White Floor	Object9	White
Yellow Floor	Object10	Yellow
Yellow Floor	Object11	Blue

According to the proposed minimum spanning tree (MST)-based clustering algorithm, 11 perceptual groups are obtained. A detailed list of the major percepts present in the video sequences (not all objects appear on the same frames simultaneously but with some objects replacing others as the training sequence progresses) is provided in Table 8.4 together with their semantic meanings and a denoting color.

The segmentation results based on the Euclidean distance, the proposed mixed metric (i.e., Eq. 8.1) and the city-block distance for eight images are shown in Fig. 8.4. The ones in the first column are the original images. The ones in the second column are the segmented images using the proposed mixed metric. The ones in the third column are the segmented images using a Euclidean metric. Finally, the one in the fourth column is the segmented images using the city-block metric. From these results, in our qualitative visual evaluation, it seems that the mixed metric is superior to the Euclidean distance and city block distance. The segmentations using the Euclidean distance consistently misclassifies the black stripe, which is believed to be a very important percept. The segmentations using the city block distance find some of the black stripe, but miss part of it from time to time. The city block distance also tends to misclassify some wood panel as black. For the rest of the percepts, they give different misclassifications. The mixed metric and the Euclidean distance misclassifies the railing as the wood panel, while the city block distance sees it as the white floor, which is not too surprising since the railing is a hard percept to model. On the whole, it can be seen that all of the metrics generally do a reasonable job and clearly identify the yellow floor, the white floor, and the wood panel. Several perceptual groups obtained by the unsupervised learning algorithm actually correspond to the same object. To make the processed image visually pleasant, several finely differentiated colors are used to refer to these different objects in the machine vision perspective.

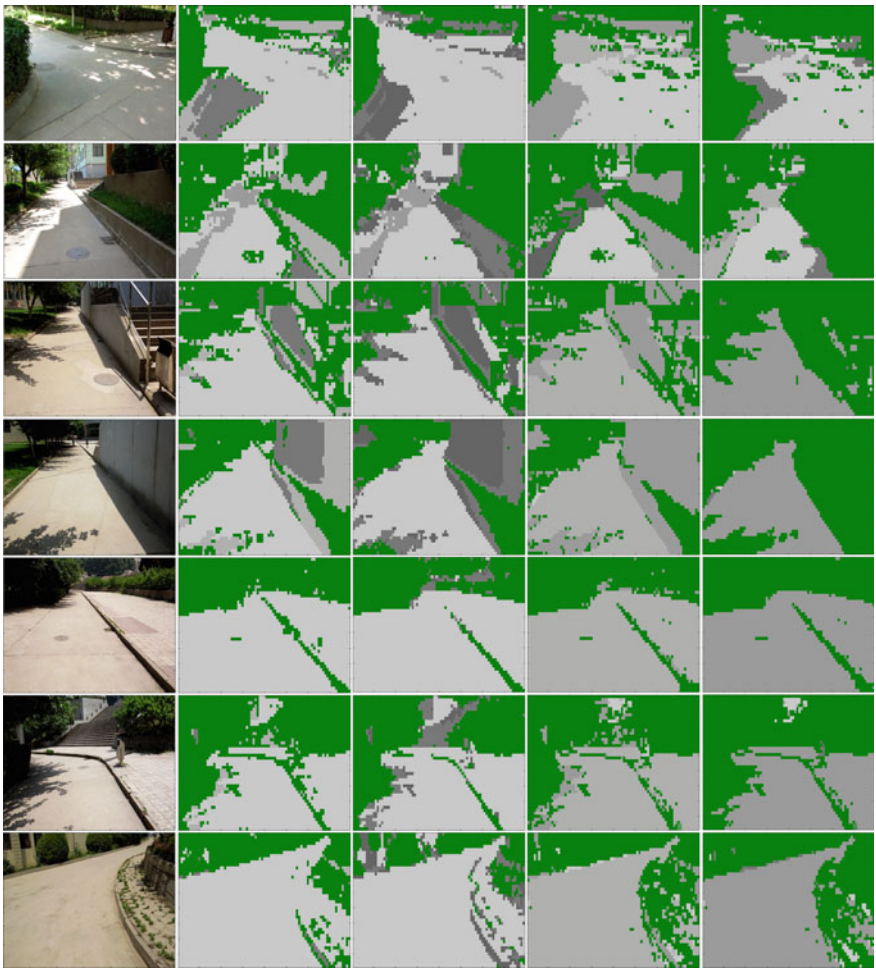


Fig. 8.5 (Left) Original images; clustering results for (middle left) HSV; (middle) OPP; (middle right) RGB; (right) transformed color histograms

8.4.2 Experiment II—Perceptual Grouping in an Outdoor Environment

The second image data was from an outdoor environment. To accurately express the image content, each of the training images is divided into small equally shifted blocks for which RGB color histograms of dimension 10,648, HSV color histograms of dimension 10,000, Opponent color histograms of dimension 10,648, and Transformed color histograms of dimension 10,648 are extracted using a window of size 24×24 pixels and a hopping size of 16 pixels resulting in totally 121,660 color histogram-based feature vectors. Next, the proposed MST-based clustering algorithm

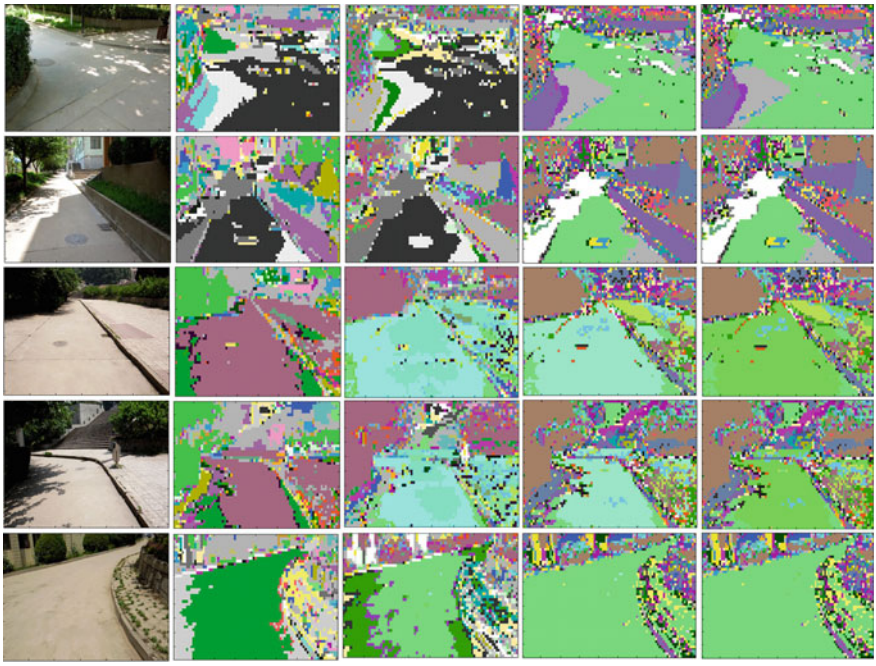


Fig. 8.6 (Left) Original images; clustering results of the LM method on (middle left) HSV histograms of dimension 10,000; (middle) OPP histograms of dimension 10,648; (middle right) RGB histograms of dimension 10,648; (right) Transformed histograms of dimension 10,648

is applied to RGB, HSV, Opponent, Transformed color histograms, respectively. The clustering results are presented in Fig. 8.5. In the figure, seven of 35 original scene images are displayed in the left column; the clustering results obtained by the proposed MST-based clustering algorithm are displayed in the second column for HSV color histogram, in the third column for OPP color histogram, in the fourth column for RGB color histograms and in the fifth column for Transformed color histogram, respectively.

It can be seen from Fig. 8.5 that, overall, the proposed approach does a reasonably good job on these very high-dimensional sparse feature data in comparison to other clustering methodologies presented in the previous three chapters which do well in relatively lower-dimensional color histograms. Specifically, the proposed method performs the best on the HSV color histograms and can identify most percepts existing in the environment. It performs the next on the OPP color histograms except for the image on the second row for which it misses the manhole cover percept. It performs less satisfactory on the RGB and Transformed color histograms by oversegmenting the concrete road percept existing in different images.

Then, two other popular graph-based clustering algorithms, Chameleon clustering algorithm (Karypis et al. 1999) and LM clustering algorithm (Laszlo and Mukherjee 2005), are applied to the RGB color histogram of dimension 10,648, HSV color

histogram of dimension 10,000, Opponent color histogram of dimension 10,648, and Transformed color histogram of dimension 10,648, and five sample clustering results are presented in Figs. 8.6 and 8.7, respectively.

In the two figures, five of 35 original scene images are displayed in the left column, the clustering results obtained by the LM algorithm are displayed in Fig. 8.5 for the four kinds of color histograms, and the clustering results obtained by the Chameleon algorithm are displayed in Fig. 8.6 for the four kinds of color histograms. From the segmented images in the five figures, it can be seen that the LM clustering algorithm did a better job on segmenting the road than the Chameleon clustering algorithm which split the roads into more segments. However, it can be seen that, overall, the segmentation results obtained by the proposed method have a better quality than those of the LM and Chameleon algorithms for percept modeling in these four high-dimensional color spaces.

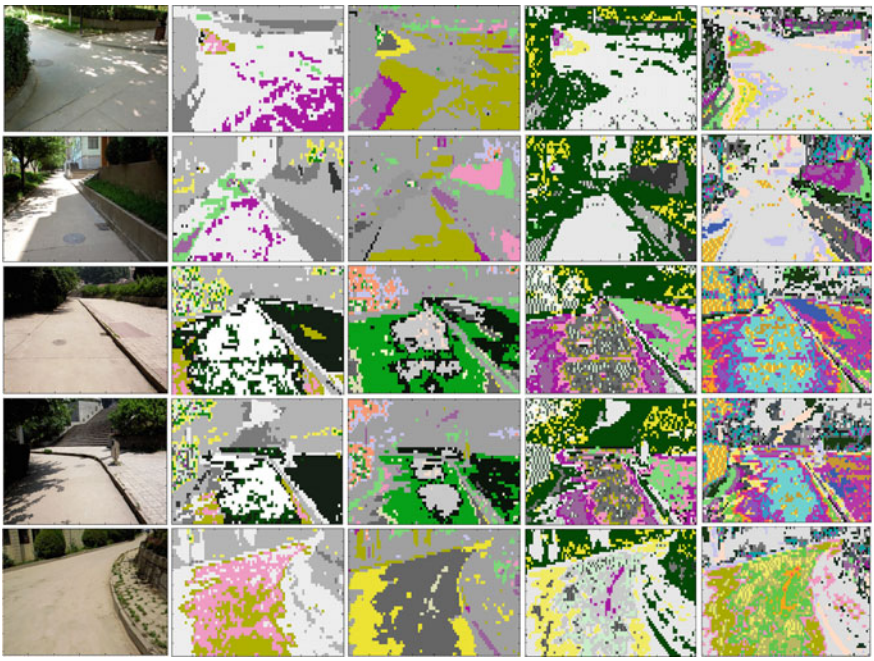


Fig. 8.7 (Left) Original images; clustering results of the Chameleon method on (middle left) HSV histograms of dimension 10,000; (middle) OPP histograms of dimension 10,648; (middle right) RGB histograms of dimension 10,648; (right) transformed histograms of dimension 10,648

Table 8.5 Running time for both datasets

	Image data 1	Image data 2-HSV	Image data 2-RGB	Image data 2-OPP	Image data 2-T
# of features	10,040	10,000	10,648	10,648	10,648
# of samples	66,740	95,550	95,550	95,550	95,550
Brute force (s)	80,784	602,197	515,797	460,062	430,508
iDistance (s)	28,369	105,887	100,987	95,925	82,273
Ours (s)	12,606	42,063	35,439	30,624	29,959

8.4.3 Experiment III—Running Time Issue

In this subsection, we evaluate the efficiency of the proposed EMST clustering algorithm. The relative results with regard to time issues are shown in Table 8.5 for these high-dimensional image data.

Since our data have such a high dimension, our algorithm is only compared with iDistance with multiple reference points. The numbers of features and the sizes of each data are listed on the second and third line of Table 8.5, respectively. The fourth line shows the total running time used to construct an EMST by Prim's algorithm. The fifth line shows the best total running time used by the iDistance algorithm, for which 8, 16, 32 and 64 reference points are used and the K-means algorithm is employed for partitioning the space. The sixth line shows the total running time used by our approach. The parameters of our algorithm were set to achieve their best performances. There is a considerable reduction in time consumed. It can tell that the proposed method takes less time than standard iDistance and brute force EMST method. This is to say, traditional Prim's method is very computationally expensive in high dimensional feature space.

8.5 Conclusions

In this chapter, an efficient EMST-based algorithm is presented for high-dimensional data. The proposed approach provides a better performance, while offering advantages of better run-time ability, which is extremely useful to assess which percepts of the environment are actually learnt by the system. The key innovations of the proposed approach are the use of a fast k-nearest neighbors search facility (as opposed to the widely used iDistance index structure). In contrast to Prim's algorithm, the proposed approach is particularly suitable for large high-dimensional data. In contrast to EMST based clustering, the proposed approach is able to autonomously identify multiple precepts in a frame. Overall the proposed approach is efficient and effective for video-analytical tasks in different environments.

References

Ajioka, S., Tsuge, S., Shishibori, M., & Kita, K. (2008). Fast multidimensional nearest neighbor search algorithm using priority queue. *Electrical Engineering in Japan, 164*(3), 69–77.

An, L., Xiang, Q. S., & Chavez, S. (2000). A fast implementation of the minimum spanning tree method for phase unwrapping. *IEEE Transactions on Medical Imaging, 19*(8), 805–808.

Angiulli, F., Basta, S., & Pizzuti, C. (2006). Distance-based detection and prediction of outliers. *IEEE Transactions on Knowledge and Data Engineering, 18*(2), 145–160.

Angiulli, F., & Fassetti, F. (2009). DOLPHIN: An efficient algorithm for mining distance-based outliers in very large datasets. *ACM Transactions on Knowledge Discovery from Data, 3*(1), 1–57, Article 4.

Angiulli, F., & Pizzuti, C. (2005). Outlier mining in large high dimensional datasets. *IEEE Transactions on Knowledge and Data and Engineering, 12*(2), 203–215.

Ankerst, M., Breunig, M., Kriegel, H. P., & Sander, J. (1999). OPICS: Ordering points to identify the clustering structure. In *Proceedings of the 1999 ACM International Conference on Management of Data (SIGMOD '99)* (pp. 49–60), Philadelphia.

Arya, S., & Mount, D. M. (2016). A fast and simple algorithm for computing approximate Euclidean minimum spanning trees. In: *Twenty-Seventh ACM-SIAM Symposium on Discrete Algorithms* (pp. 1220–1233).

Beckmann, N., Kriegel, H. P., Schneider, R., & Seeger, B. (1990). The R*-tree: An efficient and robust access method for points and rectangles. In *Proceedings of the ACM SIGMOD International Conference on Management of Data* (pp. 322–331), Atlantic City, NJ.

Bentley, J. L. (1970). Multidimensional binary search trees in database applications. *IEEE Transactions on Software Engineering, 5*(4), 333–340.

Bentley, J., & Friedman, J. (1978). Fast algorithms for constructing minimal spanning trees in coordinate spaces. *IEEE Transactions on Computers, 27,* 97–105.

Blott, S., & Weber, R. (1997). *A simple vector-approximation file for similarity in high-dimensional vector spaces* (Technical Report Nr. 19, ESPRIT Project HERMES, No. 9141).

Borůvka, O. (1926). O jist´emprobl´emuminim´aln´ım (About a certain minimal problem). In *Pr´acemoravsk´ep˘r´ırodov˘edeck´espole˘cnosti v Brn˘e*. III (pp. 37–58) (in Czech with German summary).

Callahan, P., & Kosaraju, S. (1993). Faster algorithms for some geometric graph problems in higher dimensions. In *Proceedings of 4th Annual ACM-SIAM Symposium on Discrete Algorithms* (pp. 291–300).

Chaoji, V., Hasan, M., Salem, S., & Zaki, M. J. (2008). SPARCL: Efficient and effective shape-based clustering. In *Proceedings of 8th IEEE International Conference on Data Mining* (pp. 93–102), Washington, DC.

Cormen, T. T., Leiserson, C. E., & Rivest, R. L. (2009). Introduction to algorithms. *Resonance, 1*(9), 14–24.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269–271.

Ester, M., Kriegel, H. P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference of Knowledge Discovery and Data Mining* (pp. 226–231), Portland, Oregon.

Garner, W. R. (1974). *The processing of information and structure*. New York: Wiley.

Guha, S., Rastogi, R., & Shim, K. (1998). CURE: An efficient clustering algorithm for large databases. In *Proceedings of the ACM–SIGMOD International Conference on Management of Data* (pp. 73–84), Seattle, WA.

Guha, S., Rastogi, R., & Shim, K. (1999). Rock: A robust clustering algorithm for categorical attributes. In *Proceedings of 15th International Conference on Data Engineering* (pp. 512–536), Washington, DC.

Guttman, A. (1984). R-trees: A dynamic index structure for spatial searching, In *Proceedings of the ACM SIGMOD International Conference on Management of Data* (pp. 47–57), New York, NY.

Han, J., & Kamber, M. (2001). *Data mining concepts and techniques*. Beijing: Higher Education Press.

Hinneburg, A., & Keim, D. A. (1998). An efficient approach to clustering in large multimedia databases with noise. In *Proceedings of the 1998 International Conference on Knowledge Discovery and Data Mining (KDD '98)* (pp. 58–65), New York.

Jagadish, H. V., Ooi, B. C., Tan, K. L., Yu, C., & Zhang, R. (2005). iDistance: An adaptive B+ -tree based indexing method for nearest neighbor search. *ACM Transactions on Data Base Systems (ACM TODS), 30*(2), 364–397.

Jain, A. K., Murty, M. N., & Flynn, P. J. (1999). Data clustering: A review. *ACM Computing Surveys (CSUR), 31*(3), 264–323.

Jarník, V. (1930). O jist´emprobl´emuminim´aln´ım (About a certain minimal problem). In Pr´acemoravsk´epˇr´ırodovˇedeck´espoleˇcnosti v Brnˇe VI (pp. 57–63) (in Czech).

Jothi, R., Mohanty, S. K., & Ojha, A. (2015). *Fast minimum spanning tree based clustering algorithms on local neighborhood graph*. Springer International Publishing.

Jothi, R., Mohanty, S. K., & Ojha, A. (2018). Fast approximate minimum spanning tree based clustering algorithm. *Neurocomputing, 272*.

Juszczak, P., Tax, D. M. J., Pękalska, E., & Duin, R. P. W. (2009). Minimum spanning tree based one-class classifier. *Neurocomputing, 72*, 1859–1869.

Karypis, G., Han, E. H., & Kumar, V. (1999). Chameleon: Hierarchical clustering using dynamic modeling. *IEEE Computer, 32*(8), 68–75 (Special Issue on Data Analysis and Mining).

Katayama, N., & Satoh, S. (1997). The SR-tree: An index structure for high-dimensional nearest neighbor queries. In *Proceedings of ACM SIGMOD International Conference on Management of Data* (pp. 369–380), Tucson, Arizona.

Khan, M., & Pandurangan, G. (2008). A fast distributed approximation algorithm for minimum spanning trees. *Distributed Computing, 20*(6), 391–402.

Kruskal, J. B. (1956). On the shortest spanning subtree of a graph and the traveling salesman problem. *Proceedings of the American Mathematical Society, 7,* 48–50.

Laszlo, M., & Mukherjee, S. (2005). Minimum spanning tree partitioning algorithm for microaggregation. *IEEE Transactions on Knowledge and Data Engineering, 17*(7), 902–911.

Li, C., Becerra, V. M., & Deng, J. (2004). Extension of fuzzy c-means algorithm. In *Proceedings of the IEEE Conference on Cybernetics and Intelligent Systems* (pp. 1–3), Singapore.

Liu, Z., Wang, C., Zou, P., Zheng, W., & Wang, J. (2010). iPoc: A polar coordinate based indexing method for nearest neighbor search in high dimensional space. In L. Chen et al. (Eds.), *WAIM 2010, LNCS* (Vol. 6184, pp. 345–356). Springer.

March, W. B., Ram, P. and Gray, A.G. (2010). Fast Euclidean minimum spanning tree: Algorithm, analysis, and applications. In *Proceedings of 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '10)* (pp. 603–612), Washington, DC.

Narasimhan, G., Zachariasen, M., & Zhu, J. (2000). Experiments with computing geometric minimum spanning trees. In *Proceedings of ALENEX '00* (pp. 183–196).

Ng, R., & Han, J. (1994). Efficient and effective clustering method for spatial data mining. In *Proceedings of the 20th International Conference of Very Large Databases* (pp. 144–155), Santiago, Chile.

Pappula, P. (2017). An efficient clustering algorithm of minimum spanning tree. In *Proceedings of the International Conferences on Advances in Electrical, Electronics, Information, Communication and Bio-informatics*.

Pettie, S., & Ramachandran, V. (2002). A randomized time-work optimal parallel algorithm for finding a minimum spanning forest. *SIAM Journal on Computing, 31*(6), 1879–1895.

Preparata, F. P., & Shamos, M. I. (1985). *Computational geometry*. New York: Springer.

Prim, R. C. (1957). Shortest connection networks and some generalizations. *Bell System Technical Journal, 36,* 567–574.

Sheikholeslami, G., Chatterjee, S., & Zhang, A. (1998). Wavecluster: A multi-resolution clustering approach for very large spatial databases. In *Proceedings of International Conference on Very Large Data Bases (VLDB '98)* (pp. 428–439), New York.

Shepard, R. N. (1964). Attention and the metric structure of the stimulus space. *Journal of Mathematical Psychology, 1,* 54–87.

Shepard, R. N. (1987). Toward a universal law of generalization for psychological science. *Science, 237,* 1317–1323.

Wang, X., Wang, X. L., Chen, C., & Wilkes, D. M. (2013). Enhancing minimum spanning tree-based clustering by removing density-based outliers. *Digital Signal Processing, 23*(5), 1523–1538.

Wang, X., Wang, X. L., & Wilkes, D. M. (2009). A divide-and-conquer approach for minimum spanning tree-based clustering. *IEEE Transactions on Knowledge and Data Engineering, 21*(7), 945–958.

Wang, W., Yang, J., & Muntz, R. (1997). STING: A statistical information grid approach to spatial data mining. In *Proceedings of the 1997 International Conference on Very Large Databases (VLDB '97)* (pp. 186–195), Athens, Greece.

White, D. A., & Jain, R. (1996). Similarity indexing with the SS-tree. In *Proceedings of the 12th IEEE International Conference on Data Engineering* (pp. 516–523), Washington, DC.

Xu, Y., Olman, V., & Xu, D. (2002). Clustering gene expression data using a graph-theoretic approach: An application of minimum spanning trees. *Bioinformatics, 18*(4), 536–545.

Xu, Y., & Uberbacher, E. C. (1997). 2D image segmentation using minimum spanning trees. *Image and Vision Computing, 15,* 47–57.

Yu, C., Ooi, B. C., Tan, K. L., & Jagadish, H. V. (2001). Indexing the distance: An efficient method to KNN processing. In *Proceedings of the 27th International Conference on Very large Databases* (pp. 421–430), Roma, Italy.

Zahn, C. T. (1971). Graph-theoretical methods for detecting and describing gestalt clusters. *IEEE Transactions on Computers, C20,* 68–86.

Zhang, T., Ramakrishnan, R., & Linvy, M. (1996). Birch: An efficient data clustering method for large databases. In *Proceedings of the 1996 ACM–SIGMOD International Conference on Management of Data* (pp. 103–114), Montreal, Quebec.

Zhong, C., Malinen, M., Miao D., & Fränti, P. (2015). A fast minimum spanning tree algorithm based on K-means. *Information Sciences, 295*(C), 1–17.

Zhong, C., Miao, D., & Wang, R. (2010). A graph-theoretical clustering method based on two rounds of minimum spanning trees. *Pattern Recognition, 43*(3), 752–766.

Part III
Supervised Learning and Semi-supervised Learning

Chapter 9
Supervised Learning for Data Classification Based Object Recognition



Abstract With the development of computer vision, robots need to recognize targets from image sequences by supervised learning for autonomous navigation. To identify objects, the percept learning system of autonomous robots using unsupervised learning presented in the last part has segmented the images into nonoverlapping but meaningful regions based on low-level features such as color, texture measures and so on. In this chapter, we give an overview of the field of supervised learning for data classification in general and for object recognition tasks from the perspective of an instance-based classifier in particular. We begin with an introduction to the concepts of classification, followed by instance-based classification methods in specific. The semi-supervised learning is next described. Finally, the performance evaluation metrics are reviewed.

Keywords Supervised learning · Data classification · Decision tree · Perceptron · Support vector machine · Instance-based classifier · Semi-supervised learning

9.1 Introduction

Unlike unsupervised learning where the machine is given no supervision or reward, in supervised learning, the machine is also given a sequence of desired outputs, and the goal of the machine is to learn to produce the correct output given a new input. Therefore, a model learned by unsupervised learning can be used to evaluate the probability of a new sensor reading and therefore can also be used for classification, that is, to estimate a label for a new input x given previously labeled inputs. For example, assume $P_1(x)$ is a model of the attributes for one category, and $P_2(x)$ is a model learned from another category. By evaluating the relative probabilities $P_1(x)$ and $P_2(x)$ on a new applicant x, the machine can decide to classify it into one of these two categories.

Contrary to clustering being an unsupervised learning, classification is a supervised way of learning. The classification problem is that of learning the structure of a data set which have already been partitioned into groups that are referred to as categories or classes. The database contains one or more attributes that denote the class of a tuple and are known as predicted attributes, whereas the remaining

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_9

attributes are called predicting attributes. A combination of the predicted attributes defines a class.

There are a variety of classifiers. Although in the process of classification, either the labeled training dataset is directly used, or an abstraction or model learnt from the training dataset is used, the learning of these categories is typically achieved with a model. This model is used to estimate a group identifier (or class label) for one or more previously unseen data examples with no labels. Therefore, one of the inputs to the classification problem is an example data set that has already been partitioned into different classes. This is referred to as the training data, and the group identifiers of these classes are referred to as class labels. In most cases, the class labels have a clear semantic interpretation in the context of a specific application, such as a group of data objects with a desired property of interest. The model learned is referred to as the training model. The previously unseen data points that need to be classified are collectively referred to as the test data set. The algorithm that creates the training model for prediction is also sometimes referred to as the learner.

Classification is, therefore, referred to as supervised learning because an example data set is used to learn the structure of the groups, just as a teacher supervises his or her students towards a specific goal. While the groups learned by a classification model may often be related to the similarity structure of the feature variables, as in clustering, this need not necessarily be the case. In classification, the example training data is paramount in providing the guidance of how groups are defined. Given a data set of test examples, the groups created by a classification model on the test examples will try to mirror the number and structure of the groups available in the example data set of training instances. Therefore, the classification problem may be intuitively defined. The training process is determined to be completed when all the patterns in the training set have been correctly classified. Next the system is presented with new patterns that have not been seen before but belong to the same population of patterns used to train the system (Bow 2002).

Definition 9.1 Given a set of training data points, each of which is associated with a class label, classification is the process to determine a class label for one or more previously unseen test instances. Most classification algorithms typically have two phases,

- Training phase. In this phase, a training model is constructed from the training instances. Intuitively, this can be understood as a summary mathematical model of the labeled groups in the training data set.
- Testing phase. In this phase, the training model is used to determine a class label (or group identifier) for one or more unseen test instances.

The classification problem is more powerful than clustering because, unlike clustering, it captures a user-defined notion of grouping from an example data set. The applications of classification are diverse because of the ability to learn by example. Such an approach has almost direct applicability to a wide variety of problems, in which groups are defined naturally based on external application-specific criteria.

It is assumed that the training data set is denoted by D with N data points and d features, or dimensions. In addition, each of the data points in D is associated with a label drawn from $\{1 \ldots k\}$. A training model is constructed from D, which is used to predict the label of an unseen test instance. The output of a classification algorithm can be a label prediction, in which case, a label is predicted for each test instance.

Like clustering, various models have been designed for data classification. The most well-known ones include decision trees, rule-based classifiers, probabilistic models (Bayesian classifiers), instance-based classifiers, support vector machines, and neural networks, etc. The modeling phase is often preceded by a feature selection phase to identify the most informative features for classification. Of these methods, decision trees, support vector machines, neural networks and instance-based classifiers will be briefly reviewed in this chapter.

This chapter is organized as follows. Section 9.2 introduces decision tree models. A neural network classifier, perceptron, is described in Sect. 9.3. Support vector machine approaches are introduced in Sect. 9.4. Instance-based learning methods are explained in Sect. 9.5. A brief introduction to semi-supervised learning is given in Sect. 9.6. Evaluation methods are addressed in Sect. 9.7. The summary is presented in Sect. 9.8.

9.2 Decision Trees

Classification aims to develop rules from the data in the training set. This is often done in the implicit form of a decision tree. Decision trees are a classification methodology, wherein the classification process is modeled with the use of a set of hierarchical decisions on the feature variables, arranged in a tree-like structure. There are two types of nodes, referred to as the internal nodes and leaf nodes. Each leaf node is labeled with the dominant class at that node. A special internal node is the root node that corresponds to the entire feature space. A decision tree is created by a process known as splitting on the value of attributes (or just splitting on attributes), i.e. testing the value of an attribute and then creating a branch for each of its possible values. The decision at a particular node of the tree, which is referred to as the split criterion, is typically a condition on one or more feature variables in the training data, by which the training data are divided into two or more parts. For the example as illustrated in the right plot of Fig. 9.1, the decision at the root node is an attribute Age and the split criterion is Age $\leq$50. In this case, the left branch of the decision tree contains all training examples with age at most 50, whereas the right branch contains all examples with age greater than 50. The goal of decision trees is to identify a split criterion so that the level of "mixing" of the class variables in each branch of the tree is reduced as much as possible. Each node in the decision tree logically represents a subset of the data space defined by the combination of split criteria in the nodes above it. The generic decision tree induction algorithm starts with the full training data set at the root node and recursively partitions the data into lower level nodes based on the split criterion. Only nodes that contain a mixture of different classes need to be split

Algorithm *GenericDecisionTree*(Data Set: $\mathcal{D}$)
begin
  Create root node containing $\mathcal{D}$;
  repeat
    Select an eligible node in the tree;
    Split the selected node into two or more nodes
      based on a pre-defined split criterion;
  until no more eligible nodes for split;
  Prune overfitting nodes from tree;
  Label each leaf node with its dominant class;
end

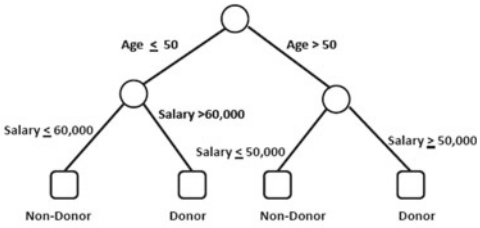


Fig. 9.1 (Left) Generic decision tree training algorithm; (right) an illustration of decision tree working principle

further. Eventually, the decision tree algorithm stops the growth of the tree based on a stopping criterion. The simplest stopping criterion is one where all training examples in the leaf belong to the same class. Like a top-down clustering algorithm which partitions the data hierarchically, the decision tree is typically constructed as a hierarchical partitioning of the training examples. Unlike clustering, the partitioning criterion in the decision tree is supervised with the class label in the training instances. Some classic decision tree algorithms include ID3 (Quinlan 1986), C4.5 (Quinlan 1993), and CART. The generic decision tree training algorithm is illustrated in the left part of Fig. 9.1.

After a decision tree has been constructed, it is used for classification of unseen test instances with the use of top-down traversal from the root to a unique leaf. The split condition at each internal node is used to select the correct branch of the decision tree for further traversal. The label of the leaf node that is reached is reported for the test instance. For the example in the right plot of Fig. 9.1, a test instance with age less than 50 and salary less than 60,000 will traverse the leftmost path of the tree. Because the leaf node of this path contains only nondonor training examples, the test instance will also be classified as a nondonor category that the leaf node of this path contains.

In general, simpler models (shallow decision trees) are preferable to more complex models (deep decision trees) if they produce the same error on the training data. Multivariate splits use more than one attribute in the split criteria and are more powerful because they lead to shallower trees. For the same level of class separation in the training data, shallower trees are generally more desirable because the leaf nodes contain more examples and, therefore, are statistically less likely to overfit the noise in the training data.

9.3 Perceptron

Being a process of classifying a pattern into the right category, the pattern classification (or pattern recognition) system is expected to have the ability to improve the performance through iterative adjustment of the system parameters in the hope

that, after an iteration of the learning process, the system will become a more knowledgeable and effective system and produce a higher recognition rate. Usually, the pattern recognition system will first undergo a training process, during which the system is repeatedly presented with a set of input patterns, the so-called training set (or prototypes). Since each input pattern is given along with the category to which this particular pattern belongs, one way to pursue the classification task is to find separating boundaries based on these input–output pairs.

For supervised learning, decision surfaces can formally be defined to be separating boundaries between different regions in the feature space. Patterns on the decision surface are of equal similarity to both regions separated by this decision surface. For d-dimensional pattern points, decision surfaces are $(d - 1)$-dimensional. The decision surface between two classes, i, j, can be defined as

$$d_i\left(\vec{x}\right) - d_j\left(\vec{x}\right) = f_i\left(\vec{w}, \vec{x}\right) - f_j\left(\vec{w}, \vec{x}\right) = 0 \tag{9.1}$$

where $d(x) = f\ (w, x)$ is called the discriminant function, $x = [x_1, x_2, \ldots, x_d, 1]^{\mathrm{T}}$ and $\vec{w} = [w_1, w_2, \ldots, w_d, w_{d+1}]^{\mathrm{T}}$ represent the augmented feature vector in pattern space and the weight vector in weight space, respectively. The weight space is a $(d + 1)$-dimensional Euclidean space in which the coordinate variables are $w_1, w_2, \ldots, w_d, w_{d+1}$. For example, if there are M classes and x is in class i, then,

$$d_i\left(\vec{x}\right) > d_j\left(\vec{x}\right) \quad \forall j \neq i, \quad j = 1, \ldots, M \tag{9.2}$$

The decision surface that separates classes i and j is then given by,

$$d\left(\vec{x}\right) = d_i\left(\vec{x}\right) - d_j\left(\vec{x}\right) = 0 \tag{9.3}$$

A linear discriminant function for a class k will be of the following form,

$$d_k\left(\vec{x}\right) = w_{k,1}x_1 + w_{k,2}x_2 + \cdots + w_{k,d}x_d + w_{k,d+1} = \vec{w}_k^{\mathrm{T}} \cdot \vec{x} \tag{9.4}$$

A nonlinear discriminant function for a class k can be described as

$$d\left(\vec{x}\right) = w_1 f_1\left(\vec{x}\right) + w_2 f_2\left(\vec{x}\right) + \cdots + w_M f_M\left(\vec{x}\right) + w_{M+1} \tag{9.5}$$

where $f_i(x), i = 1, 2, \ldots, M$ are linearly independent, real and single-valued functions which are independent of the w_i (weight).

The objective of training a system is actually to find the weight vectors with the a priori information obtained from the training samples. Initially, the weight vector is chosen as random small numbers. During the training period, patterns are presented one at a time through all prototypes (training patterns) and the weights are updated to

minimize the errors between the target value and the computed value. Each complete pass through all the patterns is called an iteration. After one iteration, all patterns are presented again in the same sequence to carry on another iteration. This is repeated until no correction is made through one complete iteration. If the prototypes are linearly separable, the algorithms may converge to a correct weight vector $\vec{w}$. Two commonly used supervised learning methods are statistical discriminant functions and neural network approaches. Detailed descriptions of these supervised learning methods are beyond the scope of this book but can be found in many pattern recognition books such as (Bow 2002), and therefore will not be repeated here. In the following, only a very simple neural network, a perceptron, is presented.

Neural networks consist of a large number of simple processing units called artificial neurons as shown in Fig. 9.2. The goal of neural networks is to process information in a way so as to map multiple inputs to a single output. The basic operation of an artificial neuron is to apply an activation function to a weighted sum of input vector plus a bias. The most commonly used activation functions are the step function, the identity function, and the sigmoid function. Often, the activation values are restricted to the range [0, 1].

Perceptron neural network models (Kulkarni 2001) use a step activation function and are designed as a classifier for linearly separable input samples. The simplest perceptron network is one with a single artificial neuron and is useful for a two-class problem or a binary decision. The weights and bias of the artificial neuron define a linear surface that separates the two classes. At the beginning of the training session, small random values are used for the weights and the bias. Then an input vector is presented, and the output is calculated. If the actual output is the same as the desired output, stop and go to the next input vector. Otherwise, update the weights and bias as

$$w_i(k+1) = w_i(k) + \alpha(\bar{o} - o)x_i \tag{9.6}$$

$$\varphi(k+1) = \varphi(k) + \alpha(\bar{o} - o) \tag{9.7}$$

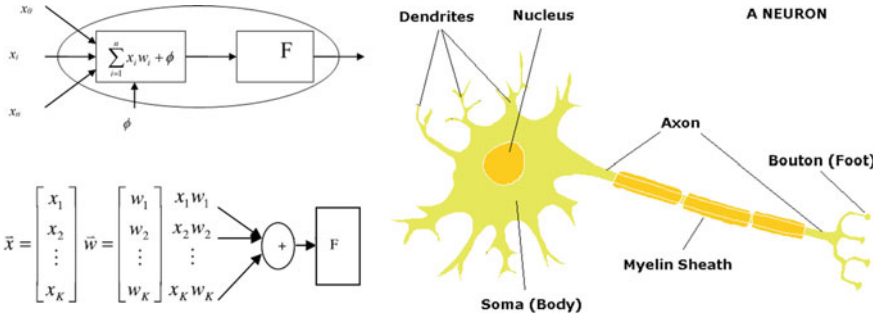


Fig. 9.2 (Left) An artificial neuron; (right) a biological neuron

$$o(t) = F = \frac{1}{1 + \exp\left(-\sum_{i=1}^{k} x_i w_i - \varphi\right)} \tag{9.8}$$

where k represents the iteration number and $(\bar{o} - o)$ represents the error between the desired output and the actual output, $o(t)$ is the output of the perceptron. This update process is repeated for each input vector until there is no change in the weights or the maximum number of iterations is reached. For problems with more than two classes, the extension from a single neuron perceptron to a multi-neuron perceptron is obvious and weight vectors exist for each artificial neuron, forming a weight matrix for all the artificial neurons.

9.4 Support Vector Machine

Being a set of related supervised learning methods belonging to a family of generalized linear classifiers for classification and regression, support vector machines (SVMs) simultaneously minimize the empirical classification error and maximize the geometric margin. For two-class classification problems in a d-dimensional space, the SVMs will construct a separating hyperplane which maximizes the "margin" between two data sets, as illustrated in the left plot of Fig. 9.3.

A classification task usually involves with training and testing data which consist of some data instances. Given a training dataset of N points of the form $\{(x_1, y_1), \ldots, (x_N, y_N)\}$ where each x_i is a d-dimensional real vector while the y_i's are either 1 or -1, each indicating the class to which the point x_i belongs, two parallel hyperplanes are constructed, one on either side of the hyperplane separating the data. The goal is to separate such points with a $(d - 1)$-dimensional hyperplane that has the largest separation, or so-called functional margin, between the two classes, since in general the larger the margin, the lower the generalization error of the classifier (Hastie et al. 2017). To find the "maximum-margin hyperplane" that divides the group of points

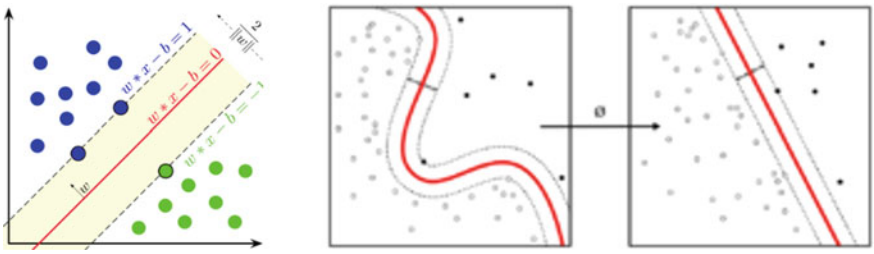


Fig. 9.3 (Left) Separating hyperplanes and the maximal "margin" (middle) nonlinear separable (right) linear separable

x_i for which $y_i = 1$ from the group of points for which $y_i = -1$, the distance between the hyperplane and the nearest point x_i from either group should be maximized. Any hyperplane can be written as the set of points x satisfying,

$$w \cdot x + b = 0 \tag{9.9}$$

where w is the (not necessarily normalized) normal vector to the hyperplane. The parameter $b/\|w\|$ determines the offset of the hyperplane from the origin along the normal vector w.

If the training data is linearly separable, two parallel hyperplanes that separate the two classes of data can be selected so that the distance between them is as large as possible. The region bounded by these two hyperplanes is called the "margin", and the maximum-margin hyperplane is the hyperplane that lies halfway between them. With a normalized or standardized dataset, these hyperplanes can be described by the equations,

$$w \cdot x + b = 1 \tag{9.10}$$

anything on or above this boundary is of one class, with label 1, and

$$w \cdot x + b = -1 \tag{9.11}$$

anything on or below this boundary is of the other class, with label -1. Geometrically, the distance between these two hyperplanes is $2/\|w\|$, so to maximize the distance between the planes we want to minimize $\|w\|$. The distance is computed using the distance from a point to a plane equation. To prevent data points from falling into the margin, the following constraint should be added. That is, for each i either,

$$w \cdot x + b \geq 1, \text{ if } y_i = 1 \tag{9.12}$$

or

$$w \cdot x + b \leq -1, \text{ if } y_i = -1 \tag{9.13}$$

These constraints state that each data point must lie on the correct side of the margin. This can be rewritten as

$$y_i(w \cdot x + b) \geq 1, \text{ for all } \; 1 \leq i \leq n \tag{9.14}$$

To put this together, the optimization problem is to minimize $\|w\|$ subject to,

$$y_i(w \cdot x + b) \geq 1 \text{ for } i = 1, \ldots, N \tag{9.15}$$

The w and b that solve this problem determine our classifier,

$$\mathrm{x} \rightarrow \mathrm{sgn}(w \cdot x + b). \tag{9.16}$$

An important consequence of this geometric description is that the max-margin hyperplane is completely determined by those x_i that lie nearest to it. These x_i are called support vectors.

The original SVM algorithm was invented by Vapnik and Chervonenkis in 1963 (Vapnik and Chervonenkis 2006). However, it often happens that the sets to discriminate are not linearly separable in the original problem. In 1992, Boser et al. suggested a way to create nonlinear classifiers by applying the kernel trick to maximum-margin hyperplanes (Boser et al. 1992). The current standard incarnation (soft margin) was proposed by Cortes and Vapnik in 1993 and published in 1995 (Cortes and Vapnik 1995). More specifically, it was proposed that the original finite-dimensional space be mapped into a much higher- or infinite-dimensional space, wherein a maximal separating hyperplane can be constructed (Cortes and Vapnik 1995). The resulting algorithm is formally similar, except that every dot product is replaced by a nonlinear kernel function. This allows the algorithm to fit the maximum-margin hyperplane in a transformed feature space. The transformation may be nonlinear and the transformed space may be high-dimensional. Although the classifier is a hyperplane in the transformed feature space, it may be nonlinear in the original input space. It is noteworthy that working in a higher-dimensional feature space increases the generalization error of support-vector machines, although, given enough samples, the algorithm still performs well (Jin and Wang 2012).

Being the solution, the support vector machines require the solution of the following optimization problem,

$$\min_{w,b,\xi} \frac{1}{2} w^{\mathrm{T}} w + C \sum_{i=1}^{l} \xi_i \quad \text{subject to } y_i\left(w^{\mathrm{T}} \phi(x_i) - b\right) \geq 1 - \xi_i \xi_i \geq 0, \tag{9.17}$$

where the training vectors x_i are mapped into a higher (may be infinite) dimensional space by the function $\phi(x)$. Then the SVM finds a linear separating hyperplane with the maximal margin in this higher dimensional space. $C > 0$ is the penalty parameter of the error term. $K\left(x_i, x_j\right) \equiv \phi(x_i)^{\mathrm{T}} \phi\left(x_j\right)$ is called the kernel function, four basic forms of which are:

- Linear: $K\left(x_i, x_j\right) = x_i^{\mathrm{T}} x_j$.
- Polynomial: $K\left(x_i, x_j\right) = \left(\gamma x_i^{\mathrm{T}} x_j + r\right),\ \gamma > 0$.
- Radial basis function: $K\left(x_i, x_j\right) = \exp\left(-\gamma ||x_i - x_j||^2\right),\ \gamma > 0$.
- Sigmoid: $K\left(x_i, x_j\right) = \tanh\left(x_i^{\mathrm{T}} x_j + r\right)$.

where γ and r are kernel parameters. The set of points x mapped into any hyperplane can be quite convoluted as a result, allowing much more complex discrimination between sets that are not convex at all in the original space. Detailed description of the SVM approach is beyond the scope of this book and will not be repeated here.

9.5 Instance-Based Learning

In the classifiers discussed in the previous sections, the classification model is constructed up front and then used to classify a specific test instance. Such classifiers are generally called eager learners. In instance-based learning, the training is delayed until the last step of classification. Such classifiers are also referred to as lazy learners. The simplest principle to describe instance-based learning is that similar instances have similar class labels. A natural approach for leveraging this general principle is to use nearest-neighbor classifiers.

9.5.1 k-*Nearest-Neighbour Classifier (kNNC)*

The Nearest-Neighbor Classifier (NNC) is the simplest and popular classifier and classifies the test pattern by using the training patterns directly. In this classifier, for a given test instance, the closest k training examples are determined. The dominant label among these k training examples is reported as the relevant class. While computing the k-nearest neighbors of a training instance x, the data point x is not included among the nearest neighbors.

The major challenge with the use of the nearest-neighbor classifier is the choice of the parameter k. Generally, a very small value of k will not lead to robust classification results because of noisy variations within the data, while large values of k will lose sensitivity to the underlying data locality. In practice, the appropriate value of k is chosen in a heuristic way. A common approach is to test different values of k for accuracy over the training data. The popular nearest-neighbor classifier has several advantages. First, it is easy to understand and implement. Second, there is no learning or training phase and the whole training data is used to classify the test pattern. Thirdly, unlike the Bayes classifier, it does not require the probability structure of the classes. An important property of the NNC is that its error rate is less than twice the error rate of the Bayes classifier when the number of training patterns is asymptotically large. Last but not least, it shows good performance. If the optimal accuracy is 99.99%, then with a large training data, it can give at least 99.80% accuracy.

Unfortunately, even though it is popular, there are some negative aspects. First, it is sensitive to noise. If the $NN(x)$ is erroneously labeled, then x will be misclassified. Second, it needs to store the entire training data. Further, it needs to compute the distances between the test pattern and each of the training patterns. In general, if there are N training patterns, then the number of distances to be calculated to classify a test pattern is $O(N)$ and also it requires $O(N)$ space. So the computational requirements can be large. On the one hand, if desired, a nearest-neighbor index may be constructed up front, to enable more efficient retrieval of instances. On the other hand, it is possible to reduce the effort by compressing the training data. There are several algorithms for performing this compression. One scheme is based on clustering. For example, the N

patterns can be clustered into k clusters using K-means algorithm and the k resulting centroids can be used instead of the N training patterns. Labeling the centroids is done by using the majority class label in each cluster. Finally, the distance between a pair of points may not be meaningful in high-dimensional spaces. It is known that, as the dimensionality increases, the distance between a point x and its nearest neighbor tends toward the distance between x and its farthest neighbor. As a consequence, NNC may perform poorly in the context of high-dimensional spaces.

A number of design variations of nearest-neighbor classifiers are able to achieve more efficient classification results. Matching an object model against object data from some other source, object recognition capabilities are a fundamental limitation of many practical robotic systems, particularly when recognition is performed by nearest neighbor search in the presence of a large database of object models. To support efficient k-nearest neighbors (kNN) search, multidimensional index structures, especially various types of tree-based index structures, have been developed to reduce the computation cost.

9.6 Semi-supervised Learning

Semi-supervised learning is a class of machine learning techniques that make use of both labeled and unlabeled data for training—typically a small amount of labeled data with a large amount of unlabeled data. Semi-supervised learning falls between unsupervised learning (without any labeled training data) and supervised learning (with completely labeled training data). Many machine-learning researchers have found that unlabeled data, when used in conjunction with a small amount of labeled data, can produce considerable improvement in learning accuracy over unsupervised learning (where no data is labeled), but without the time and costs needed for supervised learning (where all data are labeled). The acquisition of labeled data for a learning problem often requires a skilled human agent. As a result, the cost associated with the labeling process thus may render a fully labeled training set infeasible, whereas the acquisition of unlabeled data is relatively inexpensive. In such situations, semi-supervised learning can be of great practical value. Semi-supervised learning is also of theoretical interest in machine learning and as a model for human learning.

As in the supervised learning framework, we are given a set of l independently identically distributed examples $x_1, \ldots, x_l \in X$ with corresponding labels $y_1, \ldots, y_l \in Y$. Additionally, we are given u unlabeled examples $x_{l+1}, \ldots, x_{l+u} \in X$. Semi-supervised learning attempts to make use of this combined information to surpass the classification performance that could be obtained either by discarding the unlabeled data and doing supervised learning or by discarding the labels and doing unsupervised learning.

In order to make any use of unlabeled data, some structure to the underlying distribution of data should be assumed. Semi-supervised learning algorithms make use of at least one of the following assumptions (Chapelle et al. 2006),

- Continuity assumption. Points which are close to each other are more likely to share a label.
- Cluster assumption. The data tend to form discrete clusters, and points in the same cluster are more likely to share a label, giving rise to feature learning with clustering algorithms.
- Manifold assumption. The data lie approximately on a manifold of a much lower dimension than the input space. Distances and densities defined on the manifold can be used to proceed learning.

The manifold assumption is practical when high-dimensional data are being generated by some process that may be hard to model directly but only has a few degrees of freedom.

9.6.1 Learning Methods

With examples of applications starting in the 1960s (Scudder 1965), historically the oldest approach to semi-supervised learning is the heuristic approach of self-training (also known as self-learning or self-labeling) (Chapelle et al. 2006). Being the simplest form of semi-supervised classification, self-training makes use of unlabeled data within a supervised learning framework (Triguero et al. 2013). First a supervised learning algorithm is trained based on the labeled data only. This classifier is then applied to the unlabeled data to generate more labeled examples as input for the supervised learning algorithm. Generally only the labels the classifier is most confident of are added at each step (Fazakis et al. 2015).

Co-training is an extension of self-training in which multiple classifiers are trained on different (ideally disjoint) sets of features and generate labeled examples for one another (Didaci et al. 2012). Although co-training is less sensitive to errors than self-learning, a difficulty is that the assumptions for its usage may not hold true, that is, it may not be possible to split the features into mutually exclusive and class-conditionally independent sets.

Generative approaches to statistical learning first seek to estimate $p(x|y)$, the distribution of data points belonging to each class. The probability $p(y|x)$ that a given point x has a label y is then proportional to $p(x|y)p(y)$ by Bayes' rule. Semi-supervised learning with generative models can be viewed either as an extension of supervised learning (classification plus information about $p(x)$) or as an extension of unsupervised learning (clustering plus some labels). Generative models assume that the distributions take some particular form $p(x|y, \theta)$ parameterized by the vector θ. If these assumptions are incorrect, the unlabeled data may actually decrease the accuracy of the solution relative to what would have been obtained from the labeled data alone (Cozman and Cohen 2006). However, if the assumptions are correct, then the unlabeled data necessarily improves performance. A probably approximately correct learning bound for semi-supervised learning of a Gaussian mixture was demonstrated by Ratsaby and Venkatesh (1995).

The unlabeled data are distributed according to a mixture of individual-class distributions. In order to learn the mixture distribution from the unlabeled data, it must be identifiable, that is, different parameters must yield different summed distributions. Gaussian mixture distributions are identifiable and commonly used for generative models. The parameterized joint distribution can be written as $p(x, y|\theta) = p(y|\theta)\, p(x|y, \theta)$ by using the Chain rule. Each parameter vector θ is associated with a decision function,

$$f_\theta(x) = \underset{y}{\operatorname{argmax}}\, p(y|x, \theta) \tag{9.18}$$

The parameter is then chosen based on fit to both the labeled and unlabeled data, weighted by λ,

$$\underset{\theta}{\operatorname{argmax}}\big(\log p\big(\{x_i, y_i\}_{i=1}^{l}|\theta\big) + \lambda \log p\big(\{x_i\}_{i=l+1}^{l+u}|\theta\big)\big) \tag{9.19}$$

Another major class of semi-supervised learning methods is low-density separation which attempts to place boundaries in regions where there are few data points (labeled or unlabeled). Approaches that implement low-density separation include Gaussian process models, information regularization, and entropy minimization (Zhu and Goldberg 2009). Graph-based methods for semi-supervised learning use a graph representation of the data, with a node for each labeled and unlabeled example. The graph may be constructed using domain knowledge or similarity of examples. Within the framework of manifold regularization, the graph then serves as a proxy for the manifold (Belkin and Niyogi 2004; Belkin et al. 2005).

Semi-supervised learning has recently become more popular and practically relevant due to the variety of problems for which vast quantities of unlabeled data are available such as images. For a review of recent work see a survey article by Zhu (2008).

9.7 Classifier Performance Evaluation

Given a classification model, to quantify its accuracy on a given data set so as to evaluate the effectiveness of a classifier, compare different models, select the best one for a particular data set, parameter tuning and several meta-algorithms such as ensemble analysis, leads to several challenges both in terms of methodology used for the evaluation and the specific approach used for quantification. The methodological issues are associated with dividing the labeled data appropriately into training and test segments for evaluation. The choice of methodology has a direct impact on the evaluation process, such as the under-estimation or over-estimation of classifier accuracy. Several approaches are possible, such as holdout, bootstrap, and cross-validation. In this chapter, we focus on the quantification perspective.

9.7.1 Quantification Issues

The quantification issues are associated with providing a numerical measure for the quality of the method after a specific methodology for evaluation has been selected and quantifying the trade-off between true positives and false positives. Other types of numerical measures are specifically designed to compare the relative performance of classifiers. As with evaluation of unsupervised learning, several evaluation measures exist for supervised learning. In the following, these different aspects of classifier evaluation will be studied in detail.

9.7.1.1 Purity

Purity is a measure of the extent to which clusters contain a single class. Its calculation can be thought of as follows. For each cluster, count the number of data points from the most common class in a said cluster. Next take the sum over all clusters and divide by the total number of data points. Formally, given some set of clusters M and some set of classes D, both partitioning N data points, purity can be defined as

$$\frac{1}{N}\sum_{m\in M}\max_{d\in M}|m\cap d| \tag{9.20}$$

Unfortunately, this measure neither penalizes having many clusters nor works well for imbalanced data.

9.7.1.2 F-Measure

The F-measure can be used to balance the contribution of false negatives by weighting recall through a parameter $\beta \geq 0$. Let precision and recall (both external evaluation measures in themselves) be defined as follows,

$$\begin{aligned} P &= \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}} \\ R &= \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}} \end{aligned} \tag{9.21}$$

where P is the precision rate and R is the recall rate. We can calculate the F-measure by using the following formula,

$$F_\beta = \frac{\left(\beta^2+1\right)\cdot P\cdot R}{\beta^2\cdot P+R} \tag{9.22}$$

Notice that when $\beta = 0$, $F_0 = P$. In other words, recall has no impact on the F-measure when $\beta = 0$, and increasing β allocates an increasing amount of weight

to recall in the final F-measure. Also note that TN is not taken into account and can vary from 0 upward without bound.

9.7.1.3 Dice Index

The Dice symmetric measure doubles the weight on TP while still ignoring TN and is equivalent to F1—the F-measure with $\beta = 1$,

$$\text{DSC} = \frac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}} \tag{9.23}$$

9.7.1.4 Confusion Matrix

A confusion matrix for classification is a square contingency matrix where the order of rows and columns corresponds to a list of classes. Providing sufficient statistics for all clustering metrics, contingency matrix or confusion matrix reports the intersection cardinality for every true/predicted cluster pair and can be used to quickly visualize the results of a classification (or clustering) algorithm. It shows how different a cluster is from the gold standard cluster. It has a number of interesting properties. For one example, it allows to examine the spread of each true cluster across predicted clusters and vice versa. For another example, the contingency table calculated is typically utilized in the calculation of a similarity statistic between the two clusterings.

However, Contingency matrix is easy to interpret for a small number of clusters, but becomes very hard to interpret for a large number of clusters. Further, it does not give a single metric to use as an objective for clustering optimization.

9.8 Summary

In this chapter, we introduce classification methods for categorical data so as to accept a before-unseen instance and accurately position it within the categories. We begin with a short review on several supervised learning methods including decision tree, perceptron, support vector machine and instance-based method, which is followed by a short introduction to semi-supervised learning. Finally, several measures for performance evaluation of classification are described. Although a broad range of tools is available for us to use, as the complexity of our objectives and the complexity and size of our data increase, the "empty space" between clusters shrinks, this simple problem takes on manifest complexity. One challenge we have to overcome is the trade-off between the issue of accuracy and the issue of computational cost when building classification models for data mining. Fortunately, in many practical

applications, an exact classification can be generally replaced by an approximate one without degrading the quality of final application while consuming much less time. As is always the case, the challenge is to find it.

References

Belkin, M., & Niyogi, P. (2004). Semi-supervised learning on Riemannian manifolds. *Machine Learning, 56*, 209–239 (Special Issue on Clustering).

Belkin, M., Niyogi, P., & Sindhwani, V. (2005). *On manifold regularization*. AISTATS.

Boser, B. E., Guyon, I. M., & Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. In *Proceedings of the 5th Annual Workshop on Computational Learning Theory (COLT '92)* (p. 144). CiteSeerX 10.1.1.21.3818. https://doi.org/10.1145/130385.130401. ISBN 978-0897914970.

Bow, S. T. (2002). *Pattern recognition and image processing*. New York: Marcel Dekker Inc.

Chapelle, O., Schölkopf, B., & Zien, A. (2006). *Semi-supervised learning*. Cambridge, Mass.: MIT Press. ISBN 978-0-262-03358-9.

Cortes, C., & Vapnik, V. N. (1995). Support-vector networks. *Machine Learning, 20*(3), 273–297. CiteSeerX 10.1.1.15.9362. https://doi.org/10.1007/bf00994018.

Cozman, F., & Cohen, I. (2006). Risks of semi-supervised learning: How unlabeled data can degrade performance of generative classifiers. *Semi-Supervised Learning*, vol. 4, pp. 57–72.

Didaci, L., Fumera, G., & Roli, F. (2012). Analysis of co-training algorithm with very small training sets. In G. Gimel'farb, E. Hancock, A. Imiya, A. Kuijper, M. Kudo, S. Omachi, et al. (Eds.), *Lecture Notes in Computer Science* (pp. 719–726). Berlin, Heidelberg: Springer. ISBN 9783642341656.

Fazakis, N., Karlos, S., Kotsiantis, S., & Sgarbas, K. (2015). Self-trained LMT for semisupervised learning. *Computational Intelligence and Neuroscience, 2016*, 1–13. https://doi.org/10.1155/2016/3057481.

Hastie, T., Tibshirani, R., & Friedman, J. (2017). *The elements of statistical learning* (p. 134).

Jin, C., & Wang, L. (2012). Dimensionality dependent PAC-Bayes margin bound. In *Advances in Neural Information Processing Systems*.

Kulkarni, A. D. (2001). *Computer vision and fuzzy-neural systems*. Prentice Hall.

Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning, 1,* 81–106.

Quinlan, J. R. (1993). *C4.5: Programs for machine learning*. San Mateo: Morgan Kaufmann.

Ratsaby, J., & Venkatesh, S. (1995). Learning from a mixture of labeled and unlabeled examples with parametric side information. In *Proceedings of the 8th Annual Conference on Computational Learning Theory* (pp. 412–417).

Scudder, H. J. (1965). Probability of error of some adaptive pattern-recognition machines. *IEEE Transaction on Information Theory, 11,* 363–371.

Triguero, I., García, S., & Herrera, F. (2013). Self-labeled techniques for semi-supervised learning: Taxonomy, software and empirical study. *Knowledge and Information Systems, 42*(2), 245–284.

Vapnik, V., & Chervonenkis, A. (1974). *Theory of pattern recognition* (in Russian). Moscow: Nauka (Cited in Chapelle et al. 2006, pp. 3).

Zhu, X. (2008). *Semi-supervised learning literature survey*. Computer Sciences, University of Wisconsin-Madison.

Zhu, X., & Goldberg, A. B. (2009). *Introduction to semi-supervised learning*. Morgan & Claypool. ISBN 9781598295481.

Chapter 10
A Fast Image Retrieval Method Based on A Quantization Tree



Abstract Traditional content-based image retrieval technology expresses the content of images by low-level features, giving rise to the "semantic gap" problem. Therefore, it is of great significance to obtain better accuracy of retrieval by fusing the visual spatial semantic content of images into content-based image retrieval algorithms. To help retrieval algorithms improve the ability of image analysis and understanding and improve the accuracy of the retrieval algorithm, in this chapter, we propose a new image retrieval methodology that begins by partitioning images into small overlapping image patches, on which feature vectors in the form of color histograms are exacted. Next, feature vectors extracted from a small number of training images are clustered to obtain the knowledge base consisting of visual vocabulary words, and images in the whole database are parsed by the knowledge base and their size-reduced versions are stored separately and subsequently used as an index. Finally the query image is partitioned and parsed, its similarities with the indexed images in the database are calculated, and the most similar images are output. The focus of this paper is on a fast visual vocabulary tree (a fast approximate nearest neighbor search tree), Quantization Tree, to quickly parse the query images so as to partially resolve the semantic gap problem between the semantic image content and the low-level image features. Extensive experimental results for object recognition tasks in an indoor environment as well as in an outdoor environment demonstrate the effectiveness and the efficiency of the Quantization Tree.

Keywords Color histogram · Image patch · MST-based clustering algorithm · Vocabulary tree · Random quantization tree

10.1 Introduction

With the rapid development of the imaging and Internet technology, a huge amount of image data is produced every day. How to store and manage the massive image data effectively has become a hot topic in academic circles. Therefore, image retrieval technologies have become the focus of attention and the mainstream technology of the new generation of image databases. Many image retrieval methodologies, such as early Concept-Based Image Retrieval, Content-Based Image Retrieval (CBIR)

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_10

and Semantic-Based Image Retrieval (SBIR), have been developed (Veltkamp et al. 2001). Traditional content-based image retrieval technology expresses the content of each image by low-level image features such as color, shape, and texture. Then image retrieval process begins by calculating the similarity between the image to search and the images in the database in terms of the corresponding feature vectors, next ranks the images in the database by a descending order of similarity, and finally outputs the resulted top ones. This CBIR method shows good accuracy and high efficiency. However, it hardly meets modern users' needs for more semantically oriented retrieval since it employs no semantic features interpreted by human eyes in the computation of image similarities, giving rise to the so-called "semantic gap" problem. Therefore, it is of great significance to improve the accuracy of retrieval by fusing the visual semantic extraction mechanism into the CBIR algorithms.

To shorten the "semantic gap", published by Nister and Stewenius in 2006, Scalable Recognition with a Vocabulary Tree (SRVT) scales efficiently to a large number of objects with both improved retrieval accuracy and better retrieval efficiency (Nister and Stewenius 2006). In this mechanism, the SIFT descriptors (Lowe 1999) as the image characteristics are first extracted as the content features to depict the image. Next, the method of hierarchical K-means clustering algorithm is applied on the extracted image features to generate a visual word vocabulary tree as illustrated in Fig. 10.1. Finally, a hierarchical Term Frequency-Inverse Document Frequency (TF-IDF) scoring using hierarchically defined visual words that form a vocabulary tree is proposed to allow much more efficient lookup of visual words. The most significant property of the scheme is that the tree directly defines the quantization. The quantization and the indexing are therefore fully integrated, essentially being one and the same. The image retrieval based on vocabulary tree saves time for image matching, especially for large image libraries. However, this retrieval mechanism does not take the spatial location information of the SIFT image descriptors into consideration.

To meet these challenges, in this chapter, we utilize a new content-based image retrieval model which is similar to the SRVT image retrieval method but with important variations.

Natural images contain statistical regularities which distinguish objects from each other and from random noise. For successful visual recognition, each object must have attributes that can be used to differentiate it from others and to segment the whole image into meaningful objects. To allow things to be recognized through the process

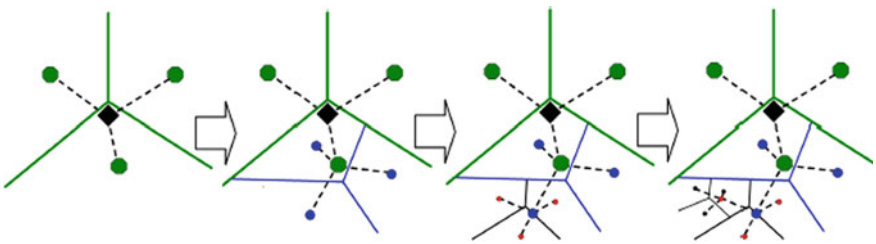


Fig. 10.1 An illustration of the process of the hierarchical quantization

of assembling sensory information into a useful and reliable representation of the world, there are many methods available for feature binding and sensory segmentation in practice. Currently, the most competitive approaches for image segmentation are formulated as clustering models for probabilistic grouping of distributional feature vectors. For image segmentation and object identification, instead of using the color information of a single pixel, the color pixels in a small local region of an image are considered. As illustrated in Fig. 10.2, to obtain feature vectors, for a given image, a moving window of size $N \times N$ hops by M pixels in the row and column directions but not to exceed the border of the image. The moving windows are overlapping to allow a certain amount of fuzziness to be incorporated so as to obtain a better segmentation performance. The window size controls the spatial locality of the result and the window hopping step controls the resolution of the result. A decrease in the step gives rise to an increased resolution but an increased processing time.

Based on the feature vectors extracted from image patches, image segmentation can be realized by partitioning the set of image patches into a number of disjoint clusters or segments. By this way, the spatial content organization of image objects can be maintained. Then the segmented images consist of object labels, which results in a much size-reduced learned version of the original ones that can be stored separately and used as indexing files. The model has been successfully applied to image segmentation tasks using feature vectors consisting of 10,000-dimensional color histograms and 80 Gabor wavelet texture measures (Wang et al. 2009a, b; Hunter et al. 2011).

To realize a more accurate and efficient image retrieval, a small number of training images from the image database are selected and partitioned into small overlapping image patches, on which feature vectors in the form of color histograms are exacted. These feature vectors are subsequently clustered to obtain a knowledge base consisting of visual vocabulary words. Next, images in the whole database can be parsed by the knowledge base and their size-reduced versions can be stored separately and subsequently used as an index. Finally the query image can be partitioned and parsed, its similarities with the indexed images in the database are calculated, and the most similar images are output.

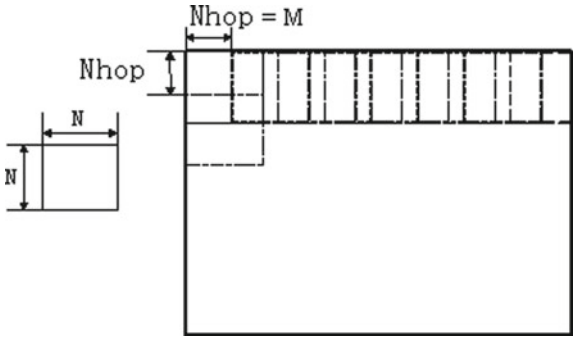


Fig. 10.2 Image segmentation model

For things in a database image or a query image to be recognized or parsed against a large knowledge base of visual vocabulary words into a reliable representation of theirs, there are many nearest neighbor based classification methods available. For modern large databases, traditional sequential brute force nearest neighbor search methods are of low efficiency and, therefore, unable to meet the requirements of the practical uses. As described in the above, for fast image retrieval, SVT is a way of both improved retrieval accuracy and better retrieval efficiency. However, due to the "curse of dimensionality" problem of high-dimensional feature space, the establishment of the Scalable Vocabulary Tree (SVT) is very computationally expensive and consumes a lot of time for modern large image databases. To partially circumvent this problem, through studying the vocabulary tree, we propose a random quantization tree (QT) method for approximate nearest neighbor search. The experimental results show that the random quantization tree is significantly better than vocabulary tree in term of the efficiency of the tree construction while maintaining similar approximate nearest neighbor search accuracy to that of SVT.

The rest of the chapter is organized as follows. In Sect. 10.2, we give a short review on some related work. We next present our proposed random quantization tree approach in Sect. 10.3. In Sect. 10.4, an empirical study is conducted to evaluate the performances of the proposed approach. Finally, conclusions are made in Sect. 10.5.

10.2 Related Work

Matching an object model against object data from some other source, object recognition capabilities are often performed by nearest neighbor search or approximate nearest neighbor search in the presence of a large database of object models. To support efficient k-nearest neighbors search, multidimensional index structures, especially various types of tree-based index structures, have been developed to reduce the computation cost.

Being the best-known and most widely used among these, the K-dimensional tree (Kd-tree) was proposed in 1970 as a generalization of a binary search tree to higher dimensions with a logarithmic time complexity (Bentley 1970). To improve, Beis and Lowe proposed to find an approximate neighbor by examining at most Emax leaves (Beis and Lowe 1997). For high-dimensional data, Duch et al. proposed a randomized generalization of Kd-trees (the randomized Kd-trees) through randomization in selecting the partitioning value (Duch et al. 1998). To further improve, Silpa-Anan and Hartly proposed a search method based on simultaneous independent searches in multiple randomized Kd-trees with different orientations (Silpa-Anan and Hartley 2008). Other tree-based index structures include R-tree (Rectangle-tree) (Guttman. 1984), SS-tree (White and Jain 1996) and SR-tree (Sphere/Rectangle-tree) (Katayama and Satoh 1997) which rely on the amount of overlap among sibling regions for efficient search. They work well for low dimensional data sets, but their performances become dramatically worse for high dimensional larger-sized data sets.

Being an interesting classifier, decision tree is a classic technique for classification of a set of data records of known classes. Standard splitting rules usually involve a single attribute (e.g., applying a threshold to obtain locally optimal decisions at each node) (Quinlan 1992). Branches from the root to the leaf nodes represent conjunctions of features that lead to those class labels. To capture the diversity and richness of high dimensional descriptors, Amit and Geman proposed to simply build multiple weakly dependent random trees of modest depth, each on a small random subset of training examples (Amit and Geman 1977). For higher dimensional data, Geurts et al. proposed extremely randomized trees with weak dependence on the dimensionality, by randomizing both attribute choices and quantization thresholds (Geurts et al. 2006). Being simple but powerful tools for classification, Randomized Trees suffer from their tree size in memory being increased exponentially with the depth and linearly with the number of trees (Lepetit and Fua 2006).

Being a good compromise between speed and loss of accuracy, vocabulary tree introduced by Nister and Stewenius is constructed off-line by repeated hierarchical K-means clustering on local descriptors until the desired number of levels is reached (Nister and Stewenius 2006). It describes the image as a vector of frequencies determined by the paths taken by the descriptors through the tree (Konolige et al. 2010) and creates a significant speedup (Riemenschneider et al. 2009; Zhao and Elgammal 2008). For perspectively distorted query images, Chen et al. proposed multiview SVTs as an efficient server-side solution (Chen et al. 2009). However, visual coding based on K-means vector quantization is effective but slow and remains hard to accelerate in high dimensional descriptor space (Moosmann et al. 2006).

10.3 Scalable Recognition with a Quantization Tree

The most significant property of the SVT scheme is that the tree directly defines the quantization in the sense that the quantization and the indexing are fully integrated, essentially being one and the same. Different from SVT, our approach defines a visual vocabulary non-hierarchically, and then devises an approximate nearest neighbor search in order to find visual words efficiently.

10.3.1 Visual Vocabulary Generation

In order to understand and manipulate the world, labeling local descriptors extracted from training images to target categories through machine learning methodologies is often a major task in the building of intelligent systems. Each feature vector extracted from the image space appears as a point in the feature space and patterns pertaining to different classes will fall into different regions of the feature space. The learning process is to partition the feature vectors into sensible clusters based on some properties in common. To ease the labeling task by grouping unlabeled feature

vectors, unsupervised learning have gained prominence and many approaches have been developed for it, including hierarchical, partition-based, density-based, model-based, and graph-based approaches, among which graph-based clustering algorithms can outperform the classic methods such as *K*-means clustering when the boundaries of clusters are irregular. To this end, Chameleon clustering algorithm (Karypis et al. 1999) and LM clustering algorithm (Laszlo and Mukherjee 2005) are employed to perform clustering on our challenging high-dimensional color histogram based image local descriptors obtained from an environment.

The color histograms of multicolored objects have been used in a technique called Histogram Intersection as stable object representations to provide a robust, efficient cue for indexing into a large database (Swain and Ballard 1991). For perceptual grouping, histogram clustering model (HCM) was proposed in (Pereira et al. 1993). To segment an image using HCM, the image is first decomposed into U not necessarily disjoint image patches, each having W features (or bins). For each image patch $i \in N$, a histogram $(h_{i,j})_{1 \le j \le M}$ is defined as a tuple $h_i \in N^M$. If u_i denotes the number of observations (e.g., pixels) belonging to image patch i, then, $u_i = \sum_j h_{i,j}$. The probability to observe features j in a given image patch i can be estimated by,

$$p_i = (p(j|i))_{1 \le j \le M} \tag{10.1}$$

where the empirical conditional probability is

$$p(j|i) = \frac{h_{i,j}}{u_i} \tag{10.2}$$

10.3.2 Quantization Tree

With the set of feature vectors being labeled, the next step is to classify new observations. Real-time processing of the information coming from video demands for efficient search structures and algorithms. Design of searching methods that scale well with the size of the database and the dimensionality of the data is a challenging task. Trees present an efficient way to index local image regions, making the nearest neighbor search more efficient by pruning. For very large databases consisting of high dimensional highly sparse feature vectors, we choose to follow the notion of Vocabulary tree but with important variations to generate a *k*-way approximate nearest neighbor search tree.

Hierarchies are a natural way to organize concepts and data. For a long time, research on object recognition has aimed at building hierarchical models (Fukushima 1980). As shown in Fig. 10.3, given the obtained database, at the first level, there is only one node, the tree root. At the second level, a set of k (which is 3 in Fig. 10.3) representative patterns are randomly selected from the whole database as the cluster centers. Then the whole database is partitioned into k subsets by assigning each

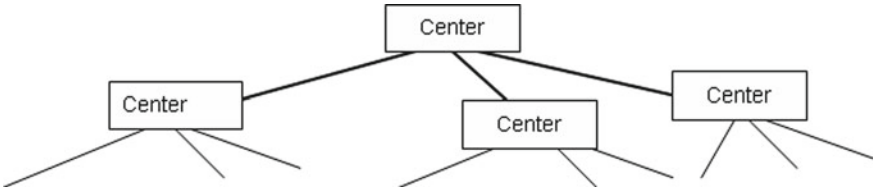


Fig. 10.3 Illustration of database random quantization tree structure

feature vector to its closest center according to a chosen similarity measure. At the third level, for each of the k clusters obtained at the second level, k feature vectors are selected randomly from its pool of feature vectors as its next level cluster centers, resulting in k^2 cluster tree nodes at this level. This procedure continues until either all the feature vectors in a leaf node (the tree node that has no children nodes) belong to the same object class (a pure node) or the number of the feature vectors in a leaf node is below some limit, e.g., 50. Every feature vector has a class label associated with it.

Given a new feature vector, to search through the tree, its distances to k cluster centers at each level along a certain branch are calculated and the winner is the center among k ones that the new feature vector is nearest to. When a leaf node is reached, if it is pure, assign the associated label to it and then stop. Otherwise, do a nearest neighbor search over the vectors in the associated cluster, and the winner is the feature vector which gives minimum distance according to the chosen metric and its label is assigned to the new feature vector.

To summarize, the proposed scalable recognition with a quantization tree method is composed of two phases, the training phase, which models the patch-based image data, and the testing phase, which identifies objects and segments the images with respect to the database tree.

10.4 Experiments and Results

In this section, we present the results of experiments conducted to evaluate the performance of our proposed recognition method. First, we took two sets of images for an indoor environment and an outdoor environment, respectively. Next, we selected a small number of training images from them and performed perceptual grouping on them by using two graph-based clustering algorithms to build up the knowledge base. Finally, we compare the accuracy and efficiency performance of our proposed k-way Quantization tree (KQTree) algorithm to that of the vocabulary tree (VTree) algorithm. To be as general as possible, our algorithm has no specific requirements on the dimensionality of the data sets and the format of the distance measure, although Euclidean distance is used in the experiments.

We implemented all the algorithms in C++. All the experiments were performed on a computer with Intel Core i5-3470 3.20 GHz CPU and 4 GB RAM. The operating

system running on this computer is Windows 7. We use the timer utilities defined in the C standard library to report the CPU time.

10.4.1 Experiment I: Visual Vocabulary Generation in an Indoor Environment

10.4.1.1 Setup

In this phase, a model of the environment is generated so that the robot vision system can identify the major percepts existing in this indoor environment. As shown in Fig. 10.4, the first working environment is an indoor hallway. Typical percepts along the hallway include Wood Panel, White Floor, Yellow Floor, Blue Wall, Blue Railing, Black Stripe Tile, Light Window.

To build a percept database, 20 pictures of size 720-by-480 along the hallway are taken by the robot, from which 66,740 unlabeled feature vectors are extracted (with $N = 15$ and $M = 10$). According to a minimum spanning tree (MST)-based clustering algorithm given in (Wang et al. 2009a, b), 11 perceptual groups are obtained. A detailed list of the major percepts present in the video sequences (not all objects appear on the same frames simultaneously but with some objects replacing others as the training sequence progresses) is provided in Table 10.1 together with their semantic meanings and a denoting color. From Table 10.1, it can be seen that several perceptual groups obtained by the unsupervised learning algorithm actually correspond to the same object. To make the processed image visually pleasant, several finely differentiated colors are used to refer these different objects in the machine vision perspective.



Fig. 10.4 The hallway

Table 10.1 Major percepts and their semantic meanings

Percept name	Percept symbol	Denoting color
Black Stripe	Object 1	Black
Wood Panel	Object 2	Light red
Light Window	Object 3	Light grey
Wood Panel	Object 4	Light red
White Floor	Object 5	Dark white
Wood Panel	Object 6	Dark red
Wood Panel	Object 7	Red
Wood Panel	Object 8	Light red
White Floor	Object 9	White
Yellow Floor	Object 10	Yellow
Yellow Floor	Object 11	Blue

10.4.1.2 Testing

To use the obtained percept database to subsequently segment any image (taken in the environment) reasonably well and fast, 10 Vocabulary trees and 10 proposed random trees are built. The average running times in seconds over 10 runs and the corresponding standard deviations are summarized in Table 10.2. It can be seen that the proposed random trees take much less time to construct.

To show the quality of these trees, segmentation results using vocabulary tree and the proposed random tree for a sample image are presented in Fig. 10.5, where the left one is the original image, the middle one is the segmented image using the vocabulary tree, and the right one is the segmented image using the proposed random tree.

From the figure, it can be seen both search trees work very well. In a qualitative visual evaluation, it can be seen that the proposed method does a reasonably good

Table 10.2 Running time for Trees' construction

	Vocabulary tree(s)	Random tree(s)
Mean	24,228	49
Std	41	2



Fig. 10.5 (Left) Original image; (middle) VT tree search result; (right) random tree search result

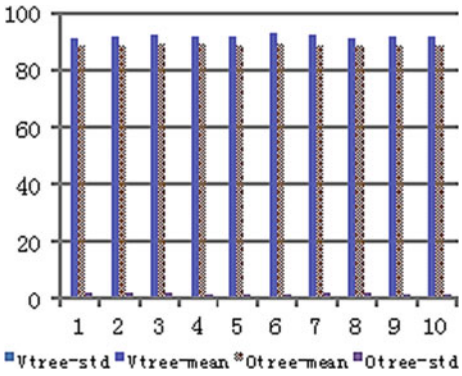


Fig. 10.6 Average percentages of correct classification over ten runs and the corresponding standard deviations

job on the whole and clearly identifies the yellow floor, the white floor, and the wood panel wall, although it misclassifies the railing as the wood panel, which is not too surprising since the railing is a hard percept to model. To further quantify the search quality and the technical robustness of both trees, 10 more images along the hallway are taken. Then for each of them, 3337 feature vectors are extracted which are classified first by an exact nearest neighbor search with respect to the whole database and then by an approximate nearest neighbor search through 10 vocabulary trees (denoted by Vtree) and 10 fast random search trees (denoted by Otree). The average percentages of correct classification (with respect to the exact search) and the corresponding standard deviations are plotted in Fig. 10.6. The horizontal axis denotes the image number. The vertical axis denotes the average percentages of correct classification over 10 runs and the corresponding standard deviation. From the figure, it can be seen both Vocabulary tree and the proposed random tree do a good job and are close in performance.

To compare the relative time issue of two approaches, for the database of size 66,740, it is observed that the proposed tree has so far at most 20 levels. As a result, at most $3 \times 20 + 50 = 110$ distance calculations are necessary. This is a considerable reduction. It takes no more than 2 seconds to find approximate nearest neighbors for the 3337 feature vectors obtained for each newly grabbed image.

10.4.2 Experiment II: Visual Vocabulary Generation in an Outdoor Environment

10.4.2.1 Setup

Although the proposed method does a good job in the last experiment, the detection of objects in the frames of an indoor structured environment simplifies the object recognition task significantly and makes the experiment somewhat artificial. For a more convincing demonstration of how the proposed model would perform detecting



Fig. 10.7 Original image

more realistic objects in a different and less structured environment, and to justify the proposed object recognition method is realistic and applicable in practice, an outdoor environment is chosen to be a second working environment, as shown in the left of Fig. 10.7. The extension to a more general environment setting is meant to substantially increase the technical soundness of the proposed approach. In this set of experiments, a Canon IXUS 50HS digital video camera recorder was used to take video around an outdoor building. From the video, 2794 images of size 1280×720 are extracted.

10.4.2.2 Matching Effectiveness

In this set of experiments, the effectiveness of the approximate nearest neighbor search (i.e., the image parse mechanism) by the quantization tree method, KQTree, and the corresponding running time efficiency are compared with two other methods, namely, brute force (BF) method and VTree. Here by brute force method, we mean that no hierarchical structure but a simple plain search is involved in the label matching process.

A detailed list of all the major percepts present in the training images (not all objects appear on the same frames simultaneously but with some objects replacing others for different training images) together with their semantic meanings and a denoting color is provided in Tables 10.3, 10.4, 10.5 and 10.6 for RGB color histograms, HSV color histograms, Opponent color histograms and Transformed color histograms, respectively. From the tables, we can see that several perceptual groups obtained by the clustering algorithms actually correspond to the same object. To make

Table 10.3 The main object categories of RGB knowledge base

Percept name	Percept label	Denoting color
Road	Object 1	Blue
Road	Object 2	Grey 1
Road	Object 3	Grey 2
Grass	Object 4	Purple
Grass	Object 5	Bright green
Shadow	Object 6	Black

Table 10.4 The main object categories of HSV knowledge base

Percept name	Percept label	Denoting color
Road	Object 1	Dark yellow
Road	Object 2	Grey 1
Road	Object 3	Grey 2
Grass	Object 4	Rose red
Grass	Object 5	Bright green
Tree Shadow	Object 6	Dark green
Grass	Object 7	Dark grey

Table 10.5 The main object categories of Opponent knowledge base

Percept name	Percept label	Denoting color
Road	Object 1	White
Road	Object 2	Light grey
Road	Object 3	Bright green
Grass	Object 4	Dark grey
Grass	Object 5	Dark green
Shadow	Object 6	Rose red

Table 10.6 The main object categories of Transformed knowledge base

Percept name	Percept label	Denoting color
Road	Object 1	Grey
Road	Object 2	Blue
Road	Object 3	Dark yellow
Grass	Object 4	Purple
Grass	Object 5	Green
Shadow	Object 6	White
Grass	Object 7	Pink

the processed image visually pleasant, we use several finely differentiated colors to refer these different objects in the machine vision perspective.

To show the quality of the trees, for each feature vector extracted from a test image, an exact nearest neighbor search with respect to the whole knowledge base and then by an approximate nearest neighbor search through VTree and the proposed fast random search trees (i.e., KOTree), respectively, are done and segmentation results using the brute force (BF) method, the VTree and the proposed KQTree for a test image are presented in the left plots of Figs. 10.8, 10.9, 10.10 and 10.11, in which the upper left one is the original image, the upper right one is the segmented image using the BF method, the lower left one is the segmented image using the VTree, and the lower right one is the segmented image using our random KQTree.

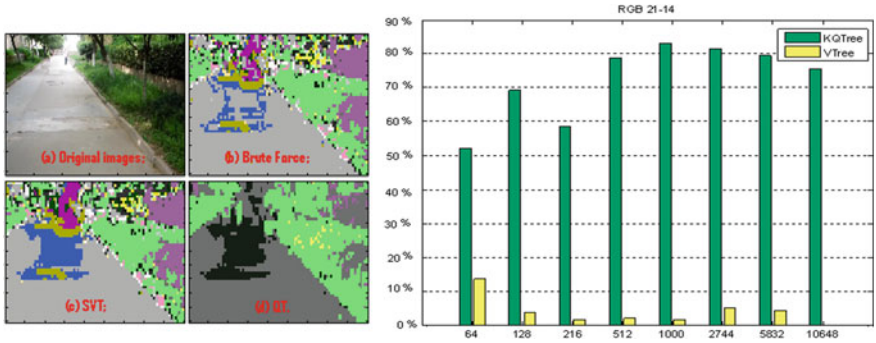


Fig. 10.8 (Left) RGB image retrieval results for dimension 1000. (Right) Correct classification percentages of random QT and VT in different dimensions of RGB histogram for $M = 21, N = 14$

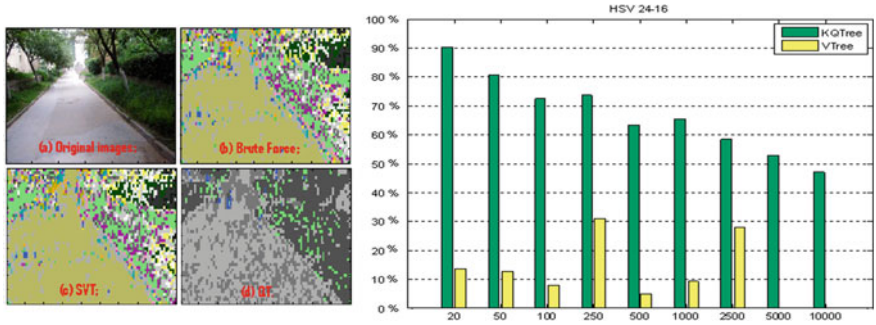


Fig. 10.9 (Left) HSV image retrieval results for dimension 512. (Right) Correct classification percentages of random QT and VT in different dimensions of HSV histogram for $M = 24, N = 16$

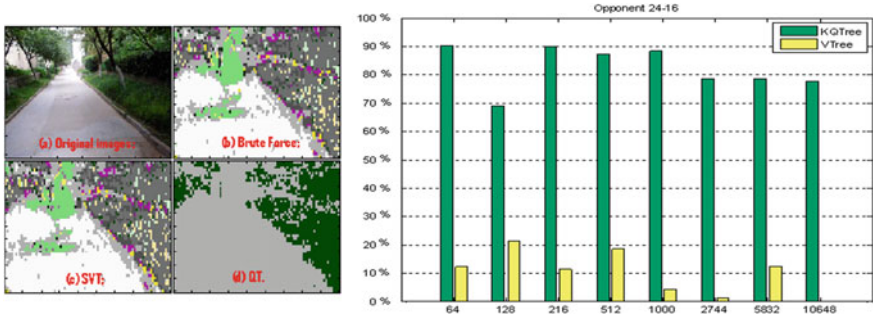


Fig. 10.10 (Left) Opponent image retrieval results of dimension 64. (Right) Correct classification accuracies of random QT and VT in different dimensions of Opponent histogram for $M = 24, N = 16$

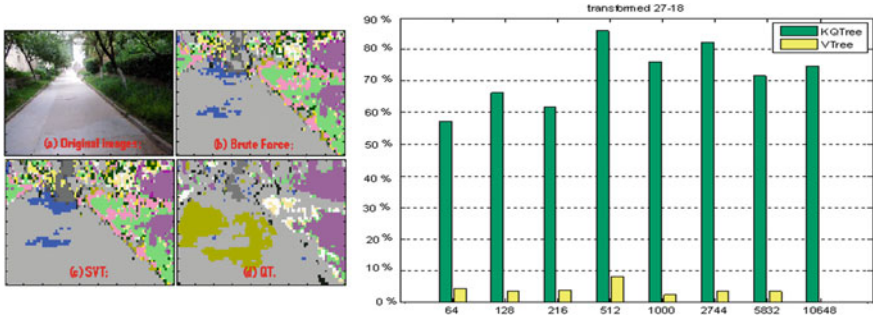


Fig. 10.11 (Left) Transformed image retrieval results of dimension 512. (Right) Correct classification percentages of random QT and VT in different dimensions of Transformed histogram for $M = 24$, $N = 16$

From the figures, we can see both search trees work very well and do a reasonably good job on the whole. For quantitative effectiveness evaluation, 10 VTrees and 10 KQTrees are built on the obtained knowledge base. The average percent of correct classification (with respect to the BF search) for a test image for all the dimensions are presented in the right plots of in Figs. 10.8, 10.9, 10.10 and 10.11. The horizontal axis denotes the histogram dimensions. The vertical axis denotes the average percentages of correct classification over 10 runs. Similar retrieval effectiveness performance results are obtained for other parameter sets of image patch sizes and hopping steps.

10.4.2.3 Matching Efficiency

To examine the construction time of the newly proposed KQTrees and the original VTrees, 10 Vocabulary trees and 10 our random trees are built. The average running times in seconds over 10 runs are summarized in Tables 10.7, 10.8, 10.9, 10.10, 10.11, 10.12, 10.13, 10.14, 10.15, 10.16, 10.17 and 10.18. There are three columns in each of these tables, representing the running time performance of constructing a vocabulary tree, i.e., the third column, and of constructing a random quantization

Table 10.7 Running times for Trees' construction of $N = 21$, $M = 14$ for RGB

Dimensions	KQTree(s)	VTree(s)
64	76	670
125	70	723
216	83	932
512	92	1820
1000	102	4362
2744	123	16,020
5832	148	288,490
10,648	162	*

Table 10.8 Running times for Trees' construction of $N = 24$, $M = 16$ for RGB

Dimensions	KQTree(s)	VTree(s)
64	46	592
125	46	695
216	49	883
512	51	1280
1000	54	3560
2744	78	7181
5832	101	24,917
10,648	113	*

Table 10.9 Running times for Trees' construction of $N = 27$, $M = 18$ for RGB

Dimensions	KQTree(s)	VTree(s)
64	23	401
125	25	423
216	28	579
512	32	1103
1000	35	3201
2744	43	6997
5832	62	17,372
10,648	69	*

Table 10.10 Running times for Trees' construction of $N = 21$, $M = 14$ for Opponent

Dimensions	KQTree(s)	VTree(s)
64	63	312
125	46	483
216	60	1134
512	66	2839
1000	78	5206
2744	81	15,176
5832	96	36,082
10,648	114	*

tree, i.e., the second column, for the corresponding dimension, i.e., the first column, respectively. For labeled high-dimensional color histograms, the construction of the vocabulary tree approach may run out of memory space and, therefore, may turn out to be infeasible (denoted by symbol '*'). From the tables, it can be seen that the constructions of our proposed KQTree approach take much less time and can be faster than that of VT by a factor of 1000 with increasing dimensionality. This is because K-means algorithm consumes more time for high-dimensional feature vectors.

Table 10.11 Running times for Trees' construction of $N = 24$, $M = 16$ for Opponent

Dimensions	KQTree(s)	VTree(s)
64	69	289
125	67	404
216	71	995
512	76	1901
1000	77	3619
2744	88	11,722
5832	101	27,069
10,648	119	*

Table 10.12 Running times for Trees' construction of $N = 27$, $M = 18$ for Opponent

Dimensions	KQTree(s)	VTree(s)
64	53	237
125	43	363
216	48	921
512	59	1492
1000	62	2989
2744	72	9102
5832	78	24,719
10,648	96	*

Table 10.13 Running times for Trees' construction of $N = 21$, $M = 14$ for Transformed

Dimensions	KQTree(s)	VTree(s)
64	40	411
125	45	502
216	54	991
512	60	2398
1000	72	4389
2744	90	12,821
5832	114	35,016
10,648	144	*

10.4.2.4 Running Time Issue

To compare the relative processing time issue of the proposed approach, the proposed tree has so far at most 18 levels for the database consisting of 95,550 labeled feature vectors from the experiments. As a result, we have had at most $3 \times 18 + 50 = 104$ distance calculations. This is a considerable reduction. It takes no more than 2 seconds to find approximate nearest neighbors for the feature vectors obtained for each newly grabbed image. For SVT, the corresponding maximum number of tree

Table 10.14 Running times for Trees' construction of $N = 24$, $M = 16$ for Transformed

Dimensions	KQTree(s)	VTree(s)
64	54	319
125	60	432
216	61	801
512	66	1993
1000	78	2919
2744	102	10,227
5832	126	25,383
10,648	148	*

Table 10.15 Running times for Trees' construction of $N = 27$, $M = 18$ for Transformed

Dimensions	KQTree(s)	VTree(s)
64	30	251
125	30	389
216	36	592
512	42	1337
1000	66	2305
2744	78	7116
5832	102	18,274
10,648	96	*

Table 10.16 Running times for Trees' construction of $N = 21$, $M = 14$ for HSV

Dimensions	KQTree(s)	VTree(s)
20	78	655
50	82	1218
100	105	2012
250	119	4426
500	138	8174
1000	174	15,293
2500	223	*
5000	281	*
10,000	316	*

levels is 15. It takes similar amount of computation time to find approximate nearest neighbors for the feature vectors obtained for each newly grabbed image. Therefore, both methods have similar running time performances.

Table 10.17 Running times for Trees' construction of $N = 24$, $M = 16$ for HSV

Dimensions	KQTree(s)	VTree(s)
20	61	584
50	70	909
100	98	1441
250	97	3295
500	113	6853
1000	139	11,945
2500	176	42,876
5000	272	*
10,000	289	*

Table 10.18 Running times for Trees' construction of $N = 27$, $M = 18$ for HSV

Dimensions	KQTree(s)	VTree(s)
20	14	496
50	15	857
100	21	1094
250	25	2989
500	40	5934
1000	59	10,008
2500	89	29,892
5000	138	39,241
10,000	175	*

10.5 Conclusions

Content-based image retrieval demands for efficient nearest neighbor search structures and algorithms. Design of searching methods that scale well with the size of the database and the dimensionality of the data is a challenging task. Trees present an efficient way to index local image regions, making the nearest neighbor search more efficient by pruning. A fast image feature vector label retrieval approach with an indexing scheme significantly more efficient than the state-of-the-art VTree has been presented. The approach is built upon a quantization tree that hierarchically organizes labeled descriptors from image color histograms. Experimental results have demonstrated the improved efficiency of the proposed method while with a similar effectiveness to that of the state-of-the-art Scalable Vocabulary Tree. In our future work, we will fulfill our image retrieval method and evaluate it by partitioning a query image into regularly spaced image patches and parsing it against the knowledge base, calculating its similarities with the similarly parsed size-reduced indexed images in the database, and outputting the most similar images.

References

Amit, Y., & Geman, D. (1977). Shape quantization and reconstruction with randomized trees. *Neural Computation, 9*(7), 1545–1588.

Beis, J. S., & Lowe, D. G. (1997). Shape indexing using approximate nearest neighbor search in high-dimensional spaces. In *Proceedings of the 1997 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR '97)* (pp. 1000–1006), Puerto Rico.

Bentley, J. L. (1970). Multidimensional binary search trees in database applications. *IEEE Transactions on Software Engineering, 5*(4), 333–340.

Chen, D., Tsai, S. S., & Chandrasekhar, V. (2009). Robust image retrieval using multiview scalable vocabulary trees. In *Visual Communications & Image Processing. International Society for Optics and Photonics*.

Duch, A., Estivill-Castro, V., & Martinez, C. (1998). Randomized *K*-dimensional binary search trees. In K.-Y. Chwa & O. H. Ibarra (Eds.), *ISAAC '98, LNCS* (Vol. 1533, pp. 199–209). Berlin Heidelberg: Springer.

Fukushima, K. (1980). Neocognition: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position. *Biological Cybernetics, 36*(4), 193–202.

Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely randomized trees. *Machine Learning, 63*(1), 3–42.

Guttman, A. (1984). R-trees: A dynamic index structure for spatial searching. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD '84)* (pp. 47–57), New York, NY.

Hunter, J. E., Tugcu, M., Wang, X., Costello, C., & Wilkes, D. M. (2011). Exploiting sparse representations in very high-dimensional feature spaces obtained from patch based processing. *Machine Vision and Applications, 22*(3), 449–460.

Karypis, G., Han, E.-H., & Kumar, V. (1999). CHAMELEON: A hierarchical clustering algorithm using dynamic modeling. *IEEE Computer, 32*(8), 68–75.

Katayama, N., & Satoh, S. (1997). The SR-tree: An index structure for high-dimensional nearest neighbor queries. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD '97)* (pp. 369–380), Tucson, Arizona.

Konolige, K., Bowman, J., Chen, J. D., Mihelich, P., Calonder, M., Lepetit, V., et al. (2010). View-based maps. *International Journal of Robotics Research, 29*(8), 941–957.

Laszlo, M., & Mukherjee, S. (2005). Minimum spanning tree partitioning algorithm for microaggregation. *IEEE Transactions on Knowledge and Data Engineering, 17*(7), 902–911.

Lepetit, V., & Fua, P. (2006). Keypoint recognition using randomized trees. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 28*(9), 1465–1479.

Lowe, D. G. (1999). Object recognition from local scale-invariant features. In *Proceedings of the International Conference on Computer Vision (ICCV '99)* (pp. 1150–1157).

Moosmann, F., Triggs, B., & Jurie, F. (2006). Fast discriminative visual codebooks using randomized clustering forest. In *Proceedings of International Conference on Neural Information Processing Systems*. MIT Press.

Nister, D., & Stewenius, H. (2006). Scalable recognition with a vocabulary tree. In *Proceedings of the 2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR '06)* (Vol. 2, pp. 2161–2168), Washington, DC, USA.

Pereira, F. C. N., Tishby, N., & Lee, L. (1993). Distributional clustering of English words. In *Proceeding of the 31st Meeting of the Association for Computational Linguistics* (pp. 183–190), Columbus, OH.

Quinlan, J. R. (1992). *C4.5: Program for machine learning*. Morgan Kaufmann.

Riemenschneider, H., Donoser, M., & Bischof, H. (2009). Bag of optical flow volumes for image sequence recognition. In *Proceedings of British Machine Vision Conference (BMVC)*.

Silpa-Anan, C., & Hartley, R. (2008). Optimised KD-trees for fast image descriptor matching. In *Proceedings of the 2008 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR '08)* (pp. 1–8).

Swain, M. J., & Ballard, D. H. (1991). Color indexing. *International Journal of Computer Vision, 7*(1), 11–32.

Veltkamp, R. C., Tanase, M., & Sent, D. (2001). Features in content-based image retrieval systems: A Survey. *Computational Imaging and Vision, 36*(03), 97–124.

Wang, X., Tugcu, M., Hunter, J. E., & Wilkes, D. M. (2009a). Exploration of configural representation in landmark learning using working memory toolkit. *Pattern Recognition Letters, 30*(1), 66–79.

Wang, X., Wang, X. L., & Wilkes, D. M. (2009b). A divide-and-conquer approach for minimum spanning tree-based clustering. *IEEE Transactions on Knowledge and Data Engineering, 21*(7), 945–958.

White, D. A., & Jain, R. (1996). Similarity indexing with the SS-tree. In *Proceedings of the 12th IEEE International Conference on Data Engineering (ICDE '96)* (pp. 516–523), Washington, DC.

Zhao, Z., & Elgammal, A. (2008). Information theoretic key frame selection for action recognition. In *Proceedings of British Machine Vision Conference (BMVC)*.

Chapter 11
An Efficient Image Segmentation Algorithm for Object Recognition Using Spectral Clustering



Abstract Being a crucial and challenging problem in computer vision, image segmentation refers to partitioning an image into several disjoint subsets such that each subset corresponds to a meaningful part of the image and is the very first step for recognizing objects in the images. A large number of available algorithms for image segmentation are prone to either time inefficiency or low-accuracy in finding objects among multiple images. In this chapter, we employ spectral clustering to address this problem. Being one of the most popular modern clustering algorithms, spectral clustering is simple to implement, can be solved efficiently by standard linear algebra software, and very often outperforms traditional clustering algorithms such as the *K*-means algorithm. However, it is not very well scalable to large datasets. To partially circumvent this problem, in this chapter, we propose an integration-based fast incremental spectral clustering algorithm which is particularly designed for object recognition oriented image segmentation tasks for robotic applications in an outdoor unknown environment. The algorithm applies spectral clustering to each image, and then integrates the clustering results using a BIRCH tree. Experiments performed on image data demonstrate the efficacy of the proposed method.

Keywords Image segmentation · Object recognition · Spectral clustering · BIRCH tree

11.1 Introduction

A human being recognizes objects in an environment using the visual system to segment color images. In computer vision, image segmentation is the partitioning of an image into homogeneous and connected regions without using additional knowledge on objects in the image. The goal of image segmentation is to simplify images and also change their representations to make them easy to understand and more meaningful. It is widely used in applications such as medical image processing (Funka-Lea et al. 2006; Grady et al. 2006; Jolly et al. 2009), object tracking (Seo et al. 2006; Lam and Yuen 1998), recognition (Yu et al. 2002; Russell et al. 2006), image reconstruction (Favaro and Soatto 2004; Bleyer and Gelautz 2005) and so on. Many techniques have been developed for it and can be divided into three major categories, namely,

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_11

edge-based approaches, region-based approaches, and clustering-based approaches. Edge-based techniques rely on the detection of boundaries between different regions of an image for segmentation using edge detectors such as the Robert edge detector, the Sobel edge detector (Gonzalez and Woods 1992) and the Canny edge detector (Canny 1986; Paul et al. 2005). Region-based techniques utilize splitting and merging to perform the segmentation and classify pixels into regions in an iterative manner until some uniformity criteria (Ning et al. 2010; Duarte et al. 2006) are satisfied. As to clustering-based approaches, it is since the very beginning that image segmentation has been closely related to perceptual grouping or data clustering. Such a relationship was pointed out by Wertheimer's gestalt theory (Wertheimer 1938) in 1938, in which, a set of grouping laws such as similarity, proximity and good continuation are identified to explain the particular way by which the human perceptual system groups tokens together.

Clustering is the process of partitioning a set of objects into subsets of similar objects called clusters. Dominating in an image, colors create dense clusters in the color space in a natural way. Pixel based clustering in three-dimensional color space on the basis of color similarity is one of the popular approaches in the field of color image segmentation. Nowadays millions of color images are stored and maintained in data sources electronically. The purpose of current research in image segmentation is to locate objects across multiple images. Exactly the same objects can exist in multiple image databases. Connections in terms of identical objects among these image databases provide better understanding of relationships of these data sources. Image segmentation algorithms for object recognition in such applications become very challenging and critical. Efficient algorithms are inevitable to address this problem.

Fortunately, a large number of algorithms are available in the real world (Peng et al. 2013). A naïve algorithm would extract the image descriptors in the form of feature vectors from all the images and perform clustering on the whole datasets. Along this line, we have previously proposed minimum spanning tree-based clustering solutions (Wang et al. 2009, 2013) for image segmentation and object recognition tasks. However, it may produce expected results but with a high time complexity. Therefore algorithms have to be devised to provide best possible results within a manageable time. To pursue this goal, in this paper, we propose an efficient integration-based spectral clustering algorithm which is particularly designed for object recognition oriented image segmentation tasks among image sequences. The algorithm first applies spectrum clustering to each image, and then integrates the clustering results using a BIRCH tree. Experiments performed on image data demonstrate the efficacy of our method.

The rest of this paper is organized as follows. In Sect. 11.2, we review some background and existing work. We next present our proposed approach in Sect. 11.3. Evaluation of our algorithm is then given in Sect. 11.4. Finally, conclusions are made in Sect. 11.5.

11.2 Background

Background with regard to the method presented in this chapter falls into two main categories, spectral clustering and BIRCH tree.

11.2.1 Spectral Clustering Algorithms

Spectral clustering goes back to Donath and Hoffman, who first suggested to compute graph partitions based on eigenvectors of the adjacency matrix (Donath and Hoffman 1973). Being a competitive clustering method, spectral clustering is based on spectral graph theory. In the machine learning community, spectral clustering became popular by the works of Shi and Malik (2000), Ng et al. (2001), Meila and Shi (2001), and Ding (2007). A huge number of papers have subsequently been published, dealing with various extensions, new applications, and theoretical results on spectral clustering (Jia et al. 2014).

Compared with traditional clustering algorithms, spectral clustering algorithms have several advantages. For one example, unlike *K*-means algorithm, the spectral clustering algorithm directly works on the Laplacian matrix of feature vectors, and therefore can identify nonconvex spherical clusters. For another example, the computational complexity of the spectral clustering algorithm depends only on the number of data points but has nothing to do with the dimensionality of the data, and therefore can deal with datasets of high dimensionalities. Finally, spectral clustering is very easy to implement, and can use the standard linear algebra methods for fast solutions.

In the current work, we utilize the *k*-way Normalized Cut (Ncut) partitioning approach. For a given data set, which contains N points, $x_1, x_2, \ldots, x_N$, the goal of clustering is to divide these N points into k clusters so that the similarities between two points belonging to a same cluster are maximized and those between two points belonging to different clusters are minimized. Treating a data point x_i in the dataset as a vertex v_i in a graph and the similarity between x_i and x_j, W_{ij}, as the weight of the edge connecting x_i and x_j, we obtain a weighted undirected graph, $G = (V, E)$ with V being the set of all the vertexes and E being the set of all the edges in graph G. Suppose that the multi-way spectral clustering partitions the graph G into k sub-graphs, $A_1, A_2, \ldots, A_k$, then the target function of Normalized Cut (Ncut) that makes the optimal partitioning can be described as:

$$\mathrm{Ncut}(A_1, A_2, \ldots, A_k) = \sum_{i=1}^{k} \frac{\mathrm{cut}\left(A_i, \overline{A}_i\right)}{\mathrm{vol}(A_i)} \tag{11.1}$$

where

$$\mathrm{cut}(A, B) = \sum_{i \in A, j \in \boldsymbol{B}} W_{ij} \tag{11.2}$$

$$\overline{A}_i = \left\{ v_p | v_p \in V,\ v_p \notin A_i \right\} \tag{11.3}$$

$$\mathrm{vol}(A_i) = \sum_{p \in A_i, q \in V} W_{pq} \tag{11.4}$$

After mathematic transformations, the solution for the above Ncut problem can be finally converted to a problem of finding eigenvalues (and eigenvectors) of a Laplacian matrix and the smallest series of eigenvalues correspond to the optimal partitioning of the graph. Thus the discrete clustering problem becomes to find the eigenvectors on a contiguous data space. In the general frame of the spectral clustering, the following three Laplacian matrices are commonly used,

$$L = D - W \tag{11.5}$$

$$L_{\mathrm{sys}} = D^{-\frac{1}{2}} L D^{-\frac{1}{2}} = I - D^{-\frac{1}{2}} W D^{\frac{1}{2}} \tag{11.6}$$

$$L_{\mathrm{rw}} = D^{-1} L = I - D^{-1} W \tag{11.7}$$

where W is the similarity matrix, D is the corresponding diagonal matrix, $D_i = \sum_{j=1}^{N} W_{ij}$, and I is the identify matrix. In the current research, we adopt the second form of Laplacian matrix. Noticing that $L_{\mathrm{sys}} = I - D^{-1/2} W D^{-1/2}$, the sums of the eigenvalues of $D^{-1/2} W D^{-1/2}$ and the corresponding eigenvalues of L_{sys} are one and the corresponding eigenvectors are equal. Therefore, finding the eigenvectors of $D^{-1/2} W D^{-1/2}$ which corresponds to the k largest eigenvalues is equivalent to finding the eigenvectors of L_{sys} which corresponds to the k smallest eigenvalues. With these ideas in mind, the general frame of spectral clustering works as following,

(1) Given a set of data points, $x_1, x_2, \ldots, x_N$, calculate the similarity between every two points according to a certain kind of similarity definition (e.g., the Gaussian kernel function) and construct the similarity matrix $\boldsymbol{W}$.
(2) Compute the corresponding Laplacian matrix L_{sys}, find k eigenvectors corresponding to the k smallest eigenvalues and use these k eigenvectors as columns to construct a feature matrix in a k-dimensional space, $H \in R^{N \times k}$.
(3) Treat each row of H as a data point in k-dimensional space and perform K-means clustering on these points. Each original d-dimensional data point is assigned the same cluster label as the k-dimensional feature vector of the corresponding row in the feature matrix H.

Although it is generally believed that spectral clustering can produce high-quality clusters on small data sets, it has limited applicability to large-scale problems due to its cubic computational complexity. To partially circumvent this drawback, Yan et al. proposed a sampling-based approach (referred to as KASP in the following) which extends the range of spectral clustering by developing a general framework for fast

approximate spectral clustering in which a distortion-minimizing local transformation is first applied to the data (Yan et al. 2009).

11.2.2 BIRCH Tree

Proposed in 1996, BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies) is a comprehensive hierarchical clustering algorithm built upon a data structure named Clustering Feature (CF) (Tian et al. 1996). BIRCH algorithm can deal with multi-dimensional data in an incremental and dynamical fashion. A clustering feature is a summary of a cluster (with n data points) which is composed of three measures, defined as,

$$\mathrm{CF} = \left(n, \overrightarrow{LS}, \overrightarrow{SS}\right) \tag{11.8}$$

$$\overrightarrow{LS} = \sum_{i=1}^{n} x_i \tag{11.9}$$

$$\overrightarrow{SS} = \sum_{i=1}^{n} x_i^2 \tag{11.10}$$

These parameters have many advantages. For example, when two clusters are merged into a new cluster, the clustering feature vector for the new cluster can be calculated incrementally. Further, given the clustering feature of a certain cluster, the cluster center, the radius, and the diameter can be calculated easily as follows,

$$x_0 = \frac{\sum_{i=1}^{n} x_i}{n} = \frac{\overrightarrow{LS}}{n} \tag{11.11}$$

$$R = \left(\frac{\sum_{i=1}^{n} (x_i - x_0)^2}{n}\right)^{1/2} = \left(\frac{n\sum_{i=1}^{d} SS_i - \sum_{i=1}^{d} LS_i^2}{n^2}\right)^{1/2} \tag{11.12}$$

$$D = \left(\frac{\sum_{i=1}^{n}\sum_{j=1}^{n} \left(x_i - x_j\right)^2}{n(n-1)}\right)^{1/2} = \left(\frac{2n\sum_{i=1}^{d} SS_i - 2\sum_{i=1}^{d} LS_i^2}{n(n-1)}\right)^{1/2} \tag{11.13}$$

As illustrated in Fig. 11.1, the clustering feature tree is a highly balanced tree, the construction of which requires two parameters, the branching factor B and the threshold value of T. The branching factor B represents the maximum number of children nodes of a parent node. The threshold T is a constraint for the leaf nodes, requiring certain parameters of the leaf node be satisfied. A common choice for T is the diameter of D as calculated from Eq. (11.13), $D < T$. Given the branch coefficient B and the threshold T, and the data set, CF tree construction algorithm runs as follows,

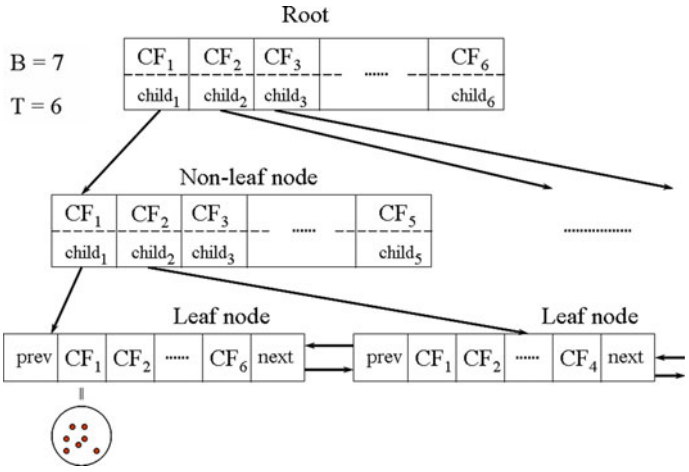


Fig. 11.1 BIRCH tree illustration

(1) For a coming data point, descend down recursively along the path from the root node to a leaf node in accordance with the principle of minimum distance.
(2) Determine whether the CF entry in the leaf node can absorb the new data point (i.e., whether the updated diameter of D when the new data point is added is less than the threshold value T); if yes, update all the CF's along the path from the root node to the leaf node, go to step (4); otherwise, go to step (3).
(3) If the leaf node still has room to accept the CF entry (i.e. the number of CF's at the leaf node is less than B), add this CF to the leaf nodes' CF list; otherwise, split the leaf node by selecting two farthest CF entries among all the CF entries in the leaf nodes as the two new CF children nodes, and then the remaining CF entries are redistributed to the new leaf nodes by the minimum distance rule, delete the original leaf node and update the tree. When the root node is split, the tree height increases by one layer.
(4) If there is any unprocessed data point left, go to step (1), otherwise the algorithm ends.

11.3 A BIRCH Clustering Feature Tree Based Spectral Clustering Algorithm

The visual words (i.e., the labeled feature vectors in the database) obtained by image segmentation correspond well to natural features in the image. In the following, the proposed methods are detailed.

11.3.1 General Idea

To apply spectral clustering algorithm to data sets too large to fit into the main memory at once, some data preprocessing comes in handy. As introduced in the background part, KASP (*K*-means-based approximate spectral clustering) algorithm uses classical *K*-means as a local data reduction step to collapse neighboring data points into a set of local "representative points." Although the algorithm makes it possible for a single machine to spectral cluster data sets with a million observations within several minutes, it may miss some small clusters existing in a large dataset. To manifest this shortcoming, suppose a data set consists of k clusters, namely, C_1, C_2, …, C_k. However, if the data set is first divided into several partitions before clustering, the elements of data points in each partition have the following three situations:

(1) The partition has all or a large number of the data points in cluster C_i.
(2) The partition has a small amount of data points in C_i.
(3) The partition has none of the data points in cluster C_i.

When clustering is performed on each of these partitions, there are three different kinds of clustering results. For the first situation, data points in the new partition will follow a similar distribution to that of data points in the original cluster C_i, and therefore, the data points in the new partition will be clustered into cluster C_i with a larger probability. For the second situation, due to the fact that there exist too few data points in the new partition, they may not follow the distribution characteristics existing in the original data set and will have a greater probability of wrong clustering results. For the third situation, there is no data point in cluster C_i being involved in the clustering process, and therefore, the cluster does not exist in the final clustering result. It is in this sense that, after the preprocessing of KASP, if it happens that the set of local "representative points" does not include all the clusters in the original dataset as the last two situations listed in the above do, some important clusters but of relatively small sizes will be missed. As an improvement, our proposed algorithm is presented in the following for image segmentation task.

11.3.2 Cluster Merging Based on BIRCH Tree

Spectral clustering receives much attention as a competitive clustering algorithms emerging in recent years and is mainly applied to image segmentation. Suppose that we are given a sequence of images taken in an unknown environment and our goal is to identify all the objects in the images in terms of patch-based local color features extracted from the images. To fulfill the task, as in the previous subsection, spectral algorithm is applied to segment each image, producing a number of clusters for each image. By this way, it is possible that data points belonging to different clusters may be labeled with the same cluster label while data points belonging to one cluster are

clustered into different clusters in different images. The next step would be to merge the clusters coming from the same object effectively and efficiently.

The labeled feature vectors obtained from different images by image segmentation correspond well to natural features in the image. It is generally agreed that data points which have natural association are contained in the same regions of the feature space. The distance to the closest region is actually an estimated confidence interval of an output given the input. The data points inside such regions significantly indicate the local fitting accuracy. Based on these observations, we propose to merge clusters in the fashion of a nearest neighbor classifier. Although simple and elegant, nearest neighbor search in a large database of object models is not very efficient. Trees present an efficient way to index local image regions. To support efficient object merging, we propose to use BIRCH tree to merge clusters identified in different images. Since the evaluation of data points only involves a relatively small portion of the tree, the proposed merge technique will first find a threshold so as to identify the same clusters that appear in the different images and separate clusters labeled with the same token.

11.3.3 Threshold Selection

Thresholding is one of the most popular methods for separating clusters. Manually tuning threshold values can be tedious. Because it is often impossible to anticipate possible abnormal situations, to use BIRCH tree as a classification tree to integrate the results of spectral clustering, either for a labeled cluster to be merged with the existing cluster or for a new class to be incorporated into existing classes, the main challenges aim to develop fully automatic systems that do not require task-specific thresholds and tuning. Being the most prominent probability distribution in statistics to describe real-valued random variables that cluster around a single mean value, a normal distribution is often used as a first approximation to complex phenomena. To detect those feature vectors with a potential possibility of coming from a novel object, a precise criterion should be obtained from empirically estimated statistical parameters (e.g., mean/median and standard deviation). Drawn from a normal distribution, about 68% of values are within one standard deviation from the mean; about 95% of the values lie within two standard deviations; and about 99.7% are within three standard deviations. Therefore, for a new cluster detection based on a distance measure with respective to the database, the sum of the unbiased mean and several standard deviations can be a good choice for the threshold value.

To find such a suitable threshold, in each newly segmented image, after its feature vectors are inserted into the BIRCH tree, distances between each feature vector and its approximate nearest neighbor in the tree are calculated and retained to calculate a intra-cluster distance mean and standard deviation. If the size of the pictures taken is Width-by-Height and a moving window of size $N \times N$ is shifted by M pixels evenly in the row and column directions, for each picture, the number of feature vectors (FV) extracted, #FV, are clustered into k categories,

$$\text{width} = (\text{Width} - N)/M + 1 \tag{11.14}$$

$$\text{height} = (\text{Height} - N)/M + 1 \tag{11.15}$$

$$\#\text{FV} = \text{width} \times \text{height} = \#\text{FV}_1 + \cdots + \#\text{FV}_k \tag{11.16}$$

where width and height denote the size of the resulting segmented image. Since the approximate nearest neighbor search through the database tree is used, in order to reduce the sensitivity to noisy outliers, the median value, instead of the mean value, of these $\#\text{FV}_1$ distances is calculated as $\text{d}_{\text{median},l}$, $1 \leq l \leq k$ for each cluster. Then, for each cluster, an L_1-norm based statistic, $\text{std}_{L,\text{l}1}$, that loosely resembles the standard deviation is calculated as,

$$\text{std}_{L1,l} = \frac{\sum_{i=1}^{\#\text{FV}_l} \left| d_{\text{FV}_l,i} - d_{\text{median},l} \right|}{\#\text{FV}_l},\ 1 \leq l \leq k \tag{11.17}$$

where $d_{\text{FV}_l,i}$ denotes the distance between the ith feature vector (FV) and its nearest neighbor in the category l, $d_{\text{median},l} = \text{median}\left(d_{\text{FV}_1,1}, \ldots, d_{\text{FV}_l,\#\text{FV}_l}\right)$, and $\#\text{FV}_l$ is the total number of feature vectors extracted per image and clustered into category l, $1 \leq l \leq k$. This results in multiple medians and multiple std_{L1}'s, each for a discovered category. The formal definition of the threshold $T_{\text{median},l}$ for category l, $1 \leq l \leq k$, is the following sum,

$$T_{\text{median},l} = d_{\text{median},1} + 1.5 \times \text{std}_{L1,l}, \quad 1 \leq l \leq k \tag{11.18}$$

11.3.4 Tree Insertion

After the first image is segmented based on spectral clustering the extracted feature vectors, all the labeled feature vectors are then inserted into clustering feature tree, that is, the BIRCH tree. For subsequent images processed by the spectral clustering algorithm, given a new labeled feature vector, we process it as follows,

(1) Recursively descends the labeled color-histogram-based feature vector from the root node to the leaf node in accordance with the minimum distance principle, locate its approximate nearest neighbor in the leaf node of the tree, and calculate their distance.
(2) Insert the feature vector into the BIRCH tree as the standard BIRCH tree insertion process does.
(3) If the majority of the members of a cluster in the subsequent image have a distance which is smaller than its cluster's threshold, these clusters are merged, and go to step (5).

(4) If the majority of the members of a cluster in the subsequent image have a distance which is larger than the threshold, $T_{\text{median,}}$ a new cluster appears and a new label is assigned to its members.
(5) If there is any unprocessed data point left, go to step (1), otherwise the algorithm ends.

11.3.5 Algorithm Description

With a set of images being the input and a set of segmented images being the output, the overall proposed clustering algorithm runs as follow,

(1) The clustering step: read in an image, extract the feature vectors, apply spectral clustering algorithm to segment it and assign a label to each feature vector.
(2) The merge step: insert the feature vectors into the BIRCH tree and merge clusters.

The clustering step and the merge step are repeated until the end of the image sequence is reached. Finally, each cluster after integration is marked by a different color, and the results of image segmentation are obtained.

11.3.6 Time Complexity Analysis

Since the number of feature vectors extracted from a single image is expected to be relatively small, the number of distance computations consumed is expected to be relatively small as well. The time complexity of our incremental spectral clustering algorithm is,

$$O_{IK} = O\left(m\left(\frac{N}{m}\right)^3 + N\log_B N\right) \tag{11.19}$$

where N is the whole data size and m is the number of partitions, that is, images. The first term is the time complexity of the original spectral clustering and the second term is the time complexity of the BIRCH algorithm.

11.4 Experiments and Results

In this section, we present the results of three sets of experiments conducted to evaluate our proposed clustering algorithms. In the first set of experiments, we perform experiments on two sets of images in outdoor environments to evaluate the proposed

Table 11.1 The set of data

Data name	Image size	# of images	N	M	Dimension
Data1	1280×720	18	27	18	$15 \times 4 \times 4$
Data2	481×321	8	10	6	$15 \times 4 \times 4$

integration-based incremental spectral clustering algorithm. Being the most prominent clustering methods, minimum spanning tree based clustering algorithm is used as the competitor in comparison to our proposed approach on these real-world image datasets. The results are also compared with those of KASP method to check the technical robustness of this study. In the second set of experiments, the run time performance of the proposed algorithm is compared to those of KASP-based and MST-based methods. In the third set of experiments, we apply the proposed approach to image segmentation in two more complex situations to test the capacity of BIRCH tree.

We implemented all the algorithms in Python and ran them on a computer with Intel Core 2 Duo Processor E6550 2.33 GHz CPU and 2 GB RAM. The operating system running on this computer is Windows 10. The data sets are briefly summarized in Table 11.1.

11.4.1 Image Segmentation

Outdoor environments are less structured. Our first dataset, Data1, consists of 18 pictures taken around an outdoor environment. The images are first converted from *RGB* color space into *HSV* color space. To obtain feature vectors, for each image, a moving window of size $N \times N$ is shifted by M pixels in the row and column directions of the image to generate image patches. For each image patch, the chromaticity H is evenly divided into 15 intervals, while the saturation S and the brightness V are evenly divided into 4 intervals. All the interval combination can generate 240 different color bins for the generation of color histogram for each image patch. Then, all the pixels in the moving window are assigned to the corresponding possibility, and a sparse feature vector (color histogram) is obtained. The second data set is from Berkeley Data Set 500 (Unnikrishnan et al. 2007). For the first set of images, $N = 27$ and $M = 18$. For the second set of images, $N = 6$ and $M = 4$.

The clustering performances are presented in Figs. 11.2 and 11.3 for five images from Data1 (in order to keep the amount of data presented reasonable but consistent performance results are obtained for other images), and all eight images from Data2. There are four columns in each figure. The left column displays the testing images used. The middle left column displays the clustering results using our proposed method. The middle right column displays the clustering results of KASP. Finally the last (right) column displays the clustering results of MST-based clustering algorithm. To ease visual examination, in the segmented images, a color is chosen to denote

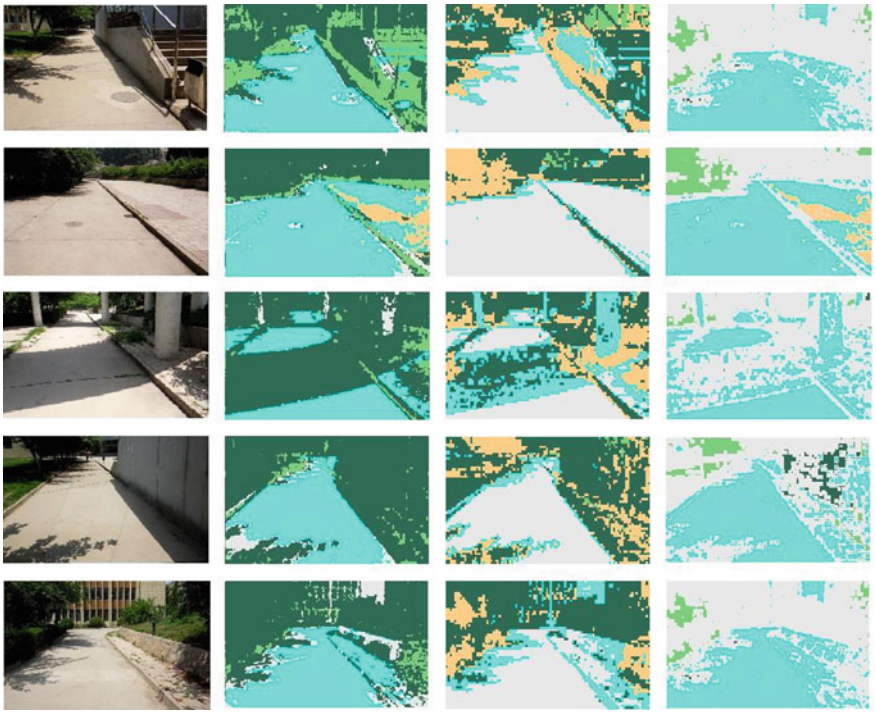


Fig. 11.2 (Left) Original image, results of (middle left) our algorithm and (middle right) KASP, (right) MST-based algorithm

the percepts. To display results from three different methods, three close but slightly different sets of colors are used. All three methods obtained 6 clusters. The results show that different methods have different optimal segmenting results. As the number of clusters increases, more percepts appear. Too many clusters will eventually make some percepts hard to recognize.

From the original images of Data1, we can see that the major percepts include grey concrete road occupying most lower-half part or lower left-half part of images and extending into the far end background, light grayish white concrete buildings and their shadows to the right of road in the first, the third and fourth scenes, concrete building steps on the right part of the first scene, manhole covers on the concrete road in the first two images, wide brick pavement on the right of the second scene in the middle of which several light pink bricks reside, several dark green trees (with shadows) and light green grass mingled in the far end of the first three scenes, on the left of the fourth scene and on the both sides of the last scene, pillars and pillar shadows in the third scene, a building in the background of and a side road on the right of the last scene, and a little garbage can standing at the lower-right corner of the first scene. In this segmentation task, the concrete road and its neighboring brick pavement together with the manhole covers and the light pink bricks are the most



Fig. 11.3 (Left) Original image, results of (middle left) our algorithm and (middle right) KASP, (right) MST-based algorithm

discriminative foreground patterns that can be easily captured and separated from each other and from the cluttered background.

From the segmented figures, it can be seen that typical stable percepts detected by our method include a bundled dark green percept (corresponding to dark green trees, green grass, concrete building walls, their shadows, and pillar shadows), concrete road, building steps, pillars, and manhole covers, and, for this outdoor environment, it is a little harder to segregate the more detailed percepts bundled into dark green. From the segmented images as shown on the first two rows of Fig. 11.2, it can be seen that our method correctly identifies the foreground road, the building steps, the background dark green trees, and even the little things such as the manhole covers and the light pink bricks, and it is in this sense that our method performs better than KASP which segments the dark green trees on the right side of background in Fig. 11.3 into two percepts. From the segmented images as shown on the fourth row of Fig. 11.2, KASP does a little better than our algorithm by segmenting the building walls from the background dark green tree. For the rest images of Fig. 11.2, it can be seen that KASP and our algorithm perform equally well one way or the other. For the second image, MST-based clustering algorithm does better than KASP by successfully detecting the light pink bricks. However, for all the images shown, MST-based clustering algorithm can only successfully identify the foreground roads and background dark green trees. To summarize, overall, our algorithm can discover all the major percepts most of time and discriminate percepts that KASP-based and MST-based methods can not.

The Berkeley Segmentation Dataset consists of natural images that contain complex layouts of distinct textures, thin and elongated shapes and relatively large illumination changes, and is therefore a challenge for segmentation task. Data2 has 8 different scenes, which are shown in Fig. 11.3.

The first scene is composed of a golden pyramid on a desert (with a dark grayish sky background) in front of which a camel carries a man. The second scene is composed of a stone pavilion in a near front of a grey brick wall and a grayish sky background. The third scene contains 6 circular stone pillars standing before two undulating mountains in different colors under a setting sun. The fourth scene has a boat on the shore and a sea separating the foreground boat from the background rock and sky. The fifth scene is composed of a small group of elephants foraging on the brown to yellowish green grass with a dark green tree in middle against a far end light blue sky. The sixth scene contains a castle in the foreground against light grayish sky. The seventh scene consists of an upper part of a big dog facing to the left and standing in front of a light grayish sky with a dark brown tree trunk to its right. The last scene has two elephants both facing to the right and standing close to each other on the brown grass with a light blue sky behind.

There are 27, 27, and 17 clusters obtained for this dataset using our method, KASP-based and MST-based clustering algorithms, respectively. For the first scene, both KASP and our method can detect the major percepts, that is, the sky, the pyramid, the camel and the rider, while MST-based method cannot set the rider and the camel apart. For the second scene, both MST-based method and our method can successfully separate the foreground stone pavilion and the wall from the background sky, while

Table 11.2 Thresholds for cluster's merger and separation

Data name	T_{median}	Mean	std
Data1	0.3994	0.2029	0.1310
Data2	0.2943	0.0971	0.1315

KASP cannot in a major part. For the third scene, it can be seen that our method is good at detecting the 6 circular pillars and in this sense does a litter better job than MST-based method, which both perform better than KASP. For the fourth scene, KASP and our method perform similarly with the former being a little better than ours. Using MST-based method, some part of the boat body and some part of the rock in the upper right part are mixed to be the same percept, and the sky and the sea are mixed to be the same percept as well.

For the fifth scene, it is clearly seen that our method does a slightly better job than MST-based method, and both perform better than KASP which oversegments the sky. For the sixth scene, it is evident that all three methods clearly separate the sky from the foreground castle. However, based on visual evaluation, KASP and our method both perform better than MST-based method. For the last two scenes, all three methods do not work reasonably well. Although they all detect the dog and the elephants to some extent, respectively, the percepts are oversegmented. To summarize, overall for all the images, our algorithm can separate most percepts apart with clearly defined boundaries while KASP-based and MST-based methods perform less satisfactorily.

To cluster a data set using our algorithm, we need to specify the threshold values for the detection of new percepts (clusters) as images are input into our algorithm. Threshold calculation is a crucial step. Using different thresholds may result in different detection performances. Hence, to test our intuition, as images come in a sequential form, threshold values extracted from them for clusters are shown in Table 11.2. In the table, T_{median} listed in the first column is used to denote the cases for which an average over 5 and 17 threshold values (with respect to 5 and 17 clusters for Data1 and Data2) are calculated from Eq. (11.11), respectively. The values in the second and third columns are the averages of the estimated values for unbiased mean and standard deviation, respectively. The thresholds based on the sum of estimated mean and 1.5 times standard deviation are used.

Overall, the results presented above offer good evidence that our method performs very well on a variety of images from different outdoor domains.

11.4.2 Running Time Performance

The parameters used for the BIRCH tree, that is, the branching factors, B's, and the threshold values, T's, are provided in Table 11.3, for Data1 and Data2, respectively. The running time performances of our algorithm, KASP, and MST-based clustering algorithm are summarized in Table 11.4. In the KASP method, 170 local

Table 11.3 The set of parameters

Data name	Branching factors, B	Threshold values, T
Data1	100	1.04
Data2	100	0.43

Table 11.4 Running time performance in seconds

Data name	Ours	KASP	MST
Data1	405	441	2580
Data2	44	96	153

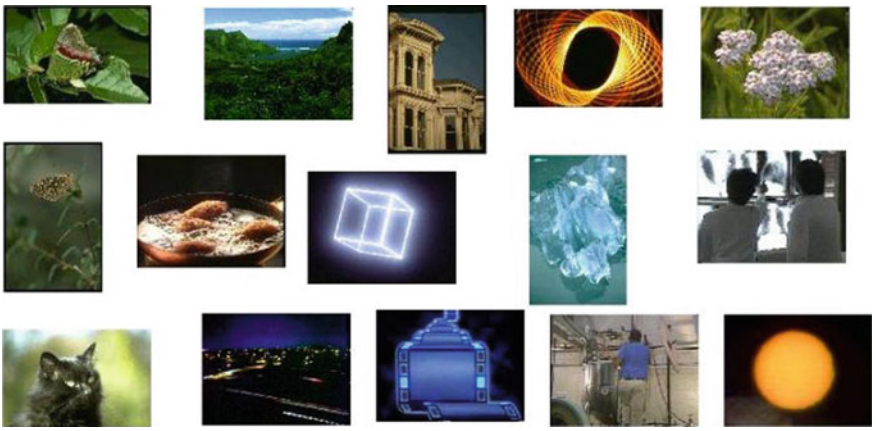


Fig. 11.4 Samples from Stanford InfoLab database

representative points are chosen. It can be seen from the table that our method and KASP use a similar amount of time to segment two image datasets, which however is significantly shorter than the MST-based method. More specifically, our algorithm and KASP outperform MST-based clustering algorithm by a factor of 5 and 4 in running time performance for the two sets of data (i.e., Data1 and Data1), respectively. This phenomenon is due to the fact that MST-based clustering algorithm requires an MST be constructed in its first place, resulting in a longer running time, that is, a quadratic time complexity. Overall, it can be seen that our algorithm outperforms the other two clustering algorithms.

11.4.2.1 Capacity of BIRCH Tree

To provide support for content-based image retrieval, we apply the proposed algorithm to two large image databases. As illustrated in Fig. 11.4, the Stanford Infolab Database consists of 9907 images and contains many things in life, such as insects, animals, plants, natural landscapes, characters, cartoons, icons, and so on.

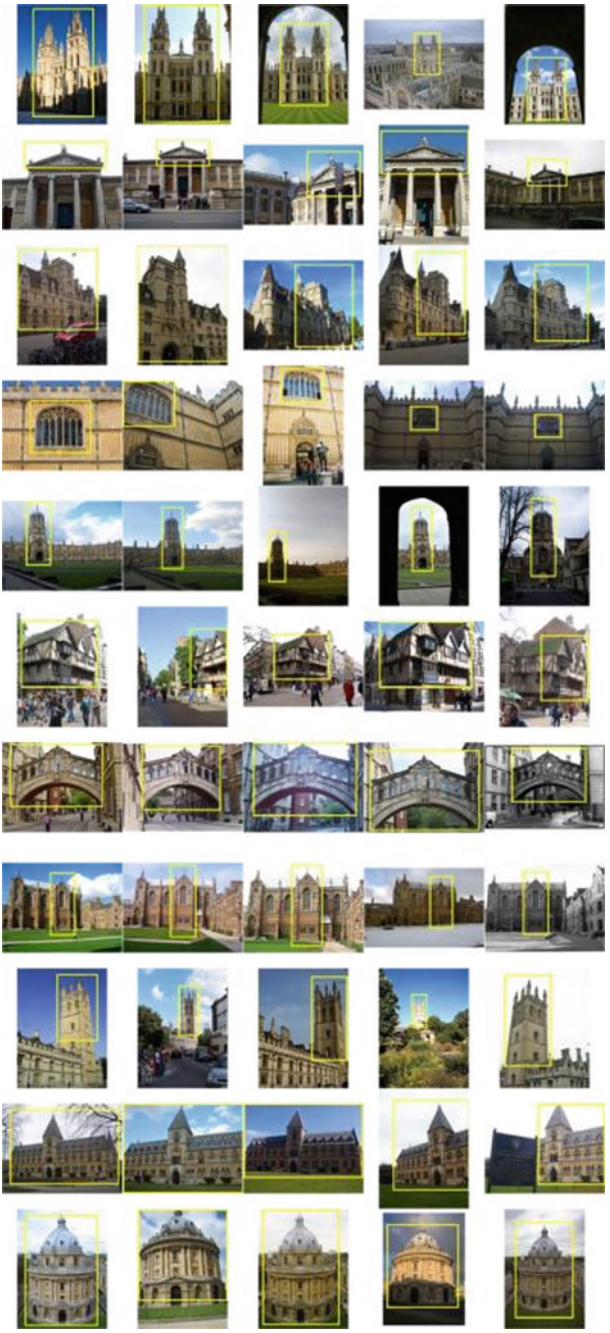


Fig. 11.5 Samples from Oxford Buildings Dataset

Table 11.5 The set of two more data and running time results (RT in seconds)

Data name	# of images	N	M	Dimension	# of inserted images	RT (second)
Oxford Buildings Dataset	5063	27	18	$15 \times 4 \times 4$	3780	3510
Stanford Infolab Dataset	9908	10	6	$15 \times 4 \times 4$	9908	583

As illustrated in Fig. 11.5, the Oxford Buildings Dataset consists of 5062 images collected from Flickr by searching for particular Oxford landmarks.

The data sets and experimental results are briefly summarized in Table 11.5. There are seven columns in Table 11.5. The first column shows the name of the datasets. In the second column, the numbers of images in each dataset are shown. In the third and fourth columns, the sizes of the moving window and the hopping step are specified. In the fifth column, the dimensions of the extracted features are shown. In the sixth column, the maximum numbers of images for which the extracted color histogram based feature vectors are successfully inserted into the BIRCH tree are presented. That is, 3780 out of 5063 images from the Oxford Building Dataset are successfully inserted into the tree, and all 9908 images of the Stanford Infolab Database are successfully inserted into the tree, in terms of the extracted feature vectors. The last column presents the running times in seconds used to insert those images into the BIRCH trees. From the experimental results, we can conclude that our method is technical sound to some extent.

11.5 Conclusions

In this chapter, we have proposed an incremental spectral clustering. Our algorithm applies spectral clustering to segment each image and uses BIRCH tree for cluster merging. The algorithm significantly reduces the expense of the matrix computation in spectral clustering, while retaining good control on the clustering accuracy. Evaluation on an image segmentation task shows that a significant speedup for spectral clustering can be achieved by our algorithm in comparison to the brute force method and that better clustering accuracy can be obtained by our algorithm with respect to the KASP method.

References

Bleyer, M., & Gelautz, M. (2005). Graph-based surface reconstruction from stereo pairs using image segmentation. In *Proceedings of SPIE—The International Society for Optical Engineering* (Vol. 5665, pp. 288–299). SPIE.

Canny, J. (1986). A computational approach to edge detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 8,* 679–698.

Ding, C. (2007). A tutorial on spectral clustering. *Journal of Statistics and Computing, 17,* 395–416.

Donath, W. E., & Hoffman, A. J. (1973). Lower bounds for the partitioning of graphs. *IBM Journal of Research and Development, 17,* 420–425.

Duarte, A., Sánchez, A., Fernández, F., & Montemayor, A. S. (2006). Improving image segmentation quality through effective region merging using a hierarchical social metaheuristic. *Pattern Recognition Letters, 27*(11), 1239–1251.

Favaro, P., & Soatto, S. (2004). A variational approach to scene reconstruction and image segmentation from motion blur cues. In *Proceedings of IEEE International Conference of Computer Vision and Pattern Recognition (CVPR '04)*.

Funka-Lea, G., Boykov, Y., Florin, C., Jolly, M.-P., Moreau-Gobard, R., Ramaraj, R., et al. (2006). Automatic heart isolation for CT coronary visualization using graph-cut. In *Proceedings of IEEE International Symposium on Biomedical Imaging (ISBI '06)* (pp. 614–617).

Gonzalez, R. C., & Woods, R. E. (1992). *Digital image processing*. Reading, MA: Addison Wesley.

Grady, L., Sun, Y., & Williams, J. (2006). Three interactive graph-based segmentation methods applied to cardiovascular imaging. In *Mathematical models in computer vision: The handbook* (pp. 453–469). Springer.

Jia, H., Ding, S., Xu, X., & Nie, R. (2014). The latest research progress on spectral clustering. *Neural Computing and Applications, 24,* 1477–1486.

Jolly, M. P., Xue, H., Grady, L., & Guehring, J. (2009). Combining registration and minimum surfaces for the segmentation of the left ventricle in cardiac cine MR images. In *Proceedings of MICCAI* (pp. 910–918).

Lam, C. L., & Yuen, S. Y. (1998). An unbiased active contour algorithm for object tracking. *Pattern Recognition Letters, 19*(5–6), 491–498.

Meila, M., & Shi, J. (2001). A random walks view of spectral segmentation. In *Proceedings of the Eighth International Workshop on Artificial Intelligence and Statistics (AISTATS '01)*.

Ng, A., Jordan, M., & Weiss, Y. (2001). On spectral clustering: Analysis and an algorithm. In *Advances in Neural Information Processing Systems* (pp. 849–856).

Ning, J., Zhang, L., Zhang, D., & Wu, C. (2010). Interactive image segmentation by maximal similarity based region merging. *Pattern Recognition, 43,* 445–456.

Paul, B., Zhang, L., & Wu, X. (2005). Canny edge detection enhancement by scale multiplication. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 27,* 1485–1490.

Peng, B., Zhang, L., & Zhang, D. (2013). A survey of graph theoretical approaches to image segmentation. *Pattern Recognition, 46*(3), 1020–1038.

Russell, B., Efros, A., Sivic, J., Freeman, W., & Zisserman, A. (2006). Using multiple segmentations to discover objects and their extent in image collections. In *Proceedings of IEEE International Conference on Computer Vision and Pattern Recognition (CVPR '06)*.

Seo, K. H., Shin, J. H., Kim, W., & Lee, J. J. (2006). Real-time object tracking and segmentation using adaptive color snake model. *International Journal of Control, Automation and Systems, 4*(2), 236–246.

Shi, J., & Malik, J. (2000). Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 22,* 888–905.

Tian, Z., Ranghu, R., & Miron, L. (1996). BIRCH: An efficient data clustering method for very large databases. In *Proceedings of the 1996 ACM SIGMOD international conference on Management of data (SIGMOD '96)* (pp. 103–114).

Unnikrishnan, R., Pantofaru, C., & Hebert, M. (2007). Toward objective evaluation of image segmentation algorithms. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 29*(6), 929–944.

Wang, X., Wang, X. L., Chen, C., & Wilkes, D. M. (2013). Enhancing minimum spanning tree based clustering by removing density based outliers. *Digital Signal Processing, 23*(5), 1523–1538.

Wang, X., Wang, X. L., & Wilkes, D. M. (2009). A divide and conquer approach for minimum spanning tree based clustering. *IEEE Transactions on Knowledge and Data Engineering, 21*(7), 945–958.

Wertheimer, M. (1938). Laws of organization in perceptual forms (partial translation). In W. B. Ellis (Ed.), *A sourcebook of gestalt psycychology* (pp. 71–88). Brace: Harcourt.

Yan, D., Huang, L., & Michael, I. J. (2009). Fast approximate spectral clustering. In *Proceedings of ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (SIGKDD '09)* (pp. 907–916).

Yu, S., Gross, R., & Shi, J. (2002, December). Concurrent object segmentation and recognition with graph partitioning. In *Proceedings of Neural Information Processing Systems (NIPS)* (pp. 1383–1390).

Chapter 12
An Incremental EM Algorithm Based Visual Perceptual Grouping



Abstract With the increasing demand for analytical calculation for datasets of different scales and types, a diversity of clustering algorithms have been developed in recent years, among which the EM clustering algorithm has a good clustering effect and has remained popular in likelihood applications. Based on the superiority of the EM algorithm on clustering small-scale datasets and to further test the performance of the BIRCH tree in classification application, in this chapter, an integration-based fast incremental EM clustering algorithm is presented for multiple-percept detection tasks in image sequences for mobile robotic applications in an unknown environment. Basically, the proposed algorithm first applies the standard EM algorithm to group image-patch-based feature vectors extracted from an image perceptually. Then, a fast tree-based approximate nearest neighbor classifier is employed to integrate the clustering results. Experiments performed in an outdoor environment demonstrate the efficacy of the proposed method.

Keywords Data clustering · EM algorithm · Instance-based classification · Approximate nearest neighbor search · BIRCH tree · Image segmentation · Perceptual grouping

12.1 Introduction

Given a set of data points and a distance or similarity measure, clustering is the process of partitioning the data set into subsets, called clusters, so that the data in each subset share some properties in common. Usually, the common properties are quantitatively evaluated by some measures of the optimality such as minimum intra-cluster distance or maximum inter-cluster distance, etc. Clustering has been extensively studied and many algorithms have been proposed in the literature, such as hierarchical, partitioning-based, density-based, model-based approaches, etc. Most of the work in clustering can be viewed in terms of learning a probabilistic model of the data. Therefore, many clustering algorithms can be seen as finding maximum likelihood parameter estimates (MLE). Being a competitive clustering method, the maximum expectation algorithm, i.e., the EM algorithm, was first proposed by Dempster et al. (1977) as an iterative algorithm for maximum likelihood estimation or maximum

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_12

posterior probability estimation for probabilistic parameter models with implicit variables and proceeds alternately in two steps, the E step and the M step (Dempster et al. 1977). The first step, that is, the E step, is to compute expectation where the maximum likelihood estimation is calculated by using the estimated values of hidden variables. The second step, that is, the M step, is to use the maximum likelihood value obtained at the E step to calculate the values of the parameters. Then the parameter estimates found in the M step are used in the next E step calculation, which is constantly alternating. The conditional likelihood converges monotonically from below to some maximum value of the sequence. In general, if this likelihood has several maxima and stationary points, the choice of starting point determines which type of point the EM sequence will converge to. Remaining popular in likelihood applications, the EM algorithm has been frequently used for data clustering in machine learning and computer vision (McLachlan and Peel 2000; Ng and McLachlan 2004a, b), and has been a popular method to determine MLE for a few decades. With the development of artificial intelligence, machine learning, and pattern recognition, while the EM algorithm has proven to be at least initially more attractive, it becomes more and more important to quickly and efficiently perform clustering on large scale datasets (Neal and Hinton 1998).

This chapter introduces the expectation-maximization (EM) clustering algorithm to perceptual grouping based image segmentation. To apply this very competitive clustering methodology for processing large image data, in this chapter, we propose an integration based fast EM clustering approach, which, when combined with a fast tree-based approximate nearest neighbor classifier, manifests its effectiveness and efficiency. First, it applies the EM algorithm to segment a coming image based on feature vectors extracted from the image taken when an autonomous mobile robot explores in an unknown outdoor environment. Next, these labeled sample feature vectors are accumulated and inserted into a balanced BIRCH tree of image vocabulary percepts, and a statistically obtained threshold is determined and used to recognize new percepts in the environment. Finally, a newly grabbed image is parsed, new objects are identified if appearing, and the tree and the threshold are updated. One important contribution of the proposed method is the use of Silhouette for the determination of the number of clusters/objects existing in an image since the convergence of an EM sequence to a local or global maxima or stationary point depends on the initial parameter estimates. Another important contribution of the proposed method is its significant efficiency in clustering large datasets while maintaining reasonably good clustering capability. This second contribution is an advantage over an approximate nearest neighbor based classifier using locality sensitive hashing. Finally, for automatically learning newly appearing objects/percepts, setting a threshold for intuitive incremental updating of the knowledge base is an important capability of any intelligent systems. This third contribution is an advantage over a previous approach presented in (Wang et al. 2017), in which a randomly generated quantization tree-based approximate nearest neighbor classifier is used. To show the technical soundness of this research, the performance of the proposed approach is compared with that of two popular clustering algorithms and previous work for a perceptual grouping task in an outdoor environment. We hope the research presented here can be

used to equip autonomous mobile robots working in an unknown environment with automatic environmental model generation capability.

The rest of the paper is organized as follows. In Sect. 12.2, we present some backgrounds on EM clustering and instance-based classifiers. In Sect. 12.3, an integration-based effective EM clustering algorithm is introduced. In Sect. 12.4, we present the results of experiments conducted to evaluate the performance of the proposed algorithm. Finally, the conclusions are made in Sect. 12.5.

12.2 Backgrounds

Background with regard to the method presented in this chapter falls into two main categories, EM algorithm and instance-based classifiers.

12.2.1 Expectation and Maximization Clustering Algorithm

The main purpose of the EM algorithm is to provide a simple iterative algorithm to compute posterior density functions. Assuming that the sample distribution conforms to the Gaussian mixture model, the aim of the algorithm is to determine the parameters of each Gaussian component so as to fit them adequately. In other words, each sample is assumed to belong to each Gaussian distribution with different probability, and the probability value will be estimated based on the above parameters.

More specifically, the Gaussian mixture model is defined as a linear combination of M Gaussian density functions,

$$P(x) = \sum_{i=1}^{M} \pi_i N_i(x, \mu_i, \Sigma_i) \tag{12.1}$$

where $N_i(x, \mu_i, \Sigma_i)$ is a Gaussian distribution with mean μ_i and covariance Σ_i, π_i are the weights of the ith Gaussian distribution and represent the prior probability, and

$$\sum_{i=1}^{M} \pi_i = 1 \quad \text{where } 0 \le \pi_i \le 1 \tag{12.2}$$

$$N_i(x, \Sigma_i, \mu_i) = \frac{1}{(2\pi)^{d/2}|\Sigma_i|^{1/2}} \exp\left\{-\frac{1}{2}(x-\mu_i)^T \sum_{i}^{-1}(x-\mu_i)\right\} \tag{12.3}$$

Maximum likelihood estimation is the most commonly used method of parameter estimation. The method is to estimate parameters by maximizing the likelihood

function. If all undetermined parameters in the Gaussian mixture density function are denoted as θ and X is defined as observation variables, the likelihood function is,

$$P(X|\theta) = \prod_{i=1}^{N} P(x_i|\theta) \Rightarrow \theta^* = \arg_\theta \max P(X|\theta) \tag{12.4}$$

In order to simplify the problem, we need to calculate the maximum of the following formula,

$$\log(P(X|\theta)) = \sum_{i=1}^{N} \log\left(\sum_{i=1}^{K} \pi_k N(x_i, \mu_k, \Sigma_k)\right) \tag{12.5}$$

Because of the logarithm of the sum, the form of derivation is complex, and the general method of finding the partial derivative and making derivative zero cannot be used. Therefore, Z is an implicit variable and the problem becomes the maximum of the following formula,

$$\log(P(X, Z|\theta)) = \sum_{i=1}^{N} \log\left(\pi_i N(x_i, \mu_{z_i}, \Sigma_{z_i})\right) \tag{12.6}$$

where the value of Z is not observed. The EM algorithm can only assume that the distribution of Z is determined by the estimated parameters of the previous iteration, and then the expected maximum is obtained. For this, we define,

$$Q(\theta, \theta^{\text{old}}) = EZ\left[\log(P(X, Z|\theta))|X, \theta^{\text{old}}\right] \tag{12.7}$$

Then we can calculate,

$$\sum_{k}^{\text{new}} = \frac{\sum_{i=1}^{N} p\left(k|x_i, \theta^{\text{old}}\right)\left(x_i - \mu_k^{\text{new}}\right)\left(x_i - \mu_k^{\text{new}}\right)^T}{\sum_{i=1}^{N} p\left(k|x_i, \theta^{\text{old}}\right)} \tag{12.8}$$

The specific flow of EM algorithm is to repeat the following two steps until convergence, the E step, where the likelihood function is calculated according to the initial set of values of the model parameters or the results of the previous iteration, and the M steps, where the likelihood function is maximized to obtain new parameter values and θ^{old} is updated with θ^{new} to maximize $Q(\theta, \theta^{\text{old}})$.

Although the EM clustering is simple and stable, the number of clusters for a given dataset has to be given as an input. To obtain a reasonable value for the number of clusters which is not given beforehand, Silhouette coefficient can be used to measure how similar an object is to its own cluster (cohesion) as to be compared to other clusters (separation) (Rousseeuw 1987; de Amorim and Hennig 2015).

12.2.2 Instance-Based Learning and a Novelty Detection Method

With the set of feature vectors being labeled, the next step is to classify new observations. In supervised learning, instance-based learning is a family of learning algorithms that compares new problem instances with instances seen in the training. More specifically, when predicting a class label for a new instance based on a stored training set, the algorithms compute distances or similarities between this instance and the training instances to make a decision (Russell and Norvig 2003). One advantage of its over other methods of supervised learning is its ability to adapt its model to previously unseen data. However, instance-based learning has a short-coming that its time complexity can grow with the data. To battle the memory complexity of storing all training instances, instance reduction algorithms have been proposed (Wilson and Martinez 2000). Examples of instance-based learning algorithms are the k-nearest neighbor algorithm, kernel machines and RBF networks (Mitchell 1997).

Real-time processing of the information coming from video demands for efficient search structures and algorithms. Trees present an efficient way to index local image regions, making the nearest neighbor search more efficient by pruning. As a result, in the current research, we are particularly interested in tree-based approximate nearest neighbor search algorithms.

For very large databases consisting of high dimensional highly sparse feature vectors, a previous work of ours chose to follow the notion of Vocabulary tree but with important variations to generate a k-way approximate nearest neighbor search tree. Given an obtained Vocabulary database, at the first level, the tree has only one node, the tree root. At the second level, a set of k representative patterns are randomly selected from the whole database as the cluster centers. Then the whole database is clustered into k subsets by assigning each feature vector to its closest center according to the chosen similarity measure. At the third level, for each of the k clusters obtained at the second level, k feature vectors are selected randomly from its pool of feature vectors as its next level cluster centers, resulting in k^2 cluster tree nodes at this level. This procedure continues until either all the feature vectors in a leaf node (the tree node that has no children nodes) belong to the same object class (a pure node) or the number of the feature vectors in a leaf node is below some limit.

Given a new feature vector, to search through the tree, its distances to k cluster centers at each level along a certain branch are calculated and the winner is the center among k ones that the new feature vector is nearest to. When a leaf node is reached, if it is pure, assign the associated label and then stop. Otherwise, do the nearest neighbor search over the vectors in the associated cluster, and the winner is the feature vector which gives minimum distance according to the chosen metric.

To detect the largest connected potential groups of novel feature vectors from new objects that are not incorporated into the database, in the following, a method for threshold selection is reviewed. Thresholding is one of the most popular methods for new percept detection. Manually tuning threshold values can be tedious. To develop fully automatic systems that do not require task-specific thresholds and tuning is

challenging, and, to detect the feature vectors with a potential possibility of coming from a novel object, a precise criterion should be obtained from empirically estimated statistical parameters (e.g., mean/median and standard deviation). Being the most prominent probability distribution in statistics to describe real-valued random variables that cluster around a single mean value, a normal distribution is often used as a first approximation to complex phenomena. Therefore, for novelty detection based on a distance measure with respect to the database, the sum of the unbiased mean and one standard deviation can be a good choice for the threshold value.

To find such a suitable threshold, in an approach presented in (Wang et al. 2017), for each picture, the number of feature vectors (FV), #FV, are extracted. Since the approximate nearest neighbor search through the database tree is used, in order to reduce the sensitivity to noisy outliers, the median value, instead of the mean value, of these #FV distances is calculated as d_{median}. Then an L_1-norm based statistic, std_{L1}, that loosely resembles the standard deviation is calculated as,

$$\text{std}_{L1} = \frac{\sum_{i=1}^{\#\text{FV}} \left| d_{\text{FV},i} - d_{\text{median}} \right|}{\#\text{FV}} \tag{12.9}$$

where $d_{\text{FV},i}$ denotes the distance between the ith feature vector and its approximate nearest neighbor in the database tree, $d_{\text{median}} = \text{median}\,(d_{\text{FV},1},\ldots, d_{\text{FV},\#\text{FV}})$, and #FV is the total number of feature vectors used per image. This results in multiple medians and multiple std_{L1}'s, each for a single training image. If l frames of normal images are used, the sum of the median of these l medians and the median of these l std_{L1}'s is used as the threshold T_{median}. The formal definition of the threshold T_{median} is the following sum,

$$T_{\text{median}} = \text{median}(d_{\text{median}\,1}, \ldots, d_{\text{median}\,l}) + \text{median}\big(\text{std}_{L1,1}, \ldots, \text{std}_{L1,l}\big) \tag{12.10}$$

where the l in the last subscript in each term is the total number of normal images used to obtain the threshold.

Since the quantization tree could be far away from being balanced when the data is highly skewed, in the current research, we are particularly interested in a popular highly balanced tree, BIRCH tree.

12.3 The Proposed Approach

12.3.1 Image Segmentation Model for Object Recognition

Empirical evidence have shown that EM clustering algorithm can be more accurate than classic clustering algorithms on small-sized datasets. In this section, we propose an integration-based fast EM clustering algorithm that applies EM clustering to data segments and uses a fast approximate nearest neighbor classifier for cluster



Fig. 12.1 Original image (left), segmentation results with SC being (middle) 0.62, (right) 0.44

merging to significantly reduce the expense of the computation in EM clustering, while retaining good control on the clustering accuracy. A concise and precise description of the proposed methodology is provided in the following four subsections.

12.3.2 EM Clustering with the Aid of Silhouette Coefficient

EM algorithm receives much attention as a competitive clustering algorithm and is applied to image segmentation in the current research. Suppose a mobile robot takes a sequence of images while moving in an unknown environment and the goal is to identify all the objects in the images in terms of patch-based local color features extracted from the images to generate a model of the environment for more advanced tasks such as mobile robot navigation tasks thereafter. To fulfill the current task, the EM algorithm is performed on the feature vectors, producing a number of clusters for each image. Since the number of clusters in a single image is not given beforehand, for the sample image shown in Fig. 12.1, to obtain an approximately optimum number of clusters, EM clustering is conducted with the input number of clusters varying in the range [2,9] and the one which corresponds to the largest Silhouette coefficient (SC) is chosen.

As in the illustration, the Silhouette coefficients of EM clustering results (on this image) set by 7 different cluster number parameters (i.e., 2, 3, 4, 5, 6, 7, 8, 9) are calculated to be 0.6178369, 0.4398569, 0.4223299, 0.3825003, 0.3617666, 0.3436575, 0.3227921, 0.3225329, correspondingly. Among them, as shown in Fig. 12.1, when the cluster number is 2, the image is segmented into two clusters, and the Silhouette coefficient value is the largest.

By calculating the Silhouette coefficients for EM clustering results, a set of approximately optimized segmented images are output. The next step would be to merge the clusters coming from the same object in different images effectively and efficiently.

12.3.3 Cluster Merging Based on BIRCH Tree

The labeled feature vectors obtained by image segmentation correspond well to natural features in the image. It is generally agreed that data points which have natural

association are contained in the same regions of the feature space. The distance to the closest region is actually an estimated confidence interval of an output given the input. The data points inside such regions significantly indicate the local fitting accuracy. Based on these observations, we propose to merge clusters in the fashion of the nearest neighbor classifier. Although simple and elegant, the nearest neighbor search in a large database of object models is not very efficient. Trees present an efficient way to index local image regions. To support efficient object merging, we propose to use the highly balanced BIRCH tree to merge clusters identified in different images. Since the evaluation of data points only involves a relatively small portion of the tree, the proposed merge technique will first find a threshold so as to identify the same clusters that appear in the different images and separate different clusters.

To use BIRCH tree as a classification tree to integrate the results of EM clustering, either for a labeled cluster to be merged with the existing cluster or for a new class to be incorporated into existing classes, the main challenges aim to develop fully automatic systems that do not require task-specific thresholds and tuning. To merge those feature vectors with a potential possibility of coming from the same object but showing up in different images into existing CF entries and to generate a new CF entry for those feature vectors coming from new objects in appearing images, a precise criterion for a threshold should be obtained from empirically estimated statistical parameters (e.g., mean/median and standard deviation). Drawn from a normal distribution, about 68% of values are within one standard deviation from the mean; about 95% of the values lie within two standard deviations, and about 99.7% are within three standard deviations. Therefore, for a new cluster detection based on a distance measure with respect to the feature vectors already in the BIRCH tree, the sum of the unbiased mean and several standard deviations can be a good choice for the threshold value.

To find such a suitable threshold, in each newly segmented image, before its feature vectors are inserted into the BIRCH tree, distances between each feature vector and those extracted from the same image and clustered into the same category are calculated, based on which, the diameters of all clusters in the image are calculated. The corresponding average and standard deviation of these diameters are then computed for the determination of the threshold T. If the number of clusters obtained so far is k, the mean value and the standard deviation of these k diameters, d_k, are calculated to be $d_{\text{mean},\,k}$ and $d_{\text{std},\,k}$. Then, the formal definition of the threshold T_k is the following sum,

$$T_k = d_{\text{mean},\,k} + \beta \times d_{\text{std},\,k} \tag{12.11}$$

where β is a factor value in the range [1,3].

With a set of images being the input and a set of segmented images being the output, the overall proposed integrated EM clustering algorithm runs as in the following.

Input: A set of n images.
Output: A set of n segmented images.

Steps:

(1) Read in an image and extract the feature vectors. For the image, the number of clusters, that is, parameter k_i, $1 \leq i \leq m$, is set and EM pre-clustering is carried out on the feature vectors.
(2) The clustering result is evaluated by clustering evaluation indicators such as the Silhouette coefficient. If the result is far from being ideal, the number of clusters k_i is adjusted, then go to Step 1, otherwise go to Step 3.
(3) Select the best clustering result and, for merging clusters subsequently, calculate the cluster separation threshold T_k according to Eq. (12.11) (which is the average plus one standard deviation of the diameters of all clusters obtained so far), set a branching factor B, initialize the CF tree, insert all the labeled feature vectors from the image into the BIRCH tree.
(4) Perform the EM clustering on the next image as in Step 1 and Step 2, after that, for each labeled feature vector, recursively descends it from the root node to the leaf node in accordance with the minimum distance principle, locate its approximate nearest neighbor in the leaf node of the tree, and calculate their distance. If their distance is smaller than the threshold T_k, assign it the label of this nearest neighbor's and go to Step 5; otherwise, if a significant number of feature vectors in a cluster have a distance value larger than T_k, a new cluster appears and go to Step 6.
(5) If the leaf node still has room to accept the CF entry (i.e. the number of CF's at the leaf node is less than B), add this CF to the leaf nodes' CF list; otherwise, split the leaf node by selecting two farthest CF entries in the leaf nodes as two new CF children nodes (and the remaining CF entries are redistributed to the new leaf nodes by the minimum distance rule), delete the original leaf node and update the tree.
(6) Update the threshold T_k and branching factor B for the portion of data processed so far, if there still exist images not processed yet, go to Step 4; otherwise, stop and return the corresponding cluster label for each feature vector as the output.
(7) Finally, each cluster after integration is marked by a different color, and the results of image segmentation are obtained.

12.3.4 Time Complexity Analysis

Since the number of feature vectors extracted from a single image is expected to be relatively small, the number of distance computations consumed is expected to be relatively small as well. The time complexity of the proposed incremental EM clustering algorithm, O_{IEM}, is,

$$O_{\text{IEM}} = O\left(n\left(d\frac{N}{n}t\right) + dN\log_B dN\right) \tag{12.12}$$

where N is the whole data size (i.e., #FV$\times$ n), n is the number of images, d is the dimension of the feature vectors, t is the number of iterations for EM algorithm on each image to converge, and B is the branching factor. The first term is the time complexity of the original EM clustering algorithm, and the second term is the time complexity of the BIRCH algorithm since it is a highly balanced tree.

12.4 A Performance Study

In this section, we empirically investigate the performance of the proposed incremental EM clustering algorithm for object detection and environmental model building by conducting a series of experiments in an outdoor environment and presenting the results as an evaluation of the proposed methodology. In the first experiment, we evaluate the performance of the proposed incremental EM clustering algorithm in comparison with those of K-means and K-medoids algorithms. In the second experiment, we evaluate the proposed integration based EM algorithm on two sets of thresholds obtained according to Eqs. (12.10) and (12.11), respectively, without any assumptions on the data distribution. In the third experiment, the results obtained with several other threshold values are also compared to check the technical robustness of this study. The experimental results show that the proposed EM algorithm is capable of perceptual grouping tasks for large-scale image data in an outdoor environment.

We implemented all the algorithms in Python. All the experiments were performed on a computer with Intel Core 2 Duo Processor E6550 2.33 GHz CPU and 2 GB RAM. The operating system running on this computer is Windows 10.

12.4.1 Performance in an Outdoor Environment

In this subsection, we investigate the relative performance of the incremental EM clustering algorithm on a real large image dataset to test the technical robustness of the proposed clustering methodology. Outdoor environments are less structured. In this experiment, the image data are extracted from 35 pictures of size 1280 $\times$ 720 taken around an outdoor building. For the set of 35 images, $N = 27$ and $M = 18$, a dataset consisting of 95,550 unlabeled feature vectors is produced and briefly summarized in Table 12.1.

The clustering performances are presented in Fig. 12.2 for five testing images out of 35 images, in order to keep the amount of data presented reasonable, but consistent

Table 12.1 The image data

Image size	# of images	N	M	Dimension
1280 $\times$ 720	35	27	18	4 $\times$ 4 $\times$ 4

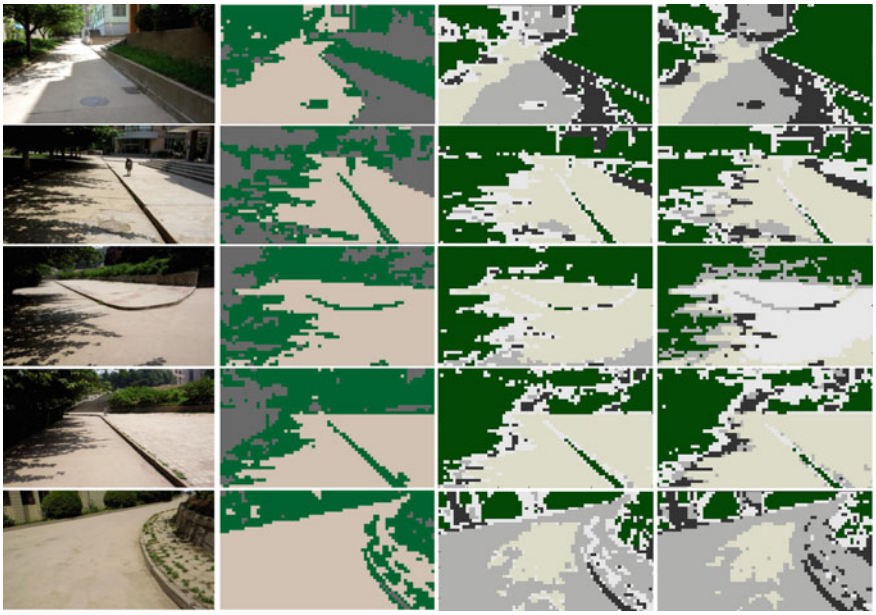


Fig. 12.2 Original image (left), segmentation results of (middle left) the proposed algorithm, (middle right) *K*-means algorithm, (right) *K*-medoids algorithm

performance results are obtained for other images. There are four columns in the figure. The left column displays the five test images used. From the original images, it is easy to identify that the major percepts include dark green trees, tree shadows, wide grey concrete road and manhole covers on the concrete road. The middle left column displays the clustering results of the proposed method. The middle right column displays the clustering results of *K*-means algorithm. The right column displays the clustering results of the *K*-medoids algorithm. For the proposed method, according to the optimization obtained by way of Silhouette coefficients and the cluster-merging threshold values, a total of 4 clusters are generated. The number of clusters is set to 5 for both the *K*-means algorithm and *K*-medoids algorithm as an input. To ease visual examination, in the segmented images, a color close to the real one is chosen to denote each percept.

From the segmented figures, it can be seen that different methods have different optimal segmenting results. For this outdoor environment, it is a little harder to segregate the more detailed percepts bundled into green (i.e., trees and tree shadows). From the segmented images as shown in the first row of Fig. 12.2, it can be seen that our method performs better than *K*-means and *K*-medoids algorithms which oversegment the concrete road in the middle of foreground into more percepts, cannot correctly separate the foreground concrete road from the wall shadows and cannot correctly separate the wall from the grass on the left (i.e., undersegment) as well as the proposed algorithm. From the segmented images, as shown in the next three

Table 12.2 The sets of parameters for segmentation using two threshold definitions

Method name	Branching factors B	Threshold values T	# of clusters
Equation (12.10)	6	0.3994	3
Equation (12.11)	10	0.3500	4

rows of Fig. 12.2, the three methods perform similarly one way or the other. In the last row of Fig. 12.2, it can be seen that our method performs better than K-means and K-medoids algorithms which oversegment the concrete road in the middle of the foreground into more percepts. To summarize, overall, our algorithm can discover all the major percepts most of the time and discriminate percepts that the K-means and K-medoids methods cannot.

12.4.2 Performance in Comparison with a Previous Threshold Model

For the performance evaluation of the threshold calculated by the proposed method in comparison with T_{median} determined by Eq. (12.10), the extracted parameters are summarized in Table 12.2 and the segmentation results for five testing images are presented in Fig. 12.3 in the following. From the segmented images, as shown in the first row of Fig. 12.3, it can be seen that, by both threshold values, the concrete road in the middle of foreground can be well segmented out from the wall shadows, however, other percepts are undersegment by the competing method of threshold computation from Eq. (12.10). From the segmented images as shown in the next three rows of Fig. 12.3, the images are undersegmented by the competing threshold with respect to the proposed threshold. In the last row of Fig. 12.3, it can be seen that the result obtained using the proposed threshold is better than the competing threshold which misclassifies the grass. To summarize, overall, by using the proposed threshold for cluster merging, our algorithm can discover all the major percepts most of the time and discriminate percepts that the competing threshold cannot.

12.4.3 Comparison with a Locality Sensitive-Hashing

We also compare the relative performance of BIRCH with respect to the locality sensitive-hashing (LSH). The segmentation results for two testing images are presented in Fig. 12.4, respectively. It can be seen from the figures in the first row of Fig. 12.4 that both nearest neighbor search methods work similarly well. However, from the figures in the second row of Fig. 12.4, it can be seen that our method can detect the edge line between the shadow of the right concrete wall and the foreground concrete road. This is to say, our method is slightly better.

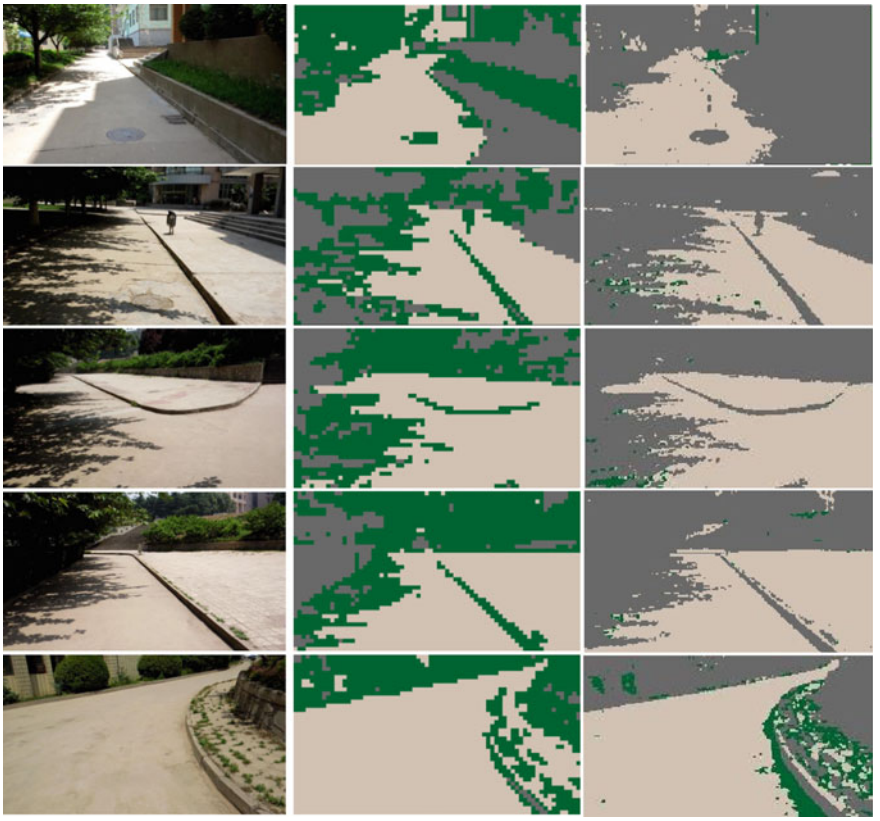


Fig. 12.3 Original image (left), segmentation results of the proposed algorithm with the threshold value calculated by (middle) Eq. (12.11), and (right) Eq. (12.10)

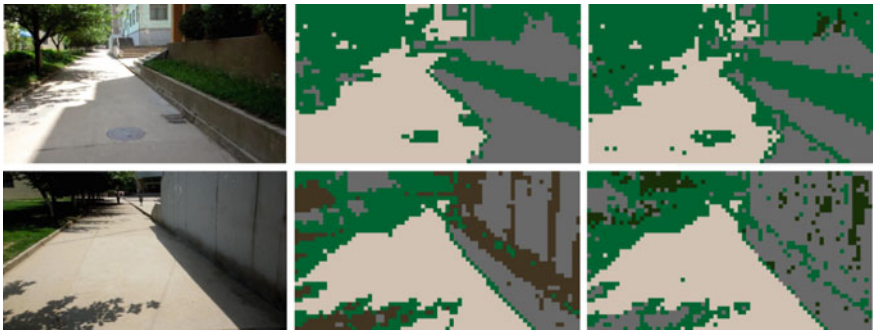


Fig. 12.4 Original image (left), segmentation results with (middle) the proposed BIRCH, (right) LSH

12.4.4 Other Threshold Values

The segmentation results for the five testing images obtained when other threshold values are considered are given in Fig. 12.5. In addition to the test images in the first column, there are three other columns in the figure, each corresponding to one of three threshold values in Table 12.3. From the figures, it can be observed that, when the threshold values become smaller, more clusters are obtained, and, as the number of clusters increases, more percepts appear (i.e., oversegment), which makes the stable ones fall apart. Too many clusters will eventually make some percepts hard to recognize. The combination with the results presented from Subsection 12.4.2, where a larger threshold value is used, indicates that our proposed threshold selection criterion works well for the BIRCH tree.

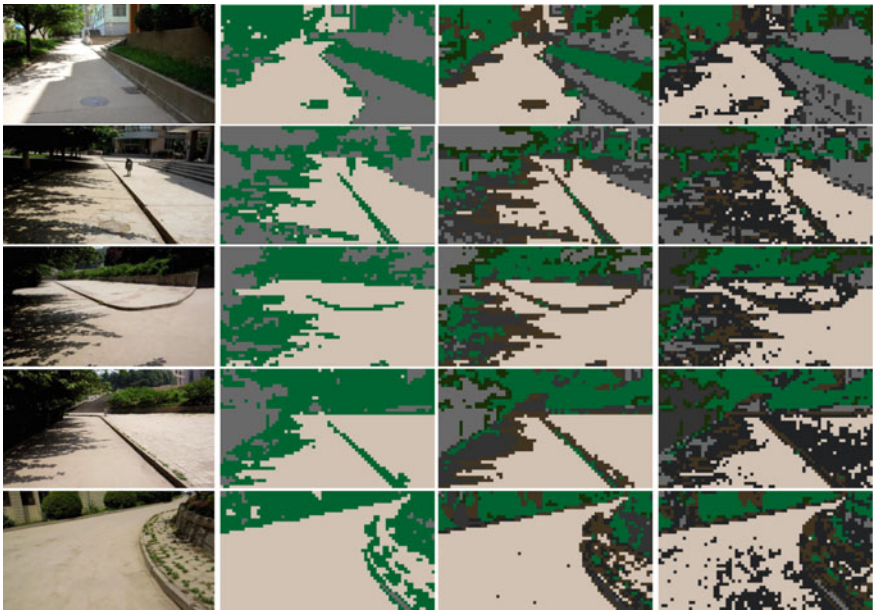


Fig. 12.5 Original image (left), results with threshold values at (middle left) 0.35, (middle right) 0.30, (right) 0.25

Table 12.3 Other threshold values and the corresponding number of clusters obtained

Threshold values T	# of clusters	Branching factors B
0.35	4	10
0.30	8	10
0.25	9	10

Table 12.4 The running time performance in seconds

Method name	Equation (12.10)	Equation (12.11)	EM algorithm
RT in seconds	92	98	195

12.4.5 Running Time Issue

We examine the relative time issue of the proposed approach in this section. For the parameters presented in Table 12.2, the running time performances of our proposed algorithm with two different sets of parameters are summarized in Table 12.4 and are also compared with that of the original EM algorithm when applied to cluster the 95,550 feature vectors as a whole into five clusters. From the results, it can be seen that the proposed method with two different sets of parameters consumes a similar amount of time and is about two times faster than the original EM algorithm.

12.4.6 Discussion

For mobile robots working in unknown outdoor environments, exact localization and navigation are usually based on the recognition and understanding of surrounding environments. Scene recognition in irregular and unstable outdoor environments is a difficult problem in the field. Being a crucial and challenging problem in computer vision, image segmentation refers to partitioning an image into several disjoint subsets such that each subset corresponds to a meaningful part of the image. A large number of available algorithms for image segmentation are prone to either time inefficiency or low-accuracy in finding objects among multiple images. In this paper, we employ EM clustering to address this problem. Being one of the most popular modern clustering algorithms, EM clustering is simple to implement and very often outperforms traditional clustering algorithms such as K-means algorithm. However, it is not very well scalable to large datasets. To partially circumvent this problem, in this chapter, we propose an integration-based fast incremental EM clustering algorithm which is particularly designed for object recognition oriented image segmentation tasks. The algorithm applies EM clustering to each image and then integrates the clustering results using a BIRCH tree. To realize self-localization in an unknown outdoor environment such as the one represented by image data in the experiments and to explore the suitability of the associations between a target location and the location's visual stimuli, the image segmentation results using the proposed fast incremental EM clustering algorithm are utilized to form the knowledge base for the unknown environment and is the very first step for recognizing objects in the newly grabbed images. From the experimental results, it can be seen that a speedup for EM clustering can be achieved by the proposed algorithm in comparison to the original EM method and that better clustering accuracy can be obtained by our algorithm with respect to several state-of-the-art clustering methods.

12.5 Conclusions

In this chapter, we have proposed an integration based incremental EM clustering algorithm. Our algorithm applies EM clustering to segment each image separately and uses a BIRCH tree-based fast approximate nearest neighbor classifier for cluster merging. The proposed algorithm can reduce the expense of the computation in EM clustering while retaining good control on the clustering accuracy. Evaluation in an image segmentation task shows that better clustering accuracy can be obtained by our algorithm with respect to *K*-means method.

References

de Amorim, R. C., & Hennig, C. (2015). Recovering the number of clusters in data sets with noise features using feature rescaling factors. *Information Sciences*, *324*, 126–145.

Dempster, A. P., Laird, N. M., & Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society*, *39*, 1–38.

McLachlan, G. J., & Peel, D. (2000). *Finite mixture models*. New York: Wiley.

Mitchell, T. (1997). *Machine learning*. McGraw-Hill.

Neal, R. M., & Hinton, G. E. (1998). A view of the EM algorithm that justifies incremental, sparse, and other variants. In M. I. Jordan (Ed.), *Learning in graphical models* (pp. 335–368). Dordrecht: Kluwer.

Ng, S. K., & McLachlan, G. J. (2004a). Speeding up the EM algorithm for mixture model-based segmentation of magnetic resonance images. *Pattern Recognition, 37,* 1573–1589.

Ng, S. K., & McLachlan, G. J. (2004b). Using the EM algorithm to train neural networks: Misconceptions and a new algorithm for multiclass classification. *IEEE Transactions on Neural Networks, 15,* 738–749.

Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Computational and Applied Mathematics*, *20*, 53–65.

Russell, S., & Norvig, P. (2003). *Artificial intelligence: A modern approach* (2nd ed.). Prentice Hall.

Wang, X., Chang, C., & Wang, X. L. (2017, December). A fast incremental spectral clustering algorithm for image segmentation. In *Proceedings of The 2017 International Conference on Computational Science and Computational Intelligence*, Las Vegas (pp. 15–27).

Wilson, D. R., & Martinez, T. R. (2000). Reduction techniques for instance-based learning algorithms. *Machine Learning, 38*(3), 257–286.

Part IV
Reinforcement Learning

Chapter 13
Reinforcement Learning for Mobile Robot Perceptual Learning



Abstract With the development of a computer vision system, the autonomous robots possess the ability to detect target objects from image sequences to build a model for an unknown environment. The next step in mobile robot autonomous navigation is for it to acquire control by some kind of learning. Reinforcement learning is of great interest because of the large number of practical applications that it can be used to address, ranging from problems in artificial intelligence to control engineering. Unlike supervised learning, in reinforcement learning, only partial feedback is given to the learner about its predictions which may have long term effects through influencing the future states of the controlled system and the goal of reinforcement learning is concerned with learning to control a system so as to maximize a numerical performance measure that expresses a long-term objective. In this chapter, we will provide an overview of the field of reinforcement learning in general and concepts that are relevant to the proposed work in specific. We begin with a review of the fundamental concepts of reinforcement learning considered from the artificial intelligence or computer science perspective on solving sequential decision-making problems. This introduction is followed by a brief introduction to one of the fundamental reinforcement learning method, the temporal difference learning algorithm, and its implementation based on a psychological model of its, working memory, to represent the learned knowledge.

Keywords Reinforcement learning · Temporal difference learning · Working memory · Conjunctive coding

13.1 Introduction

Being an important machine learning paradigm often used for sequential decision-making, reinforcement learning is a well-accepted and fundamental physiological model of learning in the Hippocampus area of human brain and easily implemented with computational and mathematical approaches in the neuroscience community (Rescorla and Wagner 1972; Dayan and Niv 2008; Maia 2009; Niv 2009). Unlike supervised learning, reinforcement learning enables the learning of optimal behavior in challenging or uncertain environments. Optimal behavior in this case can be

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_13

defined as the set of sequential decisions that result in the achievement of a goal or the best possible outcome. This learning process can be regarded as a process of trial-and-error, which is coupled with feedback provided from the environment that indicates the utility of the outcome and ultimately attempts to learn a mapping between actions and outcomes.

Reinforcement learning can be seen as somewhere between supervised and unsupervised learning. In a reinforcement learning problem, a so-called agent (that is capable of learning and making decisions) operates in an environment and gathers data from constant interaction with its surroundings (Kaelbling et al. 1996). The learning system receives a feedback, the so called a reinforcement or a reward, for its actions, but it is not provided with examples to learn from. In combination with the interaction patterns, the reinforcement provides a basis for the learning process. Therefore, it does not rely on a teacher but learns, as the name says, on the basis of received reinforcement (Russell and Norvig 2003). As the agent starts to learn from scratch, an improvement must be achieved by trial-and-error. In other words, the agent performs actions randomly at the beginning due to unavailable knowledge and adapts its behavior based on experienced rewards.

To make this learning paradigm more concrete, consider a mobile robot in an environment navigating to a goal location to perform tasks. It takes turns assessing the state, or the configuration, of the environment, and then selects the action which it believes will most likely result in an outcome that is in its best interest. Navigation therefore consists of making a sequence of decisions in order to achieve the goal, which is most often to arrive at the target location. During the process, however, the true utility and benefit of each action are not known, and only at the end of the navigation is there certain evidence of the utility of all of the actions taken during the process.

It is in this sense that reinforcement learning allows an agent to learn successful strategies in domains that are too complex to be described in a closed model or in cases where the system dynamics are only partially known. It has been shown to be effectively applicable to a large number of tasks and applications. In this chapter, we explore reinforcement learning for mobile robot location learning. Specifically, we are interested in the temporal difference learning algorithm and a successful implementation of it with neural networks on which the proposed work is based.

This chapter is structured as follows. Section 13.2 presents the fundamental concepts of the field of reinforcement learning. Section 13.3 then introduces the basic idea of the temporal difference learning. Performance measures are given in Sect. 13.4. We detail relevant topics from the field of a well-accepted and fundamental physiological model of learning in the neuroscience community in Sect. 13.5. We also discuss a successful implementation of the temporal difference algorithm to make the sequential algorithm more concrete and describe its approach for creating and analyzing metamodels of response surfaces in Sect. 13.6. Finally, we conclude this work in Sect. 13.7.

13.2 Reinforcement Learning

Whereas supervised learning problems have a static mapping between the input space X and their labels Y, the classical reinforcement learning problem can be defined by an agent that repeatedly interacts with an environment and that attempts to learn the utility of states by the receipt of (possibly infrequent) rewards in order to select a sequence of actions that maximizes the total rewards received. The reinforcement learning model consists of the state space S (i.e., a set of environment states), the action space A (i.e., a set of actions the agent can perform), and a set of reinforcement signals, usually R or a subset of it. In short, the domain of the problem is defined by S and A.

As illustrated in Fig. 13.1, in the course of interactions with the environment and with the system being in a state $s \in S$, the agent starts the learning process by observing the state of the environment. Based on s and a reinforcement signal r, the agent chooses an action $a \in A$ which it believes is likely to be beneficial and thus changes the system state s. Next the agent receives a *feedback*, or a *reward*, from the environment that provides an indication of the utility of the action (i.e., whether the action was good or bad). Then the cycle starts again. More specifically, in the most basic sense, the agent selects actions based on its perception of the value of the subsequent state. The feedback provided to the agent is used to improve its estimation of the value of being in each state. In the general reinforcement learning paradigm, rewards may be provided after each and every action. After repeated interaction with the environment, the agent's estimation of the true state values slowly improves, which enables the selection of more optimal actions in future interactions.

Largely developed as a machine learning technique by Sutton and Barto (1998), reinforcement learning is more formally described as a method to determine optimal action selection policies in sequential decision-making processes. The general framework for reinforcement learning is based on sequential interactions between an agent and its environment, where the environment is characterized by a set of states S, and the agent can pursue actions from the set of admissible actions A. In the reinforcement learning problem, actions are executed as a sequence of decisions at discrete points in time t. In a state $s_t \in S$ at a time point t, the agent chooses an action from the set of admissible actions $a_t \in$ A and receives a reward r_t. This reward may be either positive for pursuing actions that lead to beneficial outcomes, or may be negative for pursuing actions that lead to worse outcomes. The time index denotes the particular time step at which states are visited, actions are pursued, or rewards are received, and thus the sequences of states and actions can then be thought of as a progression through time. A complete process of agent-environment interaction

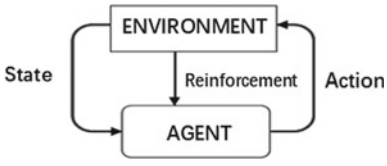


Fig. 13.1 The reinforcement learning paradigm consists of an agent interacting with an environment

from $t = 0, 1,\ldots, T$ is referred to as an *episode* consisting of the set of states visited, actions pursued, and rewards received. The total reward received for a single episode is the sum of the rewards received during the entire decision-making process,

$$R = \sum_{t=0}^{T} r_t \tag{13.1}$$

The solution to the reinforcement learning problem is the determination of which action to take for any system state. This is an optimization task. The goal is to choose the action that maximizes the received reward over the whole sequence of actions and not just the reinforcement given in a particular state. Unlike supervised learning, there are no "correct" input/output pairs and the agent has to look into the future, as the reward feedback that influences its decision may occur several time steps ahead. This is known as the concept of delayed rewards. Sutton and Barto (1998) name delayed rewards along with trial-and-error search as one of the two main characteristics of reinforcement learning. A concept to adequately express stochastic model behavior and delayed rewards is the Markov decision process (MDP) (Bellman 1957), in which the state transition probabilities depend only the most recent state information (s_t, a_t) and all information from $t = 0, 1,\ldots, t - 1$ has no effect on the state transition at time t. The probability, T, that applying a certain action $a \in A$ in a state $s \in S$ will bring the system to state $s' \in S$ is given by,

$$T\left(s, a, s'\right) = P\left(s_{t+1} = s' | s_t = s, a_t = a\right) \tag{13.2}$$

The Markov decision process (MDP) is a specific process which entails sequential decisions and consists of a set of states S, a set of actions A, and a reward function R (s, a). Each state $s \in S$ has an associated true state value $V^*(s)$. The decision process consists of repeated agent-environment interactions, during which the agent learns to associate states visited during the process with outcomes of entire process, thus attempting to learn the true value of each state. However, the true state values are unobservable to the agent, and thus the agents' estimates of the state values $V(s)$ are merely approximations of the true state values.

To solve the reinforcement learning problem, the agent has to develop a policy π which maps an action a to a system state s. A policy π is defined to be the particular sequence of actions $a = \{a_0, a_1,\ldots, a_T\}$ that is selected throughout the decision-making process. An optimal policy π^* is considered to be the set of actions pursued during the process that maximizes the total cumulative reward R. The desired optimal policy π^* is the policy that maximizes the rewards achieved over time. For finding a solution to an MDP, that is, the optimal policy π^*, the key issue is the expectation of future rewards to come. This is coupled with the question of how many of the future rewards are to be taken into account and how this is handled. In the most popular model, all following rewards are summed up, but are weighted by a discount factor γ ($0 < \gamma < 1$), so that future rewards have a smaller impact on the overall result. To mathematically formulize the solution to reinforcement learning, the function V

assigns to every system state $s \in S$ the overall expected reward when starting in s, i.e., the value of s. V is called the value function and can be defined as,

$$V(s) = E\left(\sum_{t=0}^{\infty} \gamma^t r_t\right) \tag{13.3}$$

with $E()$ denoting the expected value. The discount keeps the infinite sum bounded. In this model we have a guaranteed existence of an optimal deterministic stationary policy (Bellman 1957). The optimal value of a state is the expected reward when always following the optimal policy. The optimal value function V^* is given as,

$$V^*(s) = \max_{\pi} E\left(\sum_{t=0}^{\infty} \gamma^t r_t\right) \tag{13.4}$$

The value of the current state s is the immediate reward plus the value of the subsequent state s'. Therefore, the optimal value can be obtained by the following recursive equation considering the transition probabilities, $T(s, a, s')$. The optimal policy can be determined if the true state values $V^*(x)$ are known for every state. This equation is known as the Bellman optimality equation,

$$V^*(s) = \max_{a \in A}\left(R(s,a) + \gamma \sum_{s' \in S} T\left(s,a,s'\right) V^*\left(s'\right)\right) \tag{13.5}$$

which states that the value of being in state s and pursuing action a is a function of both the expected return of the current state and the optimal policy for all subsequent states (Bertsekas 1987). Having the optimal value function V^*, the optimal policy is to always execute the action returning the highest value,

$$\pi^*(s) = \arg\max_{a \in A}\left(R(s,a) + \gamma \sum_{s' \in S} T\left(s,a,s'\right) V^*\left(s'\right)\right) \tag{13.6}$$

Being the solution to reinforcement learning problems, a policy π^* is an action selection procedure that leads to an optimal outcome. This type of action selection is called a greedy policy. The Bellman equation provides a conceptual solution to the sequential decision-making problem in that the value of being in a state is a function of all future state values. Fortunately, when implementing reinforcement learning, however, the Bellman equation does not have to be explicitly solved. Rather, the identification of optimal policies in reinforcement learning problems is based on the notion that future information is relevant to the valuation of current states, and that accurate estimations of state values can be used to determine the optimal policy. If the transition probabilities T between states are explicitly known, these probabilities can be used to formulate a model of the system. However, if these transition probabilities are unknown, transitions between states must proceed by what could be referred to as

empirical sampling in a simulation-like manner and the agent-environment system is considered to be *model-free*. In this case, the structure of the environment can be used with the allowable actions to essentially generate the transitions between states and constrain the behavior of the agent, where the structure of the environment refers to anything that governs or has an effect on the dynamics of the environment.

There are many cases where the Markovian assumption may not be valid. For one example, in partially-observable Markov decision processes (POMDPs), the agent cannot perfectly observe the state information (Singh et al. 1994). For another example, there is a long temporal dependence between states and the feedback provided. Approaches to solving these types of problems often include retaining some form of the state history (Wierstra et al. 2007), such as by using recurrent neural networks or a more complex variant that relies on long short-term memory (LSTM) (Hochreiter and Schmidhuber 1997).

13.3 Temporal Difference Learning

Many formulations of reinforcement learning have been developed in order to either accommodate various types of environments or to utilize slightly different learning mechanisms. This section describes one of the fundamental reinforcement learning algorithms used in this work, the temporal difference learning algorithm (Sutton and Barto 1998). Being one of the fundamental reinforcement learning methods, the temporal difference learning algorithm uses information from future states and effectively propagates this information back through time in order to improve the agent's valuation of each state that is visited during an episode of agent-environment interactions. Improvements to the estimation of the state values therefore have a direct impact on the action selection processes as well. This algorithm is particularly useful in scenarios where it is not clear in advance whether an action taken is a good or a bad move and the overall consequences of this action cannot be determined immediately as rewards received in the future may have a great impact. The purpose of this task is to learn an input-output mapping between the state space S and the action set A based on the feedback from the domain R so as to evaluate the possible actions and select a single action to pursue. As a result, the temporal difference algorithm can be used to determine the utility of actions taken at an early time in the process based on the outcome of the episode, which results in a refined action selection procedure.

The goal of learning algorithms in reinforcement learning is essentially to allow the agent to learn the dynamics of the environment so that an optimal set of actions may be selected to achieve a goal or obtain the greatest total reward. The temporal difference learning algorithm is considered a value function learning algorithm. That is, in its most basic form, each state is believed to have a utility value, and the function that is learned attempts to approximate the true, underlying value of each state. States with large values are more likely to result in greater rewards or a successful outcome, whereas states with small values (potentially negative) are more likely to result in smaller (potentially negative) rewards or unsuccessful outcomes. In this work, we

focus on value function learning methods. In these methods, the agent attempts to learn a value function that approximates the value (i.e., utility) of states so as to learn about the dynamics of the domain and how the selection of actions relates to feedback delivered by the domain. Initially, the agents' state value estimations are erroneous because it has no knowledge of the environment. When trying to adapt the values of states, the idea is to update the value of state s based on current estimates of the following state values after one or more actions have been executed. The accuracy of the state value estimates improves by the agent repeatedly interacting with the environment, during which the behavior, or the action policy, of the agent emerges from its learned knowledge of the state values.

Therefore, in TD-learning, $V(s)$ is updated after the execution of an action a based on the immediate reward $r = R(s, a)$ and the current (discounted) value estimate of the subsequent state $V(s')$

$$V(s) = \alpha\delta = \alpha\big(r + \gamma V\big(s'\big) - V(s)\big) \tag{13.7}$$

The learning rate or step size α controls the speed of the update of $V(s)$. The term $\delta = r + \gamma V(s') - V(s)$ is called the temporal difference error. The prediction error δ plays an essential role in the temporal difference learning rule. It has been suggested that the activity of dopaminergic neurons in the ventral tegmental area (VTA) of the midbrain can represent this quantity. Substantial evidence has been available that dopamine is involved in reward learning. The similarity between the responses of the dopaminergic neurons and the prediction error has been observed. Therefore, the activity of the dopaminergic neurons provides a prediction error for the reward. In other words, the prediction error is an ongoing difference between the amount of reward that is delivered and the amount that is expected. The TD-learning method has been proved to converge to the optimal value function (Sutton 1998; Dayan 1992).

TD-learning methods are model-free methods because for learning a policy, no model of the underlying dynamic system is required and knowledge of the implication of actions or the transition distribution is not necessary a priori. Updates are based on sampled experience and trial-and-error. This is an important property of TD learning and one of the reasons for its success.

Therefore, for accurate assessment of progress toward reaching a goal, a wide range of different types of knowledge in terms of sensor states and internal states are required. Then the question becomes where these types of knowledge are retained and for what purpose are they stored. In cognitive psychology, according to their functionality, different kinds of memory, such as working memory (short-term memory), episodic memory, and long-term memory, have been proposed for temporal or permanent information storage on the process of personal goals. In the following, before an introduction to these types of memory is given, some performance measures are provided for a learning process.

13.4 Performance Measures

To evaluate the learning process in a practical context, some criteria are defined in the following that are useful for judging the success of reinforcement learning methods introduced in this work.

13.4.1 Success Rate

In goal-oriented tasks, it can clearly be determined whether the goal has been reached or not. Regardless of the optimality of the solution, the success rate defines the percentage of test runs that can solve the given task. Usually, we want to see a success rate near 100%. Less successful policies have to be dropped.

13.4.2 Speed to Success

Speed to reach (near-) 100% success refers the number of learning episodes until the agent is able to fulfill the task in all or almost all cases regardless of optimality of solution is of course a critical performance measure. When speed of convergence to optimality is ill-defined, it is more suitable to define a region of near-optimality and regard the time in which the system provides such a near-optimal solution.

13.4.3 Optimality of Solution

If the achieved solution is not (yet) optimal, the deviation from the optimal solution after a given learning time is an important measure for judging the quality of the learned policy.

13.4.4 Robustness of the Learning Process

Learning processes practically never show monotonous improvement, so the performance can temporarily get weaker again while learning. The robust learning measure refers to the cases where the success rate and optimality of solution do not show unstable development. Robust learning facilitates successful usage of learned policies after a short time, while non-robust learning can produce policies that perform unexpectedly badly over the learning process.

All of the given performance measures have their application in certain scenarios, and which one to focus on will generally depend on the task at hand.

13.5 Working Memory

13.5.1 Psychological Model

Developed from an earlier concept of considering short-term memory to be a unitary temporary storage system typified by the model of Atkinson and Shiffrin (1968), Baddeley and Hitch proposed the famous three-component WM model. This model accounts for a limited capacity system for temporarily storing and manipulating information used to control an ongoing behavior. In the original Baddeley and Hitch model, three components are the 'central executive' aided by the 'phonological loop' and the 'visuospatial sketchpad'. The central executive, as an attentional control system, processes the information supplied by the other two slave systems to attain a goal. The phonological loop is assumed to hold verbal and acoustic information while the sketchpad is to hold visuospatial information (Baddeley and Hitch 1974). Recently, an 'episodic buffer' has been proposed and added as a mechanism for combining information from various subsystems into a form of temporary representation. As a limited-capacity temporary storage system controlled by the central executive, the working memory model also offers a possible solution to the binding problem and the role of consciousness, and serves as an interface between the working memory and Tulving's episodic memory (Baddeley 2000). Where is the working memory located in the brain? Let's continue with some biological background next.

13.5.2 Biological Background

Anatomically, the prefrontal cortex is an important region for short-term working memory. Biologically, starting from the retinal, there are two visual pathways: the 'what' pathway and the 'where' pathway. The "what" pathway (also called the ventral system) takes care of the identification of an object while the "where" pathway (also called the dorsal system) focuses on the relative spatial position of an object. The 'what' system projects to the inferior convexity (IC) ventrolateral to the principal sulus, and the 'where' system projects to the dorsolateral prefrontal region (DL). Experimental results suggest that IC mediates working memory for objects and DL mediates spatial working memory. Although the visual system is composed of a vast number of neurons, its ability to process fully and store distinct and independent objects is strictly limited. It has been suggested that objects must compete for attention and processing 'space' in the visual system, and that this competition is influenced by both automatic and cognitive factors (Desimore 1998).

Now we move from the posterior part, which is mainly sensory, to the anterior part, which is mainly motory. In contrast to the posterior cortex being 'sensor' cortex, the entirety of the frontal cortex, in general, is 'motor' cortex devoted to organismic action, such as skeletal movement of various body parts, ocular movement, the expression of emotion, speech, or visceral control and so on, together with considerable functional cooperation between them. The cortex of the occipital, parietal, and temporal lobes largely supports perception and perceptual memory, while the cortex of the frontal lobe supports action and motor memory. Substantial evidence is available that the memory storage and perception processing are functions of large networks of interconnected neurons of the posterior cortex. Particularly, though by mechanisms still poorly understood, the hippocampus seems to play an essential role in the processing of memory formation. Being made of neurons representing the various inputs that have become associated with that memory, a network represents a memory or an item of knowledge. Since perception and memory largely share the same network and perceptions are processed through pre-established memory networks, they form the neural foundations for the intimate relationship between perception and memory. We see the world the way we know it, the way we have learned to see it (Helmholtz 1925; Hayek 1952; Bruner 1973). At the root of it, the sensory information is processed through hierarchically ordered areas of the posterior cortex and the neuronal networks (memories) they harbor.

Similar to the posterior cortical areas, the frontal areas are hierarchically organized as well for representation and processing of action. At the lowest stage of this frontal hierarchy is the primary motor cortex. Above it are the premotor areas, and above these, in the frontal motor hierarchy, are the dorsolateral areas of the prefrontal cortex. Like perceptual memory, motor memory is also hierarchical but is separated into different action domains, each providing the basic neural substrates for the representation and processing of a different category of movement.

In all instances, prefrontal cortex serves the purpose of ordering sequential actions toward a goal, whatever that goal may be, and plays a fundamental role in the temporal organization of behavior. In close cooperation with other brain structures, it provides neural support to three cognitive functions that are indispensable for conducting temporal sequential actions: (1) short-term memory (working memory), (2) motor attention or set, and (3) inhibitory control of interference (Fuster 1997) (Fig. 13.2).

Working memory is memory in the active state that the organism needs and uses for the performance of acts in a short term. The behavior of working memory goes beyond simple stimulus-response reflexes and is typified by delayed-response tasks, which was introduced ninety years ago by Hunter (1913). In the various forms of delayed-response tasks, the instruction cue disappears before the trigger appears in a given trial. The sensory stimulus and motor response are separated by a brief delay period, during which the information contained in the instruction needs to be kept in this short-term memory until it can be used for the reaction. Thus, the prefrontal cortex will sustain activation of neuronal populations during the active retention of sensory information, as during the delay of a delayed response task. In the process of reaching a goal, the old content of this short-term memory can be given renewed timelines and terms by the action at hand, which is realized by reinforcement in the

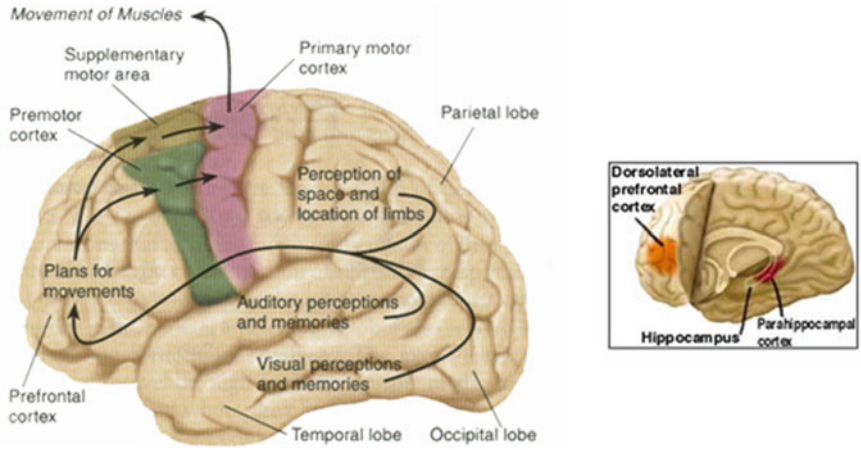


Fig. 13.2 Prefrontal cortex and hippocampus

form of either reward or punishment. When the goal is attained, the content of this short-term memory has outlived its usefulness and must be discarded since its term has expired.

The whole cerebral cortex projects to the basal ganglia, which have classically been considered as primarily motor structures. The outputs of the basal ganglia then project back to the frontal cortical areas, to the primary motor cortex, to some non-cortical motor systems, and to dopaminergic systems in the midbrain. This last pathway is important since the nucleus there contains dopaminergic neurons that project to both the basal ganglia itself and the frontal areas, and play an important role in the learning of behavior (Baldassarre 2002). A biological correlate for reward learning may be found in the activity of dopamine neurons. They respond rather uniformly to unexpected rewards which are particularly effective for learning. The selected action is maintained by the reinforcer. Being much more focused on goal-directed learning from interaction than are other approaches to machine learning, reinforcement learning is the basis of neuro-computational models of working memory.

The second function of the prefrontal cortex is motor attention, which is conventionally understood as the preparation for action or attention directed to prospective action. While the content of working memory is mainly sensory, the content of motor attention is mainly motory. Being complementary to each other, they help mediate cross-temporal contingencies. The third function of the prefrontal cortex, the inhibitory control of interference, is integrative and protects the structure of behavior, speech, or thought from interfering influences, external or internal, that may conflict with it and lead it astray.

13.5.3 Episodic Memory

Though working memory is important, it is beyond its ability to fuse co-occurring stimuli into configurations of these stimuli as temporary representations, upon which individuals can remember their personal past, and to consciously recollect experienced events and other happenings in subjectively apprehended space and time. As proposed some thirty years ago by Tulving, episodic memory refers to a neurocognitive system that enables humans to experience past events in their minds (Tulving 2002). It is about the distributed representation of contextual information such as stimulus features or internal states in particular places at a particular time, i.e., about "what" in "where" at "when" (Clayton and Dickinson 1998). The duration of episodic memory, typically a few minutes or a few hours, makes it separate from the conceptions of working memory (Baddeley 2000). But it is closely related to the 'episodic buffer' mentioned above, where inputs from different modalities are bound with information retrieved from different areas of long-term memory into a neural representation that remains in the working memory for a period of time (no more than a few minutes). The outputs of the episodic buffer are thought to retain event-specific knowledge. Therefore, functionally, episodic memories can be regarded as short-duration neural representations which selectively retain representations of states of the episodic buffer, and thus summarize the previous states of the working memory so as to keep track of the marked changes in the contents of conscious progress towards the active goals. The purpose of retaining such records of recent states of consciousness in episodic memory over a period of minutes and hours is to supply highly specific information on recent plan execution and to provide a constant evaluation of progress on current goals. This is needed because, given the heavy storage cost of the detailed content of working memory generated in a short period, long-term retention is infeasible. Episodic memory retains only sensory-perceptual summaries coming out of working memory and abstract knowledge that relates to very specific goals. In other words, episodic memory contains both abstract-conceptual and sensory-perceptual knowledge. It is possible that the former functions to organize the latter into coherent chunks of experience. It is believed to provide a link from working memory to long-term memory in such a way that momentary experience may be retained and later incorporated into consciously formed memory. Though Tulving and colleagues proposed that the right PFC was specialized for retrieval and the left for encoding, the hippocampus and associated structures are long known to be involved in the recall of recent memories, i.e., episodic memory. In humans, the structures of the hippocampal region are located in the medial temporal lobes.

Now that the mechanism by which working memory in the frontal cortex uses to control behavior is made clear, the important question is how to model the mechanism and implement it for the benefit of robots.

13.6 Working Memory Toolkit (WMtk)

Dr. David Noelle and his Ph.D. student, Joshua Phillips, at Vanderbilt University, created a set of software tools, called the NSF ITR Robot-PFC Working Memory Toolkit (WMtk), for developing working memory systems that can be easily and tightly integrated into robotic control system to perform goal-directed delayed-response tasks (Phillips and Noelle 2005). This working memory system learns to identify informational chunks worthy of retention using a model of the interactions between the brain's dopamine (DA) system and the PFC. The neuro-computational model for working memory is based on reinforcement learning, in which learning about stimuli or actions is solely based on the basis of rewards and punishments associated with them (Sutton and Barto 1990). Unlike supervised learning, reinforcement learning is minimally supervised because what actions to take in particular situations is not told explicitly but must be worked out on the basis of the reinforcement given. Though the ability of animals to learn appropriate actions in response to particular stimuli on the basis of associated rewards or punishments is a focus of behavioral psychology, there exists evidence that the midbrain dopaminergic system is associated with reward learning. There are two broad classes of reinforcement learning. In the first case, the reinforcement is delivered immediately after the action is taken. This makes learning relatively easy. In the second case, the reward or punishment is dependent on an entire sequence of actions and, therefore, is partially or wholly delayed until the sequence is completed. This makes learning more difficult since learning the appropriate action at each step in the sequence has to be based on future expectation of reward. However, this latter case is what we are most interested in.

13.6.1 WMtk Interface

Being general and flexible enough to be used on a variety of robotic platforms, the WMtk provides an extensive API that facilitates the fabrication of systems that utilize biologically-inspired working memory components. As a software library written in ANSI C++, the WMtk consists of a set of classes and methods that are designed to aid in the construction of a working memory system that uses TD learning to select working memory contents. More specifically, as a wrapper for the input information, the Chunk class is basically a simple data structure that takes a pointer to some memory location and a string as arguments, and stores some arbitrary piece of information that is believed by the user to be useful for processing by the working memory. The pointer points to any data structure the user may define. The string allows the user to store a basic amount of type information along with the pointer so that the pointer can be cast to its appropriate type later when needed. Being biologically inspired, the working memory system utilizes a neural network to make decisions about memory management. As a result, like most neural networks, when presented to the working memory, the chunks are organized in the form of a feature

vector. This vector is wrapped in a FeatureVector class, which also provides some extra functionality for filling the vector with values. It is an important class and provides the interface between the user and the working memory system. On the one side, the user provides Chunk and State information to the neural network. On the other side, the working memory system makes intelligent decisions about memory management, i.e., what to keep in working memory. Finally, as the most complex class in the toolkit, the WorkingMemory class is the workhorse of the WMtk. It is a limited store and supplies all of the functionality thought to pertain to working memory in the system. On each "time step" of the system simulation, it makes intelligent decisions such as when its contents should be updated and/or protected (erased and/or retained). In order for it to operate correctly, the required inputs to the WorkingMemory class include the size of the working memory, the size of the chunk vector (i.e., the size of the feature vector), the size of a state vector which defines the set of possible states the system will be in, the current state the system is actually in, the user-defined reward function, the user-defined state function, the user-defined chunk assignment function, and the user-defined chunk delete function (so as to release the computer memory when no longer needed).

In the following, the basic instructions for interfacing with the NSF ITR Robot-PFC Working Memory Toolkit (WMtk) is provided. The WMtk is a group of ANSI C++ classes that are designed to aid in the implementation of a biologically-inspired working memory system. In order to begin the process of developing such a system, it will be important to examin the basic purpose of a few key classes from the toolkit, what the basic purpose these classes serve in the toolkit's framework, and how these classes should interact to facilitate working memory.

13.6.2 Chunk

The Chunk class is basically a simple data structure that stores some arbitrary piece of information that you think might be useful for processing. This could be a coordinate, an angle, a plan, an action, or any other "chunk" of information. This class will act as a wrapper for this information and hold onto some basic data-type information in the form of a string. Here is the Chunk class's constructor and some of the most commonly used methods of the Chunk class:

```
Chunk(void* data, string type)
void* Chunk::getData()
void Chunk::setData(void* data)
string Chunk::getType()
void Chunk::setType(string type)
```

As can be seen, the constructor takes a pointer to some memory location and a string as arguments. The pointer may point to any data structure the user like and the string (a standard C++ string) allows the to store a basic amount of type information

along with the pointer so that the pointer can be cast to its appropriate type later when needed. Here is an example for storing an integer value in a Chunk class:

```
int* my_data = new int;
*my_data = 5;
Chunk my_chunk((void*) my_data, "Integer");
my_data = NULL;
```

Now my_chunk is storing the newly created integer. Later, the data and type information can be retrieved in order to make use of the data again:

```
double* my_double_data;
if (my_chunk.getType() == "Integer")
my_data = (int*) my_chunk.getData();
else if (my_chunk.getType() == "Double")
my_double_data = (double*) my_chunk.getData();
```

It is important to note that the type information is needed to make a decision about how to cast the void* returned by getData(). The user must cast the void* in order to make use of it. The Chunk class does not make a deep copy of any data structure, it only remembers the address of the data and its type information.

13.6.3 FeatureVector

The working memory system utilizes a neural network to make decisions about memory management. Like with most neural networks, information is provided in a vector form. A FeatureVector class is a wrapper for just such a vector and also provides some extra functionality for filling the vector with values. It is an important class to the system designer, because it will provide the interface through which he can provide Chunk and State information to the neural network so that it can make intelligent decisions about memory management. The FeatureVector class provides many useful methods for encoding information into the class. All values are stored as doubles and (in the default toolkit configuration) can only be in the range [0,1]. Typically, the user will not need to create a FeatureVector class, but will be storing and coding values into these classes. The coding strategies and how to use some of this class's methods are examined at a later time to build efficient codes for the neural network.

13.6.4 WorkingMemory

The WorkingMemory class is the workhorse of the WMtk. It provides all of the functionality thought to pertain to working memory in the system. In other words,

it is a limited capacity store with the intelligence to know when it's contents should be updated and/or protected (erased and/or retained). It will make these decisions on each "time step" of the system simulation and is the most complex class in the toolkit. However, its operation is actually very straightforward once it is clear what information it requires in order to operate correctly. It will probably be one of the first objects created in a system and will probably persist throughout the entire simulation. In most cases this will look something like the following:

```
#include<WMtk.h>
…
state_data_structure current_state;
double my_reward_function(WorkingMemory&);
void my_state_translation_function(FeatureVector&,
WorkingMemory&);
void my_chunk_translation_function(FeatureVector&,
Chunk&,
WorkingMemory&);
void my_chunk_deletion_function(Chunk&);
…
int main() {
…
int size_of_working_memory = 4; //Example value
int size_of_state_vector = 20; //Example value
int size_of_chunk_vectors = 15; //Example value
OR_CODE or_code = NOISY_OR; //Example value
…
WorkingMemory WM(size_of_working_memory,
size_of_state_vector,
size_of_chunk_vectors,
&current_state,
my_reward_function,
my_state_translation_function,
my_chunk_translation_function,
my_chunk_deletion_function,
or_code);
…
//MAIN LOOP
…
}
```

The first argument to the WorkingMemory class constructor is size_of_working_memory. This is simply the maximum number of chunks that the working memory system can hold at any given time. The value provided is just an example of a system that can hold no more than four chunks in working memory.

The argument, &current_state, is a pointer to whatever data structure holds all information about the current state of the system and the environment in one data structure. It can be anything since it is a void*. The user can retrieve this state data structure from the WorkingMemory class later on by calling it's getStateDataStructure() method. This will be of use later on.

The second argument, size_of_state_vector, and the sixth argument, my_state_translation_function, are closely related one another. The function is used by the working memory system to translate the current system state into a vector form that can be used by the neural network. Since the working memory system has no way of knowing how to do this, it will call the user function to accomplish this goal. Inside of this function should be all of the code needed to translate the current state into vector form. Notice the function takes two arguments, a FeatureVector object and a WorkingMemory object. Since the WorkingMemory class is calling the function (something you will never have to do) it supplies the arguments. It will provide a FeatureVector of the size specified by the second argument, size_of_state_vector, and also itself (*this) to the function. The user can then use the WorkingMemory::getStateDataStructure() method inside of the user function to get the state information and then translate it into the FeatureVector class.

The third and seventh arguments go hand-in-hand as well. The my_chunk_translation_function will work in a similar manner to the my_state_translation_function. However, my_chunk_translation_function, takes one more argument which is the Chunk to be translated into real values for the provided FeatureVector. The WorkingMemory class is passed into the function as an argument in case the state data structure, current contents of working memory, or anything else needs to be examined to encode the Chunk information into vector form.

The fifth argument, my_reward_function, is similar to the others in that it will be called by the WorkingMemory class. However, it will only be called when it needs reward information about the current state. The user must write this function where it takes information from the current state (accessed via WorkingMemory::getStateDataStructure()) and the contents of working memory to generate a scalar reward signal that it will then return to the WorkingMemory system.

The eighth argument, my_chunk_deletion_function, is another function that the user must implement to handle memory management issues. Once the WorkingMemory class has finished with a Chunk, it cannot properly dispose of the Chunk data on its own because it does not know how to cast the Chunk data pointer appropriately to do so. Also, the user might possibly (although not likely) want to retain some or all of the old Chunks that the WorkingMemory class is finished with. Each time the WorkingMemory class decides that it no longer needs to hold onto a Chunk, it will call this function before releasing it from the working memory store. It only takes one argument, a Chunk, and the user should write this function to either delete the data pointed to by the Chunk data pointer (void*) or manage the chunk data in some other way. Again, after this function has been called on a Chunk, the WorkingMemory

class will not contain this Chunk any longer, so either delete the data or store the pointer in some other manner via this function.

The ninth argument, or_code, specifies whether to add an addition coding scheme to the network. Possible values are NO_OR, MAX_OR, or NOISY_OR. NO_OR is the default, but the other values should not hurt performance. This option will be discussed more later when encoding schemes are examined more indepth.

The comment after creating the WorkingMemory object (//MAIN LOOP) specifies where the actual working memory processing will occur and is the topic of the next section.

13.6.5 Processing

Now that we know how to create the WorkingMemory object that will be used by the system we need a general approach to applying this object to correctly solve some problem. Here is the basic outline of the Main Loop should look like (in pseudocode):

```
do {
// Inialize the WM object for this episode
WM.newEpisode(true);
// Initialize the state data structure to match the
// start state.
initializeState();
do {
// Update the state information in the state data
// structure to match the current state of the
// world.
updateState();
// Analyze the current state of the world for
// possible chunks to provide to the working
// memory system.
list < Chunk > candidate_chunks = analyzeState();
// Provide the Working Memory system with the
// candidate chunks for this time step.
WM.tickEpisodeClock(candidate_chunks);
// Choose action(s) based on working memory
// contents and current state of the world.
decideAction();
if (no useful action could be discerned) {
// Choose to perform an action that explores
// or doesn't rely on working memory
// contents.
decideAutomaticAction();
}
```

```
// Perform action
performChosenAction();
} until the goal is obtained or the system fails to get
to the goal in time, etc.
} until the desired number of episodes have been completed
```

Notice that since the state translation, chunk translation, reward, and chunk deletion functions are automatically called by the WorkingMemory object, the processing loop is rather simple. Some of the actions in the inner loop can be rearranged without really changing the processing much.

13.6.6 Conjunctive Coding

When the working memory size is larger than 1, the toolkit creates a conjunctive code of all combinations of the chunk vectors in working memory to mimic the conjunctive representation. If the size_of_working_memory is denoted by WM_size, the size of the network weight file will be:

$$\text{State_Vector_Size} + ((\text{Chunk_Vector_Size} + 1)^{\wedge \text{WM_size}}) + \text{Bias_Unit} \quad (13.8)$$

where the first State_Vector_Size entries in the weights file will be associated with the state vector only, the next ((Chunk_Vector_Size + 1)ˆWM_size) entries will correspond to a combination of chunks in working memory, and, finally, there is one more entry for the Bias_Unit weight, that is, Bias_Unit = 1. The extra element that gets added to each Chunk_Vector is set to 1 if the WM slot is empty and 0 if a chunk is loaded in the slot. Each weight in the ((Chunk_Vector_Size + 1)ˆWM_size) portion of the file will contain the value of a particular combination of WM contents. With only a single WM slot, this equation simplifies to a single entry for each element associated with the chunk vector encoding. With a larger WM_size, the network weights do not read out so clearly. For example, if we have an experiment with three different kinds of chunks, the Chunk_Vector_Size will be 3. The chunk type #1 is then encoded (in binary) as 100, chunk type #2 as 010, and chunk type #3 as 001. Further, if the working memory of size 2, this makes for a conjunctive coding of size 16. Therefore, there will be totally 16 combinations of chunks as,

$$0123 \times 0123 = \begin{bmatrix} 00\ 01\ 02\ 03 \\ 10\ 11\ 12\ 13 \\ 20\ 21\ 22\ 23 \\ 30\ 31\ 32\ 33 \end{bmatrix} \quad (13.9)$$

If there is one chunk of type #2 (i.e., 1 in slot 2) and one chunk of type #3 (i.e., 1 in slot 3), this vector will be filled like so:

$$0100 \times 0010 = \begin{bmatrix} 0\,0\,0\,0 \\ 0\,0\,1\,0 \\ 0\,0\,0\,0 \\ 0\,0\,0\,0 \end{bmatrix} \tag{13.10}$$

If there is one chunk of type #1 (i.e., 1 in slot 1) and the second slot is empty (the empty slot is automatically given an encoding of 0001):

$$1000 \times 0001 = \begin{bmatrix} 0\,0\,0\,1 \\ 0\,0\,0\,0 \\ 0\,0\,0\,0 \\ 0\,0\,0\,0 \end{bmatrix} \tag{13.11}$$

A completely empty memory looks like:

$$0001 \times 0001 = \begin{bmatrix} 0\,0\,0\,0 \\ 0\,0\,0\,0 \\ 0\,0\,0\,0 \\ 0\,0\,0\,1 \end{bmatrix} \tag{13.12}$$

13.7 Summary

In this chapter, we give an introduction to the reinforcement learning method in general and the temporal-difference (TD) learning in particular. We next present a Psychological model with biological basis, and show how it is implemented in a software package toolkit. In the next three chapters, we will apply it to robot behavior learning problems.

References

Atkinson, R. C., & Shiffrin, R. M. (1968). Human memory: A proposed system and its control process. In K. W. Spence (Ed.) *The psychology of learning and motivation: Advances in research and theory* (pp. 89–195). Academic Press.

Baddeley, A. D., & Hitch, G. J. (1974). Working memory. In G. A. Bower (Ed.) *The psychology of learning and motivation* (pp. 47–89). Academic Press.

Baddeley, A. D. (2000). The episodic buffer: A new component of working memory. *Trends in Cognitive Science, 4*(11), 417–423.

Baldassarre, G. (2002). A modular neural-network model of the basal ganglia's role in learning and selecting motor behaviours. *Cognitive Systems Research, 3,* 5–13.

Bellman, R. E. (1957). A Markov decision process. *Journal of Mathematical Mechanics*, 6, 679–684.

Bertsekas, D. P. (1987). *Dynamic programming: Deterministic and stochastic models*. Englewood Cliffs, NJ: Prentice-Hall.

Bruner, J. S. (1973). *Beyond the information given: Studies in the psychology of knowing*. New York: W. W. Norton.

Clayton, N. S., & Dickinson, A. (1998). Episodic-like memory during cache recovery by scrub jays. *Nature, 395,* 272–274.

Dayan, P. (1992). The convergence of TD (λ) for general λ. *Machine Learning, 8,* 341–362.

Dayan, P., & Niv, Y. (2008). Reinforcement learning: The good, the bad and the ugly. *Current Opinion in Neuroscience, 18*(2), 185–196.

Desimore, R. (1998). Visual attention mediated by biased competition in extrastriate visual cortex. *Philosophical Transactions of the Royal Society B: Biological Sciences*, *353*(1373), 1245–1255.

Fuster, J. M. (1997). *The prefrontal cortex*. Philadelphia, New York: Lippincott-Raven Publisher.

Hayek, F. A. (1952). *The sensory order*. Chicago: University of Chicago Press.

Helmholtz, H. V. (1925). *Helmholtz's treatise on physiological optics* (translated from German by J. P. C. Southall). The Optical Society of America, G. Banta, Menasha, Wisconsin.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780.

Hunter, W. S. (1913). The delayed reactions in animals and children. *Behavior Monographs, 2*(1), 1–85.

Kaelbling, L. P., Littmann, M. L., & Moore, A. W. (1996). Reinforcement learning: A survey. *Journal of Artificial Intelligence Research, 4,* 237–285.

Maia, T.V. (2009). Reinforcement learning, conditioning, and the brain: Successes and challenges. *Cognitive, Affective, & Behavioral Neuroscience, 9*(4), 343–364.

Niv, Y. (2009). Reinforcement learning in the brain. *Journal of Mathematical Psychology, 53*(3):139–154.

Phillips, J. L., & Noelle, D. C. (2005). A biologically inspired working memory framework for robots. In *Proceedings of the 27th Annual Meeting of the Cognitive Science Society*, Stresa, Italy.

Rescorla, R. A., & Wagner, A. R. (1972). A theory of Pavlovian conditioning: The effectiveness of reinforcement and non-reinforcement. In A. H. Black & W. F. Prokasy (Eds.), *Classical conditioning II: Current research and theory* (pp. 64–69). New York: Appleton-Century-Crofts.

Russell, S. J., & Norvig, P. (2003). *Artificial intelligence: A modern approach* (2nd ed.). Prentice Hall Series in Artificial Intelligence. Prentice Hall.

Singh, S. P., Jaakkola, T., & Jordan, M. I. (1994). Learning without state-estimation in partially observable Markovian decision processes. In *Proceedings of the 11th International Conference on Machine Learning* (*ICML*'94), New Brunswick, NJ, 10–13 July (pp. 284–292).

Sutton, R. S., & Barto, A. G. (1990). Time-derivative models of Pavlovian conditioning. In M. Gabriel & J. W. Moore (Eds.), *Learning and computational neuroscience* (pp. 497–537). Cambridge, MA: MIT Press.

Sutton, R. S., & Barto, A. G. (1998). *Reinforcement learning*. Cambridge, MA: MIT Press.

Sutton, R. S. (1998). *Reinforcement learning: An introduction*. Cambridge, Massachusetts: MIT Press.

Tulving, E. (2002). Episodic memory: From mind to brain. *Annual Review of Psychology, 53,* 1–25.

Wierstra, D., Foerster, A., Peters, J., & Schmidhuber, J. (2007). Solving deep memory POMDPs with recurrent policy gradients. In *Proceedings of the 17th International Conference on Artificial Neural Networks* (ICANN), Paris, France, 9–13 September (Vol. 4668 of Lecture Notes in Computer Science, pp. 697–706).

Chapter 14
A Developmental Robotic Paradigm for Mobile Robot Navigation in an Indoor Environment

Abstract In general, traditional machine learning algorithms typically employ task-specific methods and only the parameters pre-determined by the human programmer are updated. These methods often fail to respond to the dynamically changing states of the uncontrolled environments. Additionally, such methods may not represent a developmental entity, such as a human mind. In contrast, an open-ended developmental robot system can learn simple behaviors and buildup more complex behaviors by utilizing the previously learned behaviors. In this chapter, we propose a basic framework for visual learning tasks that integrates a perceptual system into a biologically inspired working memory system. A main objective of this research is to provide a general framework for developmental learning and to investigate how well a neuro-computational PFC working memory model performs on a robotic platform in a real-world environment with complex tasks. Experiments conducted show impressive results.

Keywords Autonomous mental development · Developmental robotics · Machine learning · Computer vision · Working memory · Reinforcement learning · TD-learning

14.1 Introduction

The ultimate, long-term goal in robotics is to build a robotic system, which can develop new skills incrementally, in an autonomous open-ended approach through interactions with the environment without having a pre-designed, task-specific representation (Weng and Hwang 2006). However, traditional machine learning algorithms typically employ task-specific methods and often fail to respond to the dynamically changing states of the uncontrolled environments. To fill the gap that machines do not benefit from traditional machine learning and AI techniques, a relatively new area of study, called autonomous mental development (AMD), has been proposed. It was pointed out by Weng and his colleagues that a mental development process must be able to generate representation and architecture autonomously online and directly from raw sensory signals, and also must be task nonspecific (Weng et al.

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_14

Table 14.1 Main differences between traditional and developmental programs

Properties of program	Traditional programs	Developmental programs
Sensor-specific and effector-specific	Yes	Yes
Program is task-nonspecific	No	Yes
Tasks are unknown at programming time	No	Yes
Generate representation automatically	No	Yes
Animal-like online learning	No	Yes
Open-ended learning of more new tasks	No	Yes

2001). Table 14.1 shows the basic differences between the traditional engineering (manual development) paradigm and the proposed mental development paradigm.

In contrast to traditional artificial neural networks, which can only accept a series of fixed size, offline sensory data vectors, a developmental robot, which runs in the real physical world, has the ability to add new data online that it has never seen before. Furthermore, it can develop simple, basic skills for many different kinds of tasks and these skills may be utilized to learn more complex skills. For example, a human child's brain is not manually trained with data within one training session. Instead, he/she is usually taught by his/her parents from time to time through approving or rejecting behaviors. Following the same idea, to operate in the real unconstrained world, where the environment is highly complex and dynamic and a human designer cannot predict in the initial stage of programming, a developmental robot should be trained by interacting with the environment and with the user through a student-teacher relationship. It is in this sense that there is a necessity for open-ended developmental systems.

According to Weng and Hwang (2006), the complexity of a task can be considered as a product of complexity measures in five individual categories: (1) external environment, (2) input, (3) internal environment, (4) outputs, and (5) goal.

As a bridge crossing between the external world and the internal representation, perception is an awareness of things through the physical senses. Humans perform learning at the perceptual level and perceptual learning has been considered to occur at the cognitive level where identification and categorization are performed with focuses only on the relevant data that is needed by the current learning task. A main sensory modality for human beings is vision and visual object recognition is one of the most challenging problems for neuroscientists. In order to make the categorization and recognition task possible and to perform in a dynamic, noisy and uncontrolled environment, the system uses very basic features of the objects, such as color or texture information, in the perceptual level and the performance of perceptual learning is strongly dependent on the attention process. As a cognitive

process which focuses on a particular stimulus, attention mechanisms are needed in perceptual learning to guide further processing of information relevant to the current task, while ignoring other information present in the visual environment that is irrelevant. Only the information needed to be focused on is retained in a short-term storage system called the working memory while other irrelevant information is filtered out, therefore, reducing the load on cognitive processing systems.

In cognitive science, long-term memory has a very large capacity and the information is considered to be stored indefinitely, while working memory is defined as a theoretical framework which refers to a temporal type of storage that retains elements that are active and being manipulated for a short period of time. In the literature, many working memory models have been proposed, which have commonalities and differences (Miyake and Shah 1999), with the most popular one in psychology being the Baddeley and Hitch's three-component model (Baddeley and Hitch 1974) and the most famous one in computational neuroscience being the temporal difference (TD) learning model (Sutton 1988) which is a very popular machine learning algorithm describing the midbrain dopamine neurons.

In both the working memory and developmental robotics communities, there seems to be a particular interest in learning strategies that support a type of deep integrated learning. Our desire in this research is to demonstrate a developmental system that learns everything from scratch incrementally by interacting with the user and the environment. In more details, the system learns a set of basic simple behaviors, upon which more complex behaviors can be developed. For example, the perceptual system learns about the percepts and how to discriminate them. However, the system does not know what these percepts mean in the sense of movement. Therefore, in the next step, the robot should learn the meaning of the percepts with regard to movement, that is, which of them could represent open space and which could represent obstacles. The experiment of moving to open space is a very simple behavior in which the percepts could be associated with the task of motion. Once this association is complete, the system is then encapsulated with this crucial capability and, as such, the robot can further learn and encapsulate other behaviors and skills (with some aid from a human teacher) that are necessary for a complex visual-guided navigation task. However, the automatic encapsulation problem is not addressed in this work. This problem by itself is an important problem and should be addressed in future work.

A main objective of this research is to integrate a perceptual system into a biologically inspired working memory system in such a way that would yield a basic framework for visual learning tasks. Not being intended to focus on a specific task, the aim of our approach in this work is to provide a general framework for developmental learning, as well as to investigate how well a neuro-computational PFC (prefrontal cortex) working memory model performs on a robotic platform with complex tasks in a real-world environment.

In the following, Sect. 14.2 talks about some of related work and efforts that contribute to the area of developmental robotics. Section 14.3 describes our proposed vision-based autonomous mobile robot system. In Sect. 14.4, experiments are designed and results are presented. Finally, conclusions are made in Sect. 14.5.

14.2 Related Work

It was first pointed out by Weng that many traditional machine learning algorithms, including artificial neural networks, are computational frameworks but not developmental frameworks (Weng and Hwang 2006). According to Weng, two types of paradigms exist. The first one labels the traditional machine learning methods as manual development, where the programmer understands the task, chooses a representation, and maps the task to the representation. Thus, the programmer conveys his understanding into the task representation. Next, the programmer writes code which controls the machine to execute the task using the task-specific representation. The second paradigm is called the autonomous developmental paradigm where the programmer writes a developmental program and the robot learns the task by interacting with the user and the environment. The result of such a mechanism should be that the robot generates its own internal representation.

Arguing that the system must deal with high-dimensional feature space for perceptual learning, Weng developed a structure called the "Incremental Hierarchical Discriminant Regression (IHDR) engine" (Weng and Hwang 2007). To demonstrate the proposed method, a vision-guided navigation experiment using the IHDR system was conducted where a mobile robot, called SAIL and developed at Michigan State University, was trained by pushing it around the corridors of the school's Engineering Building (Weng and Hwang 2006, 2007; Weng 2004). The robot had two pressure sensors on each side. The difference of the two pressure readings was translated into heading information and associated with the images. The grayscale intensity of the pixels was used as the components of the feature vectors and the resolution of the images was 30-by-40. As a result, the number of dimensions on which the system operated was 30-by-40, i.e., 1200. One feature vector was generated from an entire image and a linear discriminant analysis was applied to the feature vectors to select the most discriminative features.

Krichmar and Edelman argued that devices, based on the working principles of the nervous systems, may provide the groundwork for the development of intelligent machines which operate on neurobiological principles rather than computational ones (Krichmar and Edelman 2006). They have designed a series of theoretical models, called brain-based devices (BBDs), and applied them to physical robots in order to investigate the functionality of different regions of the brain. The robots equipped with these models were used for studying perceptual categorization, operant conditioning, episodic, and spatial memory formation and motor control. They emphasized that models of brain function should reflect the dynamics of different brain regions, their structure and the connectivity of these regions rather than implementing them as a single neuron layer. According to them, to construct such models, detailed neuroanatomy and neural dynamics should be taken into consideration. To investigate the functional anatomy of the hippocampus region and its surrounding regions, Krichmar et al. programmed a physical robot to solve a dry variant of the Morris water maze task (Krichmar et al. 2005). In the task, the robot was meant to use its spatial and episodic memory by associating perceptual cues in the environment with a particular

location. In the experiment, the target location was hidden and can only be detected in close range by the robot's front IR sensor. The robot started a trial in one of the four locations and stopped if it detected the hidden platform or until a time limit of 1000 seconds was reached. The experimental results showed that the robot learned to navigate to the target location from multiple starting positions in about 8 trials.

Inspired by Krichmar's work, a slightly modified version of the task was performed by Bursch, et al. in a simulation environment in (Bursch et al. 2007). In contrast to Krichmar's experiment, the robot started a trial in random locations without using odometry information. In order to create a mapping between a perceptual state and an action for that state, a Self-Organizing Feature Map (SOFM) (Kohonen 1990) was used and the heights (in pixels) of the colored panels were presented as an input to the SOFM. Two navigational approaches were proposed and compared. The first method was based on creating a probabilistic graph between the nodes of the SOFM and then searching for the optimum path in the graph in order to navigate to the target position. The second approach utilized the Working Memory Toolkit (WMtk). It was reported that the first approach outperformed the WMtk with relatively little experiences. However, as the number of training samples increased, the WMtk actually performed much better than the graph search technique.

14.3 A Vision-Based Autonomous Mobile Robot System

With focuses on the issues of developmental robotics, the methodology presented in this chapter is to establish a biologically inspired system that employs perceptual knowledge along with a working memory system in order to provide broad capabilities for visual learning tasks. Such a requirement necessitates a large set of goals that the system needs to satisfy.

In general, robot vision systems tend to be designed for specific types of problems and circumstances. The resulting systems typically operate in restricted environments. Our desire in this work is to have the robot operate in real-world environments that are not modified by using artificial landmarks. In order to perceive in such a dynamic, noisy, and uncontrolled environment and to learn and recognize natural landmarks, the system learns everything from scratch.

First, the images and videos were taken as RGB (red, green, blue) color images, which were next converted to the HSV (Hue Saturation Value) color space for use. To represent the content of an environment by image segmentation, each image was divided into the same number of equally spaced small patches of the same size. A color histogram vector of size 10,000 was extracted based on the number of color components that occurred in a small image patch. Further, due to their similarity with the receptive fields of primates' V1 simple cells, 40 Gabor filters were used to convolve with each image region to form texture based features. As a result, the combination of the color histogram and Gabor texture measures resulted in 10,040-dimensional feature vectors. Then an unsupervised learning algorithm, a minimum spanning tree based clustering algorithm, was used to form percepts (i.e., objects)

for the robot vision system which would be used later for supervisedly classifying the incoming new images. For new images captured by the robot, after feature vectors were extracted, a quantization tree-based approximate nearest neighbor search process was followed to assign an object label to each newly extracted feature vector. The segmentation should yield a set of connected regions in an image. A connected-component labeling algorithm was then applied to find these regions, called blobs. A blob not only represented one specific cluster in the feature space, but also represented a clustering of pixels in the image space. This simultaneous clustering in both the feature space and the image space was usually indicative of a meaningful percept.

After the segmentation was done, the final stage of the perceptual system was to produce inputs for the working memory system, referred to as "chunks", in order for the robot to perform learning and acquire motor skills to guide its actions. It is important to maintain a strong connection between the percepts in the environment and their semantic meanings. In other words, the robot should learn what each percept means for itself. For example, a percept could mean a target object or might be perceived as an obstacle. In this context, understanding the meanings of the percepts is referred as perceptual grounding.

The limited capacity property of a working memory system provides focus for the robot to search for appropriate actions in order to accomplish a given task. In fact, the key point faced by the working memory system that is utilized in this proposed robot architecture is the determination of which chunks of information should be actively retained in the working memory and which may be safely discarded for tasks' success. It has been reported that regions of the prefrontal cortex (PFC) in the human brain are involved significantly in working memory (Goldman-Rakic 1996). During working memory operations, dopaminergic midbrain neurons physically fire (Schultz 1998) and dopamine levels in the PFC distinctively rise (Watanabe et al. 1997). The alteration of such dopamine levels suggests an adaptive mechanism of learning from experience, that is, a prediction scheme in the expected future reward.

WMtk essentially implements a neural network version of the TD learning algorithm and contains classes and methods for constructing a working memory system that intelligently decides which chunks to maintain or discard depending on the reward criterion. This criterion is dependent on the current task of the robot and is specified by the user as a requirement for the WMtk. The reward is generated in a discrete-time manner (namely "episodes") and is a real number. Depending on the selected action in the previous episode, as well as the reward signal generated for that particular action, the WMtk updates its internal contents based on which the robot can then select and execute an appropriate action.

More specifically, to build a robotic working memory system, initially a Working Memory (WM) object is created. As inputs to it, parameters such as the working memory capacity, the working memory size, learning rate, exploration rate as well as some user functions to configure the internal setup of the working memory system should be supplied by the user. Once the initial setup is complete, the WM object performs as follows. On each time step of the task, a new list of candidate chunks is provided to the WM object by the robot control system. Together with the chunks that are currently held in the system, the key objective of the WM object is to determine

which objects to retain or discard by considering every possible subset of the total collected chunks and estimating an expected future reward for each. The particular combination of chunks, which provides the highest estimate of future reward in the subset, will be maintained in the working memory system. The remaining chunks are safely discarded. Finally, the temporal difference error is then obtained from the previous episode's estimate and the current episode's estimate. The adaptive critic's weights are then changed using the calculated temporal difference error (Phillips and Noelle 2005). Even though the basis of the WMtk relies on established reinforcement learning techniques, the most distinctive feature of the toolkit is that it explores every possible combination of the collection of chunks which can be fit within the limited capacity of the working memory and selects to retain in the working memory the combination of chunks which provides the highest estimate value of the future reward.

This feature of the WMtk can be very useful in robotic applications. The robot is not limited to only one single item to consider. For instance, in many applications, a landmark location is represented by only one percept. However, a landmark location may be represented by a combination of percepts which may help the effectiveness of the robot's localization process. In addition, this would also help to increase the number of unique landmark locations in the navigation path of a robot.

The final flowchart of the system is presented in Fig. 14.1. The left part of the diagram represents the perceptual system. The right portion represents the working

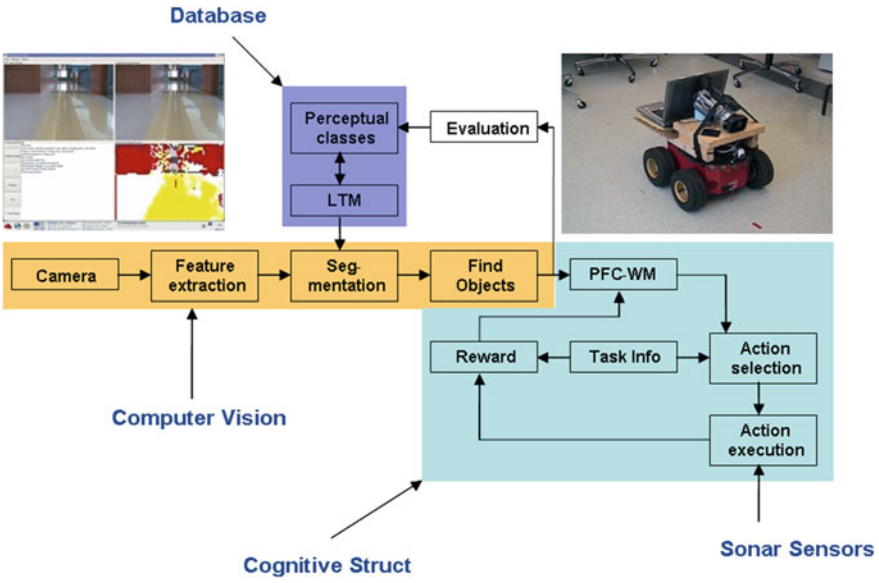


Fig. 14.1 The proposed system architecture

memory system, which is combined with the robot's sensory-motor scheme (i.e., actuators and sensors such as sonars to avoid obstacles). The combination of both the perceptual system and the working memory system yields the cognitive structure for the robot.

In order to demonstrate the methodology proposed in this research, the system is applied to a complex vision-guided navigation task. Part of the complexity of the task is that no a priori information is given to the system, such as a Cartesian map or coordinates of objects, etc. Instead, a qualitative navigation approach is taken, where the robot proceeds by detecting natural landmarks that are present in the environment. It is desired that the system is capable of learning new skills and percepts online in uncontrolled environments while performing a task and supporting explanation of its knowledge to and from the user in an interactive way. In order to achieve the navigation task, simple behaviors are learned and preserved in the system in order to learn more complex behaviors. This type of learning approach is also known as "shaping" and is extensively used in the animal experiments in psychology (Skinner 1953). The desired animal behavior is achieved by initially learning a very simple task, followed by similar, slightly more difficult tasks. Thus, the robot is trained to perform a complex behavior in stages with different reward strategies for each training phase.

14.4 Experiments and Results

The objective of this research is to provide a general framework for developmental learning and to investigate how well a neuro-computational PFC working memory model performs on a robotic platform in a real-world environment with complex tasks. To fulfill this goal, we propose to conduct several experiments that would exploit the usefulness and capabilities of this framework in mobile robotics. A fundamental property of a mobile robot is the ability to navigate safely in an uncertain environment. In order to achieve this task, the system needs to acquire motor skills to guide the behavior of the robot. Each experiment is related and built upon each other in order for the robot to navigate successfully.



Fig. 14.2 (Left) Vision-based human-robot interaction interface, (middle and right) mobile robot Skeeter with camera and laptop

As shown in Fig. 14.2, a testbed robot, Skeeter, was equipped with a single camera to perceive the environment. The front sonar array is only used for obstacle avoidance purposes. The experiments took place in the hall outside of the Intelligent Robotics Laboratory on the third floor of the Featheringill Hall. A typical view of this hall is shown in Fig. 14.3. The major objects in this hall included the yellow floor tiles in the center, the white floor tiles along the edges near the wall, and the black stripes of tiles located periodically across the center of the hall, wood panel wall sections, light blue painted wall sections and a light blue railing.

To generate a perceptual model for the environment, 20 images were collected over several days at different times to account for the different lighting conditions. Next 66,740 feature vectors were extracted from these images and clustered to obtain several percepts that constituted the training database of the system (the system's long term memory). A color was assigned to each percept for displaying them in the segmented images. Table 14.2 shows the obtained percepts and their corresponding denoting color. A sample segmented test image is shown in Fig. 14.3.

In our empirical study, three experiments were designed to accomplish the navigation task. The first two experiments have been reported (Tugcu et al. 2007) and



Fig. 14.3 A test image and its segmented version

Table 14.2 Major percepts and their semantic meanings

Percept name	Percept symbol	Denoting color
BlackStripe	Object1	Black
WoodPanel	Object2	Light red
LightWondow	Object3	Light grey
WoodPanel	Object4	Light red
WhiteFloor	Object5	Dark white
WoodPanel	Object6	Dark red
WoodPanel	Object7	Red
WoodPanel	Object8	Light red
WhiteFloor	Object9	White
YellowFloor	Object10	Yellow
YellowFloor	Object11	Blue

are briefly repeated here for reference. The third experiment is a further extension built upon the first two. The objective of these experiments is to reveal the capabilities of the system in uncontrolled environments. In the experiments, the system learns new skills so as to ultimately perform a qualitative vision-guided navigation task. Since the task is qualitative navigation, no metric information is involved in the computations.

14.4.1 Experiment I: Learning Open Space

In order for a robot to navigate safely, it has to move in an open space. The first step in the navigation task is to learn the percepts that would yield an open path for the robot through the robot's own interaction with the world. Once meaningful percepts are obtained from the images acquired from the camera, the robot needs to learn the association of the percepts with motion. In other words, the robot needs to determine whether the percept represents an obstacle or an open space, since the robot does not have any prior information about these percepts. To do so, in this task, the reward criterion for the WM system is selected to be the distance moved by the robot per trial. The procedure for the first experiment is as follows,

- The WM size is set to 1 so that only one of the input chunks is selected per episode.
- The robot captures an image from the video stream and segments it into percepts, which are then presented to the WM as chunks. The WM system randomly selects one of them.
- Next, the robot directs itself to the selected chunk and tries to move toward it for 5 meters or until sonar sensor indicates an obstacle happens, forcing it to stop.
- The distance moved by the robot is returned as a reward for the WM system. That is, if the robot moves 5 meters, it will receive a maximum reward of 5, otherwise, it will receive a reward which is the distance it has moved.

The above procedure is followed multiple times, allowing the WM system to intelligently update its internal parameters associated with each chunk in the environment until the percepts corresponding to an open space eventually dominates the trials. The WMtk has also a reserved position for the situation when none of the chunks is selected. With a WM size of 1, the weight parameters inside the WMtk are directly associated with the percepts and the received rewards. In order to evaluate the system's learning performance, two sets of experiments were conducted. In the first set, the robot was put in the middle of the hallway and let go freely without any user interference. In the second set, the robot started moving from wherever it was. Figure 14.4 shows the development of the weights associated with each percept versus the number trials, where all the initial weights for percepts were set to 0.5 (Wang and Wilkes 2009).

In terms of behavior, at the beginning, the robot moved about at random, improving its chances of encountering anything it needs. The ability to learn enables the

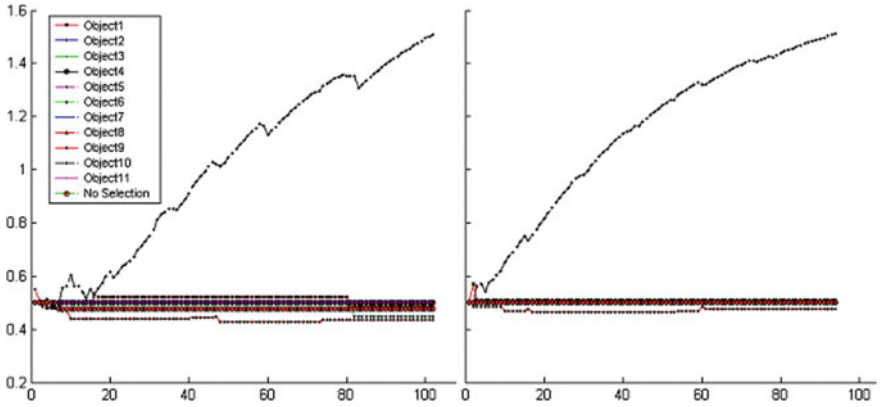


Fig. 14.4 The weight logs for each percept (left) when the robot was let to go freely (right) when the robot was forced back to the center after each trial

robot to remember what it has seen and where it has been. After a percept (stimulus) has been approached and examined several times during the learning, the robot will evolve to activate responses that are most likely to reduce the need, that is, to find open space, by more and more switching attention to the sensory input necessary for effective response, that is, always being able to move 5 meters without being stopped. Unattended signals or percepts that are not immediately necessary for the performance of a response may also be presented by the sensory systems and delivered, but will eventually be excluded from the parts of the motor system engaged in performing the current task.

14.4.2 Experiment II: Learning Landmarks

In a navigation task, the robot not only needs to know which percepts are associated with the open space to move safely, but also to learn about landmark locations in order to localize itself in the environment. In this experiment, the appearance of black floor tiles was found to be a convenient indication of a landmark location. To learn the landmark location, the following steps were taken in this experiment,

- The WM size is set to values of 1, 2, and 3 to determine which percept or combination of percepts should be selected for recognizing a landmark location.
- A set of total 80 training images representing landmark and non-landmark locations (40 for each kind of locations) as seen in Fig. 14.5 were collected. As mentioned, the presence of a black floor tile in the foreground is an indication of a landmark.
- The training images from the set were selected randomly and processed in order to obtain the chunks for the WM system.
- The percepts generated for each image were duplicated and presented to the WM system. In other words, one chunk was marked as a landmark chunk and the other



Fig. 14.5 (Left) A landmark image and (right) a non-landmark image

Table 14.3 Percentage of correctness for landmark classification

WM size	1000 trials (%)	5000 trials (%)
1	47.90	50.06
2	95.50	97.42
3	98.00	99.38

as a non-landmark chunk. Once the WM system selected a set of such chunks, a majority vote was taken to determine whether the location is a landmark one or a non-landmark one. If there was a tie in the vote, it was interpreted as a non-landmark one by default. Since the training images were labeled, the WM system received a reward of 1 if the vote agreed with the label of the image, otherwise, it receives a reward of 0.

For each WM size, 1000 and 5000 trials of experiments were conducted. The results are summarized in Table 14.3 and quite good for all the cases.

14.4.3 Experiment III: Learning the Navigation Task

From the first experiment, the robot acquired the ability to navigate safely through the environment. From the second experiment, the robot acquired the ability to localize itself in the environment. At this point, the robot knows which percepts yield open space, as well as which group of percepts is related to a landmark location. Now the question becomes how the robot can utilize its acquired knowledge to reach a goal position.

The objective of this experiment is for the robot to utilize its previously learned knowledge to learn a complex task. Specifically, it is assumed that the robot is positioned in front of the Intelligent Robotics Laboratory, looking towards the window side of the hallway. The black stripes along the hallway indicate the presence of a landmark. The goal of the robot is to navigate along the hallway until it reaches the third landmark. Furthermore, the delayed response task is widely used in cognitive

neuroscience in order to evaluate the properties of working memory (O'Reilly et al. 1999; Phillips and Noelle 2005).

Therefore, there are two situations to be considered. If the robot recognizes a green ball (which has been added to the system (Wang et al. 2008)) which is shown in Fig. 14.6 as an unexpected contingent percept along the way as it reaches the third landmark, it should remember the green ball and come back to the starting position. If no green ball is detected along the navigation path, the robot is required to learn to stop at the third landmark. During the process, the robot moves with a small increment distance, stops and makes a decision. The time required for this cycle is referred to as one time step. Then the cycle repeates. This process continues until the navigation task is complete. The procedure for Experiment III is as follows,

- Initially in the environment, the robot captures an image and segments it to obtain the percepts. Three actions, such as "move", "stop", and "turn around", are also and always presented to the working memory as chunks at each time step. The working memory system is rewarded or punished by the possible actions chosen by the robot along the navigation path. In addition to the action chunks, the working memory is presented with the percepts detected in the environment. However, only "YellowFloor", "BlackFloor", and "GreenBall" percepts are presented, if recognized by the perceptual system. Since the task requires only remembering the "GreenBall" chunk, the other percepts are treated as distractors. The working memory has a reserved a "Nothing" chunk when the system does not make any selection. Therefore, the maximum number of chunks that can be presented to the working memory is seven.
- The working memory size is set to 2. One slot is for an action chunk and the other one is for a percept chunk. Thus, the working memory is required to learn to select an action chunk as well as a percept chunk.
- Once the candidate input chunk vector is presented to working memory, a decision is made. The robot either moves, stops or turns around depending on the action selection made by the working memory at each time step. Furthermore, if a "GreenBall" percept is detected, the working memory is expected to put the "GreenBall" chunk in its second output slot.

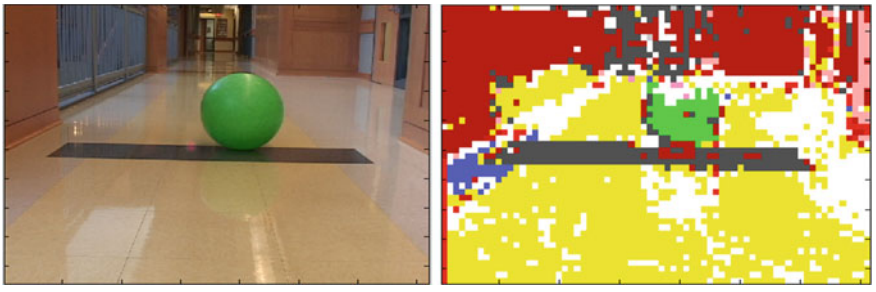


Fig. 14.6 (Left) A test image and (right) its segmented version

- The robot starts by moving towards the open space for one meter, stops and makes a decision. Then the cycle is repeated until the third landmark is reached. However, if a green ball is detected, it needs to remember that it has detected an indication for a contingent situation. Therefore, as the task requires, it needs to come back to its initial position by passing through each landmark again, in order to demonstrate the bootstrapping effect.

With the size of input chunk vectors being set to seven, the training is performed in a simulated environment and all the possible chunks are presented to the robot at each time step. The robot is trained for each region of the navigation path as shown in Fig. 14.7. Each region may be thought of as the state of the robot (i.e., state 0 is being located at region 1, and state 1 is at region 2, etc.). Furthermore, the detection of the contingent percept is also represented as a state of the robot. However, during the training, this state is either randomly set to true once in one of the regions or not set to true at all. This means that the contingent percept has been detected at a particular region if set to true or not detected if the state stays false in the entire trial. In the real world experiment (not in simulation), this state is set true once the perceptual system detects the contingent percept. The robot is required to choose the correct action chunk at each state. In addition, it must learn to remember the contingent percept if recognized along the navigation path. The learning criteria faced by the working memory are not trivial, indeed they are challenging. The robot needs to learn a considerable amount of information from a large combinatorial chunk space in four different states. The robot is trained for each region (each of the states from zero to three) of the navigation path and a reward value is given.

However, in order to speed up the learning, the training state is switched from one to another, only if multiple numbers of positive rewards are received in a row at each state. For instance, the number of positive rewards required to switch the state from one to another is assumed to be ten. A successful trial is defined as not receiving any negative reward. Therefore, the system must receive 40 positive rewards (10 for each region) in a row in order for the trial to be considered as successful. The number of received positive rewards in a row is referred to as the positive reward threshold

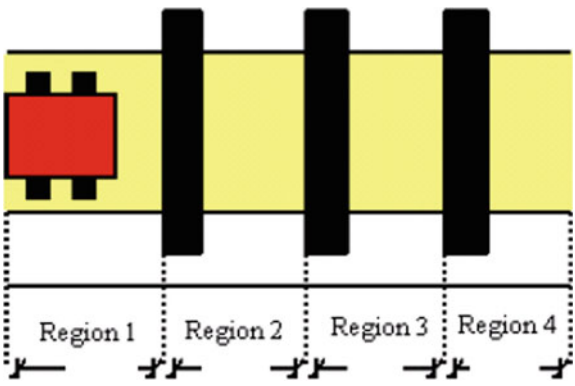


Fig. 14.7 An illustration of each region of the navigation path

(PRT). Figures 14.8, 14.9, 14.10 and 14.11 show the learning curve of the system, as a plot of the average reward values over a sliding window of size 25 over 10,000 trials for several PRTs.

Table 14.4 shows the number of successful trials for each plots in Figs. 14.8, 14.9, 14.10 and 14.11.

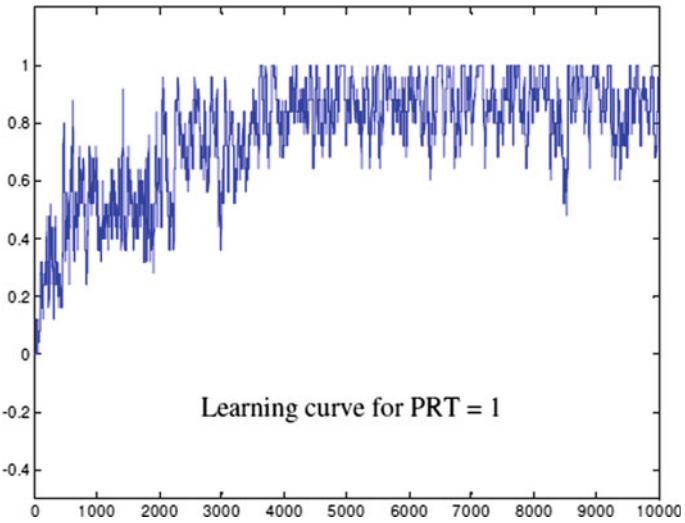


Fig. 14.8 Learning curve of the system for PRT of value 1

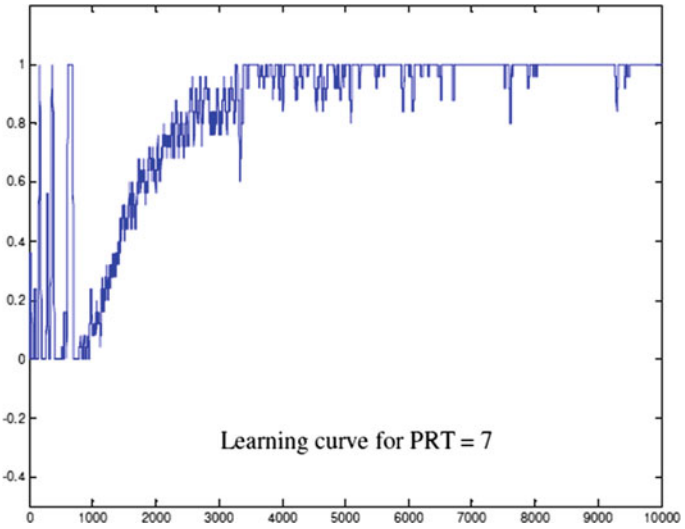


Fig. 14.9 Learning curve of the system for PRT of value 7

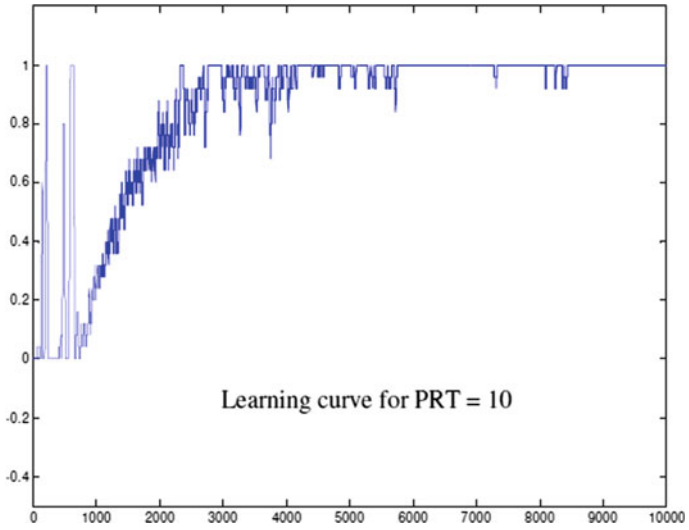


Fig. 14.10 Learning curve of the system for PRT of value 10

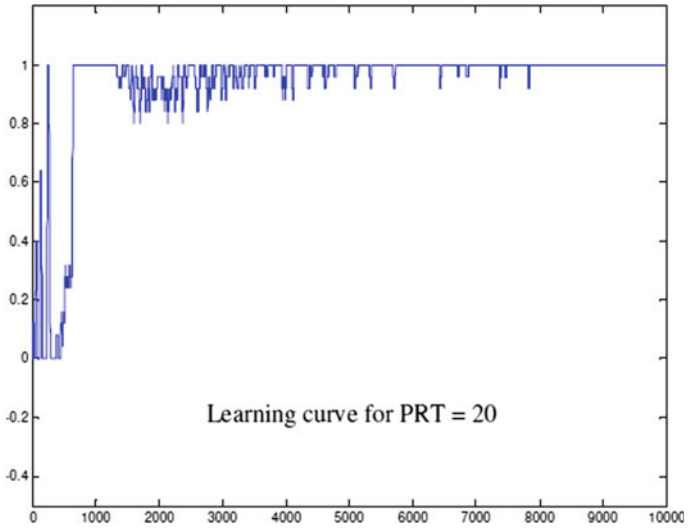


Fig. 14.11 Learning curve of the system for PRT of value 20

Table 14.4 Number of successful trials for 10,000 trials and for different reward thresholds

Positive reward threshold	Successful trials (out of 10,000)	Percentage of successful trials (%)
1	7591	75.91
7	8275	82.75
10	8502	85.02
20	9330	93.30

These numbers are reasonable under the very strict rule, that is, receiving multiple positive rewards in a row. Moreover, the initial training stage is also included in these results. If, at any time step of the training, the system fails to choose the correct chunk combination, the whole trial is considered to be an unsuccessful trial. In all of the cases, there is a positive learning trend. As the PRT is increased, the system makes fewer mistakes overall and the speed of the training is significantly increased in terms of episodes. With the system settings described in the above, the time required to train the system is 12 minutes when PRT is set to 10 and 15 minutes for a PRT being set to 20. Thus, the training time is relatively short in a simulated environment.

In this final experiment, we particularly demonstrated the delayed response effect by holding on to a percept for a period of time. The robot successfully navigated in the hallway and it came back to its initial position when a green ball percept was presented. The robot chose the correct actions in the action selection slot and chose the "Nothing" percept only when no percept was held in the working memory. However, once the contingent percept was presented along the navigation, the robot remembered the contingent percept in the percept selection slot of the working memory at each time step. Thus, the robot has successfully learned a complex task based on previously learned fundamental behaviors.

14.5 Conclusions

In this chapter, the robot successfully learned a complex, vision-guided navigation task by means of a powerful and reliable perceptual system combined with a biologically inspired prefrontal cortex working memory model. The task faced by the robot is indeed a challenging task to be learned. The robot initially learned the percepts in the environment and robustly performed segmentation. Next, the robot associated the meanings relative to motion and landmark detection with the percepts in the environment and encapsulated them as simple behaviors. These behaviors were then used to build up a more complex navigation behavior.

References

Baddeley, A. D., & Hitch, G. J. (1974). Working memory. In G. A. Bower (Ed.), Rece*nt advances in learning and motivation* (Vol. 8, pp. 47–90). New York: Academic Press.

Bursch, M. A., Skubic, M., Keller, J. M., & Stone, K. E. (2007). A robot in a water maze: Learning a spatial memory task. In *Proceedings of the IEEE International Conference on Robotics and Automation (IRCA)*, Rome, Italy, April 10–14.

Goldman-Rakic, P. S. (1996). Regional and cellular fractionation of working memory. *Proceedings of the National Academy of Science USA, 93,* 13473–13480.

Kohonen, T. (1990). The self-organizing map. *Proceedings of the IEEE, 78,* 1464–1480.

Krichmar, J. L., & Edelman, G. M. (2006). Principles underlying the construction of brain-based devices. In T. Kovacs & J. A. R. Marshall (Eds.), *Adaptation in artificial and biological systems* (pp. 37–42). Bristol, UK: Society for the Study of Artificial Intelligence and the Simulation of Behavior.

Krichmar, J. L., Nitz, D. A., Gally, J. A., & Edelman, G. M. (2005). Characterizing functional hippocampal pathways in a brain-based device as it solves a spatial memory task. *Proceedings of National Academy Science USA, 102,* 2111–2116.

Miyake, A., & Shah, P. (Eds.). (1999). *Models of working memory: Mechanisms of active maintenance and executive control*. Cambridge: Cambridge University Press.

O'Reilly, R. C., Braver, T. S., & Cohen, J. D. (1999). A biologically based computational model of working memory. In A. Miyake & P. Shah (Eds.), *Models of working memory: Mechanisms of active maintenance and executive control* (pp. 375–411).

Phillips, J. L., & Noelle, D. C. (2005). A biologically inspired working memory framework for robots. In *Proceedings of the 27th Annual Meeting of the Cognitive Science Society*, Stresa, Italy, July.

Schultz, W. (1998). Predictive reward signal of dopamine neurons. *Journal of Neurophysiology, 80,* 1–27.

Skinner, B. F. (1953). *Science and human behavior*. New York: Colliler-Macmillian.

Sutton, R. S. (1988). Learning to predict by the method of temporal differences. *Machine Learning, 3,* 9–44.

Tugcu, M., Wang, X., Hunter, J. E., Phillips, J., Noelle, D., & Wilkes, D. M. (2007). A computational neuroscience model of working memory with application to robot perceptual learning. In *IASTED Computational Intelligence*, Canada, July 2–4.

Wang, X., & Wilkes, D. M. (2009). An autonomous vision system based sensor-motor coordination using working memory toolkit. In *Proceedings of the 2009 International Conference on Artificial Intelligence*, Las Vegas, July.

Wang, X., Wang, X. L., & Wilkes, D. M. (2008). Visual novel object detection for mobile robots. In *Proceedings of the 2008 International Conference on Data Mining*, Las Vegas, July 14–17.

Watanabe, M., Kodama, T., & Hikosaka, K. (1997). Increase of extracellular dopamine in primate prefrontal cortex during a working memory task. *Journal of Neurophysiology, 78,* 2795–2798.

Weng, J. (2004). Developmental robots: Theory and experiments. *International Journal of Humanoid Robotics, 1*, 199–236.

Weng, J., & Hwang, W. S. (2006). From neural networks to the brain autonomous mental development. *IEEE Computational Intelligence Magazine, 1*(3), 15–31.

Weng, J., & Hwang, W. (2007). Incremental hierarchical discriminant regression. *IEEE Transactions on Neural Networks, 18*(2), 397–415.

Weng, J., McClelland, J., Pentland, A., Sporns, O., Stockman, I., Sur, M., & Thelen, E. (2001). *Computational autonomous mental development: A white paper for suggesting a new initiative* (Technical Report).

Chapter 15
An Automatic Scene Recognition Using TD-Learning for Mobile Robot Localization in an Outdoor Environment

Abstract Navigation is an important ability of mobile robots. Localization in an environment is the very first step to achieve it. In this chapter, based on the extensive research already conducted for known indoor environments, we are going to utilize a natural landmark-based localization strategy for mobile robot working in an outdoor unknown environment. Particularly, we are going to pursue a real-time scene recognition scheme so as to use objects segmented in it as the natural landmarks and to explore the suitability of configure representation for automatic scene recognition in robot navigation. Experiments designed to infer semantic prediction of a scene from different configurations of its stimuli using TD-learning are conducted and the results demonstrate the effectiveness of the proposed location learning method.

Keywords Localization · Visual landmarks · Scene recognition · Configural representation · TD-learning

15.1 Introduction

Human eyes not only have unique biological forms, but also possess special psychological functions. Environmental cognition is the first step of a mobile robot to understand the environment and realize the control of autonomous motion. Its accuracy directly affects the results of subsequent cognition. How to make robots have similar cognitive ability as intelligent life is one of the most challenging research subjects that researchers have been paying close attention to and actively discussing for a long time. The difficulties of self-localization research of mobile robots based on machine learning methodology mainly exist in the following aspects.

Since more and more random and diverse factors exist in outdoor unstructured environments, the working ability of mobile robots is greatly influenced by scene recognition. Due to the lack of prior information, visual navigation is still a difficult problem in the field. From the current level of development of machine vision, without any priori information, to accurately separate the effective object or area from natural images, real-time image analysis and understanding are still the basis of the research in the field and in close relation with the problem of pattern recognition. In an unknown environment, the improvement of the autonomous learning ability of a

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_15

mobile robot is one of the urgent problems to solve in this field so as to perform tasks independently. In theory, due to the lack of prior information, the mobile robot is unable to obtain model parameters of a learning system. However, reinforcement learning in general and TD-learning algorithm in particular can be explored through the interaction with the environment and used directly to learn the optimal strategies through value function estimation (Sutton and Barto 1998). But with the increasing complexity and uncertainty of the controlled objects, how to compactly calculate and store the solution space is still a hot and difficult problem in this emerging discipline.

Existing robot localization systems, such as Simultaneous Localization and Mapping (SLAM) systems, have shown their remarkable performance on mapping and localization in real time. However, some conventional vision-based SLAM algorithms are developed for a single camera and tend to accumulate error when robots run a long distance. Others are designed for high-level cameras and consume a period of time to draw a map. These shortcomings of SLAM systems may make a negative influence on the subsequent accuracy of mobile robot localization and navigation. Compared with map building and matching-based localization methods, how to realize autonomous localization for mobile robots in an unknown environment by self-learning is a challenging task, which has important theoretical and practical significances. To partially solve the problem, in this chapter, we propose a machine learning based methodology of automatic scene recognition for mobile robot localization which does not need to construct a map explicitly. A preliminary work of the approach has been presented for mobile robot localization in an indoor environment (Wang et al. 2009). The proposed approach began by taking pictures through on board sensors while moving in an indoor environment and constructing a model of the environment by object recognition based image segmentation. It subsequently identified one or several external target locations from which landmark elements were extracted and bounded together into a configure association that can be used to guide the robot's behaviors continuously so as to reach a goal location. The main contribution of current work is that it extends the proposed mobile robot localization approach to an outdoor environment based on automatic landmark-based scene identification with the aid of a reinforcement learning based software package, called the working memory toolkit (WMtk), which allows reward associations between a target location and the conjunctive representations of its stimuli. Experiments are conducted and the performance is compared with those of two other classification mechanisms, a support vector machine approach and a simple linear two-class classifier, perceptron.

15.1.1 Scene Representation

Navigation in an outdoor environment has been a subject of researches for years. Although, despite efforts put into researching the problem, lack of reliable and universal solutions is still evident. Localization and navigation requires knowledge about the environment and information regarding robot's current position. The prior information is especially important when operating in an unstructured outdoor environment,

where robot has to be prepared to deal with a variety of different situations. The vast amount of possible cases combined with the noisy nature of sensors result in difficulty of outdoor mobile robot navigation. Therefore, the robot's perception is a crucial part of navigation. Data extracted from available sources have to be chosen carefully and are usually combined together to fuse available information in order to achieve more reliable estimate. Efficient navigation of autonomous robots depends on the system providing precise information about current location. Vision-based localization has been an area of active research for the last 20 years. Successful localization has been realized by many research groups (Civera et al. 2010; Mouragnon et al. 2009; Se et al. 2001; Tardif et al. 2008; Wolf et al. 2005). To improve the robustness of vision-based localization, the properties of the features used for localization must be examined. Two feature extraction techniques that are prevalent in vision-based localization algorithms are the Scale Invariant Feature Transform (SIFT) (Lowe 2004) and Features from Accelerated Segment Test (FAST) corner detection (Rosten and Drummond 2006). SIFT has been proven to be a robust technique for extracting and matching features across multiple images (Mikolajczyk and Schmid 2005), whereas FAST corner detection is designed to work with real-time video frame rates. Filtering the features extracted by these two techniques can improve localization as well as partially improve illumination robustness. SIFT is scale invariant and partially invariant to orientation and slight changes in illumination (Lowe 2004). A significant problem experienced by those that use SIFT for feature extraction and localization is the large number of features extracted from each image. Improving the efficiency of feature matching between images is still an area of active research. In this chapter, instead of using single features, image-patch based segmentation is used to reduce the number of features involved in scene representation for mobile robot localization. More specifically, color histogram based feature vectors are first extracted from equally spaced small image patches and next clustered to form abstracted high-level percept concepts which are then used to represent a scene and take part in mobile robot localization.

15.1.2 Scene Recognition

To reach a target location, animals use external landmarks for goal identification (Wehner 1992), which makes the ability to recall the appropriate memories of contexts a significant part of navigating through familiar terrains. However, an isolated natural landmark on its own may easily be mistaken for another similar isolated natural landmark in a different location. A scene in which events occur can provide a place, space, or context for an experience. To reduce ambiguity, elements that comprise a scene, including the landmarks, should be bound together into a configural association for it to be more resistant to interference from conflicting information acquired in another context. There are two ways to represent a context which is made up of stimulus features, a feature representation where the scene is represented as a set of independent individual features associated with the scene, and a configural

representation (also referred to as unitary representation and configural association) where the scene is represented as a set of features combined into a unitary representation that encodes their conjunction. The formation of configural association between a particular set of landmark cues and a particular set of contextual cues is called compound learning, configural learning, or contextual learning. Unlike the feature representation which associates a discrete cue with a response, the configural representation associates a conjunctive representation formed from the combination of multiple cues with a response and different configurations of the stimuli have different responses. There is sufficient evidence that neocortical systems can represent the independent individual features of an environment (Nadel and Willner 1980) whereas the hippocampus (together with the cortex) uses compound stimuli constructed through conjunctive representation to control performance (Sutherland and Rudy 1989).

An important source of evidence supporting the view that the hippocampus contributes to memory by associating some features of a scene with their context to form conjunctive representation comes from the studies of the transverse patterning problem, which requires representing the individual stimulus in the context of other stimuli (Dusek and Eichenbaum 1998). In the transverse patterning problem, two of three stimuli, A, B, and C, are presented on each trial and reward is issued according to the following rule, A^+B^-, B^+C^-, and C^+A^-. The rule A^+B^- says that A will be rewarded when presented with B. The rule B^+C^- says that B will be rewarded when presented with C. The rule C^+A^- says that C will be rewarded when presented with A. When all the patterns are presented equally often, each stimulus will receive the same total rewards. To solve dilemma, each of the conjunctive representations, AB, BC, and CA, must be uniquely associated with the appropriate response. It is the unique conjunctive coding of stimuli that configural representations rely on. Further, the conjunctive representation can have advantages over the feature representation by solving nonlinear discrimination problems. For example, in a bi-conditional discrimination task of form AB+, CD+, AC−, BD−, a reward is issued for responding in the presence of AB and CD compounds and a punishment is issued for responding in the presence of AC and BD compounds. However, since each element (A, B, C, D) is equally associated with rewarded and punished trial outcomes, their linear combinations will end up with equal associative strengths across the compounds. To get over this problem, conjunctive representation in the form of the compounds should uniquely be associated with its respective trial outcome. In addition, when two events have overlapping features, such as ABCDE and ABCEF, if each combination is treated as a unique representation, there will be less interference than the case where the feature representation scheme is used. It is in this sense that conjunctive representation can produce pattern separation for similar experiences or input patterns (Rudy and O'Reilly 2001).

The rest of this paper is organized as follows. We review some existing work on robot localization in Sect. 15.2. In Sect. 15.3, we present details of the proposed solution to some existing problems in robot localization area. In Sect. 15.4, experiments are conducted to evaluate the performances of the proposed approach. Finally, conclusions are made in Sect. 15.5.

15.2 Related Work

To solve the localization problem for a mobile robot working in an unknown environment, there exist many difficulties, such as sensor noise interference and dynamic impact of the environment and so on. Therefore, the research on autonomous location learning of a mobile robot in an unknown environment is still in the development stage, and many theories need to be further improved. For example, to navigate through an unknown terrain, a mobile robot must realize precise localization in its working environment through internal and external sensors. There are mainly two kinds of modeling methods at present, environmental modeling and intelligent agent modeling. The former needs to establish the representation of the working environment within the robot (that is, the environment model or map). The latter is modeled by a partially observable Markov decision process by the finite perceptive information provided by the sensor.

The basic method of most mobile robot positioning systems in an unknown environment is based on map building and matching. The method of map building is developed on the basis of the SLAM problem (Smith et al. 1990; Thrun et al. 1998). It is the synchronous self positioning for the incremental modeling of the unknown environment and the robot. The map matching method matches the feature or roadmap that the sensor can perceive with the environment map. Information fusion based on Bayes rules and probability distribution updating with new environment data produced by robot motion are the mainstream research methods for the mobile robot SLAM problem in unknown environments. So far, extensive research has been conducted in the field and methods proposed include extended Kalman filter location (Guivant and Nebot 2001), Monte Carlo localization (Dellaert et al. 1999), multi-hypothesis tracking localization (Jensfelt and Christensen 2001) and Markov positioning (Fox 1998). However, the extended Kalman filter method will cause unrecoverable failure in robot localization due to the existence of nonlinear noise. Although Monte Carlo method can evaluate the nonlinear and non-Gauss process, it can not solve problems such as sample degradation. Multiple hypothesis tracking method has to solve the problem of data association in tracking. Markov methods need to store the whole distribution space of pose probability and, therefore, result in high memory consumption and high computational cost. In addition, these methods have difficulties achieving real-time performance. In recent years, SLAM algorithms are popular and generally implemented in complex localization and path planning systems (Williams et al. 2017). However, the methods mentioned above may not be particularly suitable for the mobile robot localization problem in outdoor unknown environments.

Machine learning has been the core of intelligent computing research. Robot learning system is one of the main research areas in the field of robotics in recent years. The integration of different machine learning methods (supervised learning, unsupervised learning and reinforcement learning) with visual processing methods is becoming a major trend in this field. Among the existing learning methods in combination with vision, supervised learning and unsupervised learning algorithms can

be used to identify stable regions of interest. Unlike supervised learning and unsupervised learning, reinforcement learning utilizes the external stimuli to strengthen the implementation of behavior learning and is a data-driven learning method via interaction with the environment (Murray et al. 2002). Tempo-difference learning (Xin 2006) is not only the key algorithm in reinforcement learning, but also a high order learning process with evidence found in the human brain functions (Seymour et al. 2004).

For the application of reinforcement learning to the problem of robot navigation, many scholars have done a thorough and detailed study. In 2005, Michels et al. proposed first to use supervised learning to accurately estimate the relative depth of obstacles in an outdoor scene according to the output of visual system and then to use reinforcement learning to choose the driving direction to complete the obstacle avoidance task (Michels et al. 2005). In 2007, Conn and Peters put forward a supervised reinforcement learning method to complete a navigation task which was capable of withstanding environmental changes when there were obstacles in the pathway (Conn and Peters II 2007). In the same year, Busch et al. adopted the TD-learning model to solve the Morris water maze task (Busch et al. 2007). They first discretized the perceptual space, and then made the mobile robot learn to associate the perception information with a specific location to navigate to the target location. In 2008, Duan et al. integrated the TS fuzzy neural network and reinforcement learning to solve the navigation problem in a complex unknown environment (Duan et al. 2008). In the same year, Kollar and Roy proposed an application of reinforcement learning to identify the track of data acquisition in order to achieve an automatic data collection process for robots so as to maximize the accuracy of map (Kollar and Roy 2008). In 2009, Juang et al. proposed a fuzzy reinforcement learning algorithm which can reduce the dimension of a fuzzy input set, and applied it to the wall tracking technique of a mobile robot (Juang and Hsu 2009). When mobile robots move fast, blur would be introduced into acquired images, resulting in the pose estimation distortion. To solve this problem, in 2010, Hornung et al. proposed a new fast reinforcement learning algorithm and applied it to visual navigation strategy (Hornung et al. 2010). In 2011, Deisenroth and Rasmussen proposed a data-effective model-based reinforcement learning algorithm, which can achieve learning successfully through limited trials (Deisenroth and Rasmussen 2011). In 2012, Quintia et al. proposed a fast learning algorithm based on dynamic programming and reinforcement learning for robots to learn continuously through interaction with environment, and used it in the task of wall tracking (Quintia et al. 2012). In the same year, Hester et al. proposed a new real-time reinforcement learning algorithm based on a parallel architecture model, and applied it to in real-time operations of mountain bikes and unmanned vehicles (Hester et al. 2012). In 2013, Kober et al. at Carnegie Mellon University performed a comprehensive summary of the contributions of reinforcement learning in robotics (Kober et al. 2013). Unfortunately, no systematic research with regard to the application of reinforcement learning method in vision-based mobile robot self localization in an unknown outdoor environment has been conducted in the literature.

15.3 Mobile Robot Localization with Configural Representation

A primary goal of the human visual system is to recognize surroundings and locate target objects. In order to obtain useful information in the obtained images, a computer vision system is established, which simulates the human visual system to help the mobile robot to recognize objects in an unknown environment quickly and facilitate the subsequent localization with no need to construct a map. For successful visual recognition, each object must have attributes that can be used to differentiate it from others and to segment the whole image into meaningful objects. In the current research, 18 images were taken in an outdoor environment, and the final database obtained consists of 454,104 labeled feature vectors by 5 perceptual classes (Wang et al. 2017). Each class is assigned a different color. The major percepts and their semantic meanings are shown in Table 15.1.

When a mobile robot moves in an unknown environment without knowing its exact position relative to the environment nor the exact position of the obstacle or target location, the robot makes intelligent decisions with limited images and automatically localizes itself with the help of learning by the WMtk. More specifically, an autonomous mobile robot first needs to identify one or several external goal objects as landmarks through on board sensors to guide their behaviors continuously to reach a goal location.

Learning the correct correlation between the percepts and the goal location is the premise of robot localization. A specific location along the path around a building is chosen as a target location to be taught to the robot. This target location will have at least one clearly distinguishing characteristic, such as a manhole cover which is common and has clearly distinguishing features in the chosen outdoor environment. As a result, images with a manhole cover are selected to be the target location and the robot is trained so that it could identify a favorable landmark from multiple objects. To do so, as shown in Fig. 15.1, two categories of images along the path in the outdoor environment, denoted as non-target images and target images, were taken for our experiments. Their segmented versions are shown on the right sides of Fig. 15.1.

Further, for a mobile robot to navigate in the real world outdoor environments, using an isolated natural landmark to identify a location may easily be mistaken for another similar isolated natural landmark in a different location. The different

Table 15.1 Major percepts and their semantic meaning

Percept name	Label	Denoting color
ConcreteFloor	Object0	Light blue
Stairs	Object1	Dark green
RedBricks	Object2	Light green
Underbrush	Object3	Green
ManholeCover	Object4	Yellow

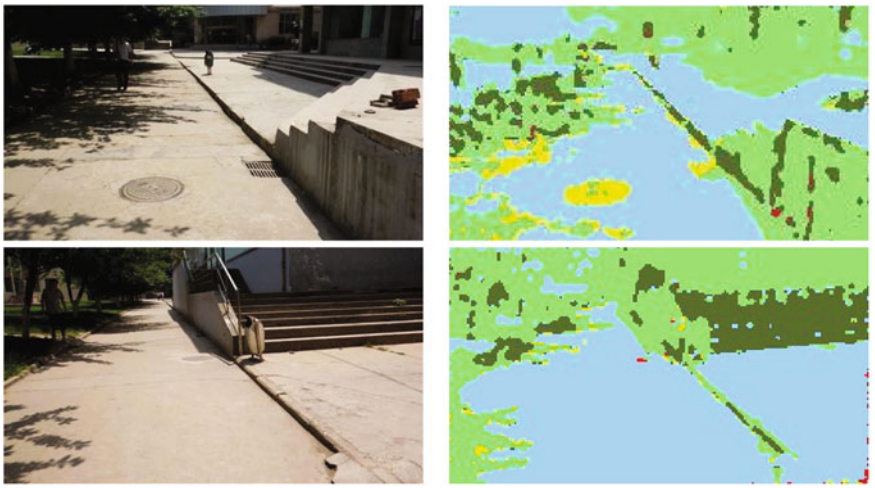


Fig. 15.1 (Upper left) The image of the target location, (upper right) its segmented image, (lower left) the image of the non-target location, (lower right) its segmented image

landmark elements of the environment when bounded together into a configure association can be more resistant to interference from confusing scenes. As a result, in our system, we propose to use configural representation to increase the system robustness and accuracy for mobile robot localization. A series of particular landmark cues combined together with contextual cues can be learned and memorized in terms of working memory to guide the mobile robots to navigate through a more complex scene.

The target location learning process is illustrated in Fig. 15.2 and described in the following. In the training period, we choose 15 images which are taken at target locations and labeled as "target", as typified by the upper part of Fig. 15.1, and 15

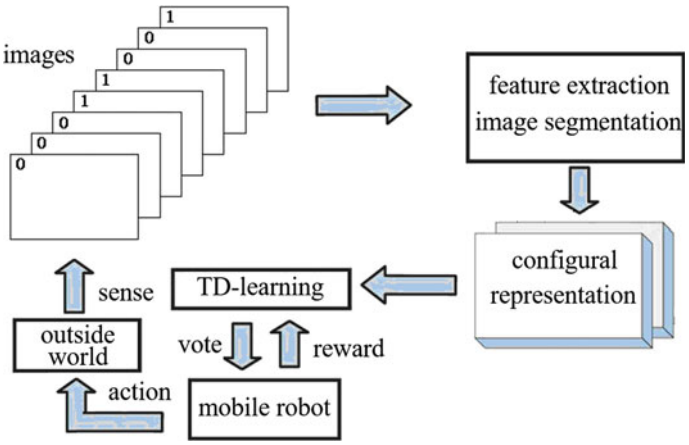


Fig. 15.2 The flow chart of landmark learning

images are taken elsewhere which have no view of the manhole cover and are labeled as "non-target", as illustrated in the lower part of Fig. 15.1. The two sets of images are combined alternately, that is, all the odd-numbered training samples are from the target images and all the even-numbered training samples are from the non-target images. But during the training and testing, the images are randomly selected and presented to the Working Memory. Each image will be segmented into blobs from which candidate working memory chunks are obtained. This collection of blobs, i.e., chunks, is duplicated so that we have two identical sets. All of the blobs in the first set are given the additional voting label of "landmark", for example, Object1_L. All those in the second set are labeled "non-landmark", for example, Object1_NL.

Each training trial consists of two steps, a voting step followed by a reward-generating step. In the first step, the collection of blobs from the two sets is presented to the WM which selects up to L of them, where L is the size of the WM. As shown in Fig. 15.2, the robot will use the labels of the selected chunks to vote for a target or non-target label for this image. For example, if there are three percepts in the target image, Object1, Object3 and Object4, after the duplication operation, there are six chunks that are going to be presented to the WM, namely, Object1_L, Object1_NL, Object3_L, Object3_NL, Object4_L, Object4_NL. If, for example, L=3 and Object1_L, Object3_L and Object4_NL are chosen by the working memory, the final vote will be that it is a target location. In the second step, if the vote agrees with the label of the image, a reward of 1 is returned. Otherwise, a reward of 0 is issued. For another example, if Object1_NL, Object3_L and Object4_NL are chosen, the final vote will be that it is a non-target location. Since it disagrees with the label of the image, the reward of 0 is returned. In the case of a tie (including 0 vote), the decision defaults to non-target.

15.4 Experiments and Results

In this section, we present the results of experiments conducted to investigate the relative performance of the proposed algorithm on mobile robot localization and to demonstrate that the mobile robot can learn one or more percepts to define a target location from the image sequences without a priori knowledge of these representations.

Each experiment consists of an 80-trial training period followed by a 1000-trial and 5000-trial testing period. In both the training period and the testing period, the images are selected randomly and presented to the WM.

15.4.1 Experiment I—Single Percept

Table 15.2 shows the total accuracy of the WMtk after 1000 and 5000 testing trials with 3 different working memory sizes and two exploration rate parameters, 0.00 and

Table 15.2 Percentage of correct location classification with two WM exploration rates for three working memory sizes

WM size	Exploration rate 0.05		Exploration rate 0.00	
	1000 trials (%)	5000 trials (%)	1000 trials (%)	5000 trials (%)
1	50.40	49.50	49.70	50.46
2	95.60	95.70	99.00	99.74
3	96.60	96.86	98.90	99.82

0.05, respectively. The exploration rate parameter reflects the percentage of noise introduced intentionally into the neural network to give other possible solutions some chances so that the final solution will not be stuck in a local minimum. From the table, it can be seen that, for working memory size of one, the neural network implementation inside the WMtk degenerates to the linear case, i.e., the elemental percept coding, and the learning accuracy is only about 50% and unstable. But for working memory sizes of more than one, the conjunctive coding becomes possible and the performances increase largely after proper training. The poor performance of the WMtk with size one is because using the vote from one single percept to predict the label of an image is not as reliable as using the vote from multiple percepts.

In order to see the learning performance of the system, the plots of the average correct classifications over a sliding window of 25 points for 1000 training episodes with two WM exploration rates (i.e., 0.00 and 0.05, respectively) are shown in Fig. 15.3.

It can be seen from the plots that, at the beginning, the Working Memory system starts with random guesses and randomly selecting targets. With TD-learning, the

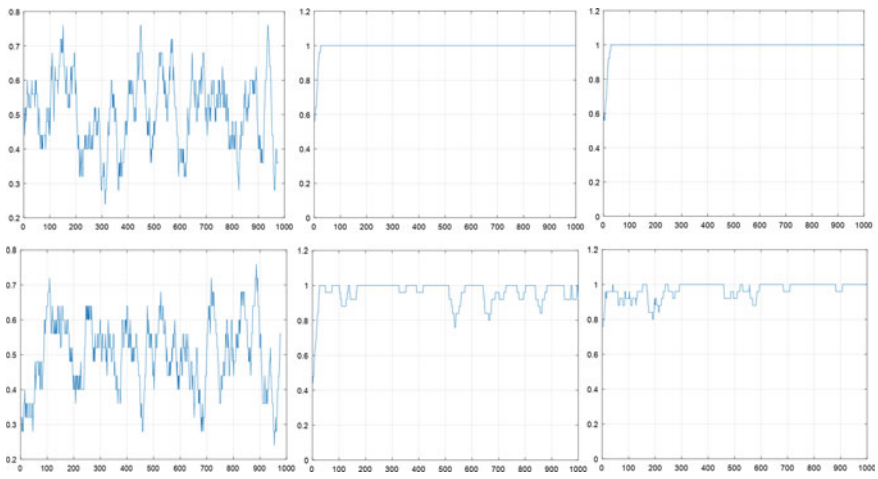


Fig. 15.3 Learning curves of correct classification after 1000 trials with two WM exploration rates. (Upper left) WM size = 1, (upper middle) WM size = 2, (upper right) WM size = 3 with rate set at 0.00; (lower left) WM size = 1, (lower middle) WM size = 2, (lower right) WM size = 3 with rate set at 0.05

system gradually begins to learn the correct choices and the accuracy gradually converges to 1 in about 50–60 episodes on average. Thus, the system is capable of learning both elemental and configural representations with the performances of the latter being increased dramatically.

To see that the performance of the WMtk improves with learning and its behaviors in the weight space, typical plots of the weight distributions for WM of sizes up to 3 for 1000 trials and 5000 trials are shown in Figs. 15.4 and 15.5 for two exploration rate parameters, i.e., 0.05 and 0.00, respectively.

It can be seen from the results that, although all the weights were initialized to be 0.25, they change after learning. When the WM size is one, Object4 (in position 5 on the *x*-axis in the plot), that is, the ManholeCover, has the highest weight. This is because when the WM size is one, the neural network in WMtk degenerates into linear, and its associated largest accumulated reward makes Object4 dominate the output calculation. For WMs of sizes larger than one, two distinctive weight peaks appear of which one peak for each case corresponds to a conjunctive code for the images of a target location, that is, with Object4, while the other peak corresponds to a conjunctive code for the images of a non-target location, such as Object0 and Object3. The combinations of objects which correspond to the peak weights are listed in Tables 15.3 and 15.4 for two exploration rate parameters (i.e., 0.05 and 0.00), respectively. From these tables, we can see that the WMtk can identify the correct landmark(s) and realize the configural representation successfully when the size of the WM is larger than one and most of the results agree with the design intentions of the experiments.

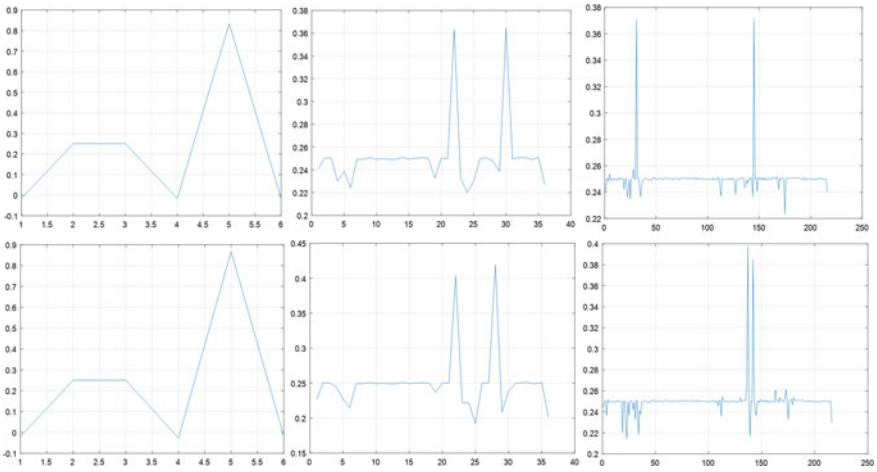


Fig. 15.4 The weight distributions with WM exploration rate set to 0.05. (Upper left) WM size = 1, (upper middle) WM size = 2, (upper right) WM size = 3 after 1000 trials; (lower left) WM size = 1, (lower middle) WM size = 2, (lower right) WM size = 3 after 5000 trials

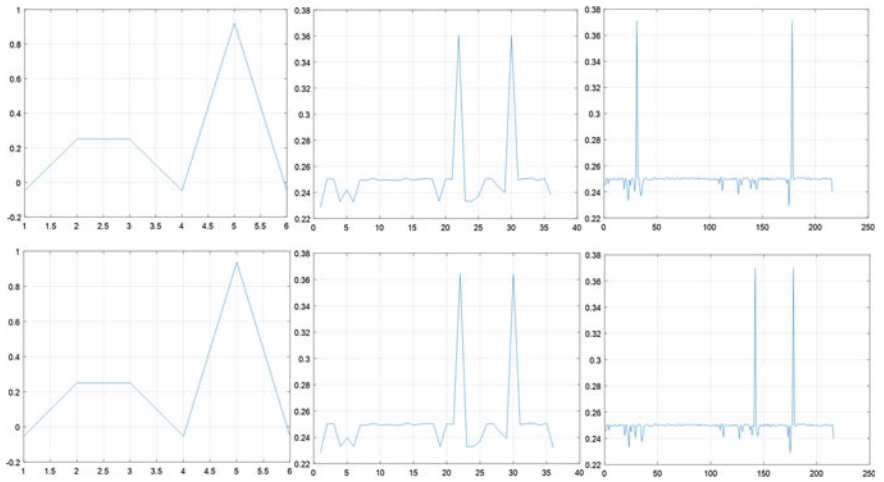


Fig. 15.5 The weight distribution with WM exploration rate set to 0.00. (Upper left) WM size = 1, (upper middle) WM size = 2, (upper right) WM size = 3 after 1000 trials; (lower left) WM size = 1, (lower middle) WM size = 2, (lower right) WM size = 3 after 5000 trials

Table 15.3 WM conjunctive codes for WM exploration rate set at 0.05

WM size	1000 trials		5000 trials	
	Peak1	Peak2	Peak1	Peak2
1	Object4	–	Object4	–
2	Object3	Empty	Object3	Object3
	Object3	Object4	Object3	Object4
3	Empty	Object0	Object3	Empty
	Object0	Object1	Object4	Object3
	Object0	Object4	Object4	Object3

Table 15.4 WM conjunctive codes for WM exploration rate set at 0.00

WM size	1000 trials		5000 trials	
	Peak1	Peak2	Peak1	Peak2
1	Object4	–	Object4	–
2	Object3	Empty	Object3	Object3
	Object3	Object4	Object3	Object4
3	Empty	Object0	Object3	Empty
	Object0	Object0	Object4	Object3
	Object0	Object4	Object4	Object3

15.4.2 Experiment II—Multiple Percepts

Further, we would like to perform another experiment to demonstrate that the conjunctive coding strategy can identify multiple key percepts associated with a target location. As can be seen from the plots in Fig. 15.6, if the robot moves forwards along the path around a building to return to the starting point, at some point of time, the manhole cover appears together with a few red bricks which happen to exist to its left. Suppose the robot is fully disoriented at this location. If it can identify in its acquired pictures the manhole-cover-red-brick pair with the manhole cover on the right and the red bricks on the left, it can figure out its moving direction from this combination. So for this experiment, the upper plot of Fig. 15.6 indicates a target location for the robot to move toward (because it has both the manhole cover percept and the red bricks percept) while the lower part of Fig. 15.6 indicates a non-target location. Therefore, 15 pictures with clearly recognized manhole cover percept and red bricks percept (denoted by the red color in the left part of the segmented image) are collected as the target location and 15 pictures with no such a percept combination are collected as the non-target location for the WM system to learn. These two sets of images are combined to form a labeled training set. Each image will be segmented into blobs from which duplicated candidate working memory chunks are obtained. Then the same voting procedure as described in the previous experiment is followed to train and test the robot.

In order to see the learning performance of the system, the plots of the average correct classifications for three WM of sizes, that is, 1, 2 and 3, over a sliding window of 25 points for 5000 training episodes with WM exploration rate set to 0.00



Fig. 15.6 (Upper left) The original image of the target image, (upper right) the target image after segmentation, (lower left) the original image of the non-target image, (lower right) the non-target image after segmentation

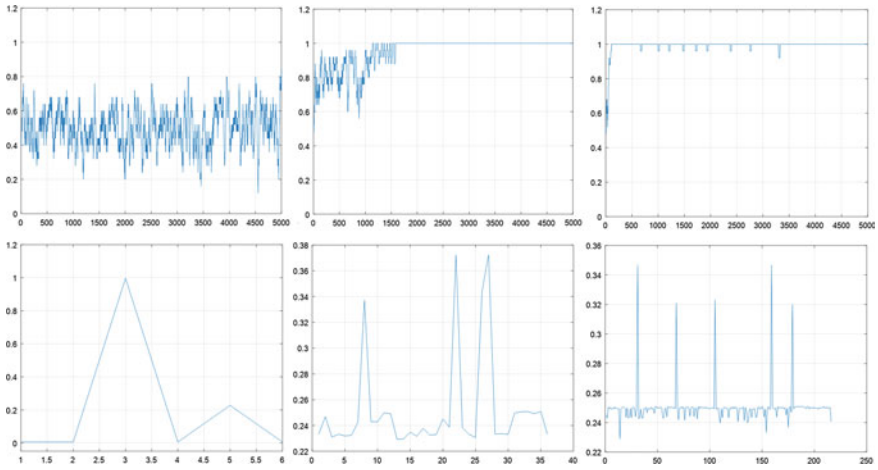


Fig. 15.7 Learning curves of correct classification after 5000 trials with WM exploration rate set to 0.00 for (upper left) WM size = 1, (upper middle) WM size = 2, and (upper right) WM size = 3; the weight distribution at episode 5000 with WM exploration rate set to 0.00 for (lower left) WM size = 1, (lower middle) WM size = 2, (lower right) WM size = 3

are shown on the upper part in Fig. 15.7. To see that the performance of the WMtk improves with learning and its behaviors in the weight space, typical plots of the weight distributions for the three WM of sizes after 5000 trials are shown in lower part of Fig. 15.7 for the exploration rate parameter of 0.00.

Table 15.5 shows the total accuracy of the WMtk after 5000 testing trails with 3 different working memory sizes and one exploration rate parameter set to 0.00, while similar results are observed for the exploration rate parameter set to 0.05. From the table, it can be seen that, for working memory size of one, the neural network implementation inside the WMtk degenerates to the linear case, i.e., elemental percept coding, and the accuracy of learning is only about 50% and unstable. But for working memory size of more than one, the conjunctive coding becomes possible and the performances increase largely after proper training. The poor performance of the WMtk with size one is because using the vote from one single percept to predict the label of the image is not as reliable as using the vote from multiple percepts.

It can be seen from the results that, although all the weights were initialized to be 0.25, they change after learning. When the WM size is one, Object2 (in position 3 on the x-axis in the plot), that is, the RedBricks, has the highest weight and the

Table 15.5 Percentage of correct location classification with two WM exploration rate of 0.00 for three working memory sizes

WM size	5000 trials (%)
1	49.90
2	95.15
3	99.18

Table 15.6 WM conjunctive codes for WM exploration rate set at 0.00

WM size	Peak1	Peak2	Peak3	Peak4	Peak5
1	Object2	Object4	–	–	–
2	Object1	Object3	Object2	–	–
	Object1	Object3	Object4	–	–
3	Object0	Object1	Object2	Object4	Object4
	Empty	Empty	Empty	Object2	Empty
	Object0	Object3	Object2	Object2	Object4

ManholeCover has the second highest weight. This is because when the WM size is 1, the neural network in WMtk degenerates into linear, and its associated largest accumulated reward makes Object2 dominate the output calculation. For WMs of sizes larger than one, several distinctive weight peaks appear of which one peak for each case corresponds to a conjunctive code for the images of a target location, that is, with Object2 and Object4, while the other peak corresponds to the non-target locations, such as with Object0 and Object3. The combinations of objects which correspond to the peak weights are listed in Table 15.6 for the exploration rate parameter set to 0.00. From these tables, we can see that that the WMtk can identify the correct landmark(s) and realize the configural representation successfully when the size of the WM is larger than one and most of the results agree with the design intentions of our experiments.

The execution time of landmark learning for each image depends on the size of the WM and, in our current setting, is about 4.07 s for WM of size one, 7.15 s for WM of size two, 42.74 s for WM of size three. Overall the execution time can be reduced if a much faster workstation computer is available.

To conclude, the WMtk can identify the correct landmarks and realize the configural representation successfully in an unknown and complicated outdoor environment when the size of the WM is larger than one.

15.4.3 Experiment III—Competing Methods

A competing system, based on a single perceptron, is used as the first performance comparison for our experiments. The perceptron will be trained on the same test image sets as used in the previous two experiments. The results are used as a comparison with those of the proposed system. To further evaluate the learning capability of the WMtk, a support vector machine approach is used to do the same classification task as described for the perceptron. The classification results of the perceptron and SVM (using a linear kernel) over training and testing are summarized in Table 15.7. For the perceptron, the outputs are thresholded by 0.5. That is, if the output is larger than 0.5, the image is believed to be the target image. Otherwise, it is believed to be the non-target image.

Table 15.7 Percentage of correct location classification of the perceptron and SVM

	Perceptron	SVM
Trials	5000	5000
Correctness (%)	98.68	100

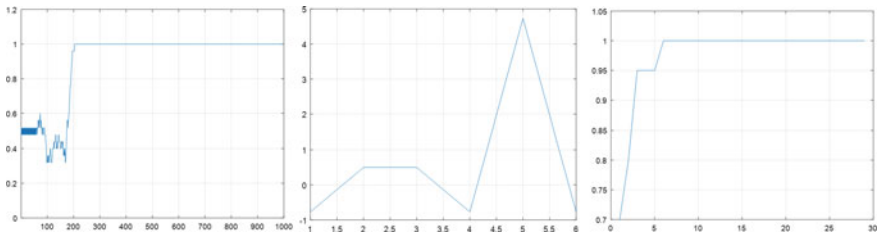


Fig. 15.8 (Left) The learning curve of the perceptron and (middle) the weight distribution of the perceptron at episode 1000; (right) The learning curve of SVM at episode 30

The weight distribution and the learning curve of the perceptron at episode 1000 are shown in left and middle plots of Fig. 15.8, respectively. The learning curve of the linear kernel SVM is shown in the right plot of Fig. 15.8. It can be seen that the perceptron and the linear kernel SVM have done a better job than the WMtk when the WM size is set to one. The weight distribution and the learning curve of the perceptron have similar structures to those of the WMtk with a WM of size one, but it undergoes a much longer (about 10 times longer than the WMtk) learning periods. The SVM method is faster than the WMtk.

15.5 Conclusions

In this chapter, the integration of a computer vision system and a nonlinear learning model, TD-learning which is implemented in WMtk, is proposed for the landmark-based mobile robot localization problem in an outdoor unknown environment. Experiments are designed to infer semantic prediction of a scene from different configurations of its stimuli. From the experimental results, we can conclude that recognizing a target location visually for robot's localization is feasible, and using the conjunctive coding for automatic scene recognition can produce a better performance. Although the WMtk has better classification and running time performances than the single perceptron, the SVM approach is competitive. However, the WMtk has a better performance than the perceptron and the SVM in the sense of its being capable of high-level semantic information prediction in addition to being a good classifier. Therefore, in our future work, efforts will be devoted to provide solution for questions with regard to how to find the target locations automatically.

References

Busch, M. A., Skubic, M., Keller, J. M., & Stone, E. E. (2007). A robot in a water maze: Learning a spatial memory task. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)* (pp. 1727–1732).

Civera, J., Grasa, O., Davison, A., & Montiel, J. (2010). 1-point RANSAC for EKF filtering: Application to real-time structure from motion and visual odometry. *Journal of Field Robot, 27*(5), 609–631.

Conn, K., & Peters II, R. A. (2007). Reinforcement learning with a supervisor for a mobile robot in a real-world environment. In *Proceedings of the IEEE International Symposium on Computational Intelligence in Robotics and Automation (CIRA)* (pp. 73–78).

Deisenroth, M. P., & Rasmussen, C. E. (2011). PILCO: A model-based and data-efficient approach to policy search. In *Proceedings of the 28th International Conference on Machine Learning (ICML)* (pp. 465–472).

Dellaert, F., Burgard, W., Fox, D., et al. (1999). Monte Carlo localization for mobile robots. In *Proceedings of the IEEE/RSJ International Conference on Robotics and Automation (ICRA)* (pp. 1322–1328).

Duan, Y., Cui, B., & Yang, H. (2008). Robot navigation based on fuzzy RL algorithm. In *International Symposium on Neural Networks (ISNN). Advances in Neural Networks. ISNN 2008 Lecture Notes in Computer Science* (Vol. 5263, pp. 391–399).

Dusek, J., & Eichenbaum, H. (1998). The hippocampus and transverse patterning guided by olfactory cues. *Behavioral Neuroscience, 112*(4), 762–771.

Fox, D. (1998). *Markov localization: A probabilistic framework for mobile robot localization and navigation*. Bonn, Germany: University of Bonn.

Guivant, J., & Nebot, E. (2001). Optimization of simultaneous localization and map building algorithm for real time implementation. *IEEE Transactions on Robotics and Automation, 17*(3), 242–257.

Hester, T., Quinlan, M., & Stone, P. (2012). RTMBA: A real-time model-based reinforcement learning architecture for robot control. In *Proceedings of IEEE International Conference on Robotics and Automation (ICRA)* (pp. 85–90).

Hornung, A., Bennewitz, M., & Strasdat, H. (2010). Efficient vision-based navigation: Learning about the influence of motion blur. *Autonomous Robots, 29*(2), 137–149.

Jensfelt, P., & Christensen, H. I. (2001). Active global localization for a mobile robot using multiple hypothesis tracking. *IEEE Transactions on Robotics and Automation, 17*(2), 748–760.

Juang, C. F., & Hsu, C. H. (2009). Reinforcement ant optimized fuzzy controller for mobile-robot wall-following control. *IEEE Transactions on Industrial Electronics, 56*(10), 3931–3940.

Kober, J., Bagnell, J. A., & Peters, J. (2013). Reinforcement learning in robotics—A survey. *The International Journal of Robotics Research, 32*(11), 1238–1274.

Kollar, T., & Roy, N. (2008). Trajectory optimization using reinforcement learning for map exploration. *International Journal of Robotics Research, 27*(2), 175–197.

Lowe, D. (2004). Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision, 60*(2), 91–110.

Michels, J., Saxena, A., & Ng, A. Y. (2005). High speed obstacle avoidance using monocular vision and reinforcement learning. In *Proceedings of the 22 International Conference on Machine Learning (ICML)* (pp. 593–600).

Mikolajczyk, K., & Schmid, C. (2005). A performance evaluation of local descriptors. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 27*(10), 1615–1630.

Mouragnon, E., Lhuillier, M., Dhome, M., Dekeyser, F., & Sayd, P. (2009). Generic and real-time structure from motion using local bundle adjustment. *Image and Vision Computing, 27*(8), 1178–1193.

Murray, J. J., Cox, C., Lendaris, G. G., et al. (2002). Adaptive dynamic programming. *IEEE Transactions on Systems of Man and Cybernetics—Part C: Applications and Reviews, 32*(2), 140–153.

Nadel, L., & Willner, J. (1980). Context and conditioning: A place for space. *Physiology & Behavior, 8,* 218–228.

Quintia, P., Iglesias, R., Rodriguez, M. A., Regueiro, C. V., & Valdes, F. (2012). Learning in real robots from environment interaction. *Journal of Physical Agents, 6*(1), 43–51.

Rosten, E., & Drummond, T. (2006). Machine learning for high-speed corner detection. In *Proceedings of European Conference on Computer Vision, Lecture Notes Computer Science* (Vol. 3951, pp. 430–443).

Rudy, J.W., & O'Reilly, R.C. (2001). Conjunctive representations, the hippocampus, and contextual fear conditioning. *Cognitive, Affective, & Behavioral Neuroscience, 1*(1), 66–82.

Se, S., Lowe, D., & Little, J. (2001). Vision-based mobile robot localization and mapping using scale-invariant features. *Proceedings IEEE International Conference on Robotics and Automation, 2,* 2051–2058.

Seymour, B., O'Doherty, J. P., et al. (2004). Temporal difference models describe higher-order learning in humans. *Nature, 429*(10), 664–667.

Smith, R., Self, M., & Cheesman, P. (1990). Estimating uncertain spatial relationships in robotics. In *Autonomous Robot Vehicles* (pp. 167–193). New York, USA: Springer-Verlag.

Sutherland, R. J., & Rudy, J. W. (1989). Configural association theory: The role of the hippocampal formation in learning, memory, and amnesia. *Psychology, 17,* 129–144.

Sutton, R. S., & Barto, A. G. (1998). *Reinforcement learning: An introduction*. Cambridge: MIT Press.

Tardif, J., Pavlidis, Y., & Daniilidis, K. (2008). Monocular visual odometry in urban environments using an omnidirectional camera. In *Proceedings of International Conference on Intelligent Robots and Systems* (pp. 2531–2538).

Thrun, S., Burgard, W., & Fox, D. (1998). A probabilistic approach to concurrent mapping and localization for mobile robots. *Machine Learning, 31*(1), 29–53.

Wang, X., Tugcu, M., Hunter, J. E., & Wilkes, D. M. (2009). Exploration of configure representation in landmark learning using working memory toolkit. *Pattern Recognition Letters,* 66–79.

Wang, X, Chang, C., & Wang, X. L. (2017). A fast incremental spectral clustering algorithm for image segmentation. In *Proceedings of the 2017 International Conference on Computational Science and Computational Intelligence* (pp. 15–27), Las Vegas, December.

Wehner, R. (1992). Arthropods. Animal Homing (ed. Papi, F.). Chapman and Hall, London, pp 45–144.

Williams, H., Browne, W. N., & Carnegie, D. A. (2017). Learned action SLAM: Sharing SLAM through learned path planning information between heterogeneous robotic platforms. *Applied Soft Computing,* 313–326.

Wolf, J., Burgard, W., & Burkhardt, H. (2005). Robust vision-based localization by combining an image retrieval system with Monte Carlo localization. *IEEE Transactions on Robotics, 21*(2), 208–216.

Xin, X. (2006). A sparse kernel-based least-squares temporal difference algorithm for reinforcement learning. In *Proceedings of the 2006 International Conference on Natural Computation, Lecture Notes in Computer Science* (Vol. 4221, pp. 47–56).

Chapter 16
An Autonomous Vision System Based Sensor-Motor Coordination for Mobile Robot Navigation in an Outdoor Environment

Abstract Autonomous mobile robots are intelligent machines capable of performing tasks in the real world without explicit human control for extended periods of time. For a mobile robot to navigate from a starting position to a goal position within its environment, a path should be identified to generate an obstacle-free trajectory between the starting point and the goal point. Based on a vision-based autonomous percept acquisition system, this chapter discusses the basic ideas of the proposed autonomous navigation strategy of mobile robots by using vision as the sensing mechanism in achieving the desired objectives. The success of the proposed computer vision-based robot system is demonstrated by its application to a simple sensor-motor coordinated navigation task.

Keywords Computer vision · Pattern recognition · Clustering · Classification tree · Machine learning · Sensor-motor coordination · Working memory toolkit

16.1 Introduction

After four robot design generations, from the early days' application of control theory to complex systems, followed by the sense-think-act cycle of the functional approach, then to the Brooks' behavior-based robotics, and finally to the recent cognitive robotics, different branches, such as industrial robots, mobile robots, and humanoid robots, have come to a remarkable level of maturity as evidenced in the literature. However, the ability to build and realize such artificial intelligent creatures, capable of moving and operating autonomously so as to replace trained individuals in a variety of applications, still remains as the continuous desire of mankind. As an absolute prerequisite for the design of such robots, human-like intelligence refers to the ability to self-organize, to learn and adapt to varying, complex, dynamic, and uncertain environment, and to interact with the environment and perform tasks efficiently and safely through using their on-board problem-solving capabilities. To imitate humans, any intelligent robot should be made up of elements such as sensing, perception, cognition, planning, control, and actuation (Fukuda et al. 2001).

For an autonomous mobile robot to perform tasks successfully in an environment, the path planning and navigation must be handled simultaneously. Being considered

© Xi'an Jiaotong University Press 2020

X. Wang et al., *Machine Learning-based Natural Scene Recognition for Mobile Robot Localization in An Unknown Environment*,
https://doi.org/10.1007/978-981-13-9217-7_16

to be the main challenge in the field, the path planning problem of mobile robots deals with the identification and generation of the obstacle-free trajectories between a starting position and a goal position within an environment. When a map of an environment is known, the robot can plan its trajectory before navigation. However, many times in an unknown environment, the mobile robot can not decide the entire trajectory beforehand. In this situation, a robot divides the path planning task into a set of subgoals in terms of subgraphs, plans a subpath and thus navigates to that point. Then the process continues until the robot reaches the destination. In this process, the environment representation, that is, the environment modeling, is an important aspect of path planning. It is in this sense that the representation of the space plays an important role in the navigation of an intelligent system.

Not having a human's sophisticated vision system, to operate in realistic environments and to recognize objects in a given image, a robot must have the ability to form visual percepts on its own so as to recognize objects in a given image by segmenting the image into non-overlapping but meaningful regions whose union is the entire image (Buhmann et al. 2005). In general, by meaningful, the regions in the segmented image should be homogeneous and have well-defined boundaries, denoting reliable objects. With high spatial frequencies being filtered out, image segmentation reduces the amount of storage and makes behavior resistant to the loss of information. Our motivation is to develop an object recognition vision system based on a set of features associated with each object in the images to assist in the analysis of data that may be applied successfully across different application domains (Hunter 2005).

In this chapter, a new approach is developed for solving the problem of mobile robot navigation in an unknown environment based on TD-learning. TD-learning algorithms have been used widely for solving real-world problems especially in robotics since it has been proved to give reliable and efficient solutions due to its simple and well developed theory. The proposed method does not use the coordinate information. The experimental simulation conducted demonstrates that a mobile robot equipped with such perceptual learning and behavior learning systems can learn to perform a navigation task successfully in realistic environments.

In the following, Sect. 16.2 provides a short overview of the current work on path planning research. Then based on the visual percept learning system that has been developed, Sect. 16.3 introduces a visual perceptual learning model for a mobile robot navigation task in this work. Experiments are designed in Sect. 16.4 to demonstrate the learning capability of an autonomous mobile robot system. Finally, conclusions are given in Sect. 16.5.

16.2 Related Work

Path planning is one of the most-studied tasks in autonomous mobile rootics (Arkin 1998). The navigation problem of a mobile robot can be divided into two main categories (Amato and Wu 1996), namely a global path planning (GPP) and a local

path planning (LPP). Global path planning refers to the path planning in fully known environments. The first step of global path planning is to build a global map of the environment, and then use an optimization algorithm to search the optimal path under the global map model. In general, GPP methods seek for "the shortest", "the most energy efficient", "the fastest" path and etc. In fact, since GPP belongs to NP-hard, its solution is in practical systems intractable because of the time and space complexity and path optimality is not a necessity for a GPP. Metaheuristic methods such as evolutionary computing is one of proper methods used for solving it. Traditional approaches to global path planning utilize roadmap representation based on computation geometry, resulting in a group of Visibility Graph methods (Lumelsky and Stepanov 1987), shortest path methods (Lee and Preparata 1984), and Voronoi diagrams (Klein et al. 2009). For these methods, as a binary one-dimensional feature, a map contains information whether a location contains obstacle or not (Zhang et al. 2013). Relatively new sampling techniques used in roadmap representation include Probabilistic Roadmaps (Amato and Wu 1996; Bohlin and Kavraki 2000; Kavraki 1995; Kavraki and Latombe 1994) or Rapidly Exploring Random Trees (RERT) methods (Lavalle 1998). These methods are composed of two recursively performed steps: (1) sampling of Configuration Space (usually a position), (2) building a graph by simple planners. However, there is no one-and-only method for these steps. One has to rather choose a specific one that will perform best in a particular case considered. Several corresponding sampling approaches exist in the literature with the most important ones being Uniform Sampling, Random Sampling, Halton Sampling (Branicky et al. 2001), random Halton Sampling, Gaussian sampling (Boor et al. 1999), and Obstacle based sampling (Amato et al. 1998). The latter group—RERT is a graph-based method that can be used for the holonomic, non-holonomic (Frazzoli et al. 2002) and kinodynamic planning (LaValle and Kuffner 2001) of moving objects including the mobile robots (Liu et al. 2012).

When the environment is unknown, the mbile robot performs localization and perception for a local path planning so as to generate a feasible path in real time according to the obtained sensor information. The commonly used local path planning algorithms mainly include cell decomposition (Ramer et al. 2013), artificial potential field (Yu et al. 2013) and graph representation. Since they are totally discrete models, cellular automata (CA) (Mitchell 1996; Oliveira et al. 2009) have also been recently considered for path planning (Behring et al. 2000; Soofiyani et al. 2010; Rosenberg 2007; Akbarimajd and Lucas 2006; Akbarimajd and Hassanzadeh 2011; Ioannidis et al. 2008, 2011; Shu and Buxton 1995; Marchese 2005; Tzionas et al. 1997). First introduced by Khatib in 1986, Artificial Potential Field method was originally developed as a real-time obstacle avoidance method when emphasis was put on online computations. In this approach, a robot is treated as an electron in a potential field on the map where attractive and repulsive forces act on it (Barraquand et al. 1992). The last group of methods are bases on graph representation. The most popular such methods include Depth First Search algorithm, Breadth-First Search algorithm, Uniform Cost Search algorithm, Dijstra algorithm, Trulla algorithm (Mitchell and Papadimitriou 1991), A* algorithm (Hart et al. 1968), and D* algorithm (Stentz 1997).

16.3 Vision-Based Perceptual Learning

16.3.1 A Behaviour Learning Model

Navigation in an outdoor environment has been a subject of researches for years. Although, despite efforts put into researching the problem, lack of reliable and universal solutions is still evident. As Sperry pointed out long ago, the perceptual system evolved to improve the performance of the motor system, but not the other way around (Sperry 1952; Butler and Hodos 1996). That is to say, behavior is not always determined by sensory stimulation. For example, when two sources of sensory inputs try to attract the subject in different directions, in order to function effectively, the subject must be able to ignore all stimuli except the one it regards as the most important. This sort of dilemma happens all the time. In other words, if several stimuli are present, the subject may have to choose between possible courses of action and, when one has been decided on, must prevent any irrelevant input from interfering with the behavior. It is in this way that attention comes into play. It is in this sense that behavior learning is the process that forms association between perception and action.

Attention is a process that highlights a particular stimulus. It selects the items relevant to the current task from the vast array of sensory inputs and determines which associations of those items should follow. For an explanation of how attention identifies and targets the relevant stimuli, the sensitization of the appropriate sensory pathways is an essential component of any response. At the beginning, a goal to perform some action comes first, which exerts a major influence on the type of stimulus to which the sensory system is attuned. Next, after the goal has been established, attention is focused on stimuli needed to proceed with the task. For example, when a two-year-old baby finds some new toy which is out of his reach, he may have the desire in the first place to approach it. Subsequently, the location of the toy guides the direction of his movement. At the center of the model, a goal must be established, often felt as a need, usually in the form of internal stimuli, which will be satisfied by external sources, while other stimuli are often called reinforcers. During learning, the goal will evolve to activate responses that are most likely to reduce the need. Therefore, in this model, it is obvious that the stimuli play a dual role, serving to motivate sometimes while to inform and guide responses at other times.

Since any basic behavior defines a category of action in response to sensory inputs—a stimulus-response category, perception and action are tightly coupled. This coupling was given a name, sensory-motor coordination, by American philosopher, John Dewey. In the process of explaining human behavior, as a relatively recent evolutionary development from the accumulated experiences of sensory-motor coordination, learning plays a major part and allows animals to transfer the motivational properties of innately reinforcing stimuli to accompanying or perceptibly related stimuli. To survive, animals must recognize sources of nourishment and dangerous situations, and respond accordingly by approaching the former and staying away from the latter. In other words, learning makes attention initially directed only to an innate reinforcer being directed to objects associated with the reinforcer. For example,

when you are hungry and out of your house, you will look for a restaurant. When you finish eating in the restaurant and want to get out, you will look for the doors. It is learning that allows initially neutral objects to arouse a motivational state and links the motivation system to the perceptual systems. As another example, when you see wall, you will stop going toward it and turn to open space. For higher animals such as humans, evidence suggests that the occurrence of a response depends on its predicted outcome and behaviors are planned on the basis of future possibilities rather than present contingencies alone (Butler and Hodos 1996; Striedter 2004). If the outcome of an action satisfies this expectation, an innate reward is issued, facilitating the intended behavior. Conversely, if the expectation is not met, an innate punishment inhibits this behavior.

Based on the same principle, the robot learning problem addresses the question of making a robot perform certain tasks in the world successfully. As a result, every learning process has some "goal", which can be most precisely defined in terms of perceived states of the world, i.e., the sensor states, obtained by transforming the true state of the world. Suppose G is an identifiable goal state, X and Y are two different perceived states of the world, the learning problem can be formally described by,

$$G : X \rightarrow Y : R \tag{16.1}$$

which can be interpreted as, suppose the robot finds itself in a state satisfying condition X, if, in reaching the goal G, a state satisfying condition Y becomes active, a reward R is received (Nehmzow 2003). If a task can be defined in term of a set of such reward receiving goals, a qualitative measure of robot performance can be the sum of the rewards it receives over time. Given this definition of robot performance, the robot learning problem is to improve robot performance through experience. Since this type of situation is common in robotics, reinforcement learning may be applied to many robotics tasks.

16.3.2 An Incremental Percept Learning System for Mobile Robot Navigation

Localization and navigation requires knowledge about the environment and information regarding robot's current position. The prior information is especially important when operating in an unstructured outdoor environment, where robot has to be prepared to deal with a variety of different situations. The vast amount of possible cases combined with the noisy nature of sensors result in the difficulty of outdoor mobile robot navigation. Therefore, the robot's perception is a crucial part of navigation. Data extracted from available sources has to be chosen carefully and usually combined together to fuse available information in order to achieve a more reliable estimate. The data sources are generally divided into two groups—on-line and off-line. The online sources are available only during the operation time. The data from

them usually reflects actual states of the robot and the environment, but their scope is limited. The second type are sources prepared before the operation time. Information obtained from that group, in general, characterizes the environment or parametrizes the robot's structure.

Efficient navigation of autonomous mobile robots depends on the system providing precise information about current location. In this section, the problem of acquisition and utilization of information about the general structure of an outdoor environment is tackled. We present a localization and navigation solution utilizing an incremental percept learning system.

Suppose that we are given a sequence of images taken in an unknown environment and our goal is to identify all the objects in the images in terms of patch-based local color features extracted from the images. To fulfill the task, a clustering algorithm is applied to segment each image, producing a number of clusters for each image. By this way, it is possible that data points belonging to different clusters may be labeled with the same cluster label while data points belonging to one cluster are clustered into different clusters in different images. The next step would be to merge the clusters coming from the same object effectively and efficiently. Trees present an efficient way to index local image regions. We propose to use BIRCH tree to merge clusters identified in different images. After the first image is segmented based on clustering the extracted feature vectors, all the labeled feature vectors are then inserted into the BIRCH tree.

For subsequent images processed by the a clustering algorithm, given a new labeled feature vector, we process it as,

(1) recursively descends the labeled color-histogram-based feature vector from the root node to the leaf node in accordance with the minimum distance principle;
(2) determines whether the nearest CF entry in the leaf node can absorb the new data point (i.e., whether the updated diameter of D when the new data point is added is less than the threshold value T);
(3) if it is smaller than T and the CF entry and the new data point have different cluster labels, merge the two cluster labels, update all the CF information from the root node to the path of the leaf node, if the leaf node still has room to accept the CF entries (i.e. the number CF of the leaf node is less than B), then adding the CF to the leaf nodes of the CF list;
(4) otherwise, split the leaf node and a new cluster appears by selecting the original leaf node and the new data point as the two seeds of nodes, delete the original leaf node, and update the tree (when the root node split, add a layer and the tree height increases by one);
(5) if there is any unprocessed data point left, go to step (1), otherwise the algorithm ends.

To use BIRCH tree as a classification tree to integrate the results of a clustering, either for a labeled cluster to be merged with the existing cluster or for a new class to be incorporated into existing classes, a cluster's threshold should be given or determined. If the majority of the members of a cluster in the subsequent image have a distance which is smaller than its cluster's threshold, these clusters are merged. If

the majority of the members of a cluster in the subsequent image have a distance which is larger than the threshold, a new cluster appears and a new label is assigned to its members. Given a set of images, BIRCH tree branch factor B and threshold T, a set of segmented images are obtained by two steps.

(1) The clustering step: read in an image, extract the feature vectors, apply a clustering algorithm to segment it and assign a label to each feature vector;
(2) The merge step: insert the feature vectors into the BIRCH tree and merge clusters.

The clustering step and the merge step are repeated until the end of the image sequence is reached. Finally, each cluster after integration is marked by a different color, and the results of image segmentation are obtained.

16.4 Experiments and Results

To evaluate the quality of the proposed approach, our working environment is a path around a building. Images were taken around this outdoor environment using SONY digital video camera recorder. The images were first converted from RGB color space into HSV color space. To obtain feature vectors, for each image, a moving window of size 10×10 is shifted by 6 pixels in the row and column directions but not to exceed the border of the image. The chromaticity H is evenly divided into 16 intervals, while the saturation S and the brightness V are evenly divided into 4 intervals. All the interval combination can generate 240 different color bins for the generation of color histogram for each image patch. Then, all the pixels in the moving window are assigned to the corresponding possibility, and a sparse feature vector (color histogram) is obtained. From the current implementation of the incremental percept learning system, five clusters were obtained to form our database and the corresponding percepts are denoted in Table 16.1. Three sample images and their segmented versions are shown in Fig. 16.1. Each class was assigned a different color for displaying purposes.

It can be seen that typical percepts perceived by our robot vision system include percept groups for ConcreteRoad, Stairs, RedBricks, Underbrush and ManholeCover.

Table 16.1 Autonomously learned percepts

Denoting symbol	Percept name identified by human
Object0	ConcreteRoad
Object1	Stairs
Object2	RedBricks
Object3	Underbrush
Object4	ManholeCover

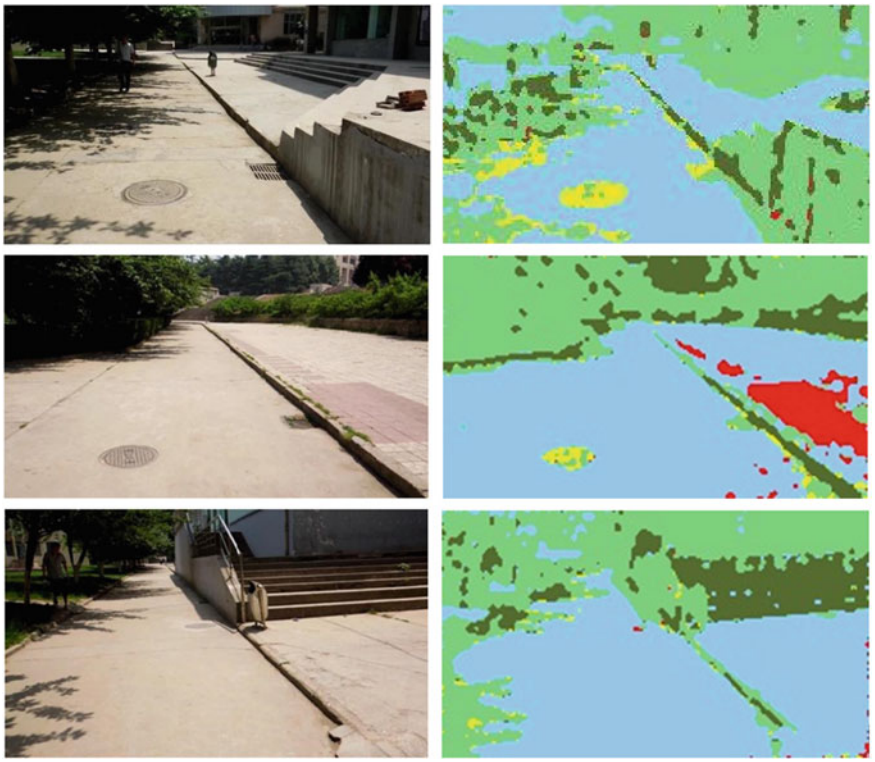


Fig. 16.1 (Left) Test images and (right) their segmented versions

16.4.1 Experiment I: Learning the Road

In this experiment, a road learning task is implemented using WMtk. For a mobile robot to navigate through its environment, path planning means to search an optimal obstacle free trajectory from a starting position to a goal position. Therefore, the first step in the navigation task is to learn the percepts that would yield an open path for the robot through its own interaction with the environment. To simplify the search area into a manageable number of node, a path should be divided into a set of subgoals in terms of straight path segments between two target locations to simplify the search area in path finding. Once meaningful percepts are obtained from the images acquired by the camera, the robot needs to determine whether each percept represents an obstacle, an open space or the road to follow. Since it does not have any prior information about these percepts, the mobile robot needs to learn the association of the percepts with motion. Unlike the indoor case where there are only open space and obstacles, in the outdoor environment as illustrated in Fig. 16.1, multiple percepts other than the concrete road are associated with the open space. Therefore, in the next step, the robot should learn the meaning of the percepts with regard to motion using the WMtk, that is, which of the detected percepts could represent the road to follow and which could represent others. Therefore, in this

task, there are two situations to consider. In the first situation, while moving towards a percept for several meters long to realize a subgoal, if the robot is stopped by an obstacle percept, it will receive a reward 0 and retrace back to the previous point and then replans for an alternative subpath. In the second situation, if the robot is not stopped, the corresponding percept is meant to be open space. However, it may not necessarily be the road to follow to reach the subgoal. To recognize the road percept, the robot is put in the middle of the road to start and to move toward the subgoal target location, and, in this case, the reward criterion for the WM system is selected to be the case that the robot is not stopped and the percept chosen is in agreement with the lower part percept before its moving per trial. The goal in this methodology is to see whether the robot will follow the road consistently by some blobs but not by others. The procedure for the first experiment is as follows,

- The robot takes pictures in the environment while moving, partitions the pictures into blobs, and remembers each blob together with their position in the image. Since the robot is only interested in the blobs in the lower half of the image, blobs in that region are used as input chunks to the working memory system created by WMtk.
- The WM size is set to 1 so that only one of the input chunks is selected per episode. The WM system selects one and asks the robot to move toward it for 5 meters long.
- Next, the robot directs itself to the selected chunk and tries to move toward it for 5 meters or until sonar sensor indicates an obstacle happens, forcing it to stop. If the robot moves $x < 5$ meters long and then stops since it would bumps onto that object, it will receive a reward 0 and gather evidence that the object is associated with an obstacle. The robot retraces back to the previous location and then replans for the next trial.
- If the percept chosen and presented to the working memory agrees with the percept that is in the middle of the lower half of the starting position and the robot moves 5 meters long without being stopped, it will receive the maximum reward, 1, and gather evidence that the object is associated with the road. Otherwise, if the robot moves 5 meters long without bumping onto the percept but the percept it moves toward does not agree with the percept in the middle of the lower half of the starting position, it will receive a reward 0 and gather evidence that the object is associated with a open space percept other than the road. The robot retraces back to the previous location and then replans for the next trial.

The process of retracing and navigating may continue until the robot reach a given goal point, allowing the WM system to intelligently update its internal parameters associated with each chunk in the environment until the percept corresponding to the Concrete Road percept eventually dominates the trials. The WMtk has also a reserved position for the situation when none of the chunks is selected. With a WM size of 1, the weight parameters inside the WMtk are directly associated with the percepts and the received rewards. In order to evaluate the system's learning performance, the robot was put in the middle of the road to start.

Two sets of experiments were conducted to show the effectiveness of the computer vision system with two exploration rates, 0.00 and 0.05. Because the weights

of the neural network associated with each percept reflect the strength that percept is associated with the reward, to show the performance of this experiment, the weights associated with each percept were recorded at each trial for two sets of initial weights, 0.25 (pessimistic) and 2.5 (moderately optimistic). The results are presented in Figs. 16.2, 16.3, 16.4 and 16.5, respectively. The *x*-axis denotes the number of trials while the *y*-axis denotes the weight associated with each percept.

From Fig. 16.2, it can be seen that, at the beginning, the weights may not always appear rational, but they were always based on the data available at the time and were modified by any relevant information the individual had absorbed along the way. Therefore, at the beginning, the model generated random responses based on the sensory inputs but it learned the right percept gradually. Although still competing with each other to become active, the ConcreteRoad percept was always chosen and eventually recognized as being associated with the road. Thus, its weight took off quickly. It took approximately 30 trials for the ConcreteRoad to become the most likely one to be activated.

The reason why there were more or less large fluctuations at the beginning is that random noise was introduced by the toolkit so that the algorithm would not be locked at a local minimum, an example of the so-called exploration versus exploitation trade off. After a percept (stimulus) has been approached and examined several times during the learning, the robot will evolve to activate responses that are most likely to reduce the need, that is, to find the road to follow, by more and more switching attention to the sensory input necessary for effective response, that is, always being able to move to the percept that agrees with the road without being stopped. Unattended signals or percepts that are not immediately necessary for the performance of a response may also be presented by the sensory systems and delivered, but will eventually be excluded from the parts of the motor system engaged in performing the current task. It can be seen that the weights of some percepts didn't change much.

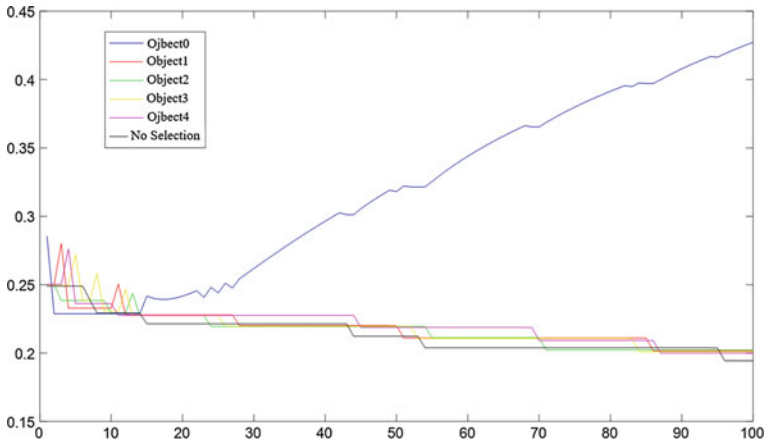


Fig. 16.2 The weight logs for each percept with an exploration rate of 0.00 when all the initial average weights are set to 0.25

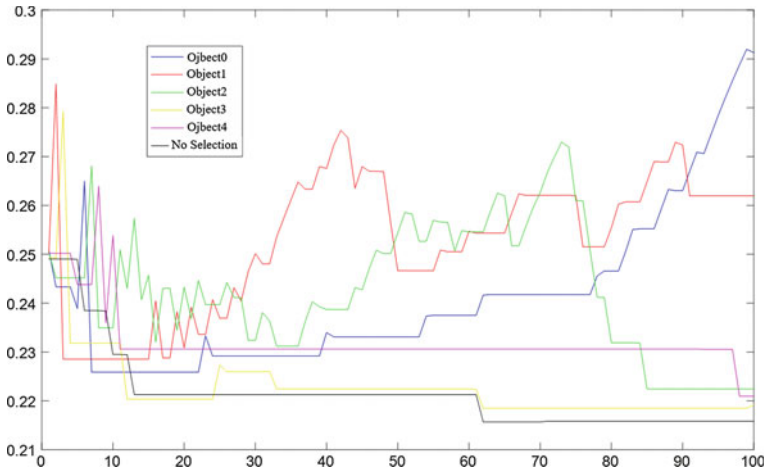


Fig. 16.3 The weight logs for each percept with an exploration rate of 0.05 when all the initial average weights are set to 0.25

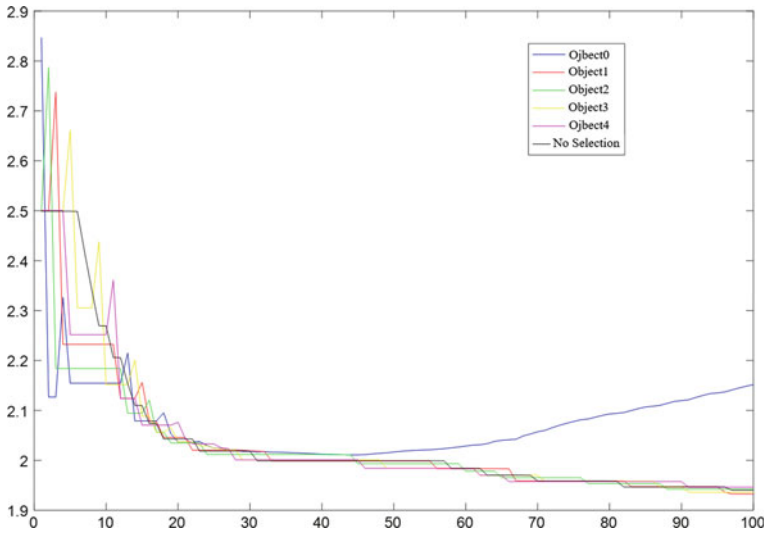


Fig. 16.4 The weight logs for each percept with an exploration rate of 0.00 when all the initial average weights are set to 2.5

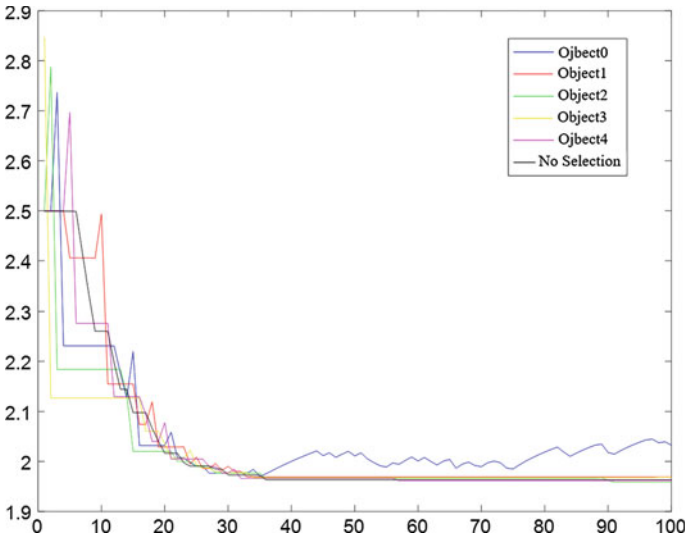


Fig. 16.5 The weight logs for each percept with an exploration rate of 0.05 when all the initial average weights are set to 2.5

This is because those percepts were rarely observed in the lower half of the image, even though they were considered when the database was built.

16.4.2 Experiment II: Learning the Subgoal Target Location

From the previous experiment, the robot identifies the road to follow. The current experiment tries to reduce our search area to a simple target locations to reach the subgoal, which has been conducted in the previous chapter and is briefly repeated here for reference. To learn the landmark-based target location, the following steps were taken in this experiment,

- The WM size is set to values of 1, 2, and 3 to determine which percept or combination of percepts should be selected for recognizing a landmark location.
- A set of total 30 training images representing landmark (i.e., target) and non-landmark (i.e., non-target) locations (15 for each kind of locations) as seen in Fig. 16.6 were collected. As mentioned, the presence of a ManholeCover in the foreground concrete road is an indication of a landmark.
- The training images from the set were selected randomly and processed in order to obtain the chunks for the WM system.
- The percepts generated for each image were duplicated and presented to the WM system. In other words, one chunk was marked as a landmark chunk and the other as a non-landmark chunk. Once the WM system selected a set of such chunks, a majority vote was taken to determine whether the location is a landmark one or



Fig. 16.6 (Left) A landmark image and (right) a non-landmark image

Table 16.2 Percentage of correctness for landmark classification

WM size	1000 trials (%)	5000 trials (%)
1	47.90	50.06
2	95.50	97.42
3	98.00	99.38

a non-landmark one. If there was a tie in the vote, it was interpreted as a non-landmark one by default. Since the training images were labeled, the WM system received a reward of 1 if the vote agreed with the label of the image, otherwise, it receives a reward of 0.

For each WM size, 1000 and 5000 trials of experiments were conducted. The results are summarized in Table 16.2 and quite good for all the cases.

16.4.3 Experiment III: Learning the Navigation Task

From the first experiment, the robot acquired the ability to identify each percept to be walkable or unwalkable so as to move toward the subgoal safely through the environment. From the second experiment, the robot can localize itself in the environment. In this experiment, the robot is meant to acquire skills by learning so as to chain the subgoal target locations to reach a goal position. More specifically, it is assumed that the robot is positioned in the middle of the road around the outdoor building looking forward. The ManholeCovers along the road indicate the presence of a landmark. The goal of this learning task is for the robot to navigate along the road in a straight line fashion until it reaches the third landmark. Three sample images corresponding to the subgoal target locations are shown in Fig. 16.7.

Therefore, the success of this experiment is fulfilled when the robot learns to stop at the third landmark. Three actions, such as “move”, “stop”, and “turn around”, are also and always presented to the working memory as chunks at each time step. The working memory system is rewarded or punished by the possible actions chosen by the robot along the navigation path. In addition to the action chunks, the working memory is presented with the percepts detected in the environment. However, only “ConcreteRoad” and “ManholeCover” percepts are presented, if recognized by the



Fig. 16.7 (Left) The image with the first landmark; (middle) the image with the second landmark; and (right) the image with the last landmark

perceptual system while other percepts are treated as distractors. The working memory has a reserved "Nothing" chunk when the system does not make any selection. Therefore, the maximum number of chunks that can be presented to the working memory is six. The working memory size is set to 2. One slot is for an action chunk and the other one is for a percept chunk. Thus, the working memory is required to learn to select an action chunk as well as a percept chunk. During the process, the robot moves with a small increment distance, stops, takes a picture and makes a decision. The time required for this cycle is referred to as one time step. Then the cycle repeats. This process continues until the navigation task is complete. The procedure for Experiment III is as follows,

- Initially in the environment, the robot is positioned in the middle of the road around the outdoor building looking forward. To begin learning, it captures an image and segments it to obtain the percepts.
- The working memory size is set to 2. One slot is for an action chunk and the other one is for a percept chunk. Once the candidate input chunk vector is presented to working memory, the working memory is required to learn to select an action chunk as well as a percept chunk and a decision is made. The robot either moves, stops or turns around depending on the action selection made by the working memory at each time step.
- The robot starts by moving towards the road for one meter, stops and makes a decision. Then the cycle is repeated until the third landmark is reached.

With the size of input chunk vectors being set to six, the training is performed in a simulated environment and all the possible chunks are presented to the robot at each time step. The robot is trained for each of the three regions of the navigation path as shown in Fig. 16.7. Each region may be thought of as the state of the robot (i.e., state 0 is being located at region 1, state 1 is at region 2, state 2 is at region 3, and state 3 is at region 4). The robot is required to choose the correct action chunk at each state. In addition, it must learn to remember the landmark percept if recognized along the navigation path. The robot needs to learn a considerable amount of information from a large combinatorial chunk space in four different states. The robot is trained for each region (each of the states from zero to three) of the navigation path and a reward value is given.

However, in order to speed up the learning, the training state is switched from one to another, only if multiple numbers of positive rewards are received in a row at each state. For instance, the number of positive rewards required to switch the state from one to another is assumed to be ten. A successful trial is defined as not receiving any

negative reward. Therefore, the system must receive 40 positive rewards (10 for each region) in a row in order for the trial to be considered as successful. The number of received positive rewards in a row is referred to as the positive reward threshold (PRT). Figures 16.8 and 16.9 shows the learning curve of the system, as a plot of the average reward values over a sliding window of size 25 over 5000 trials for several PRTs.

Table 16.3 shows the number of successful trials for each plots in Figs. 16.8 and 16.9.

These numbers are reasonable under the very strict rule, that is, receiving multiple positive rewards in a row. Moreover, the initial training stage is also included in these results. If, at any time step of the training, the system fails to choose the correct chunk combination, the whole trial is considered to be an unsuccessful trial. In all of the cases, there is a positive learning trend. As the PRT is increased, the system makes fewer mistakes overall and the speed of the training is significantly increased in terms of episodes. With the system settings described in the above, the time required to train

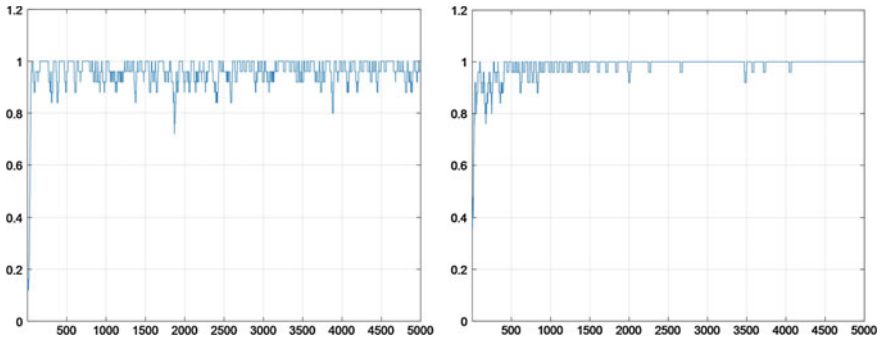


Fig. 16.8 (Left) Learning curve of the system for PRT of value 1; (right) learning curve of the system for PRT of value 10

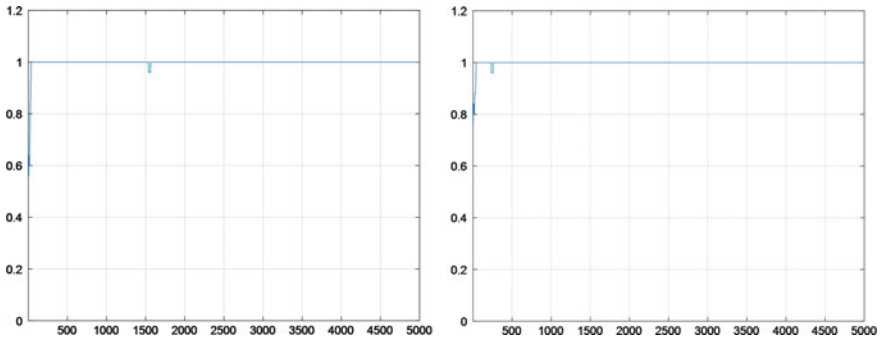


Fig. 16.9 (Left) Learning curve of the system for PRT of value 15; (right) learning curve of the system for PRT of value 20

Table 16.3 Number of successful trials for 5000 trials and for different reward thresholds

Positive reward threshold	Successful trials (out of 5000)	Percentage of successful trials (%)
1	3765	75.30
10	4907	98.14
15	4986	99.72
20	4991	99.82

the system is 21 minutes when PRT is set to 10, and 34 minutes for a PRT being set to 20. Thus, the training time is relatively short in a simulated environment. In this final experiment, we have particularly demonstrated that the robot has successfully learned a complex navigation task based on previously learned fundamental behaviors.

16.5 Conclusions

In this chapter, we investigated a machine learning based sensor-motor coordination model for local path planning of an autonomous mobile robot working in an unknown outdoor environment. Particularly, an incremental percept acquisition system was used for near real-time percept generation and the working memory toolkit has been incorporated into robot control system to learn the association between percepts and motion. Based on the model, we implemented a visual landmark based navigation approach to provide a solution to one of the most important problem in mobile robotics, navigation in an unknown outdoor environment. Experiments conducted validated the proposed vision-based perceptual learning model by successfully accomplishing a simple sensor-motor coordinated navigation task in a real-world outdoor environment.

References

Akbarimajd, A., & Hassanzadeh, A. (2011). A novel cellular automata based real time path planning method for mobile robots. *Internatioal Journal of Engineering Research and Applications*.

Akbarimajd, A., & Lucas, C. (2006). A new architecture to execute CAs-based path-planning algorithm in mobile robots. In *Proceedings of the IEEE International Conference on Mechatronics* (pp. 478–482).

Amato, N. M., & Wu, Y. (1996). A randomized roadmap method for path and manipulation planning. In *Proceedings of IEEE International Conference on Robotics and Automation* (Vol. 1, p. 7).

Amato, N. M., Bayazit, O. B., Dale, L. K., Jones, C., & Vallejo, D. (1998). OBPRM: An obstacle-based PRM for 3D workspaces. In *Proceedings of the Third Workshop on the Algorithmic Foundations of Robotics on Robotics: The Algorithmic Perspective* (pp. 155–168).

Arkin, R. C. (1998). *Behavior-based robotics*. MIT Press.

Barraquand, J., Langlois, B., & Latombe, J.-C. (1992). Numerical potential field techniques for robot path planning. *IEEE Transactions on Systems, Man, and Cybernetics, 22*(2), 224–241.

Behring, C., Bracho, M., Castro, M., & Moreno, J. A. (2000). An algorithm for robot path planning with cellular automata. In *Proceedings of the 4th International Conference on Cellular Automata for Research and Industry* (pp. 11–19).

Bohlin, R., & Kavraki, L. E. (2000). Path planning using lazy PRM. In *Proceedings of IEEE International Conference on Robotics and Automation* (Vol. 1, pp. 521–528). San Francisco, CA.

Boor, V., Overmars, M. H., & van der Stappen, A. F. (1999). The Gaussian sampling strategy for probabilistic roadmap planners. In *Proceedings of 1999 IEEE International Conference on Robotics and Automation* (pp. 1018–1023).

Branicky, M. S., LaValle, S. M., Olson, K., & Yang, L. (2001). Quasi-randomized path planning. In *Proceedings of the IEEE International Conference on Robotics and Automation* (pp. 1481–1487).

Buhmann, J. M., Lange, T., & Ramacher, U. (2005). Image segmentation by networks of spiking neurons. *Neural Computation, 17,* 1010–1031.

Butler, A. B., & Hodos, W. (1996). *Comparative vertebrate neuroanatomy: Evolution and adaptation*. Wiley.

Frazzoli, E., Dahleh, M. A., & Feron, E. (2002). Real-time motion planning for agile autonomous vehicles. *Journal of Guidance, Control and Dynamics, 25,* 116–129.

Fukuda, T., Michelini, R., Potkonjak, V., Tzafestas, S., Valavanis, K., & Vukobratovic, M. (2001). How far away is "artificial man"? *IEEE Robotics & Automation Magazine, 8*(1), 66–73.

Hart, P., Nilsson, N., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100–107.

Hunter, J. (2005). Human motion segmentation and object recognition using fuzzy rules. In *Proceedings of 14th Annual IEEE International Workshop on Robot and Human Interactive Communication (RO-MAN'05)* (pp. 210–216), Nashville, TN, August 13–15.

Ioannidis, K., Sirakoulis, G. C., & Andreadis, I. (2008). A cellular automaton collision-free path planner suitable for cooperative robots. In *Panhellenic Conference on Informatics*. IEEE Computer Society.

Ioannidis, K., Sirakoulis, G., & Andreadis, I. (2011). Cellular ants: A method to create collision free trajectories for a cooperative robot team. *Robotics and Autonomous Systems*.

Kavraki, L. E. (1995). *Random networks in configuration space for fast path planning*. Stanford, CA: Stanford University.

Kavraki, L., & Latombe, J. C. (1994). Randomized preprocessing of configuration space for fast path planning. In *Proceedings of IEEE International Conference on Robotics Automation (ICRA'94)* (p. 7).

Klein, R., Langetepe, E., & Nilforoushan, Z. (2009). Abstract voronoi diagrams revisited. *Computational Geometry: Theory and Applications, 42*(9), 885–902.

Lavalle, S. (1998). *Rapidly-exploring random trees: A new tool for path planning* (Research Report 9811). Department of Computer Science, Iowa State University.

LaValle, S. M., & Kuffner, J. J., Jr. (2001). Randomized kinodynamic planning. *International Journal of Robotic Research, 20,* 378–400.

Lee, D. T., & Preparata, F. P. (1984). Euclidean shortest paths in the presence of rectilinear barriers. *Networks, 14,* 93–410.

Liu, F., Liang, S., Xian, X., & Bi, H. (2012). Optimal path planning for mobile robot in consideration of road attributes. *ICIC Express Letters, 6,* 281–287.

Lumelsky, V. J., & Stepanov, A. A. (1987). Path-planning strategies for a point mobile automaton moving amidst unknown obstacles of arbitrary shape. *Algorithmica, 2,* 403–430.

Marchese, F. (2005). A reactive planner for mobile robots with generic shapes and kinematics on variable terrains. In *Proceedings of International Conference on Advanced Robotics (ICAR'05)* (pp. 23–30).

Mitchell, M. (1996). Computation in cellular automata: A selected review. In *Non-standard computation* (pp. 385–390).

Mitchell, J. S. B., & Papadimitriou, C. H. (1991). The weighted region problem: Finding shortest paths through a weighted planar subdivision. *Journal of the ACM, 38*(1), 18–73.

Nehmzow, U. (2003). *Mobile robotics: A practical introduction*. London, New York: Springer.

Oliveira, G., Martins, L. G. A., de Carvalho, L. B., & Fynn, E. (2009). Some investigations about synchronization and density classification tasks in one-dimensional and two-dimensional cellular automata rule spaces. *Electronic Notes in Theoretical Computer Science, 252,* 121–142.

Ramer, C., Reitelshofer, S., & Franke, J. (2013). A robot motion planner for 6-DOF industrial robots based on the cell decomposition of the workspace. In *Proceedings of the 2013 44th International Symposium on Robotics (ISR'13)* (pp. 1–4).

Rosenberg, A. (2007). Cellular automata. In *Parallel and distributed processing and applications* (pp. 78–90).

Shu, C., & Buxton, H. (1995). Parallel path planning on the distributed array processor. *Parallel Computing, 21*(11), 1749–1767.

Soofiyani, F. R., Rahmani, A. M., & Mohsenzadeh, M. (2010). A straight moving path planner for mobile robots in static environments using cellular automata. In *Proceedings of International Conference on Computational Intelligence, Communication Systems and Networks* (pp. 67–71).

Sperry, R. W. (1952). Neurology and the mind-brain problem. *American Scientist, 40,* 291–312.

Stentz, A. (1997). Optimal and efficient path planning for partially known environments. In M. H. Hebert, C. Thorpe, & A. Stentz (Eds.), *Intelligent unmanned ground vehicles* (pp. 203–220). US: Springer.

Striedter, G. (2004). *Principles of brain evolution*. Sinauer Associates.

Tzionas, P., Thanailakis, A., & Tsalides, P. (1997). Collision-free path planning for a diamond shaped robot using two-dimensional CA. *IEEE Transactions on Robotics and Automation, 13*(2), 237–250.

Yu, J., Du, H., & Zhou, L. (2013). Research about local path planning of moving robot based on improved artificial potential field. In *Proceedings of Chinese Control and Decision Conference (CCDC'13)* (pp. 2861–2865).

Zhang, Y., Fattahi, N., & Li, W. (2013). Probabilistic roadmap with self-learning for path planning of a mobile robot in a dynamic and unstructured environment. In *Proceedings of the IEEE International Conference on Mechatronics & Automation (ICMA'13)* (pp. 1074–1079).